ARM 嵌入式 Linux 系统构建与驱动开发范例

周立功　陈明计　陈　渝　著

高清PDF藏
gqpdf.com

北京航空航天大学出版社

内 容 简 介

本书介绍 μClinux 内核编程的一个方面——使用内核模块编写设备驱动程序。主要内容包括：介绍如何建立基于 ARM7 的交叉开发环境以及如何向 ARM7 芯片上移植 μClinux；分析 μClinux 上一个已在 ARM7 上成功运行的最简单的内核模块；介绍 μClinux 的字符设备驱动程序、块设备驱动程序和网络设备驱动程序的编写、编译及使用，这些范例均在 ARM7 平台编译和使用，但可以方便地移植到其他平台。

本书通俗易懂，是一本面向初学者的 μClinux 内核编程图书，对资深 μClinux 内核编程人员也有参考价值。可作为高等院校相关专业师生以及从事嵌入式系统应用开发工程师的参考资料。

图书在版编目（CIP）数据

ARM 嵌入式 Linux 系统构建与驱动开发范例/周立功等著. —北京：北京航空航天大学出版社，2006.1

ISBN 7-81077-749-1

Ⅰ.A… Ⅱ.周… Ⅲ.①微处理器，ARM②Linux 操作系统 Ⅳ.①TP332②TP316.89

中国版本图书馆 CIP 数据核字（2006）第 002835 号

©2006，北京航空航天大学出版社，版权所有。
未经本书出版者书面许可，任何单位和个人不得以任何形式或手段复制或传播本书内容。侵权必究。

ARM 嵌入式 Linux 系统构建与驱动开发范例

周立功 陈明计 陈 渝 著

责任编辑 王慕冰

*

北京航空航天大学出版社出版发行

北京市海淀区学院路 37 号（100083） 发行部电话：010-82317024 传真：010-82328026

http://www.buaapress.com.cn E-mail：bhpress@263.net

涿州市新华印刷有限公司印装 各地书店经销

*

开本：787×960 1/16 印张：25.25 字数：566 千字

2006 年 1 月第 1 版 2006 年 1 月第 1 次印刷 印数：5 000 册

ISBN 7-81077-749-1 定价：38.00 元

更多高清PDF图书 www.gqpdf.com

前　言

简洁就是美。

研究开源软件的热潮已经遍及神州大地，作为开源软件的重头戏 Linux 自然成为大家的研究重点，而其嵌入式分支 μClinux 也备受关注。对于大多数开源软件来说，由于开发者不能从中获利，因此造成大多数开源项目缺少文档。同样，多人协作开发的 Linux 也具有这个特点，即缺少“官方”文档。对于其他非官方写作的文档、资料等，由于其作者本身已经是高手，他们编写的资料对已入门者具有重要的参考价值，所以只可作为“手册”来使用。不过，这些资料大多是分析内核的，对于应用涉及不多。作为 Linux 的分支 μClinux 也继承了这些特点。

对于 μClinux 的应用来说，主要有 3 方面：编写应用软件、编写设备驱动程序和移植 μClinux。

因为 Linux 和 μClinux 的应用程序编程接口与 Unix 兼容，所以这方面的资料较多，也比较容易入门。而移植 μClinux 由专业人员完成，一般的应用工程师接触不到。对于设备驱动程序来说，尽管它在 Linux 和 μClinux 的源代码中比例最大，但仍不能包括所有设备驱动程序。在一个应用中，这些（还不存在的）设备驱动程序与应用密切相关，只能由应用工程师完成，至少是部分完成。即使有现成的设备驱动程序，也未必适合这个应用，也需要重写。

编写驱动程序是如此重要，但针对初学者的资料却很少，大多数读者对于编写驱动程序始终没有清晰的脉络。可以想象，这样编写的程序会存在多少隐患。笔者针对这种情况，通过阅读大量的资料和源码，斧去大量的细枝繁叶，使其脉络清晰，并设计多个实际的设备驱动程序作为例子，便成此书。

希望读者在阅读本书后，由衷地说：“编写 μClinux 设备驱动程序原来是如此简单！”

本书的各章节安排如下：

第 1 章是 μClinux 内核简介。本章主要讲述了 μClinux 和 Linux 的渊源及差别，并且对 μClinux 的开发环境、如何编译 μClinux 内核及其建立文件系统进行了详细的描述。

第 2 章是使用 GNU Tools 建立 μClinux 开发环境。本章主要介绍了在嵌入式 Linux 开发过程中所用到的 GNU 工具。

第 3 章是移植 μClinux 到 LPC2200 实例。本章以 LPC2200 为例，简单地介绍了如何把 μClinux 移植到一个新系统中。

第 4 章是 μClinux 设备驱动程序概述。本章首先介绍了 Linux（μClinux）设备驱动程序相关知识，包括简单的使用；然后主要介绍了一个最简单的设备驱动程序，包括源码分析和 Makefile 文件的编写和分析。

第 5 章是字符设备驱动程序初步。本章首先介绍了 Linux(μClinux)字符设备驱动程序的相关概念;然后主要介绍了一个最简单的字符设备驱动程序,主要包括源码分析和编译使用;最后介绍程序员如何使用字符设备。

第 6 ~ 10 章是 5 个字符设备驱动实例。这 5 章分别以 GPIO、PWM、A/D、SPI 和 I^2C 等硬件为例,通过分析这些设备驱动程序源码,详细介绍了编写字符设备需要了解的一些基本知识及设计思想。

第 11 章是块设备驱动程序初步。本章首先介绍了 Linux(μClinux)块设备驱动程序的相关概念;然后主要介绍了一个最简单的块设备驱动程序,主要包括源码分析和编译使用;最后介绍如何使用块设备。

第 12 章是简单的块设备编写范例。本章以 NAND FLASH K9f2808 为例介绍了实际的 Linux(μClinux)块设备驱动程序的相关知识。首先简单介绍这个驱动,然后对源码进行分析。

第 13 章是基于 ZLG/FS 驱动程序的通用块设备驱动程序。本章以一个通用块设备驱动程序为例介绍了实际的 Linux(μClinux)块设备驱动程序的相关知识。首先介绍这个驱动的使用方法,然后对源码进行分析。

第 14 章是转换 ZLG/FS V1.10 驱动为 μClinux 驱动。本章是第 13 章的延续,主要介绍如何把 ZLG/FS V1.10 的驱动转换成 μClinux 的驱动程序。

第 15 章是网络设备驱动程序初步。本章主要介绍了 Linux(μClinux)网络驱动程序的一般编写方法和相关知识,包括源码分析和编译使用。

第 16 和 17 章是 2 个网络设备驱动实例。这 2 章分别以 RTL8019 和 DM9000 等硬件为例,通过分析这些设备驱动程序源码,详细介绍了编写网络设备需要了解的一些基本知识及设计思想。

本书是面向初学者的入门书籍,不可能也没有必要介绍编写设备驱动程序的各个方面。如果读者需要深入学习,可参考本书参考文献中列出的资料及其他相关书籍和源码。

本书第 1 ~3 章由清华大学计算机系陈渝博士及其学生康烁、杨坚编写,其他各章主要由陈明计编写。另外,陈锡炳、黄邵斌、甘达、周立山、郑明远等参与了部分具体工作。全书由周立功负责规划、统稿与审核。

本书介绍的设备驱动程序均由广州周立功单片机发展有限公司设计,是基于 μClinux V2.4.x 版本的,均在其出品的开发套件上测试通过。软件作者会尽力保证软件的正确性,但限于作者的理论水平和经验,本书及软件中难免有疏忽的地方。因为软件的发展时间不长,所以不可能十分完善。不过,软件作者会不断地升级软件,力求软件越来越完善。

感谢北京航空航天大学出版社的大力支持,他们为规划本书付出了很大的心血。与此同时,也感谢清华大学邵贝贝教授的帮助。如果没有他们的努力,我想这项工作很难及时完成。

周立功

2005 年 10 月 26 日

更多高清PDF图书 www.gqpdf.com

目　录

第 1 章　μClinux 内核简介

1.1　μClinux 简介 …… 1
1.2　μClinux 内核与传统 Linux 内核的比较 …… 2
1.3　μClinux 运行条件 …… 5
1.4　μClinux 的目录结构 …… 5
1.5　μClinux 的开发流程 …… 6
1.5.1　编译 μClinux 内核 …… 6
1.5.2　创建文件系统 …… 8
本章小结 …… 10

第 2 章　使用 GNU Tools 建立 μClinux 开发环境

2.1　GNU Tools 简介 …… 11
2.2　GCC 编译器 …… 13
2.2.1　GCC 使用示例 …… 14
2.2.2　编译内核时使用的编译选项 …… 20
2.3　binutils 二进制工具集 …… 21
2.3.1　ar 软件 …… 22
2.3.2　nm 软件 …… 24
2.3.3　objcopy 软件 …… 27
2.3.4　objdump 软件 …… 29
2.3.5　readelf 软件 …… 32
2.4　ld 链接器 …… 33
2.4.1　ld 选项 …… 34
2.4.2　链接描述文件介绍 …… 35
2.4.3　链接描述文件的语法 …… 36
2.4.4　链接描述文件例子 …… 39
2.5　make 介绍 …… 42
2.5.1　GNU make 简介 …… 42
2.5.2　Makefile 及其语法 …… 43

高清 PDF gqpdf.com

2.5.3 μClinux-dist2.4 中的 Makefile 分析 …… 52
2.6 GDB 调试器 …… 58
2.6.1 GDB 简介 …… 58
2.6.2 GDB 调试程序示例 …… 59
2.6.3 gdbserver 介绍 …… 62
2.7 GNU Tools 的制作和安装 …… 63
2.7.1 从源码制作交叉编译环境 …… 63
2.7.2 安装二进制的交叉工具链 arm-elf 工具 …… 63
2.8 使用 GNU Tools 编译 μClinux 内核和应用程序 …… 64
2.8.1 使用 GNU Tools 编译 μClinux 内核 …… 64
2.8.2 使用 GNU Tools 在 μClinux 中编译应用程序 …… 68
本章小结 …… 70
第 3 章 移植 μClinux 到 LPC2200 实例
3.1 选择开发环境 …… 71
3.2 移植内核 …… 71
3.2.1 增加体系架构和机型相关的代码 …… 72
3.2.2 添加中断系统 …… 74
3.2.3 增加其他代码 …… 83
3.2.4 修改 Makefile 和配置菜单 …… 95
3.3 移植驱动程序 …… 98
3.3.1 时钟驱动程序的移植 …… 98
3.3.2 串口驱动程序的移植 …… 100
本章小结 …… 101
第 4 章 μClinux 设备驱动程序概述
4.1 设备驱动程序基础知识 …… 102
4.1.1 驱动程序分类 …… 102
4.1.2 加载和删除驱动程序 …… 103
4.2 最简单的内核模块 …… 104
4.2.1 内核模块的概念 …… 104
4.2.2 编写内核模块 …… 104
4.2.3 关于内核模块初始化(加载)函数 …… 106
4.2.4 关于内核模块清除(卸载)函数 …… 106
4.2.5 关于 printk()函数 …… 106
4.3 编写 Makefile 文件 …… 107

更多高清PDF图书 www.gqpdf.com

4.3.1　Makefile 文件整体介绍 …… 107
4.3.2　Makefile 文件中的变量 …… 108
4.3.3　Makefile 文件中的规则 …… 109
本章小结 …… 111
第 5 章　字符设备驱动程序初步
5.1　几个概念 …… 112
5.1.1　关于主设备号和从设备号 …… 112
5.1.2　关于设备节点、节点文件和设备文件 …… 112
5.2　最简单的字符设备驱动程序 …… 113
5.2.1　驱动程序代码 …… 113
5.2.2　驱动程序的初始化函数 …… 116
5.2.3　驱动程序的清除函数 …… 116
5.2.4　保存设备信息的结构体 …… 117
5.2.5　关于 file 结构 …… 122
5.2.6　内核空间与用户空间交换数据 …… 125
5.2.7　内核空间与 I/O 空间交换数据 …… 126
5.2.8　ioctl() 方法的编写 …… 127
5.2.9　一些宏定义 …… 130
5.2.10　关于头文件 …… 130
5.2.11　其他说明 …… 131
5.3　使用字符设备驱动程序 …… 131
5.3.1　建立设备文件 …… 131
5.3.2　加载驱动程序 …… 131
5.3.3　接口函数 …… 131
5.3.4　示例代码 …… 137
5.3.5　编写 Makefile 文件 …… 138
本章小结 …… 139
第 6 章　GPIO（字符设备）驱动程序编写范例
6.1　在应用程序中使用 GPIO 驱动程序 …… 140
6.1.1　建立设备文件 …… 140
6.1.2　可使用的函数 …… 140
6.1.3　提供给应用程序的头文件 …… 141
6.1.4　设置 GPIO 口数据传输方向 …… 142
6.1.5　让 GPIO 口输出指定电平 …… 144

gqpdf.com

6.1.6 读 GPIO 口输出电平 …… 146
6.1.7 读 GPIO 口引脚上的电平 …… 147
6.2 编 译 …… 148
6.3 源码分析 …… 149
6.3.1 初始化函数 …… 149
6.3.2 清除函数 …… 150
6.3.3 open() 方法 …… 150
6.3.4 release() 方法 …… 151
6.3.5 ioctl() 方法 …… 151
6.3.6 其他说明 …… 158
本章小结 …… 159
第 7 章 PWM(字符设备)驱动程序编写范例
7.1 在应用程序中使用 PWM 驱动程序 …… 160
7.1.1 建立设备文件 …… 160
7.1.2 可使用的函数 …… 161
7.1.3 提供给应用程序的头文件 …… 161
7.1.4 PWM 总体控制 …… 163
7.1.5 允许某路 PWM 输出 …… 164
7.1.6 禁止某路 PWM 输出 …… 164
7.1.7 设置某路 PWM 高电平宽度 …… 165
7.2 编 译 …… 166
7.3 源码分析 …… 167
7.3.1 为驱动程序增加中断服务程序 …… 167
7.3.2 关于临界区 …… 169
7.3.3 初始化函数 …… 170
7.3.4 清除函数 …… 170
7.3.5 open() 方法及中断处理 …… 171
7.3.6 release() 方法 …… 172
7.3.7 ioctl() 方法 …… 173
7.3.8 其他说明 …… 179
本章小结 …… 179
第 8 章 A/D(字符设备)驱动程序编写范例
8.1 在应用程序中使用 A/D 驱动程序 …… 180
8.1.1 建立设备文件 …… 180

更多高清PDF图书 www.gqpdf.com

8.1.2　可使用的函数 …… 181
8.1.3　提供给应用程序的头文件 …… 181
8.1.4　设置 A/D 工作频率 …… 182
8.1.5　设置 A/D 转换精度 …… 182
8.1.6　读取 A/D 转换结果 …… 183
8.2　编　译 …… 183
8.3　源码分析 …… 184
8.3.1　初始化函数 …… 184
8.3.2　清除函数 …… 185
8.3.3　open () 方法 …… 186
8.3.4　release () 方法 …… 187
8.3.5　ioctl () 方法 …… 188
8.3.6　read () 方法 …… 189
8.3.7　其他说明 …… 191
本章小结 …… 191
第 9 章　SPI (字符设备) 驱动程序编写范例
9.1　在应用程序中使用 SPI 驱动程序 …… 192
9.1.1　建立设备文件 …… 192
9.1.2　使用 SPI 的特殊注意点 …… 193
9.1.3　可使用的函数 …… 193
9.1.4　提供给应用程序的头文件 …… 194
9.1.5　设置 SPI 分频值 …… 195
9.1.6　SPI 时钟相位控制 …… 195
9.1.7　SPI 时钟极性控制 …… 196
9.1.8　SPI 数据传输顺序控制 …… 197
9.1.9　开始和结束 SPI 操作 …… 198
9.1.10　发送同时接收一字节数据 …… 198
9.1.11　写数据到 SPI 从器件 …… 199
9.1.12　从 SPI 从器件读数据 …… 199
9.2　编　译 …… 200
9.3　源码分析 …… 201
9.3.1　初始化函数 …… 201
9.3.2　清除函数 …… 201
9.3.3　open () 方法 …… 202

9.3.4 release()方法 …… 204
9.3.5 ioctl()方法 …… 205
9.3.6 write()方法 …… 209
9.3.7 read()方法 …… 211
9.3.8 其他说明 …… 212
本章小结 …… 212
第 10 章 I^2C(字符设备)驱动程序编写范例
10.1 在应用程序中使用 I^2C 总线驱动程序 …… 213
10.1.1 建立设备文件 …… 213
10.1.2 可使用的函数 …… 214
10.1.3 提供给应用程序的头文件 …… 214
10.1.4 设置 I^2C 总线频率 …… 215
10.1.5 写数据到 I^2C 从器件 …… 215
10.1.6 从 I^2C 从器件读数据 …… 216
10.2 编 译 …… 216
10.3 源码分析 …… 217
10.3.1 初始化函数 …… 217
10.3.2 清除函数 …… 218
10.3.3 open()方法 …… 218
10.3.4 release()方法 …… 220
10.3.5 I^2C 中断处理 …… 220
10.3.6 ioctl()方法 …… 223
10.3.7 write()方法 …… 225
10.3.8 read()方法 …… 226
10.3.9 其他说明 …… 227
本章小结 …… 227
第 11 章 块设备驱动程序初步
11.1 几个概念 …… 228
11.1.1 关于主设备号和从设备号 …… 228
11.1.2 关于设备节点、节点文件和设备文件 …… 228
11.2 最简单的块设备驱动程序 …… 229
11.2.1 驱动程序代码 …… 229
11.2.2 注册初始化函数和清除函数 …… 231
11.2.3 驱动程序的初始化函数 …… 232

更多高清PDF图书 www.gqpdf.com

11.2.4　驱动程序的清除函数 …… 232
11.2.5　保存设备信息的结构体 …… 233
11.2.6　一些宏定义 …… 238
11.2.7　关于头文件 …… 238
11.2.8　其他说明 …… 239
11.3　使用块设备 …… 239
11.3.1　建立设备文件 …… 239
11.3.2　加载驱动程序 …… 239
11.3.3　把块设备(的某个分区)映射到指定的目录 …… 240
11.3.4　取消目录映射 …… 240
11.3.5　卸载驱动程序 …… 240
本章小结 …… 240
第 12 章　简单的块设备编写范例
12.1　使用范例 …… 241
12.1.1　范例简介 …… 241
12.1.2　编译驱动程序 …… 241
12.1.3　使用驱动程序 …… 242
12.2　范例程序源码分析 …… 244
12.2.1　关于主设备号 …… 244
12.2.2　几个数据结构 …… 246
12.2.3　驱动程序的初始化函数 …… 246
12.2.4　驱动程序的清除函数 …… 247
12.2.5　open () 方法 …… 248
12.2.6　release () 方法 …… 250
12.2.7　ioctl () 方法 …… 251
12.2.8　check_media_change () 方法 …… 253
12.2.9　revalidate () 方法 …… 253
12.2.10　关于数据传输 …… 254
12.2.11　其他说明 …… 260
本章小结 …… 260
第 13 章　基于 ZLG/FS 驱动程序的通用块设备驱动程序
13.1　背景知识 …… 261
13.1.1　驱动程序简介 …… 261
13.1.2　ZLG/FS 简介 …… 261

高清PDF 收藏 gqpdf.com

13.2 ZLG/FS V1.10 驱动程序使用指南 …… 263
13.2.1 接口函数 …… 263
13.2.2 参 数 …… 263
13.3 使用范例 …… 266
13.3.1 编译驱动程序 …… 266
13.3.2 使用驱动程序 …… 267
13.4 范例程序源码分析 …… 269
13.4.1 块设备信息结构体 …… 270
13.4.2 注册 ZLG/FS V1.10 驱动程序 …… 271
13.4.3 注销 ZLG/FS V1.10 驱动程序 …… 271
13.4.4 关于分区设备 …… 272
13.4.5 检测分区 …… 273
13.4.6 驱动程序的初始化函数 …… 276
13.4.7 驱动程序的清除函数 …… 277
13.4.8 open () 方法 …… 278
13.4.9 release () 方法 …… 280
13.4.10 ioctl () 方法 …… 282
13.4.11 check_media_change () 方法 …… 284
13.4.12 revalidate () 方法 …… 285
13.4.13 数据传输 …… 286
13.4.14 其他说明 …… 290
本章小结 …… 290
第 14 章 转换 ZLG/FS V1.10 驱动为 μClinux 驱动
14.1 转换步骤 …… 291
14.2 转换实例之 CF 卡驱动 …… 295
14.3 转换实例之 SD 卡驱动 …… 297
14.4 转换实例之 ZLG/FFS 驱动 …… 299
14.5 模板源码分析 …… 302
14.5.1 驱动程序的初始化函数 …… 302
14.5.2 驱动程序的清除函数 …… 303
14.5.3 open () 方法 …… 303
14.5.4 release () 方法 …… 304
14.5.5 get_info () 方法 …… 304
14.5.6 其他说明 …… 306

更多高清PDF图书 www.gqpdf.com

本章小结 …… 306
第 15 章　网络设备驱动程序初步
15.1　最简单的网络设备驱动程序 …… 307
15.1.1　驱动程序代码 …… 307
15.1.2　驱动程序的初始化函数 …… 310
15.1.3　驱动程序的清除函数 …… 311
15.1.4　保存设备信息的结构体 …… 311
15.1.5　数据接收与中断服务程序 …… 319
15.1.6　关于头文件 …… 321
15.1.7　其他说明 …… 321
15.2　使用网络设备驱动程序 …… 321
本章小结 …… 322
第 16 章　RTL8019 驱动程序编写范例
16.1　硬件原理 …… 323
16.2　使用源代码 …… 325
16.3　源码分析 …… 326
16.3.1　初始化函数 …… 326
16.3.2　清除函数 …… 327
16.3.3　init () 方法 …… 327
16.3.4　open () 方法 …… 329
16.3.5　stop () 方法 …… 332
16.3.6　set_config () 方法 …… 333
16.3.7　hard_start_xmit () 方法 …… 333
16.3.8　set_mac_address () 方法 …… 335
16.3.9　中断服务程序 …… 336
本章小结 …… 340
第 17 章　DM9000 驱动程序编写范例
17.1　硬件原理 …… 341
17.2　使用源代码 …… 341
17.3　源码分析 …… 344
17.3.1　头文件 DM9000.h …… 344
17.3.2　底层硬件操作函数 …… 346
17.3.3　初始化函数 …… 348
17.3.4　清除函数 …… 349

17.3.5 init () 方法 …… 349
17.3.6 open () 方法 …… 351
17.3.7 stop () 方法 …… 354
17.3.8 set_config () 方法 …… 355
17.3.9 hard_start_xmit () 方法 …… 356
17.3.10 set_mac_address () 方法 …… 357
17.3.11 中断服务程序 …… 358
本章小结 …… 360
附录 A 版权声明及许可协议
附录 B SmartARM2200 教学实验开发平台
B.1 产品概述 …… 363
B.2 产品目标 …… 364
B.3 产品特点 …… 364
B.4 产品组成 …… 368
B.5 硬件结构说明 …… 369
B.6 应用环境 …… 369
B.7 产品部件规格描述 …… 369
附录 C MagicARM2200 教学实验开发平台
C.1 产品概述 …… 370
C.2 产品目标 …… 371
C.3 产品特点 …… 371
C.4 产品组成 …… 376
C.5 硬件结构说明 …… 377
C.6 应用环境 …… 377
C.7 产品部件规格描述 …… 377
附录 D MagicARM2200 - S 教学实验开发平台
D.1 产品概述 …… 378
D.2 产品目标 …… 379
D.3 产品特点 …… 379
D.4 产品组成 …… 382
D.5 硬件结构说明 …… 384
D.6 应用环境 …… 384
D.7 产品部件规格描述 …… 384
参考文献

更多高清PDF图书 www.gqpdf.com

第1章　μClinux 内核简介

1.1　μClinux 简介

在 μClinux 这个英文单词中，μ 表示 Micro，“小的”之意，C 表示 Control，“控制”之意，所以 μClinux 就是 Micro-Controller-Linux，字面上的理解就是“针对微控制领域而设计的 Linux 系统”。由于 μClinux 和 Linux 渊源很深，因此为了介绍 μClinux，需要先介绍一下 Linux。事实上，μClinux 是由 Linux 衍生而来，广泛应用于嵌入式系统。

Linux 是著名的自由软件运动的产物，并充分体现了自由软件的精神，因此，对 μClinux 的介绍，就要从自由软件运动开始。自由软件运动的组织者是 FSF（Free Software Foundation）自由软件基金会。FSF 坐落在美国的 Boston，它是非营利组织，依靠广大自由软件爱好者的支持才得以发展壮大。FSF 现任主席是 Richard Stallman。FSF 的宗旨就是要保护人们自由使用、学习、拷贝、修改和发布软件的权利。FSF 最著名的项目就是 GNU 项目，它简直就是自由软件的代表。早在 1984 年，Richard Stallman 打算自己开发一个全新的操作系统，用来解放那些必须为购买商用操作系统付费的人们。他把这个新的操作系统取名为 GNU，即“GNU’s Not Unix”的递归缩写。为了完成 GNU 操作系统，Richard Stallman 首先完成了 C 语言编译器 GCC 和 C 语言库 glibc，还编写了编译工具 make 和编辑工具 Emacs。这些软件已成为今天广泛使用的自由软件代表作。随后在对操作系统的开发过程中，Richard Stallman 遇到了诸多困难，以致于进展缓慢。直到 1991 年，GNU 操作系统的内核依然没有完成。就在这时，芬兰的一位大学生 Linus Torvalds 创造了 Linux 内核的早期版本。Linus Torvalds 本人鼓励大家参与 Linux 的开发，并且充分地使用互联网来组建自己的团队，使得 Linux 系统逐步完善。Linus Torvalds 开放的思想和 GNU 项目的初衷一拍即合，于是 GNU 项目的各种软件很快被移植到 Linux 内核之上，这样 GNU 的软件和 Linux 内核形成了今天人们常说但又常误解的 GNU/Linux。这段传奇故事，在今天被人们用来和当年 Bill Gates 辍学创办微软的经历相提并论。

接下来，有必要澄清 GNU/Linux 的概念。Linux 只是一个操作系统内核，提供对内存、中断、文件系统、网络和进程的管理。Linux 本身没有提供任何应用软件，比如网页浏览器、E-mail 收发软件和 shell；它甚至没有提供系统软件：编译器和 C 语言库，因此一个纯粹的 Linux 内核是无法单独运行的。GNU 项目的软件，正好弥补了上述问题。因此，只有由 Linux

内核搭配 GNU 项目的软件组成的完整可用的操作系统才称为 GNU/Linux。很可惜，大多数情况下，我们使用的不是 GNU/Linux，因为严格的 GNU/Linux 系统的安装和维护需要高超的技术。为了方便 Linux 系统的使用，很多商业公司提供了商业发行版，比如 Redhat 等。商业发行版虽然使用 Linux 内核，但是打了很多补丁，即做了修改，因此内核的特性会有所变化；同时应用软件的选取也不受 GNU 项目的限制。这些 Linux 商业发行版有着自己的版本命名方式，比如 Redhat9.0。这个版本号(9.0)不是 Linux 内核的版本号，请严格区分。为了表示两者的不同，一般称 Redhat 商业发行版为“Red Hat GUN/Linux”，其他商业发行版的称呼类似。

Linux 最开始只运行在 x86 系统上面，后来逐步支持其他微处理器。在 Linux 2.0 发布后，有人就开始对 Linux 源码进行修改，使得它能在没有内存管理单元(Memory Management Unit，简称 MMU)的微控制器上运行。修改后的内核代码就是 μClinux。时至今日，μClinux 已得到广泛的认可和完善，它包含 2.0、2.4 和 2.6 的 Linux 内核，并且集成了很多有用的应用程序和库。同时 μClinux 的网站(http://www.uClinux.org)也发布针对 μClinux 平台的各种基于 GNU Tools 的交叉编译器。这些相关的软件极大地方便了嵌入式开发者的工作。因此，目前在嵌入式 Linux 领域中，已经产生了明显的选择：如果微控制器具有 MMU，就选择 Linux；否则，选择 μClinux。

接下来，将会给大家详细介绍 Linux 与 μClinux 的不同之处。

1.2 μClinux 内核与传统 Linux 内核的比较

μClinux 是针对工业控制领域的嵌入式 Linux 操作系统，它从 Linux 2.0/2.4 内核派生而来，沿袭了主流 Linux 的绝大部分特性，适合不具备 MMU 的微处理器/微控制器。支持没有 MMU 的 CPU 是 μClinux 与主流 Linux 的基本差异。

标准 Linux 是针对有 MMU 的 CPU 设计的。在这种处理器上，CPU 把虚拟地址送到 MMU，由 MMU 把虚拟地址映射为物理地址，通过赋予每个进程不同的虚拟-物理地址转换映射，支持不同进程之间的内存保护，所以标准的 Linux 访问的是虚拟内存。而对 μClinux 来说，其设计针对没有 MMU 的处理器，所以无法使用处理器的虚拟内存管理技术，只能直接访问物理内存。

μClinux 在对物理内存的管理上仍然采用分页管理方式，即系统在启动时把实际存储空间按页大小(在 32 位 CPU 中，页大小一般为 4 KB)进行分页，在加载应用程序时采用分页加载的方式。但由于 μClinux 运行在没有 MMU 管理的 CPU 上，所以实际上 μClinux 采用物理存储器管理策略。μClinux 系统中所有进程访问的地址都是实际的物理地址。操作系统对内存空间没有保护，各个进程实际上共享一个运行空间。一个进程在执行前，系统必须为进程分配足够的连续地址空间，然后全部载入主存储器的连续空间中。

同时，由于采用了一系列的技术，μClinux 内核和用户软件对内存大小的占用很小。如果

更多高清PDF图书 www.gqpdf.com

熟悉主流 Linux 的开发者注意到在 μClinux 下工作的一些差异，也就可以很快地熟悉 μClinux 的一些特性。对于设计内核或系统空间的应用程序的开发者，要特别注意 μClinux 既没有内存保护，也没有虚拟内存模型。另外，在有些内核系统调用的使用上，与标准 Linux 也有一些不同。

下面分 3 方面来分析它们的不同之处。

1. 内存保护

没有内存保护（memory protection）的操作会导致这样的结果：即使由无特权的进程来调用一个无效指针，也会触发一个地址错误，并潜在地引起程序崩溃，甚至导致整个系统的挂起。显然，在这样的系统上运行的代码必须仔细编程，并深入地测试，以确保健壮性和安全性。

对于普通的 Linux 来说，需要运行不同的用户程序，如果没有内存保护，将大大降低系统的安全性和可靠性；而对于嵌入式 μClinux 系统而言，由于所运行的程序往往是在出厂前已经固化的，不存在危害系统安全的程序侵入的隐患，因此只要应用程序经过较完整的测试，出现问题的概率就可控制在有限的范围内。

2. 虚拟内存

没有虚拟内存（virtual memory）主要会导致下面几个后果：

首先，由内核所加载的程序必须能独立运行，与它们在内存中的位置无关。实现这一目标的第一种办法是，一旦程序被加载到 RAM 中，程序的基准地址就“固定”下来；另一种办法是，产生只使用相对寻址的代码（称为“位置无关代码”，Position Independent Code，简称为 PIC）。μClinux 对这 2 种模式都支持。

其次，要解决在扁平（flat）的内存模型中的内存分配和释放问题。通常动态的内存分配会造成内存碎片，并可能耗尽系统的资源。对于使用了动态内存分配的那些应用程序来说，增强健壮性的一种办法是，用预分配缓冲区池（preallocated buffer pool）的办法来取代 malloc（）调用。由于 μClinux 中不使用虚拟内存，进出内存的页面交换也没有实现，因此不能保证页面会被加载到 RAM 中的同样位置。在常见的计算机上，操作系统允许应用程序使用比物理内存更大的内存空间，这往往是通过硬件虚拟内存管理和在硬盘上设立的交换分区来实现的。但是，在嵌入式系统中，通常都用 FLASH 存储器来代替硬盘，很难高效地实现内存页面交换的存取，因此，对运行的应用程序都限制其可分配空间不大于系统的物理内存空间。

最后，μClinux 支持的无 MMU 的 CPU 缺乏内存管理的硬件单元，使得 μClinux 的少数系统调用接口需要做些改变，其中最大的不同就是没有 fork 和 brk 系统调用。父进程调用系统调用 fork，fork 将复制父进程来创建一个子进程。在 Linux 下，系统调用 fork 是使用 copy-on-write 机制实现按需分配页来实现进程复制的，这样保证了复制的开销很小。由 μClinux 支持的是无 MMU 的 CPU，μClinux 无法实现按需分配页。为了弥补这一缺陷，μClinux 实现了系统调用 vfork。当父进程调用系统调用 vfork 来创建子进程时，两个进程共享它们的全部内存空间，包括堆栈。这样，子进程或者代替父进程执行（此时父进程已经挂起睡眠）直到子进程调

用系统调用 exit 退出执行，或者调用系统调用 exec（此时将加载可执行文件）执行一个新的进程。当子进程执行系统调用 exit 或者系统调用 exec 后，子进程会把父进程唤醒，父进程将继续往下执行。

在标准 Linux 上的某些程序依赖子进程来执行基本任务，使得即使在进程负载很重时，系统仍可保持一种“可交互”的状态。例如，有一个简单的网络后台程序 daemon，大量使用了系统调用 fork 创建子进程来完成工作。这个 daemon 总监听一个知名端口（或套接字）等待网络客户端来连接。当客户端连接时，这个 daemon 给它一个新的连接信息（新的 socket 编号），并调用系统调用 fork 产生子进程。子进程接下来就会与客户端在新的 socket 上进行连接，并完成事务处理。而此时父进程可与子进程并发执行，继续监听新的连接。

但在 μClinux 中，由于有上述情况，所以父进程和子进程无法并发执行，导致这些程序可能需要实质上的修改才能在 μClinux 下完成同样的任务。如果一个关键的应用程序非常依赖这样的结构，那就不得不对它重新编写了。

由于没有 MMU 的硬件支持，所以 μClinux 既没有实现可自动生长的堆栈，也没有实现系统调用 brk。这样，用户空间的程序必须使用系统调用 mmap 来分配内存。为了方便，在 μClinux 的 C 语言库中所实现的函数 malloc（）实质上就是调用了系统调用 mmap；而且，μClinux 在编译时可指定程序的堆栈大小，从而满足特定的需要。

3. 通用架构的内核变化

在 μClinux 的发布中，/linux/mmnommu 目录取代了/linux/mm 目录。前者就是修改后的内存管理子系统被修改，去除了 MMU 硬件的依赖，并在内核软件自身提供基本的内存管理函数。

很多子系统需要被重新修改、添加或重写。内核和用户内存分配及释放进程必须重新实现。对透明交互/页面调度的支持也被去除。

内核中，加入了支持“位置无关代码（PIC）”的程序加载模块，并使用了新的二进制目标码格式，称为扁平格式，用来支持 PIC（有非常紧凑的头部）。内核也提供了支持 ELF 格式的程序加载模块，用来支持使用固定基准地址的可执行程序。两种模式各有利弊，传统的 PIC 运行快，代码紧凑，但有代码大小限制。例如，Motorola 68K 架构的 16 位相对跳转限制了 PIC 程序不能超过 32 KB 大小，而采用运行期固定基址的方法使得没有了代码大小限制。不过，当程序被内核加载后导致了较多的系统开销。对于内核开发者来说，μClinux 基本上与 Linux 没有区别，惟一的区别就是不能利用 MMU 提供的内存管理。实际上，这对内核并没有影响。

Linux 下所有标准的可执行文件的格式在 μClinux 并不被支持，因为这些格式也用到了虚拟内存的一些功能。μClinux 使用的是另外一种扁平格式。扁平格式是一种简洁高效的可执行文件格式，它只包含可执行的代码和数据，还有一些把可执行文件加载到内存任意位置所需要的可重定位的信息。

更多高清PDF图书 www.gqpdf.com

1.3 μClinux 运行条件

μClinux 运行在没有 MMU 的处理器上。目前，μClinux 支持的 CPU 体系结构有 ARM、MIPS、PowerPC 和 m68k 等，并且支持多种流行的处理器及开发板。

在 μClinux 运行期间，嵌入式开发板最少提供 1 MB 的内存（更小的内存有可能也可运行 μClinux，但很不方便，需要对裁剪工作精打细算）。一般 2 MB 以上的内存比较适合 2.4 版本的 μClinux 内核的运行；如果要运行 2.6 版本的 μClinux 内核，最好拥有 4 MB 以上的内存。

μClinux 编译完后的内核映像文件不超过 1 MB。在 40 MHz 频率的处理器上运行 μClinux 没有问题，处理器的快慢只影响应用程序的快慢，对内核本身没有影响。

1.4 μClinux 的目录结构

在 μClinux 的发行包的顶层目录中包含以下几个目录。

- linux-2.0.x：μClinux 的内核，与 Linux-2.0.x 内核兼容。
- linux-2.4.x：μClinux 的内核，与 Linux-2.4.x 内核兼容。
- freeswan：来自 www.freeswan.org 的软件。
- lib：用户层应用程序用到的一些库，包括 libc、libm 等。
- uClibc：一个 libc 库，但它比通常使用的 glic 更小、更精简。
- user：μClinux 自带的用户层的应用程序，来自开源社区，可从互联网上免费获取。

下面几个目录包含了编译后的映像文件及输出结果。

- config：一个基本的 Linux 配置脚本，用来进行默认的配置。
- vendors：包含平台特定的一些编译信息和一些支持文件。这里还存放了对每一个 μClinux 支持的平台的默认文件。
- romfs：ROM 文件系统的目录结构。
- images：包含编译好的二进制内核文件，文件系统的映像文件及一个包含文件系统和内核的映像。

其中在 Linux-2.4 目录下，包含了与 μClinux 内核相关的所有文件，其目录如下。

- arch：包含硬件相关的代码，也就是 μClinux 内核的硬件抽象层。因为 μClinux 要支持很多不同的硬件，而它们的结构差别很大，所以就需要有一些代码屏蔽掉这些差别，让上层代码可以不受硬件差异性的影响。arch 目录就包含这些代码。
- kernel：包含 μClinux 内核的一些核心功能，最终用于生成内核映像文件。
- fs：μClinux 内核的文件系统。
- init：μClinux 内核的初始化代码。

- include：μClinux 内核的头文件。其中 include/asm - <arch>是硬件相关的头文件，include/linux 是内核核心使用的头文件。
- mm：μClinux 内核的内存管理。
- ipc：μClinux 内核的进程间通信。
- lib：μClinux 内核需要使用的库。
- net：μClinux 内核的网络系统。其中 net/ipv4 就是 TCP/IP 的协议栈。
- Documentation：μClinux 内核的帮助文档。

1.5 μClinux 的开发流程

μClinux 的开发流程与标准 Linux 的开发流程类似，而且还更简洁。下面简要介绍 μClinux 的两个重要开发过程：编译 μClinux 内核和创建文件系统。在第 2 章中还会进一步介绍 μClinux 的开发细节。

1.5.1 编译 μClinux 内核

编译 μClinux 前，首先要安装编译目标平台的编译器（在本例中，需要安装 arm-elf-tools 交叉编译环境）。在 μClinux 的官方网站一般都可免费下载到相应的交叉编译开发环境安装程序（如 arm-elf-tools-20040714. sh），然后在 root 用户下执行安装程序，就会自动把 arm-elf-tools 交叉编译环境安装好。假设已安装好交叉编译开发环境，那么就开始编译 μClinux 内核。

首先，把下载的 μClinux 内核解压，在当前目录下会生成 uClinux-dist 目录。进入 uClinux-dist 目录，在当前目录下运行“make menuconfig”，将会出现配置菜单（参见图 1.1）。

图 1.1 配置菜单

然后，进入 Vendor/Product Selection 菜单，选择开发板的厂商和相应的开发板型号。假设

更多高清PDF图书 www.gqpdf.com

选择了 GDB/Armulator(参见图 1.2)。

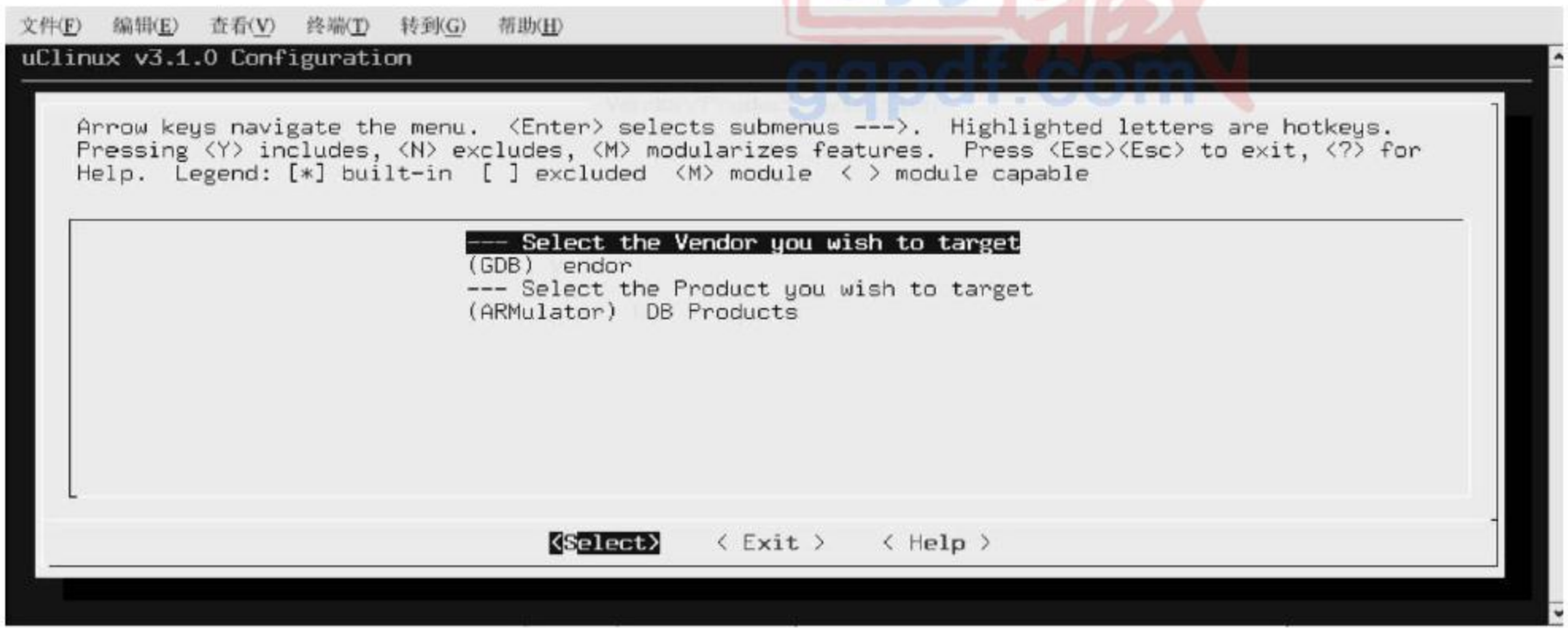

图 1.2 选择开发板

然后,回到上级菜单,选择内核和 C 库,在这里选择了 linux-2.4.x 和 uClibc(参见图 1.3)。

图 1.3 选择内核和 C 库

选择完毕,再选择 Exit,并且保存退出(参见图 1.4)。

然后在当前目录下运行“make dep”和“make”(这里假定开发系统中已安装并可执行 arm-elf-gcc 等基于 arm-elf-tools 的交叉编译工具),就生成了 Linux 的内核映像文件,其中二进制的内核文件位于 linux-2.4.x 目录下,文件名为 linux.bin。

1.5.2 创建文件系统

仅有 μClinux 内核是不够的,还需要各种应用程序才能完成各种功能。μClinux 发行包中

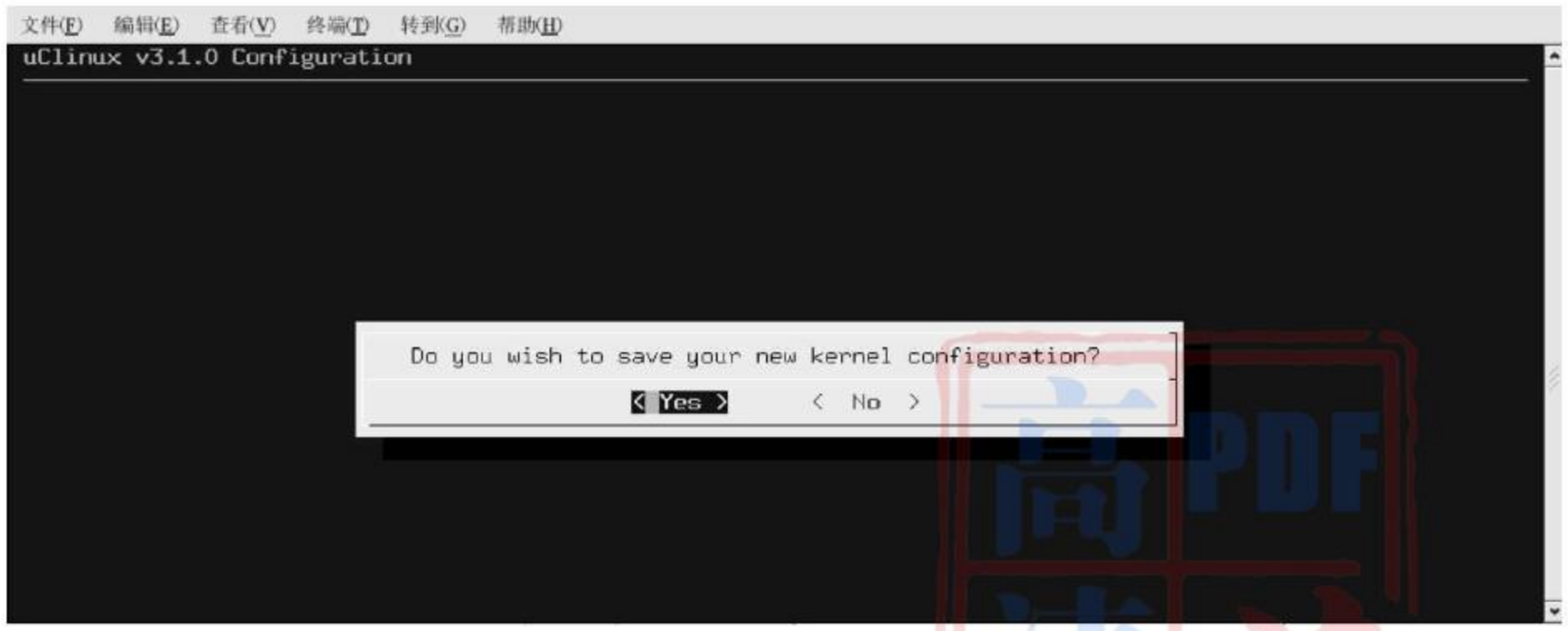

图 1.4 保存设置

包含了丰富的应用程序，其中包括 WEB 服务器、MySQL 数据库等，而这些应用程序必须位于某个文件系统上，μClinux 才能找到各应用程序的位置。可很轻松地利用 μClinux 提供的环境来创建文件系统，在解压后的 uClinux-dist 目录下运行“make menuconfig”，进入一个配置菜单（参见图 1.5）。

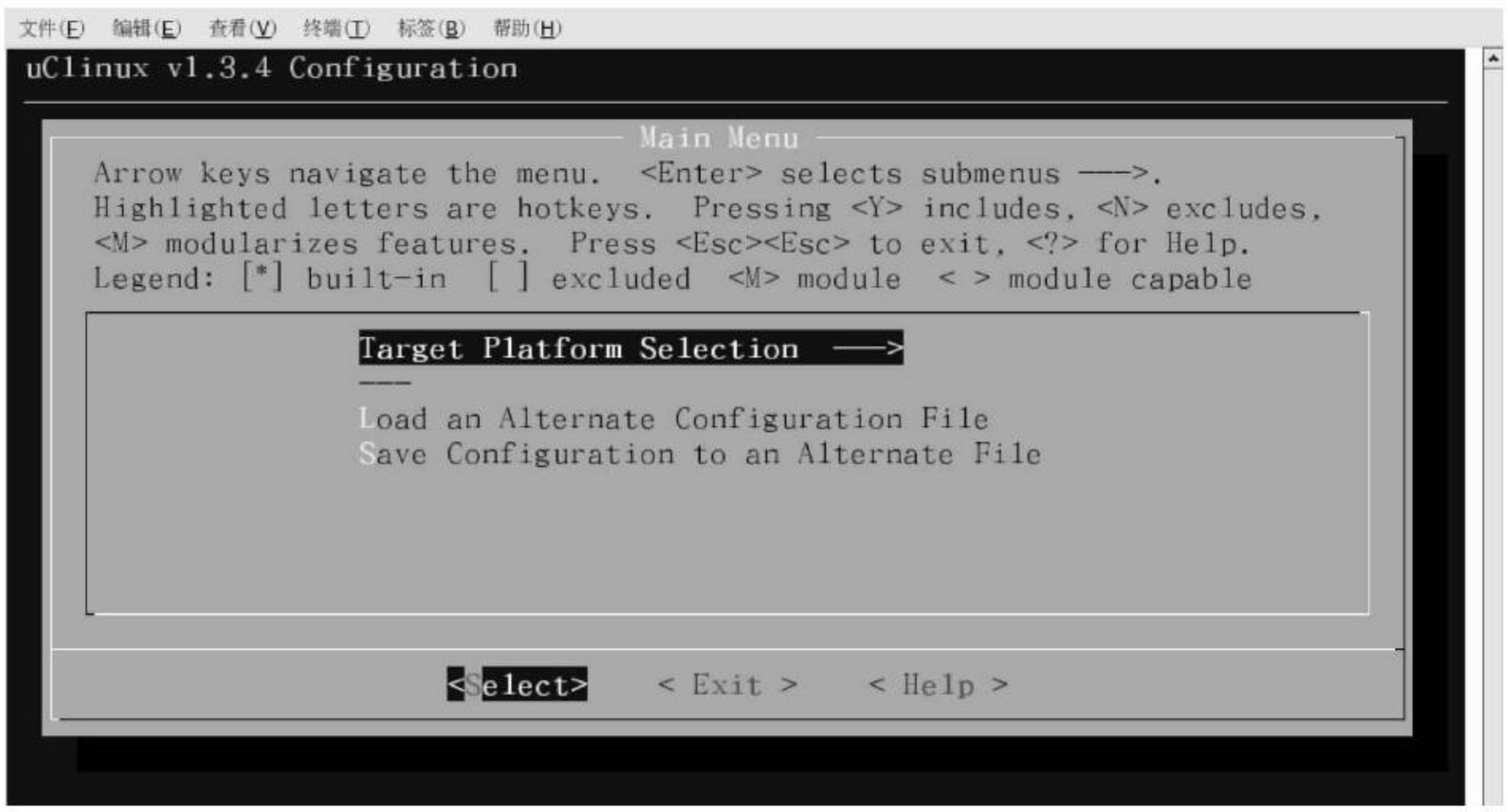

图 1.5 进入配置菜单

在 Vendor/Product 项中选择 GDB/ARMulator，并选择 Customize Vendor/User Settings (NEW)项（参见图 1.6）。

然后进入应用程序配置菜单，选择需要的应用程序（参见图 1.7）。

更多高清PDF图书 www.gqpdf.com

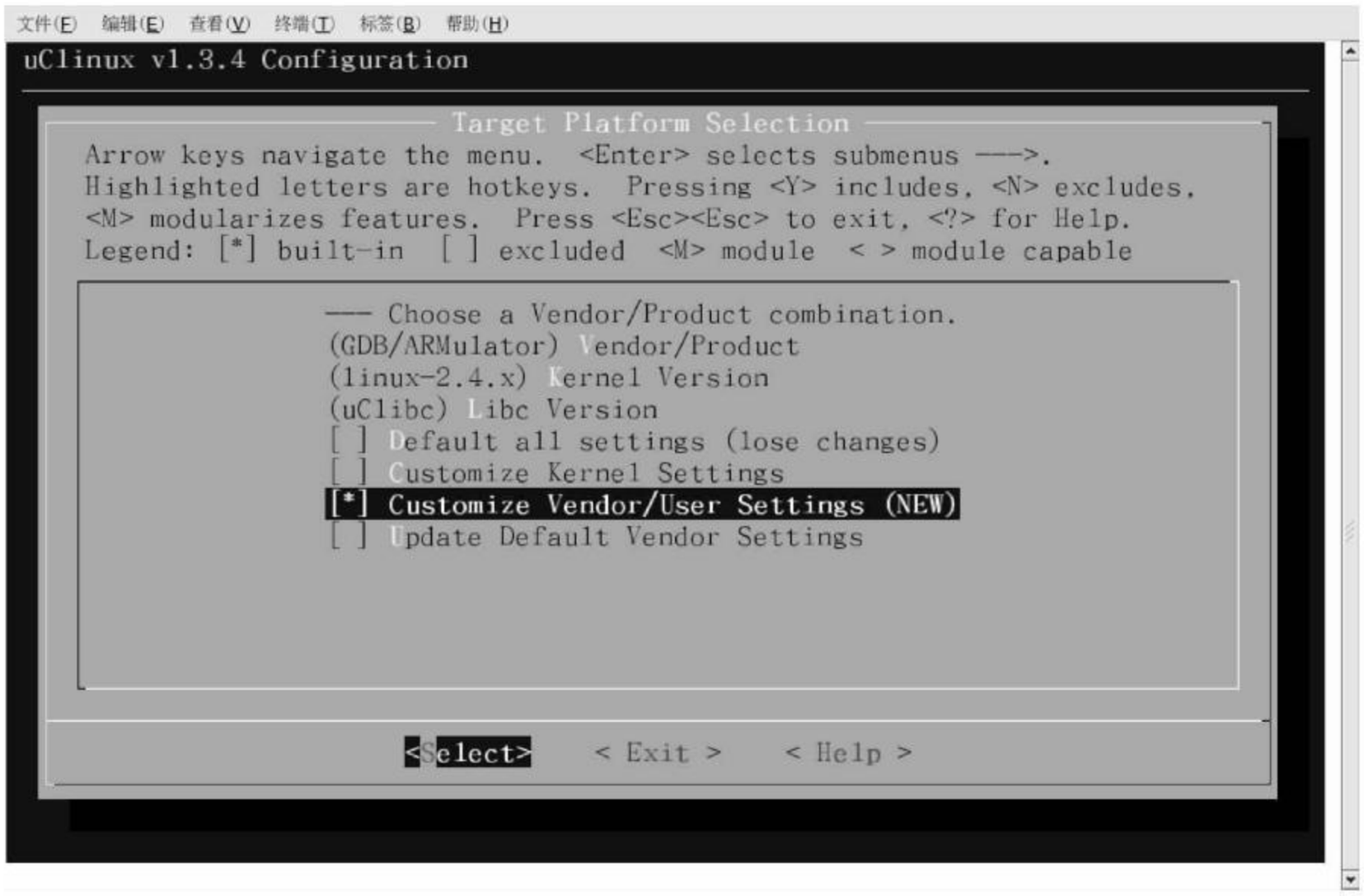

图 1.6 配置主菜单

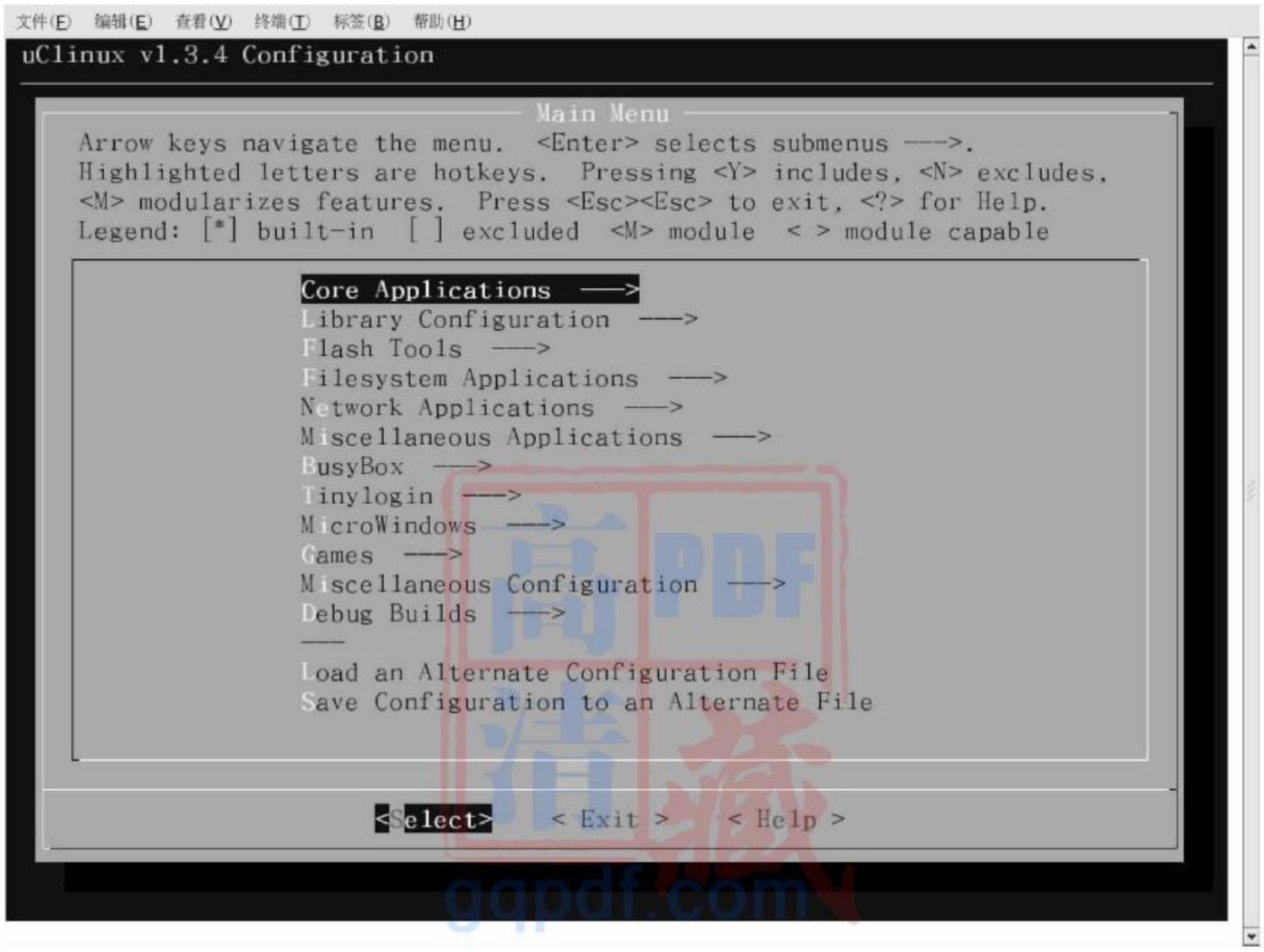

图 1.7 选择应用程序

选择完后，保存退出（参见图 1.8）。

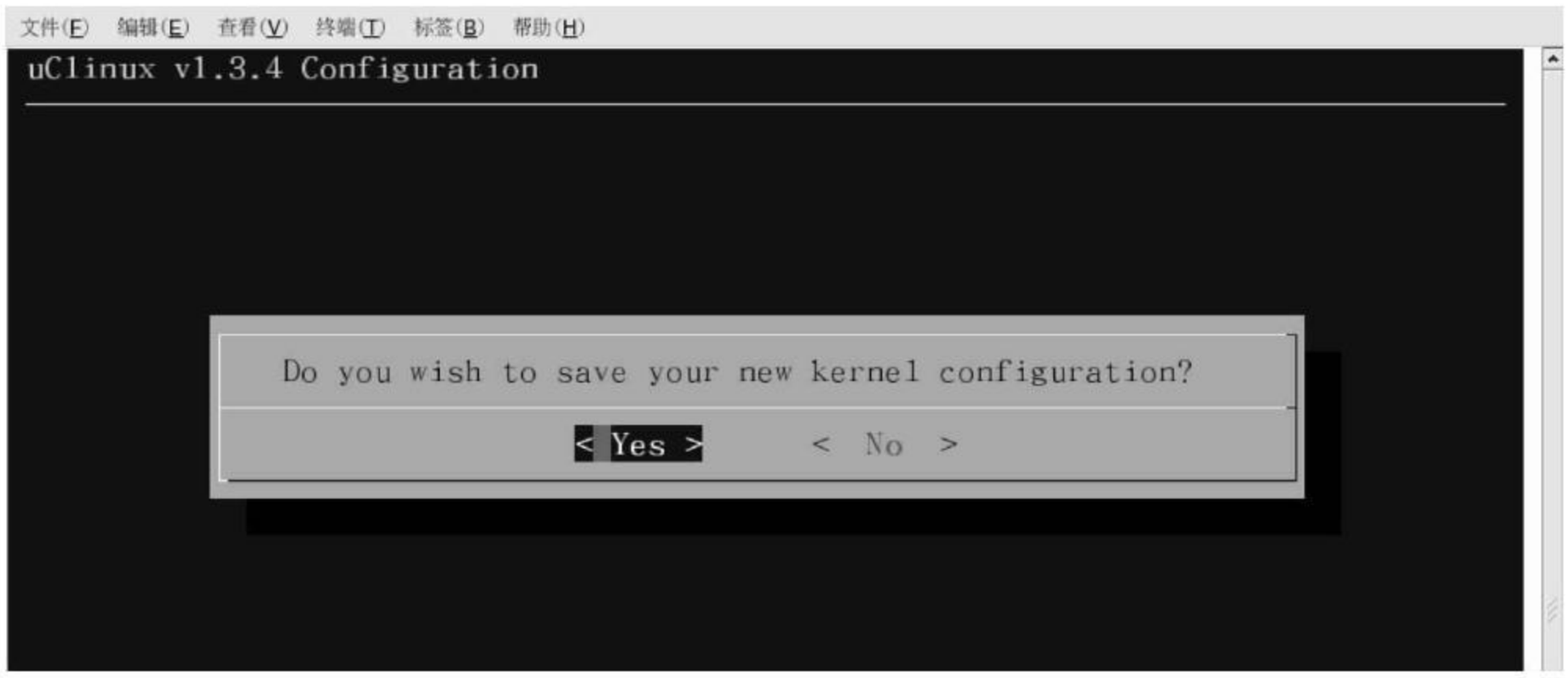

图 1.8 保存退出

选择配置完毕，保存设置退出；然后在 uClinux-dist 目录下运行“make romfs”，得到文件系统。可在 μClinux 顶层目录的 images 目录下看到 romfs. img，这就是生成的文件系统。

本章小结

本章主要讲述了 μClinux 和 Linux 的渊源及差别，并且对 μClinux 的开发环境、如何编译 μClinux 内核及建立文件系统进行了详细的描述。

高清PDF藏 gqpdf.com

更多高清PDF图书 www.gqpdf.com

第2章　使用 GNU Tools 建立 μClinux 开发环境

GNU Tools 是 Linux/μClinux 环境下最主要的开发工具，而掌握这些开发工具是一个嵌入式开发人员在 Linux/μClinux 下进行软件开发所必备的技能。

本章首先花了较多篇幅介绍了 GNU Tools 开发工具中的核心软件及一些调试开发工具的使用，为后面的开发做好准备。这些工具软件的选项很多，作者从实用的角度，给大家讲解一些开发中用到的功能。至于更多的选项，读者可自行参考相关手册。在本章的后面讲解了如何编译安装这些工具，以及如何使用这些工具进行内核及应用程序的编译和开发，这是嵌入式 Linux 开发需要掌握的基本技能。

2.1　GNU Tools 简介

嵌入式软件开发本质上与普通的软件开发并没有什么不同，但是由于嵌入式系统的可配置性和定制要求，需要对嵌入式系统的每个环节都应有所了解。为了更有效地在 Linux 下开发嵌入式系统，至少需要了解和掌握如下 4 类与开发相关的工具。

- 编译工具：能把一个源程序编译生成为一个可执行程序，如 GCC 等。
- 调试工具：能对执行程序进行源码或汇编级调试，如 GDB 等。
- 软件工程工具：用于协助多人开发或大型软件项目的管理，如 make、CVS 等。
- 其他工具：诸如用于把多个目标文件链接成可执行文件的链接器，或者用作格式转换的工具等。

而 GNU Tools 和其他一些优秀的开源软件可完全覆盖上述类型的软件开发工具。在此逐一介绍在嵌入式 Linux 开发中所用到的这些软件工具，后续内容中还会进行更详细的介绍。

GNU Tools 中包含了一个编译器 GCC。GCC 是 GNU Compiler Collection 的简称，事实上，它是一个编译器家族，可支持 C、C + +、ADA、Object C、Java、Fortran、Pascal 等多种高级语言。下面介绍它包括的一些工具。

- cpp：GNU 预处理器(preprocessor)，主要用于扩展头文件和宏定义。cpp 是 GNU C 兼容编译器预处理器。cpp 是一个宏指令处理器，C 编译器使用它在程序被编译前改变

它。它之所以被称为宏指令处理器，是因为它允许自定义宏指令（长结构的简写）。

- GCC：符合 ISO 标准的 C 编译器。
- g++：基本符合 ISO 标准的 C++编译器。
- gcj：gcj 是 GCC 的 Java 前端，可生成执行速度更快的二进制本地执行码，而不是 java binary code。gcj 为把 Java 程序和字节代码编译成机器代码提供了试验性的支持。要做到这一点，还需要安装包括在 gcc-libgcj 中的 Java 运行时库。

binutils 是一组二进制工具程序集，它包括 addr2line、ar、as、gprof、ld、nm、objcopy、objdump、ranlib、size、strings、strip 等工具，是辅助 GCC 的主要软件。

- as：GNU 汇编器（assembler），主要用于把汇编代码转换成二进制执行代码，并存放到一个 object 文件中。
- ld：GNU 链接器（linker），主要用于确定相对地址，把多个 object 文件、起始（startup）代码段、库（library）等链接起来，并最终形成一个可执行文件。
- addr2line：把执行程序中的一个地址映射到源文件中的对应行。
- ar：创建归档文件（archive）、修改/替换库中的 object 文件，向库中添加/提取 object 文件。
- C++file：解码 C++符号名。
- nm：列出 object 文件中的符号。
- objectcopy：拷贝和转换 object 文件。
- objdump：用来显示对象文件的信息。
- ranlib：根据归档文件中的内容建立索引。
- readelf：显示 ELF 格式执行文件中的各种信息。
- size：显示 object 文件和执行文件各节（section）的大小和总的大小。
- strings：显示文件中可打印的字符（基本上是英文字符串）。
- strip：去掉执行文件中多余的信息（如调试信息等），可减少执行文件的大小。
- gprof：用来显示调用图表档案数据。

除了上面介绍的这些 GNU Tools 工具外，还应掌握以下工具。

- make：一个用来控制对可执行程序的生成过程，从其他源码程序文件中生成可执行程序的 GNU 工具。GNU make 允许用户生成和安装软件包，而无须了解生成、安装软件包的具体执行过程。
- GDB：GNU 调试器。它允许调试用 C、C++和其他语言编写的程序，其基本运行方式是在一个 shell 环境下用命令行方式调试程序和显示数据。如果加上一些图形前端（如 DDD 等软件），则可在一个更方便的图形环境下调试程序。
- SkyEye：一个硬件模拟器，可用来对 μClinux 内核进行调试。
- diff/diff3/sdiff：比较文本差异的工具，也可用来产生补丁。

更多高清PDF图书 www.gqpdf.com

- patch：补丁安装程序，可根据 diff 生成的补丁来更新程序。
- CVS：一个版本控制系统。它能记录对文件的修改历史（通常但并不总是包括源码）。CVS 只存储版本间的区别，而不是所创建文件的每一个版本。CVS 还保留一个记录改变者、改变时间以及改变原因的日志。CVS 对于管理发行版本和控制在多位作者间同时编辑源码文件很有帮助。CVS 为一个层次化的目录提供版本控制，目录由修订控制的文件组成，而不是在一个目录中为一组文件提供版本控制。这些目录和文件可被合并起来构成一个软件发行版本。

2.2 GCC 编译器

GCC 是一个强大的工具集合，它包含了预处理器、编译器、汇编器、链接器等组件。它在需要时调用其他组件（预处理器、编译器、汇编器、链接器）。输入文件的类型和传递给 GCC 的参数决定了 GCC 具体调用哪些组件。对于一般或初级的开发者，GCC 可提供简单的使用方式，即只要给它提供 C 源码文件，它将完成预处理、编译、汇编、链接所有工作，最后生成一个可执行文件。而对于中高级开发者，它提供了足够多的参数，可让开发者全面控制代码的生成，这对于嵌入式系统级软件开发相当重要。

GCC 识别的主要文件扩展名如表 2.1 所列。这里需要清楚地区别 GCC 使用预处理器、编译器和汇编器用源码文件来产生某种形式的目标文件，在这一系列的处理过程中，外部的符号参考并没有被替换。然后 GCC 使用链接器来链接这些目标文件和一些标准的程序库，并生成一个可执行程序。在链接阶段，一个目标文件中对其他文件中符号的参考引用（reference）被解释和替换，且如果有不能被解释的参考引用，则一般以错误信息的形式报告出来。在 GCC 处理的各个阶段中，常用的编译选项如表 2.2 所列。

表 2.1 GCC 识别的主要文件扩展名列表

文件扩展名	文件类型
.c	C 语言代码
.C、.cc	C++语言代码
.i	预处理后的 C 语言代码
.s、.S	汇编语言代码
.o	目标代码
.a	静态链接库（程序编译时使用）
.so	动态链接库（程序运行时使用）

高清PDF gqpdf.com

表 2.2　GCC 命令的常用编译选项

选　项	含　义
-ansi	只支持 ANSI 标准的 C 语法，禁止 GNU C 的某些特色，例如 asm 或 typeof 关键词
-c	只编译并生成 object 文件
-DMACRO	定义 MACRO 宏为字符串 1
-DMACRO = DEF	定义 MACRO 宏为字符串“DEF”
-UMACRO	取消对 MACRO 宏的定义
-E	只运行 C 预编译器 cpp
-g	生成可被 GDB 使用的标准调试信息
-ggdb	生成只被 GDB 使用的扩展调试信息
-I DIRECTORY	除缺省情况外，指定其他头文件搜索路径 DIRECTORY
-L DIRECTORY	除缺省情况外，指定其他函数库搜索路径 DIRECTORY
-l LIBRARY	确定链接时需要的其他函数库 LIBRARY
-mXXX	针对 XXX CPU 进行优化，如 XXX 可以是 486、586 等
-o FILE	生成指定的输出文件名 FILE
-O0	不进行代码优化
-O 或 -O1	进行一般的代码优化，减少执行代码大小和执行的时间
-O2	比 -O1 进行更多的代码优化，一般在内核编译中会使用
-O3	比 -O2 更进一步优化
-fomit-frame-pointer	忽略帧指针，即在函数调用中不需要保存、安装和恢复帧指针（在内核编译中用得较多）
-shared	生成支持动态共享库的执行文件
-static	不支持动态共享库，把函数库中的内容静态链接到执行文件中
-MM	输出源文件的依赖关系
-fPIC	产生位置无关代码（英文缩写为 PIC），一般创建共享库时需要
-w	不生成任何警告信息
-Wall	生成所有级别的警告信息
-Werr or	把所有的警告当成错误，并中止编译过程

2.2.1　GCC 使用示例

这一小节介绍在 Linux 系统上 GNU C 编译器的使用以及 GCC 编译器的一些常用选项。尽管 GCC 的选项非常多，但主要的用法还是很简单，如果熟悉其他一些 C 编译器的用法，相信能很快掌握。

更多高清PDF图书　www.gqpdf.com

高清PDF藏 gqpdf.com

1. 一般使用方法

下面是一个相当简单的 C 程序示例代码(参考程序清单 2.1),通过编译运行这个小程序来举例说明 GCC 的编译参数和使用方法。

程序清单 **2.1** 测试程序 **1**

```
/*test.c*/
#include <stdio.h>
void main ()
{
  int i,j;
  j=0;
  i=j+1;
  printf("Hello World\\n");
  printf("the result is %d\\n",i);
}
```

利用如下的命令可编译生成可执行文件:

```
$ gcc -o  test test.c
```

然后执行程序,并输出了结果:

```
$./test
Hello World
the result is 1
```

如果使用如下命令,则可看到更详细的信息:

```
$ gcc -v -o test test.c
Reading specs from /usr/lib/gcc-lib/i386-redhat-linux/3.2.2/specs
Configured with: ../configure --prefix=/usr --mandir=/usr/share/man --infodir=/
usr/share/info --enable-shared --enable-threads=posix --disable-checking --with-
system-zlib --enable-__cxa_atexit --host=i386-redhat-linux
Thread model: posix
gcc version 3.2.2 20030222 (Red Hat Linux 3.2.2-5)
 /usr/lib/gcc-lib/i386-redhat-linux/3.2.2/cc1 -lang-c -v -D__GNUC__=3 -D__GNUC_MI-
NOR__=2 -D__GNUC_PATCHLEVEL__=2 -D__GXX_ABI_VERSION=102 -D__ELF__ -Dunix -D__gnu
_linux__ -Dlinux -D__ELF__ -D__unix__ -D__gnu_linux__ -D__linux__ -D__unix -D__linux -
Asystem=posix -D__NO_INLINE__ -D__STDC_HOSTED__=1 -Acpu=i386 -Amachine=i386 -
Di386 -D__i386 -D__i386__ -D__tune_i386__ test.c -quiet -dumpbase test.c -version -o
/tmp/cc1p2eNK.s
```

```
GNU CPP version 3.2.2 20030222 (Red Hat Linux 3.2.2-5) (cpplib) (i386 Linux/ELF)
GNU C version 3.2.2 20030222 (Red Hat Linux 3.2.2-5) (i386-redhat-linux)
        compiled by GNU C version 3.2.2 20030222 (Red Hat Linux 3.2.2-5).
ignoring nonexistent directory "/usr/i386-redhat-linux/include"
#include "..." search starts here:
#include <...> search starts here:
 /usr/local/include
 /usr/lib/gcc-lib/i386-redhat-linux/3.2.2/include
 /usr/include
End of search list.
test.c: In function 'main':
test.c:3: warning: return type of 'main' is not 'int'
as -V -Qy -o /tmp/ccyH0xPy.o /tmp/cc1p2eNK.s
GNU assembler version 2.13.90.0.18 (i386-redhat-linux) using BFD version 2.13.90.
0.18 20030206
 /usr/lib/gcc-lib/i386-redhat-linux/3.2.2/collect2 --eh-frame-hdr -m elf_i386 -dy-
namic-linker /lib/ld-linux.so.2 -o test /usr/lib/gcc-lib/i386-redhat-linux/3.2.
2/../../../crt1.o /usr/lib/gcc-lib/i386-redhat-linux/3.2.2/../../../crti.o /usr/
lib/gcc-lib/i386-redhat-linux/3.2.2/crtbegin.o -L/usr/lib/gcc-lib/i386-redhat-
linux/3.2.2 -L/usr/lib/gcc-lib/i386-redhat-linux/3.2.2/../../.. /tmp/ccyH0xPy.o -
lgcc -lgcc_eh -lc -lgcc -lgcc_eh /usr/lib/gcc-lib/i386-redhat-linux/3.2.2/crtend.o
/usr/lib/gcc-lib/i386-redhat-linux/3.2.2/../../../crtn.o
```

通过上面的输出可以看到，不带-v 参数的 GCC 编译过程实际上屏蔽了许多一般用户没有必要看到的处理过程。一般而言，对于 C 程序的 GCC 编译过程主要分为以下 4 个阶段：

① 预处理阶段，即完成宏定义和 include 文件展开等工作；

② 根据编译参数进行不同程度的优化，编译成汇编代码；

③ 用汇编器把上一阶段生成的汇编代码进一步生成目标代码；

④ 用链接器把上一阶段生成的目标代码、其他一些相关的系统提供的目标代码（如 crtx.o）和系统或用户提供的库链接起来，生成最终的执行代码。

2. 生成预处理代码

如果想让 GCC 在预处理后停止编译，生成完成预处理的 C 程序，则可执行如下代码：

```
$ gcc -E -o test.cpp test.c
```

如果用 wc 命令，则可看到这两个阶段代码大小：

更多高清PDF图书 www.gqpdf.com

```
$ wc test.c test.cpp
     9      16     127 test.c
   842    1934   16498 test.cpp
   851    1950   16625 总用量
```

可以看到，test. cpp 比 test. c 增加了不少内容。通过看 test. cpp 代码，可看到它相对于 test. c 增加了不少定义，而这些定义实际上是放在系统提供的 include 文件中的。

3. 生成汇编代码

GCC 编译器生成汇编代码的过程的具体命令如下：

```
$ gcc -S test.c
```

通过看汇编代码，可以很快地了解 GCC 编译器支持的标准汇编代码的写法。有一定水平且对机器语言有一定了解的嵌入式开发人员通过分析 GCC 编译器生成的汇编代码，相信可以很快地写出地道的 GNU 汇编代码。

4. 生成目标代码

生成目标代码也有多种方式，这里只介绍 2 种。

① 用 gcc 直接从 C 源程序生成目标代码，具体命令如下：

```
$ gcc -c test.c
```

② 用汇编器从汇编代码生成目标代码，具体命令如下：

```
$ as -o test.o test.s
```

这种方式实际上是其他 2 种方式的最后一个阶段必须完成的工作，在 gcc 命令后加上-v 参数，即可看到上述 2 种方式实际的处理过程。

5. 生成最终的执行代码

生成最终的执行代码的方式也有多种，这里介绍 2 种。

① 从 C 源程序直接生成执行代码，具体命令如下：

```
$ gcc -o test test.c
```

实际的执行细节可通过加参数-v 后的输出结果看到。

② 从汇编代码生成执行代码，具体命令如下：

高PDF库 gqpdf.com

```
$ ld -dynamic-linker /lib/ld-linux.so.2 -o test \\
/usr/lib/gcc-lib/i386-redhat-linux/3.2.2/../../../crt1.o \\
/usr/lib/gcc-lib/i386-redhat-linux/3.2.2/../../../crti.o \\
/usr/lib/gcc-lib/i386-redhat-linux/3.2.2/crtbegin.o \\
-L/usr/lib/gcc-lib/i386-redhat-linux/3.2.2 \\
-L/usr/lib/gcc-lib/i386-redhat-linux/3.2.2/../../.. \\
test.o -lgcc  -lc /usr/lib/gcc-lib/i386-redhat-linux/3.2.2/crtend.o \\
/usr/lib/gcc-lib/i386-redhat-linux/3.2.2/../../../crtn.o
```

可以看到，上述命令生成了一个支持动态链接库的执行代码 test，它实际上是由多个目标文件（crt1.o、crti.o、crtbegin.o、test.o、crtend.o、crtn.o）和库文件（libgcc、libc）中的部分函数内容构成的。libc 库中有实际的 printf() 函数的实现，而 test 执行文件要调用这个函数实现输出功能，所以 ld 需要链接 libc 才能生成正确的执行文件。参数-L 表示 ld 搜索库文件的路径。参数-dynamic-linker 用来设定动态链接器的文件名，这只是用于生成 ELF 格式的执行文件，如果生成静态链接的 a.out 格式的文件，则不需要这个文件。

6. 其他说明

GCC 可同时用来编译 C 程序和 C++程序。一般来说，C 编译器通过源文件的后缀名来判断是 C 程序还是 C++程序。在 Linux 中，C 源文件的后缀名为.c，而 C++源文件的后缀名为 .C 或 .cc。gcc 命令只能编译 C++源文件，而不能自动和 C++程序使用的库链接。因此，通常使用 g++命令来完成 C++程序的编译和链接，该程序会自动调用 GCC 实现编译。假设有一个如程序清单 2.2 所示的 C++源文件（test.C）：

程序清单 **2.2**　测试代码 **2**

```
/*test.C*/
#include <iostream.h>
int main (void)
{
    int i,j;
    j=0;
    i=j+1;
    cout << "Hello world!" << endl;
    cout << "the result is " << i <<endl;
    return 0;
}
```

则可调用如下 g++命令编译、链接并生成可执行文件：

更多高清PDF图书　www.gqpdf.com

```
$ g++ -o test2 test.C
$ ./test2
Hello world!
the result is 1
```

通过如下命令，也可看到 g++编译器实际编译 test.C 的过程。

```
\[chy@ localhost test\]$ g++ -v -o test2 test.C
Reading specs from /usr/lib/gcc-lib/i386-redhat-linux/3.2.2/specs
Configured with: ../configure --prefix=/usr --mandir=/usr/share/man --infodir=/
usr/share/
```

局 PDF 清 藏 gqpdf.com

```
info --enable-shared --enable-threads=posix --disable-checking --with-system-zlib
--enable-__cxa_atexit --host=i386-redhat-linux
Thread model: posix
gcc version 3.2.2 20030222 (Red Hat Linux 3.2.2-5)
 /usr/lib/gcc-lib/i386-redhat-linux/3.2.2/cc1plus -v -D__GNUC__=3 -D__GNUC_MINOR__
=2 -D__GNUC_PATCHLEVEL__=2 -D__GXX_ABI_VERSION=102 -D__ELF__ -Dunix -D__gnu_linux_
_ -Dlinux -D__ELF__ -D__unix__ -D__gnu_linux__ -D__linux__ -D__unix -D__linux -Asystem=
posix -D__NO_INLINE__ -D__STDC_HOSTED__=1 -D_GNU_SOURCE -Acpu=i386 -Amachine=i386
-Di386 -D__i386 -D__i386__ -D__tune_i386__ test.C -D__GNUG__=3 -D__DEPRECATED -D__EX-
CEPTIONS -quiet -dumpbase test.C -version -o /tmp/ccg4I4XD.s
GNU CPP version 3.2.2 20030222 (Red Hat Linux 3.2.2-5) (cpplib) (i386 Linux/ELF)
GNU C++ version 3.2.2 20030222 (Red Hat Linux 3.2.2-5) (i386-redhat-linux)
        compiled by GNU C version 3.2.2 20030222 (Red Hat Linux 3.2.2-5).
ignoring nonexistent directory "/usr/i386-redhat-linux/include"
#include "..." search starts here:
#include <...> search starts here:
 /usr/include/c++/3.2.2
 /usr/include/c++/3.2.2/i386-redhat-linux
 /usr/include/c++/3.2.2/backward
 /usr/local/include
 /usr/lib/gcc-lib/i386-redhat-linux/3.2.2/include
 /usr/include
End of search list.
In file included from /usr/include/c++/3.2.2/backward/iostream.h:31,
                 from test.C:1:
/usr/include/c++/3.2.2/backward/backward_warning.h:32:2: warning: #warning
This file includes at least one deprecated or antiquated header. Please consider
using one of the 32 headers found in section 17.4.1.2 of the C++ standard. Exam-
ples include substituting the <X> header for the <X.h> header for C++ in-
cludes, or <sstream> instead of the deprecated header <strstream.h>. To disa-
ble this warning use -Wno-deprecated.
 as -V -Qy -o /tmp/ccHUw6He.o /tmp/ccg4I4XD.s
GNU assembler version 2.13.90.0.18 (i386-redhat-linux) using BFD version 2.13.90.
0.18 20030206
 /usr/lib/gcc-lib/i386-redhat-linux/3.2.2/collect2 --eh-frame-hdr -m elf_i386 -dy-
namic-linker /lib/ld-linux.so.2 -o test2 /usr/lib/gcc-lib/i386-redhat-linux/3.2.
2/../../../crt1.o /usr/lib/gcc-lib/i386-redhat-linux/3.2.2/../../../crti.o /usr/
lib/gcc-lib/i386-redhat-linux/3.2.2/crtbegin.o -L/usr/lib/gcc-lib/i386-redhat-
linux/3.2.2 -L/usr/lib/gcc-lib/i386-redhat-linux/3.2.2/../../.. /tmp/ccHUw6He.o -
lstdc++ -lm -lgcc_s -lgcc -lc -lgcc_s -lgcc /usr/lib/gcc-lib/i386-redhat-linux/3.2.
```

更多高清PDF图书 www.gqpdf.com

由上可知，在编译一个文件时有很多的编译选项，上面已介绍了一些常用的编译选项，下面以基于 ARM CPU 的 μClinux 内核中一个文件为例介绍一些编译选项。介绍的这些选项无论是内核开发人员还是应用程序开发人员都已足够。

2.2.2 编译内核时使用的编译选项

下面以编译内核 kernel/printk.c 为例说明每个编译选项的含义。假设使用如下命令行来编译内核中的 kernel 目录下的 printk.c 文件：

```
arm-elf-gcc -D__KERNEL__ -I/home/oyangjian/develop/tmp/lpc-uclinux-bin/uClinux-dist/linux-2.4.x/include -Wall -Wstrict-prototypes -Wno-trigraphs -O2 -fno-strict-aliasing -fno-common -pipe -fno-builtin -D__linux__ -g -DNO_MM -mapcs-32 -march=armv4 -mtune=arm7tdmi -mshort-load-bytes -msoft-float -nostdinc -iwithprefix include -DKBUILD_BASENAME=printk -c -o printk.o printk.c
```

-Dsymbol=y：表示定义了符号 symbol，其值等于 y。

-c：表示只编译，不链接该函数。

-o：表示输出目标文件。

-g：表示有调试信息。

-I：表示头文件的搜索路径，即 Linux 中的 include 目录。这里是绝对目录。

-Wall：表示打开所有通用的警告选项。

-nostdinc：表示不使用标准 C 的包含路径。

-iwithprefix include：增加包含路径。

-Wstrict-prototypes：表示如果函数没有指明参数类型，则提示警告。

下面解释一下“-Wstrict-prototypes”的含义。假设编写了如程序清单 2.3 所示的测试程序（main.c）。

程序清单 **2.3** 测试程序 **3**

```
/*main.c*/
int foo()
{
 return 0;
}

int main(void)
{
  foo();
}
```

高清藏 PDF gqpdf.com

然后加-Wstrict-prototypes 进行编译：

```
oyangjian@ 9 \[test \] $ gcc -Wstrict-prototypes -O2 -o main main.c
```

会产生如下错误信息：

```
main.c:3: warning: function declaration isn't a prototype
```

如果在程序的首行对 foo () 函数进行声明：

```
int foo(void)
```

则不会产生错误信息。下面继续介绍其余的 gcc 选项及其含义。

-Wtrigraphs：表示如果在代码中碰到三图符就警告，不过该选项只有在-trigraphs 选项打开时才有效。-trigraphs 表示处理三图符。三图符，就是三个字符在一起表示一个字符，比如“?? (”表示“\[”，“?? /”表示“\\”。

```
Trigraph:      ??(   ??)   ??<   ??>   ??=   ??/   ??'   ??!   ??-
Replacement:   \[    \]    {     }     #     \\    ^     |     ~
```

-fstrict-aliasing -fno-strict-aliasing

允许编译器进行严格的类型匹配检查。

-fno-common：表示为每个变量分配自己的空间。每个全局变量，即使没有初始值，也会分配空间，而且对于全局变量的检查更加严格。

-pipe：表示在编译阶段变量之间的通信是通过管道而不是临时文件。

-fno-builtin：表示不要识别 built-in 函数，除非函数是以__builtin_作为前缀。例如：

```
#define likely(x)       __builtin_expect((x),1)
#define unlikely(x)     __builtin_expect((x),0)
```

-mapcs-32、-march = armv4、-mtune = arm7tdmi、-mtune = cpu_type、-mshort-load-bytes、-malignment-traps、-mno-alignment-traps、-mno-short-load-words 和-msoft-float：能产生对浮点数的处理。

上面这些参数就是编译器根据特定的 CPU 进行特有的优化，比如有 ARM 系列 CPU 可使用上述的编译选项。

2.3 binutils 二进制工具集

binutils 是一组二进制工具程序集，它包括了在 2.1 节提到的一些工具，如 addr2line、ar、gprof、nm、objcopy、objdump、ranlib、size、strings、strip 等。下面介绍这些常用的工具及其用法。

更多高清PDF图书 www.gqpdf.com

2.3.1 ar 软件

ar 软件用于建立、修改、提取归档文件(archive)。一个归档文件是包含多个文件的单个文件(也可把它认为是一个库文件),其结构保证了可从中检索并得到原始的被包含文件(称作此归档文件的成员)。被包含的原始文件的内容、模式(权限)、时间戳、所有者和组等属性都保存在归档文件中,并且可在提取后恢复原始文件的相关属性。

ar 可以维护成员名具有任意长度的归档文件。ar 被认为是一种二进制工具,这是因为这种类型的归档文件一般被当作库,而库中包含有通常所需的子程序执行代码。ar 的一般执行格式如下:

```
ar \[-\] p\[ mod \[ relpos\]\] archive \[ member ...\]
```

以 UNIX 风格使用 ar 时,ar 需要至少 2 个参数才能运行:一个参数是关键字母(keyletter),用来指定具体的操作,在关键字母后面可加上修饰符(modifier flags,简称 mod),可进一步确定需要执行的操作;另一个参数 archive 是归档文件名,指定要归档的文件。参数 member 是相关的被包含文件序列,也可称为成员序列。关键字母"p"指定要执行的操作,可以是下列字母中的任意一个,但是一次只能指定其中的一个。

d: 删除归档文件中的被包含文件,用参数"member ..."指定要删除的成员名字;如果没有指定要删除的成员,那么归档文件将不会改变。

r: 将参数"member..."指定的文件列表插入归档文件。该操作不同于"q"的地方在于,如果先前已存在的成员与要添加的成员重名,那么以前的成员将会被删除。如果"member..."指定的文件名不存在,ar 将显示一条出错消息,然后退出。在默认情况下,新成员被添加到文件的末尾;但是可使用修饰符"a"、"b"或"i",确定相对于一些已存在的成员的插入位置。如果该操作带有修饰符"v",将为每个被插入的文件引出一行输出,并伴有字母"a"或"r",表示该文件是被附加的(没有老成员被删除)还是被替换。

t: 显示所有归档文件的内容,或那些仅在参数"member..."中列出并在归档文件中存在的文件。一般情况下,只有成员名被显示出来;但如果指定了修饰符"v",那么将会显示成员的模式(权限)、时间戳、所有者、组和大小。如果没有指定 member 参数,将列出归档文件中的所有文件。

x: 从归档文件中提取成员(由 member 指定)。如果该操作使用了修饰符"v",则在提取成员时,ar 将列出每个成员的名字。如果没有指定 member 参数,将提取文件中所有文件。

许多修饰符可直接加在关键字母后面,用于指定一个操作的行为变化。下面是一些常见的修饰符说明。

a：添加新文件到归档文件中的一个已存在的成员后面。如果使用修饰符“a”，那么那个已存在的归档文件成员的名字就必须以参数 relpos 出现在参数 archive 的前面。

c：创建归档文件。当请求一次更新时，即使指定的 archive 不存在，它也总是会被创建，但会得到警告消息。除非使用这个修饰符，事先指定要创建的归档文件，这样就不会产生警告信息。

f：截短归档文件中的成员名。GNU ar 一般允许任意长度的成员文件名，但这将创建与一些系统中的原有 ar 程序不兼容的归档文件。考虑到这一点，当添加归档文件成员时，可使用修饰符“f”截短成员文件名。

s：即使在归档文件没有发生任何更改的情况下，也会将目标文件索引写入归档文件，或更新一个已存在的索引。该修饰符标记可与任何其他操作一起使用，也可单独使用。对归档文件运行“ar s”等价于对其运行“ranlib”。

v：该修饰符会请求得到一个操作的详细版本。当该修饰符被使用时，很多操作将显示附加的信息，例如被处理的文件名字。

V：该修饰符显示 ar 的版本号。

下面介绍一个简单的使用，编写两个文件 add. c（见程序清单 2.4）和 minus. c（见程序清单 2.5）。

程序清单 **2.4　add. c**

```
int Add(int a, int b)
{
    int result;
    result = a + b;
    return result;
}
```

程序清单 **2.5　minus. c**

```
int Minus(int a, int b)
{
    int result;
    result = a - b;
    return result;
}
```

执行命令：

```
$ gcc -c add.c minus.c
```


高清藏 PDF gqpdf.com
更多高清PDF图书　www.gqpdf.com

后生成 add.o 和 minus.o 文件。

输入以下命令：

```
$ ar rv libtest.a add.o minus.o
```

生成一个 libtest.a 库文件，将这个文件拷贝到/usr/lib 目录下，再编写一个测试程序 test.c（见程序清单 2.6）。

程序清单 **2.6　test.c**

```
#include <stdio.h>

int main( int argc,  char *argv\[\])
{
    int a=8;
    int b=3;
    printf("a=%d\\tb=%d\\n", a, b);
    int sum=Add(a, b);
    printf("a+b=%d\\n", sum);
    int cha=Minus(a, b);
    printf("a-b=%d\\n", cha);
    return 0;
}
```

这个程序用到了 Add() 和 Minus() 函数，所以用如下命令编译：

```
$ gcc  - o test test.c  - ltest
```

-ltest 就是链接到刚才生成的库文件，这样就可使用 Add() 和 Minus() 函数了。编译生成可执行文件 test，其运行结果如下：

```
a=8     b=3
a+b=11
a-b=5
```

2.3.2　nm 软件

nm 软件的功能是列出目标文件中的符号，这样可帮助程序员分析可执行程序，获得目标文件中的符号信息及其属性。如果没有目标文件作为参数被列出，则 nm 默认目标文件名为 a.out，其一般执行格式如下：

```
nm \[ -a | --debug-syms \] \[ -g | --extern-only \]
\[ -B \] \[ -C | --demangle \] \[ -D | --dynamic \]
\[ -s | --print-armap \] \[ -A | -o | --print-file-name \]
\[ -n | -v | --numeric-sort \] \[ -p | --no-sort \]
\[ -r | --reverse-sort \] \[ --size-sort \] \[ -u | --undefined-only \]
\[-t radix | --radix=radix\] \[ -P | --portability \]
\[ --target=bfdname\] \[-f format | --format=format\]
\[ --defined-only \] \[ -l | --line-numbers \]
\[ --no-demangle \] \[ -V | --version \]
\[ --help \]
\[ object-file ...\]
```

通过如下命令可看到 nm 的一般帮助信息：

```
$ nm -h
```

对于每一个符号，nm 将显示如下内容。

符号的值：用某种基数表示的数值，默认情况下用十六进制表示。

符号的类型：至少要用到下面的这些类型。也可用其他类型，这依赖于目标文件的格式。

A：符号的值是绝对值，并且不会被将来的链接所改变。

B：符号位于未初始化数据部分（被认为是 BSS）。

C：符号是公共的。公共符号是未初始化的数据。在链接时，多个公共符号可能以相同的名字出现。如果符号在其他地方被定义，该符号会被当作未定义的引用来处理。

D：符号位于已初始化数据部分。

N：符号是调试符号。

R：符号位于只读数据部分。

T：符号位于文本（代码）部分。

U：符号未被定义。

W：符号是弱定义符号（weak symbol），也称为弱符号。当一个弱定义符号和一个已定义的普通符号链接时，使用该已定义的普通符号不会引起错误。当一个未定义的弱符号被链接且该符号未被定义时，该 weak 符号的值被无错误地变为零。

o：符号名。

nm 的命令行参数选项的长格式和短格式是等价的，下面列出的是可供选择的命令行参数格式。

-D/--dynamic：显示动态符号而不是普通符号。该任选项仅对于动态目标文件有意义，例如特定类型的共享库。

-g/--extern-only：只显示外部符号。

更多高清PDF图书 www.gqpdf.com

-l/--line-numbers：对每个符号，使用调试信息去试图找到文件名和行号。对于已定义的符号，查找符号地址的行号。对于未定义符号，查找指向符号重定位入口的行号。如果可找到行号信息，在其他符号信息之后显示行号信息。

-n/-v/--numeric-sort：按符号对应地址的顺序排序，而不是按符号名的字符顺序。

-p/--no-sort：不以任何顺序对符号进行排序；按目标文件中遇到的符号顺序显示。

-s/--print-armap：当列出归档文件中成员的符号时，包含索引，即名字和包含该名字定义的模块的映射（通过 ar 或 ranlib 保存在归档文件中）。

-r/--reverse-sort：反转排序的顺序（按数字或字母顺序）显示。

--size-sort：按大小排列符号顺序。该大小是按照一个符号的值与它下一个符号的值进行计算的。

-u/--undefined-only：仅显示没有定义的符号（那些每个目标文件的外部符号）。

--defined-only：仅显示每个目标文件中已定义的符号。

-V/--version：显示 nm 的版本号，然后退出。

--help：显示 nm 的所有选项，然后退出。

下面介绍一个简单的使用，按照刚才介绍 ar 命令时的例子，执行如下命令：

```
$ nm test.o
```

输出结果如下：

```
         U Add
00000000 T main
         U Minus
         U printf
```

执行命令：

```
$ nm add.o
```

输出结果如下：

```
00000000 T Add
```

执行命令：

```
$ nm minus.o
```

输出结果如下：

```
00000000 T Minus
```

命令“nm test.o”的输出说明 test.o 定义了 main() 函数，但没有定义 Add()、Minus() 和

高清PDF藏 gqpdf.com

printf()函数符号。命令“nm add. o”的输出说明 add. o 定义了 Add()函数符号。命令“nm minus. o”的输出说明 minus. o 定义了 Minus()函数符号。test. o 中没有定义但使用了 printf()函数符号,printf()函数实际上定义在 libc. a 库(属于 glibc 软件库)中。

2.3.3 objcopy 软件

objcopy 软件用来把某种格式的目标文件内容进行转换,并输出为另一种格式的目标文件。objcopy 使用 GNU BFD 库读/写目标文件,通过这个 BFD 库,objcopy 能以一种不同于源目标文件的格式来生成目标文件。

objcopy 提供了很多命令行选项来方便用户使用。objcopy 通过访问 BFD 库中描述的所有格式的目标文件来识别大部分可执行文件的格式。例如,指定 srec 的输出目标格式(使用“-O srec”),objcopy 就能生成 S-records 格式的目标文件;指定 binary 的输出目标格式(使用“-O binary”),objcopy 就能生成原始的二进制文件,即通常所说的 image 文件。当 objcopy 生成一个原始的二进制文件时,它必然会产生输入目标文件内容的一个内存信息转储。所有符号和重定位信息将被丢弃,且内存信息转储将从已拷贝到输出文件的最低端的虚拟地址开始。当生成 S-record 或原始二进制文件时,使用-S 有助于 objcopy 工具去除包含调试信息的部分。在某些情况下,“-R”有助于去除二进制文件并不需要的信息。objcopy 的一般执行格式如下:

更多高清PDF图书 www.gqpdf.com

```
objcopy \[ -F bfdname | --target = bfdname \]
\[ -I bfdname | --input-target = bfdname \]
\[ -O bfdname | --output-target = bfdname \]
\[ -S | --strip-all \] \[ -g | --strip-debug \]
\[ -K symbolname | --keep-symbol = symbolname \]
\[ -N symbolname | --strip-symbol = symbolname \]
\[ -L symbolname | --localize-symbol = symbolname \]
\[ -W symbolname | --weaken-symbol = symbolname \]
\[ -x | --discard-all \] \[ -X | --discard-locals \]
\[ -b byte | --byte = byte \]
\[ -i interleave | --interleave = interleave \]
\[ -R sectionname | --remove-section = sectionname \]
\[ -p | --preserve-dates \] \[ --debugging \]
\[ --gap-fill = val \] \[ --pad-to = address \]
\[ --set-start = val \] \[ --adjust-start = incr \]
\[ --change-address = incr \]
\[ --change-section-address = section{ = , + , - } val \]
\[ --change-warnings \] \[ --no-change-warnings \]
\[ --set-section-flags = section = flags \]
\[ --add-section = sectionname = filename \]
\[ --change-leading char \] \[--remove-leading-char \]
```

高清PDF gqpdf.com

```
\[ --weaken \]
\[ -v | --verbose \] \[ -V | --version \] \[ --help \]
input-file \[ outfile\]
```

通过如下命令可看到 objcopy 的一般帮助信息：

```
$ objcopy -h
```

其中各命令行参数的含义描述如下。

input-file/outfile：分别指定源和输出文件。如果不指定输出文件，objcopy 将创建一个临时文件，然后破坏性地以 input-file 的名字重命名输出文件。

-O bfdname/--output-target = bfdname：使用 bfdname 作为写入输出文件的目标格式。

-R sectionname/--remove-section = sectionname：从输出文件中去除名字为 sectionname 的任何节。可以多次使用这个选项。

注意：如果不恰当地使用这个选项，可能导致输出文件不可用。

-S/--strip-all：不复制源文件中的重定位和符号信息。

-g/--strip-debug：不复制源文件中的调试信息。

--strip-unneeded：去除重定位处理不需要的所有符号。

-K symbolname/--keep-symbol = symbolname：只从源文件中复制符号 symbolname。该选项可多次使用。

-x/--discard-all：不从源文件中复制非全局符号。

-X/--discard-locals：不复制编译器生成的局部符号。（这些符号通常以“L”或“.”开始。）

--set-start val：设置新文件的起始地址为 val。

注意：并非所有目标文件格式都支持设置起始地址。

--weaken：弱化文件中的所有全局符号。利用打开了-R 选项的链接器创建与其他目标链接的目标时，该选项是有用的。该选项只对支持弱符号的目标文件格式有效。

-V/--version：显示 objcopy 的版本号。

-v/--verbose：详细输出，则列出更改过的所有目标文件。对于归档文件，“objcopy -V”列出该归档文件的所有成员。

--help：显示 objcopy 的所有选项。

下面介绍使用 objcopy 的例子。对于前面生成的可执行文件 test，输入如下命令：

```
$ file test
```

可看到输出结果为：

```
test: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), for GNU/Linux 2.2.
5, dynamically linked (uses shared libs), not stripped
```


更多高清PDF图书 www.gqpdf.com

可以看到，test 是 elf 格式的。输入如下命令：

```
$ objcopy -O srec test ts
```

执行这条命令后，得到一个 ts 文件。输入命令：

```
$ file ts
```

输出结果为：

```
ts: Motorola S-Record; binary data in text format
```

可以看到，通过 objcopy 命令将 elf 格式的文件转换为 srec 格式的文件，srec 就是 Motolora S-Record 格式的文件，这种格式主要用于主机和目标机间传输数据。

2.3.4 objdump 软件

objdump 显示一个或多个目标文件的信息。由其选项来控制显示哪些特定的信息。这些信息只对那些编写编译工具的程序员有帮助，而对于那些只想让自己编写的程序编译和运行起来的程序员来说没有更多意义。但在嵌入式系统级开发时，通过使用这个软件可方便地查看执行文件或库文件的信息，如可通过 objdump 软件反汇编执行程序，看到执行程序的汇编格式。objdump 的一般执行格式如下：

```
objdump \[ -a | --archive-headers \]
\[ -b bfdname | --target = bfdname \] \[ --debugging \]
\[ -C | --demangle \] \[ -d | --disassemble \]
\[ -D | --disassemble-all \] \[ --disassemble-zeroes \]
\[ -EB | -EL | --endian = {big | little } \]
\[ -f | --file-headers \]
\[ -h | --section-headers | --headers \] \[ -i | --info \]
\[ -j section | --section = section \]
\[ -l | --line-numbers \] \[ -S | --source \]
\[ -m machine | --architecture = machine \]
\[ -p | --private-headers \]
\[ -r | --reloc \] \[ -R | --dynamic-reloc \]
\[ -s | --full-contents \] \[ --stabs \]
\[ -t | --syms \] \[ -T | --dynamic-syms \] \[ -x | --all-headers \]
\[ -w | --wide \] \[ --start-address = address \] \[ --stop-address = address \]
\[ --prefix-addresses \] \[ --\[no \]show-raw-insn \]
\[ --adjust-vma = offset \]
```

```
\[ --version \] \[ --help \]
\[ object-file...\]
```

通过如下命令可看到 objdump 的一般帮助信息：

```
$ objdump -H
```

参数“object-file...”是需要操作的目标文件序列。当目标文件指定的是归档文件时，objdump 显示的是归档文件中每个成员文件的信息。选项的长格式和短格式是等价的。下面描述了作为可供选择的参数格式（除“-l”之外，至少要给出一个参数选项）。

-a/--archive-header：如果任何一个由 object-file 指定的文件是归档文件，则显示该归档文件的头信息（类似于“ls -l”的格式）。除了“ar -tv”能显示的信息外，“objdump -a”还可以显示每个归档文件成员的目标文件格式。

--debugging：显示调试信息。该选项试图解析保存在文件中的调试信息，并且用 C 语言风格的语法打印这些信息。仅能对特定类型的调试信息实现这个功能。

-d/--disassemble：显示目标文件中的机器指令使用的汇编语言。该选项仅仅反汇编那些应含有指令机器码的节。

-D/--disassemble-all：类似于“-d”，但是反汇编所有节的内容，仅对那些应含有指令机器码的节。

--disassemble-zeroes：通常情况下，反汇编的输出会跳过大块的零。该选项将指引反汇编器去反汇编那些大块的零，就像处理其他数据一样。

-EB/-EL/--endian = {big | little}：指定目标文件的字节顺序。该选项只会影响反汇编。当反汇编像 S-records 这样没有描述字节顺序的文件格式时，该选项是有用的。

-f/--file-header：显示每个由 object-file 指定的所有目标文件的文件头概要信息。

-h/--section-header/- -header：显示目标文件的节头概要信息。ld 使用了“-Ttext”、“-Tdata”或“-Tbss”这些选项时，可能会使得目标文件的各节被重定向到非标准的地址。这样，通过这个选项，可查出目标文件的各节的起始地址。然而，一些像 a. out 这样的目标文件格式并没有存储文件段的起始地址。在这些情况下，尽管 ld 正确地重定位了每个节，但使用“objdump -h”列出文件节头时，并不能显示其正确的地址。

--help：显示 objdump 的所有选项的概要信息，然后退出。

-j name/--section = name：只显示由 name 指定的节。

-l/--line-numbers：用对应于目标代码的文件名和行号来标注要显示的信息（使用调试信息），仅仅与-d、-D 或-r 一起使用才有效。

-r/--reloc：显示文件的重定位入口。如果与-d 或-D 一起使用，则重定位部分以反汇编后的格式显示出来。

-R/--dynamic-reloc：显示文件的动态重定位入口，仅仅对于动态目标文件有意义，比如特

更多高清PDF图书 www.gqpdf.com

定类型的共享库。

-s/--full-contents：显示任何指定节的全部内容。

-S/--source：尽可能显示与反汇编混和的源代码。隐含了参数-d。

--start-address = *address*：从指定地址开始显示数据，该选项影响打开-d、-r 和-s 选项时的输出。

--stop-address = *address*：显示数据直到指定地址为止，该选项影响打开-d、-r 和-s 选项时的输出。

-t/--syms：显示文件中的符号表入口。类似于 nm 程序提供的信息。

-T/--dynamic-syms：显示文件中的动态符号表入口。仅仅对于动态目标文件有意义，比如特定类型的共享库。类似于打开了-D（--dynamic）选项的 nm 程序提供的信息。

--version：显示 objdump 的版本号，然后退出。

-x/--all-header：显示所有可用到的头信息，包括符号表和重定位入口，使用-x 等价于指定了“-a -f -h -r-t”参数。

-w/--wide：对超过 80 列的输出设备指定一些行的格式。

下面是使用 objdump 的简单介绍。对于刚才生成的 test 和 ts 执行文件，分别执行

```
$ objdump -f test
```

和

```
$ objdump -f ts
```

得到的结果为：

```
test:     文件格式 elf32-i386
体系结构：i386，标志 0x00000112：
EXEC_P，HAS_SYMS，D_PAGED
起始地址 0x08048298
ts:       文件格式 srec
体系结构：UNKNOWN!，标志 0x00000000：
起始地址 0x08048298
```

可通过参数-d 或-D 反汇编生成的目标代码，比如对于前面生成的目标文件 add.o，执行如下命令：

```
$ objdump -d add.o
```

输出结果如下：

```
add.o:    文件格式 elf32-i386
```

反汇编 .text 节：

```
00000000 <Add>:
   0:   55                      push   %ebp
   1:   89 e5                   mov    %esp,%ebp
   3:   83 ec 04                sub    $0x4,%esp
   6:   8b 45 0c                mov    0xc(%ebp),%eax
   9:   03 45 08                add    0x8(%ebp),%eax
   c:   89 45 fc                mov    %eax,0xfffffffc(%ebp)
   f:   8b 45 fc                mov    0xfffffffc(%ebp),%eax
  12:   c9                      leave
  13:   c3                      ret
```

还可自己试试参数-D 和其他参数的功能，看看 objdump 的输出。

2.3.5 readelf 软件

readelf 软件用来显示一个或多个 ELF 格式的目标文件的信息。可通过各种参数选项来控制 readelf 软件显示目标文件中的特定信息。关于更多的信息，可参看 Linux 系统所带的随机文档。

通过如下命令可看到 readelf 的一般帮助信息：

```
readelf -H
```

参数“elffile...”指定了要查看的目标文件序列。注意，readelf 软件不支持归档文件的信息显示，也不支持 64 位的 ELF 文件。选项的长格式和短格式是等价的（例如，-a 和 --all 的结果是一样的）。下面列出了可供选择的参数格式和意义（除-v 或-H 之外，至少要给出一个选项）。

-a/--all：等价于同时指定--file-header、--program-headers、--sections、--symbols、--relocs、--dynamic 和--version-info 这些选项。

-h/--file-header：显示包含在文件开始处的 ELF 头中信息。

-l/--program-headers/--segments：如果存在，则显示包含在文件的任何一个段头中的信息。

-S/--sections/--section-headers：如果存在，则显示包含在文件的任何一个节头中的信息。

-s/--symbols/--syms：如果存在，则显示文件的符号表的入口。

-e/--headers：显示文件中的所有头。等价于“-h -l -S”。

-r/--relocs：如果存在，则显示文件的重定位节中的内容。

-d/--dynamic：如果存在，则显示文件动态节的内容。

-V/--version-info：如果存在，则显示文件的版本节中的内容。

-D/--use-dynamic：当显示符号时，该选项可使得 readelf 软件使用文件动态节中的符号

更多高清PDF图书 www.gqpdf.com

表,而不是符号节中的符号表。

-x < number >/--hex-dump = < number >: 显示指定节的内容(由 < number > 指定)作为十六进制信息转储。

-v/--version: 显示 readelf 的版本号。

-H/--help: 显示能被 readelf 识别的所有命令行选项。

下面举例介绍 readelf 的使用。假设 test 是一个可执行文件,执行如下命令:

```
$ readelf -h test
```

则输出结果为:

```
ELF 头:
  Magic:   7f 45 4c 46 01 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF32
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              EXEC (可执行文件)
  Machine:                           Intel 80386
  Version:                           0x1
  入口点地址:                        0x8048298
  程序头起点:                        52 (bytes into file)
  Start of section headers:          2140 (bytes into file)
  标志:                              0x0
  本头的大小:                        52 (字节)
  程序头大小:                        32 (字节)
  程序头数量:                        7
  节头大小:                          40 (字节)
  节头数量:                          27
  字符串表索引节头:                  24
```

2.4 ld 链接器

ld 软件的作用是把各种目标文件(.o 文件)和库文件链接在一起,并定位数据和函数的地址,最终生成执行程序。ld 软件可使用链接命令语言(linker command language)表示的链接描述文件(linker script)来显式地控制链接的过程。通过 BFD(Binary Format Description)库,ld

可读取和操作 COFF、ELF、a.out 等各种执行文件格式的目标文件。

ld 的一个简单使用例子如下：

```
$ ld -o hello /lib/crt0.o hello.o -lc
```

它的意思是把 crt0.o、hello.o 和库文件 libc.a 链接起来产生 hello 执行程序。这里的 crt0.o 是应用程序编译链接时需要的启动文件，在程序链接阶段被链接，主要工作是初始化应用程序栈，初始化程序的运行环境和在程序退出时清除和释放资源。libc.a 中包含了 C 语言程序中常用的函数实现，如 printf(gets)等。

GCC 可间接地调用 ld，这主要通过给 GCC 传递一个参数 -Wl 来完成。GCC 不会处理放到-Wl 后面的参数，而是交给 ld 进行处理。例如：

```
$ gcc -Wl,--startgroup foo.o bar.o -Wl,--endgroup
```

等同于执行如下的 ld 命令：

```
$ ld --startgroup foo.o bar.o --endgroup
```

2.4.1 ld 选项

ld 命令具有许多选项，执行如下命令：

```
$ ld --help
```

就可列出 ld 常用的一些选项。

-b TARGET，--format TARGET：设置目标文件的文件格式。

-e ADDRESS，--entry ADDRESS：设置执行程序的开始地址。

-EB：链接大端格式(big-endian)的目标文件。

-EL：链接小端格式(small-endian)的目标文件。

-l LIBNAME，--library LIBNAME：创建执行程序时要链接的库文件。

-L DIRECTORY，--library-path DIRECTORY：寻找要链接的库文件时搜索的文件路径。

-M，--print-map：把符号的地址映射关系输出到屏幕上。

-o FILE，--output FILE：设置输出文件的名字。

-O LEVEL：如果 LEVEL 大于 0，则优化输出文件。

-r，-i，--relocatable：生成可重定位(relocatable)的输出文件，这种文件可作为 ld 的输入文件。

-s，--strip-all：去除输出文件中的所有符号信息。

-S，--strip-debug：去除输出文件中的调试符号信息。

--strip-discarded：去除 discarded 节(section)中的符号信息。

更多高清PDF图书 www.gqpdf.com

--no-strip-discarded：不去除 discarded 节(section)中的符号信息。

-t，--trace：输出 ld 处理的输入文件名。

-T FILE，--script FILE：读链接描述文件名，以确定符号等的定位地址。

-u SYMBOL，--undefined SYMBOL：不定义 SYMBOL。

-v，--version：输出 ld 的版本信息。

-x，--discard-all：去掉所有局部(local)符号信息。

-X，--discard-locals：去掉临时的局部符号信息，缺省情况会设置这个选项。

--discard-none：不去掉所有局部符号信息。

-(archives -)或 --start-group archives --end-group：在两个参数中的目标文件会被 ld 重复搜索并解析它们中的未定义符号，这对相互交叉引用的目标文件很有用。

--check-sections：检查各节之间是否地址重叠，缺省情况会设置这个选项。

--no-check-sections：不检查各节之间是否地址重叠。

--help：输出选项帮助信息。

-Map FILE：把符号映射关系输出到文件 FILE 中。

-pie，--pic-executable：创建位置无关(position independent)的执行程序。

--stats：输出内存使用统计。

--section-start SECTION = ADDRESS：设置节 SECTION 的起始地址为 ADDRESS。

-Tbss ADDRESS：设置节 bss 的起始地址。

-Tdata ADDRESS：设置节 data 的起始地址。

-Ttext ADDRESS：设置节 text 的起始地址。

--verbose：输出链接过程的详细信息。

2.4.2 链接描述文件介绍

链接描述文件用来显式地控制 ld 的链接过程。ld 的选项“-T FILE，--script FILE”可指定链接描述文件名。链接描述文件遵循特定的链接命令语言(linker scripts)的语法，语法会在后面介绍。链接描述文件描述了各个输入文件的各节如何映射到输出文件的各节中，并控制输出文件中各节或符号的内存布局。如果不指定链接描述文件，则 ld 会使用一个缺省的描述文件来产生执行文件。

目标文件是由多个节(section)构成的，如 data 节一般保存了有初值的全局变量，text 节保存了执行代码，bss 节保存了无初值的全局变量。目标文件中的每节都有名字和大小，且节可标识为 loadable，表示该节中的内容可加载到内存中。如果节不是 loadable 或 allocatable，则这种节可能包含调试信息。

通过如下命令可看到执行程序 test 的各节的信息：

```
objdump -h test
```

每个有 loadable 或 allocatable 标识的输出节有两种地址：一种地址是 VMA（Virtual Memory Address），这种地址是输出文件运行时节的运行地址；另一种地址是 LMA（Load Memory Address），这种地址加载输出文件时节的加载地址。一般情况下，这两种地址是相同的，但在嵌入式系统中，经常存在执行地址和加载地址不一致的情况，如把输出文件加载到开发板的 FLASH 存储器中（地址由 LMA 指定），但运行时，要把 FLASH 存储器中的输出文件拷贝到 SDRAM 中运行（地址由 VMA 指定）。

每个目标文件有许多符号，每个符号都有一个名字和一个地址。符号可以是定义的（defined）或未定义的（undefined）。符号可以是函数名、全局变量、静态变量等。可通过 nm 命令或“objdump -t”命令来查看目标文件或执行文件中的符号信息。

链接描述文件是一个文本文件，它主要由一系列的命令组成，每个命令可以是一个带参数的关键字或赋值语句。各个命令通过分号分隔。“/*”和“*/”之间的字符是注释。

2.4.3 链接描述文件的语法

链接描述文件的语法主要包括如下几类：

- 设置入口点（entry point）的命令（执行程序的第一条执行指令称为入口点）；
- 处理文件的命令；
- 处理文件格式的命令；
- 其他命令。

下面就一些常用的命令进行介绍。

ENTRY（symbol）：设置 symbol 的值为执行程序的入口点。ld 有多种方法设置执行程序的入口点，它按如下的顺序来确定程序的入口点。

① ld 的命令行选项-e 指定的值；

② ld 的链接描述文件中的 ENTRY（symbol）指定的值；

③ .text 节的起始地址；

④ 入口点为 0。

INCLUDE filename：包含其他 filename 的链接描述文件。

INPUT（file，file，...）：指定多个输入文件名。

OUTPUT（filename）：指定输出文件名。

SEARCH_DIR（path）：指定输入文件搜索路径。

STARTUP（filename）：指定第一个链接的输入文件名。

OUTPUT_FORMAT（bfdname）：指定输出文件的 BFD 格式。

TARGET（bfdname）：指定输入文件的 BFD 格式。

简单的赋值语句：

更多高清PDF图书 www.gqpdf.com

```
symbol = expression ;
symbol += expression ;
symbol -= expression ;
symbol *= expression ;
symbol /= expression ;
symbol <<= expression ;
symbol >>= expression ;
symbol &= expression ;
symbol |= expression ;
```

PROVIDE (symbol = expression)：定义了一个变量 symbol，且其值为 expression，这个变量可被输入文件引用。

MEMORY 命令：MEMORY 命令在用于嵌入式系统的链接描述文件中经常出现，它描述了各个内存块的起始地址和大小。其格式如下：

```
MEMORY
  {
    name \[(attr)\] : ORIGIN = origin, LENGTH = len
    ...
  }
```

name 可理解为描述的内存块名字 (region)。len 表示这个内存块的大小。attr 表示内存块的属性，它的值可以是：

R：只读。

W：可读/写。

X：可执行。

A：可分配 (allocatable)。

I：用于初始化的内存，初始化后，可回收为空闲内存。

!：表示“非”，与上述各个值配合使用。

SECTIONS 命令：告诉 ld 如何把输入文件的各个输出节映射到输出文件的各个输入节中，其格式如下：

```
SECTIONS
{
  sections-command
  sections-command
  ...
}
```

高清 PDF 藏 gqpdf.com

每个 SECTIONS 相关的命令可以是如下命令：ENTRY 命令、符号赋值（symbol assignment）命令、输出文件节描述（output section description）和覆盖描述（overlay description）。其中输出文件节描述的格式如下：

```
section \[address\] \[(type)\] : \[AT(lma)\]
  {
    output-section-command
    output-section-command
    ...
  } \[>region\] \[AT>lma_region\] \[:phdr :phdr ...\] \[=fillexp\]
```

上述“\[”和“\]”之间的属性是可选的，一般可以不用。常用属性的介绍如下：

address：指定输出节的起始地址。

type：每个输出节可以有一个属性 type。整个属性的格式如下：

```
(type_keyword)
```

type_keyword 可以是如下值：

NOLOAD：表示这个节在输出文件运行时是不加载到内存中的（not loadable）。

DESCT、COPY、INFO、OVERLAY：这几个值的含义都是一样的，都表示这个节在输出文件运行时是不分配内存的（not allocatable）。

region：表示可输出节的起始地址和大小由 MEMORY 命令中的 region 定义指定。下面是一个简单的例子：

```
MEMORY { rom : ORIGIN=0x1000, LENGTH=0x1000 }
SECTIONS { ROM : { *(.text) } >rom }
```

输出文件节描述可包括如下命令：符号赋值（symbol assignment）命令、输入文件节描述、输出节数据值定义和输出节关键字（一般使用很少）。

输出文件节的名字一般包括. text、. data 和. bss 等常见的节名字，也可定义其他一些特定的名字。如果输出节的名字是“/DISCARD/”，则它表示所有映射到这个输出节的输入节都不会在输出文件中存在。输入文件描述的格式如下：

```
filename(sectionname)
```

filename 可由通配符“*”表示，即表示所有输入文件，也可以是多个或单个输入文件的名字。sectionname 是输入文件中输入节的名字。下面是一些例子：

```
data.o(.data)
*(.text)
*(.init)
```


更多高清PDF图书 www.gqpdf.com

2.4.4 链接描述文件例子

很多链接描述文件都比较简单，有时可能只包含一个链接命令 SECTIONS。假设 ld 要输出的一个执行程序包含初始化数据的.data 节、未初始化数据的.bss 节和包含代码的.text 节，加载地址（LMA 为 0x1000000）和运行地址（即 VMA 为 0x8000000），而且 ld 处理的输入目标文件也只包含这 3 个节，则这个简单的链接描述文件例子如程序清单 2.7 所示。

程序清单 **2.7** 链接描述文件例子 **1**

```
SECTIONS
{
  . =0x1000000;
  .text : { *(.text) }
  . =0x8000000;
  .data : { *(.data) }
  .bss : { *(.bss) }
}
```

第一行是关键字 SECTIONS，紧接着是一系列的符号赋值，而输出文件中的节描述包含在“(”和“)”中。

大括号中的第一条语句是一个赋值语句，它表示起始的加载地址为 0x1000000，这里“.”表示当前位置的地址。第二条语句定义了输出文件的.text 节包含的内容，*（.text）表示把所有输入文件中的.text 收集起来，构成输出文件的.text 内容。“*”表示 ld 命令行上描述的所有输入文件。

第三条语句是一个地址赋值语句，它定义了输入文件接下来的.data 节的起始地址为 0x8000000，而输入文件的.bss 节的起始地址为 0x8000000 加上所有输入文件的.data 节的大小总和。

程序清单 2.8 是描述用于某个嵌入式设备的 μClinux 内核布局的链接描述文件（“//”后的语句是附加的注释，在实际的文件中不存在）。

程序清单 **2.8** 链接描述文件例子 **2**

```
//标注嵌入式设备中各个内存块的地址划分情况
MEMORY
{
  //表示 flash 中断向量表的起始地址为 0x01000000，长度为 0x00400
romvec : ORIGIN =0x01000000, LENGTH =0x00400
//标注 flash 的起始地址为 0x01000400，长度为 0x011fffff -0x01000400
```

```
flash : ORIGIN =0x01000400, LENGTH =0x011fffff -0x01000400
//标注 flash 的结束位置在 0x011fffff
eflash : ORIGIN =0x011fffff, LENGTH =0
//标注 ram 中中断向量表的起始地址为 0x00000000,长度为 1 024
ramvec : ORIGIN =0x00000000, LENGTH =1024
//标注 ram 其他可用的内存,长度为 0x003fffff -0x00000400
ram : ORIGIN =0x00000400, LENGTH =0x003fffff -0x00000400
//标注 ram 内存的结束位置
eram : ORIGIN =0x003FFFFF, LENGTH =0
}

SECTIONS
{
  //定义输出文件的.ramvec 节
.ramvec :
{
  //设定一个变量_ramvec 来代表当前的位置,即 ramvec 节的开始处
ramvec = . ;
} > ramvec
//把该节定义到 MEMORY 中定义的 ramvec 所代表内存块中,即从 0x00000000 开始的 1 024 字节
//定义输出文件的.data 节
.data :
{
//设定一个变量_data_start 来代表当前的位置,即 data 节的开始处
__data_start = . ;
//把所有输入文件中.data 节的数据放在此处
*(.data)
__data_end = ALIGN( 0x10 ) ;
//定义 edata 变量的值为.data 节的结束地址
edata = . ;
//把 edata 按 16 位对齐
edata = ALIGN( 0x10 ) ;
} > ram
//把.data 节的内容放到 ram 定义的 MEMORY 中
//定义未初始化数据的.bss 节
.bss :
{
```


高清PDF藏 gqpdf.com
更多高清PDF图书 www.gqpdf.com

```
    //记录该节的起始位置
__bss_start = ALIGN( 0x10 ) ;
//记录.bss 节的结束位置

//把所有.bss 数据放在此处
*(.bss)
//把所有 COMMON 的数据放在此处
*(COMMON)
//记录.bss 节结束的位置,16 位对齐
end = ALIGN( 0x10 );
//记录.bss 节结束的位置,16 位对齐
_end = ALIGN( 0x10 ) ;
} > ram
//把.bss 节的内容放到 MEMORY 中定义的 ram 所代表的内存块中

//定义输出文件的.eram 节
.eram :
{
  //记录该节结束的位置
_ramend = . ;
} > eram
//把.eram 节的内容放到 MEMORY 中定义的 eram 所代表的内存块中
  //定义输出文件的.romvec 节,用于记录 flash 中中断向量表
.romvec :
{
  //记录该节的位置
_romvec = . ;
*(.data.initvect)
} > romvec
//将该节内容放到 MEMORY 中定义的 romvec 中
//定义输出文件.text 节,即程序段
.text :
{
//记录.text 节的起始位置
text_start = . ;
//将所有输入文件.text 节的内容放到这里
*(.text)
```

高清藏 PDF gqpdf.com

```
//记录.text 节结束的位置
_etext = . ;
//标志.text 节中数据结束的位置
} > flash
//把.text 节的内容放到 MEMORY 中定义的 flash 表示的内存块中
//定义 flash 结束的节
.eflash :
{
//记录当前的位置
_flashend = . ;
} > eflash
//记录该节放入 MEMORY 中的位置
}
//描述文件结束
```

2.5 make 介绍

2.5.1 GNU make 简介

GNU make(简称 make)是一种代码维护工具,在大中型项目中,它将根据程序各个模块的更新情况,自动维护和生成目标代码。make 的主要任务是读入一个文本文件(缺省文件名是 makefile 或 Makefile),并根据这个文本文件定义的规则和步骤,完成整个软件项目的维护和代码生成等工作。这个文本文件里主要定义了依赖关系(即有关哪些文件的最新版本是依赖于哪些其他文件产生或组成的)和需要用什么命令来产生文件的最新版本或管理各种文件等工作。有了这些信息,make 会检查文件的修改或生成时间戳。如果目标文件的修改或生成时间戳比它的任意一个依赖文件旧,则 make 就执行 Makefile 文件中描述的相应命令,以便更新目的文件。其特点如下:

- 适合于支持多文件构成的大中型软件项目的编译、链接、清除中间文件等管理工作;
- 只更新那些需要更新的文件,而不重新处理那些并不过时的文件;
- 提供和识别许多缺省规则,方便对大型软件项目的管理;
- 支持对多目录的软件项目进行递归管理;
- 对软件项目,具有渐进式的可维护性和扩展性。

GNU make 程序具有的使用形式为:

```
make   \[选项\] \[变量定义\] \[目标\]
```


更多高清PDF图书 www.gqpdf.com

make 的选项指出 make 的工作方式，其常用的选项如表 2.3 所列。

表 2.3　make 命令常用选项

make 选项	含　义
-f filename	使用指定的文件作 Makefile
-C dirname	make 在开始运行后的工作目录为指定目录
-e	不允许在 Makefile 中替换环境变量的赋值
-k	执行命令出错时，放弃当前目标，继续维护其他目标
-n	按实际运行时的执行顺序模拟执行命令（包括用@ 开头的命令），没有实际执行效果，仅用于显示执行过程
-p	显示 Makefile 中所有变量和内部规则
-r	忽略内部规则
-s	执行但不显示命令，常用来检查 Makefile 的正确性
-S	如果执行命令出错，则退出
-t	修改每个目标文件的创建日期
-I	忽略运行 make 中执行命令的错误
-V	打印 make 的版本号

在使用 make 维护目标时，为了调试错误和看到输出选项，常使用如下方式：

```
$ make 2 >&1 |tee errofile
```

这样，make 的输出和错误信息就都写入了 errofile，同时在屏幕上也可看到输出和错误信息。

由于 make 主要根据 Makefile 中定义的规则执行各种命令，所以下面将主要介绍 Makefile 的用法。make 使用了一些难懂的语法条件，其中最重要且最容易出错的一条语法就是“每一个命令列的开头都要是一个 tab 字符”。一个常犯的错误就是在每个命令行的开头省略了 tab 字符。在每个命令行中按空白键插入 4 个空白来代替 tab 字符是行不通的，因为 make 无法辨别出这就是 tab 字符，而且在这种情况下，make 也无法提供有用的信息来定位这种错误。由于 make 是靠开头的那个 tab 字符来辨识命令行，所以一定要注意不要在其他不是命令行的那一行的开始加上 tab 字符。

2.5.2　Makefile 及其语法

Makefile 中的内容大致可分成宏定义区和基于依赖关系的指令区两大部分。宏定义区主要定义了一些变量，Makefile 里的变量就像 UNIX 的环境变量一样，这些变量名区分大小写，但一般使用大写字母。它们可在 Makefile 的任何地方被引用，也可被用来做如下事情：

- 保存文件名列表。如果生成可执行文件的一些规则包含一些目标文件名，定义一个变量来存储所有目标文件名，且在相关规则中引用这个变量，则可减少规则描述的数据量。如果需要加入新的目标文件名，则只修改这个变量即可。这导致在项目管理中加入新的目标文件会变得简单且较不易出错。
- 保存可执行文件名。如果软件项目需要用于一个非 GNU Tools 编译环境工具集，则必须将 Makefile 中所有使用编译器的地方改成新的编译器名。但如果使用一个变量来代替编译器名，那么只需改变变量定义的地方，其他所有引用这个变量的地方不用改变。
- 保存参数。如果需要给编译命令定义一组相同的编译选项，且这组编译选项会被多个规则引用，那么可把这组选项赋值给一个变量，把这个变量放在所有引用它的地方。这样当需要改变选项时，只需在变量赋值的地方改变即可。

在 Makefile 中定义一个变量，无须声明，直接赋值就隐含地在系统中定义了这个变量。以后要引用这个变量，表示法为：以 $ 符号开始，后面是围在小括号里的变量名。下面是一个简单的例子：

```
MYTEST = test
MYTEST2 = $ (MYTEST)
```

除了 Makfile 中定义的变量外，make 还识别一些预先设定的内部变量。其中比较常用的变量包括：

$ *　不包含文件扩展名的依赖文件名。
$?　比目标文件新的依赖文件。
$ @　当前规则的目的文件名。
$ <　当前规则中依赖文件列表中的第一个依赖文件。
$ ^　当前规则中依赖文件列表中的所有依赖文件。
$ %　当目标是归档文件（库文件）时，该变量是目标成员名。例如，如果目标是 foo. a (bar. o)，则 $ % 的值是 bar. o，$ @ 的值是 foo. a。如果目标不是归档文件，则 $ % 是空值。

Makefile 中的规则一般用于解释如何和何时重建或处理特定文件，一般这些特定文件是规则的目标。make 需要先调用命令对依赖关系进行检查，然后才能创建或更新目标。当然，一个规则也可用于解释如何和何时执行一个动作。

一个 Makefile 文件可包含规则以外的其他文本，但一个简单的 Makefile 文件仅需要包含规则。虽然真正的规则比这里展示的例子复杂，但格式却完全一样。规则的一般形式如下：

```
目标/文件：依赖文件列表;命令
```

如一个规则可写成：

更多高清PDF图书 www.gqpdf.com

```
test: test.c;gcc -O -o test test.c
```

或是可将命令写在下一行，但需要有定位字元 TAB 作为前导，因此可写成：

```
test: test.c
(TAB)gcc -O -o test test.c
```

“(TAB)”表示用户按了 TAB 键后产生的空白区域，通常以此方式编写较好。若依赖文件列表中由两个以上的文件组成，则各文件名称间以空白隔开即可。例如：

```
test: test1.c test2.c
(TAB)gcc -O -o test test1.c test.c
```

目标通常是要产生的文件名称，有时就是一个要执行的虚拟目标，目标的例子是可执行文件或 OBJ 文件。目标也可以是一个执行的动作名称，如 clean 或 cleanall 等。依赖文件列表是用来输入从而产生目标的文件，一个目标经常有多个依赖文件。命令是 make 执行的动作，一个规则可以含有几个命令，每个命令占一行。

每个命令行前面必须是一个 TAB 字符，即命令行第一个字符是 TAB。这是容易出错的地方。

如果一个依赖关系发生变化，即依赖文件的修改时间比目标文件的修改时间更靠前或目标文件不存在，则需要规则调用命令对相应依赖和服务进行处理，从而更新或创建目标。但是，指定命令更新目标的规则并不都需要依赖，例如，目标 clean 和 cleanall 相联系的删除命令的规则就没有依赖。

编译并生成执行代码并不是编写 make 规则的惟一事情。Makefile 文件可让 make 完成编译程序以外的其他任务，例如删除中间文件和可执行文件等。设执行文件是 exec_file，中间文件是以“.o”结尾的文件，下面是一个这样的规则例子：

```
clean:
        $ rm * .o exec_file
```

如果在一个实际应用中，存在 clean 这个文件，则可采用下面的方法：

```
.PHONY : clean
clean :
(TAB)rm * .o exec_file
```

.PHONY 确保下面的 clean 是一个虚拟目标，而不是一个文件，这样可防止 make 因为存在名为 clean 的文件而发生混乱，避免了在执行 rm 命令时可能发生的错误。可把 Makefile 中使用到的规则分为两类：隐含规则和显式规则。下面分别介绍这两种规则。

1. 隐含规则

Makefile 中的隐含规则能告诉 make 如何使用传统的技术完成任务，而不需要开发人员详

高清PDF gqpdf.com

细地指定它们。例如，有一条编译 C 语言源程序的隐含规则指出，编译 C 语言程序一般是使用“.c”文件（依赖文件），产生“.o”文件（目标文件），用的编译器是 cc（执行的命令）。因此，make 据此和文件名的后缀就可决定使用编译 C 语言源程序的隐含规则。一系列的隐含规则可按顺序应用，例如，make 借助“.c”文件可从一个“.y”文件重建一个“.o”文件，这需要用到隐含规则链。内建隐含规则的命令需要使用变量，通过改变这些变量的值，就可改变隐含规则的工作方式。例如，变量 CFLAGS 控制隐含规则用于编译 C 程序传递给 C 编译器的标志。可参阅节中关于隐含规则使用的变量的说明。通过编写格式规则，开发人员可创建自己的隐含规则。

下面是编译 C 程序和 C++程序的隐含规则定义：

- Compiling C programs（编译 C 程序）。
- “filename.o”自动由“filename.c”使用命令“$（CC）-c $（CPPFLAGS）$（CFLAGS）”生成。
- Compiling C++ programs（编译 C++程序）。
- “filename.o”自动由“filename.cc”或“filename.C”使用命令“$（CXX）-c $（CPPFLAGS）$（CXXFLAGS）”生成。

在隐含规则中使用了许多预定义的变量。内建隐含规则的命令对预定义变量的使用是开放的，开发人员可在 Makefile 文件中改变变量的值，也可使用 make 的运行参数或在环境变量中改变这些预定义的变量，这样在不对这些规则本身重新定义的情况下，就可以改变这些规则的工作方式。还可使用选项“-R”或“—no-builtin-variables”删除所有隐含规则使用的变量。

例如，编译 C 程序的隐含规则的命令实际是“$（CC）-c $（CFLAGS）$（CPPFLAGS）”，变量“$（CC）”缺省的值是 cc。如果重新定义变量“$（CC）”的值为 ncc，则所有隐含规则将使用 ncc 作为编译 C 语言源程序的编译器。通过重新定义变量“$（CFLAG）”的值为“-g”，则可将“-g”选项传递给编译器。

隐含规则使用的变量可分为两类：一类是程序名变量（像“$（CC）”）；另一类是包含程序运行参数的变量（像“$（CFLAG）”）。程序名变量可能也包含一些命令参数，但它必须以一个实际可执行的程序名开始。如果一个变量值中包含多个参数，则它们之间用空格隔开。

下面是内建规则中常用的程序名变量列表。

- AR：档案管理程序，缺省为 ar。
- AS：汇编编译程序，缺省为 as。
- CC：C 语言编译程序，缺省为 cc。
- CXX：C++编译程序，缺省为 g++。
- CPP：带有标准输出的 C 语言预处理程序，缺省为“$（CC）-E”。
- LD：链接程序，缺省为 ld。
- RM：删除文件的命令，缺省为“rm -f”。

更多高清PDF图书 www.gqpdf.com

下面是为上述程序附加参数的变量列表。在没有注明的情况下，所有变量的值为空值。

- ARFLAGS：用于档案管理程序的标志，缺省为“rv”。
- ASFLAGS：用于汇编编译器的额外标志（当具体调用“.s”或“.S”文件时）。
- CFLAGS：用于 C 编译器的额外标志。
- CXXFLAGS：用于 C++编译器的额外标志。
- CPPFLAGS：用于 C 预处理以及使用它的额外标志（C 和 Fortran 编译器）。
- LDFLAGS：用于调用 linker(ld)的编译器的额外标志。

2. 显式规则

开发人员可通过编写格式规则定义新的隐含规则或取代旧的隐含规则。该规则看起来与普通规则类似，不同之处在于格式规则的目标中包含字符“%”。目标是匹配文件名的格式；字符“%”可以匹配任何非空的字符串，而其他字符仅仅与它们自己相匹配。依赖用“%”表示它们的名字和目标名关联。格式“%.o：%.c”是将任何“.c”结尾的文件编译为“.o”结尾的文件的格式规则。在格式规则中，使用的“%”扩展是在所有变量和函数扩展以后进行的，它们是在 Makefile 文件读入时完成的。

例如，“%.c”匹配任何以“.c”结尾的文件名；“s.%.c”匹配以“s.”开始并且以“.c”结尾的文件名，该文件名至少包含 5 个字符（因为“%”至少匹配一个字符）。要使用格式规则，文件名必须匹配目标的格式，而且符合依赖格式的文件必须存在或可以创建。下面规则：

```
% .o : % .c ; command...
```

表明要创建目标文件 filename.o，使用 filename.c 作为它的依赖文件，而且文件 filename.c 必须存在或可以创建。

在格式规则中，依赖有时不含有“%”，这表明采用该格式规则创建的所有文件都是采用相同的依赖。这种固定依赖的格式规则在有些场合十分有用。

格式规则在 Makefile 文件中的次序很重要，所以必须考虑它们次序的写法。对于多个都能使用的规则，使用最先出现的规则，即开发人员编写的规则比内建的规则优先。注意，依赖存在或被提及的规则优先于依赖需要经过隐含规则链生成的规则。

下面是一个实际在 make 中预定义的格式规则例子。编译“.c”文件生成“.o”文件的规则：

```
% .o : % .c
    $(CC) -c $(CFLAGS) $(CPPFLAGS) $< -o $@
```

该格式规则定义了一条编译 filenamex.c 文件生成 filename.o 文件的规则，命令使用内部变量“$@”和“$<”替换任何情况使用该规则的目标文件和源文件。

3. Makefile 中的自动生成依赖关系

最容易想到的编写规则的方法是人工查看源码文件，把它们的目标文件作为目标，把源码

文件和被它#include 的头文件作为依赖文件列表，把编译过程作为命令，但可能要把其他被这些头文件#include 的其他头文件也列为依赖文件。实际上，在较大的软件项目中，这种方法很容易出错。可让编译器（GCC）帮助自动完成分析依赖关系的工作。

在 GCC 编译每一个源码文件时，GCC 知道应包括哪些头文件。使用 GCC 时，用“-M”参数，它会为“.c”文件输出一个规则，即把目标文件（“.o”文件）作为目标，而这个 C 文件和所有应被#include 的头文件将作为依赖文件。如果使用“-M”参数，则这个规则会加入所有 header 文件，包括被角括号（“<”、“>”）和双引号（“""”）所包围的文件。如果用“-MM”参数来代替“-M”参数传递给 GCC，那些用角括号包围的头文件将不会被包括。

可使用自动生成依赖的方式简化依赖关系的生成。例如，对每一个源程序文件 filename.c 有一个名为 filename.d 的 Makefile 文件和它对应，该 Makefile 文件中列出了名为 filename.o 的目标文件所依赖的源文件和头文件。这种方式的优点是，仅在源程序文件改变的情况下才有必要重新扫描生成新的依赖。

这里有一个根据 C 语言源程序 name.c 生成名为 name.d 依赖文件的格式规则：

```
%.d: %.c
    set -e; $(CC) -M $(CPPFLAGS) $< \\
    | sed 's/\\($*\\)\\.o\[:\]*/\\1.o $@ : /g' > $@; \\
    \[ -s $@ \] || rm -f $@
```

关于定义格式规则的信息参阅定义与重新定义格式规则。“-e”开关是告知 shell，如果“$(CC)”命令运行失败（非零状态退出），则立即退出。正常情况下，shell 退出时带有最后一个命令在管道中的状态（sed），因此 make 不能注意到编译器产生的非零状态。

对于 GNU C 编译器，可使用“-MM”开关代替“-M”，这样省略了有关系统头文件的依赖关系。

命令 sed 的作用是翻译：

```
main.o : main.c defs.h
```

到

```
main.o main.d : main.c defs.h
```

这使每一个“.d”文件和与之对应的“.o”文件依靠相同的源程序文件和头文件，据此，make 可以知道如果任一个源程序文件和头文件发生变化，则必须重新构造依赖文件。

一旦定义了重新构造“.d”文件的规则，则可使用 include 命令直接将它们读入。例如：

```
sources = foo.c bar.c
include $(sources:.c=.d)
```

这个例子中使用一个代替变量参照从源程序文件列表“foo.c bar.c”翻译到依赖文件列表

更多高清PDF图书 www.gqpdf.com

“foo. d bar. d”。详细内容参阅替换引用。因此,“. d”的 Makefile 文件与其他 Makefile 文件一样,即使没有使用任何进一步的指令,make 也同样会在必要时重新构建它们。

4. Makefile 中的函数

Makefile 里内建的函数允许开发人员在 Makefile 文件中处理文本、计算文件、操作使用命令等。在函数调用时,必须指定函数名以及函数操作使用的参数。函数处理的结果将返回到 Makefile 文件中的调用处。使用函数的方式与使用变量的方式类似,用一个“$”符号和“(”或“{”开始,然后是函数名,接下来是空格,后跟一列由逗号分隔的参数,最后用“)”或“}”结束。其通用格式如下:

```
$(function arguments)
```

或

```
${function arguments}
```

例如,有一个名为 wildcard 的函数,它有一个参数,功能是展开成一列所有符合由其参数描述的文件名,文件间以空格间隔。可按如下所示使用这个命令:

```
SOURCES = $(wildcard * .c)
```

这样函数 wildcard 产生一个所有以“. c”结尾的文件列表,然后存入变量 SOURCES 里。另一个有用的函数是 patsubst(patten,substitude, var)函数。它需要 3 个参数:第 1 个是一个需要匹配的模式;第 2 个表示用什么来替换它;第 3 个是一个需要被处理的由空格分隔的字符列表。例如,处理那个经过上面定义后的变量:

```
OBJS = $(patsubst % .c,% .o, $(SOURCES))
```

这行将处理所有在 SOURCES 字列中的字(一列文件名),如果它的结尾是“. c”,则用“. o”取代“. c”。这里的符号“%”将匹配一个或多个字符,而它每次所匹配的字符串即是第 2 个参数里的符号“%”。

Makefile 中的 shell 函数很有用,它是 make 与外部环境的通信工具。函数 shell 和在大多数 shell 中后引号(`)执行的功能一样用于命令的扩展。这意味着它起着调用 shell 命令和返回命令输出结果的参数的作用。make 仅仅处理返回结果,在返回结果替换调用点之前,make 将每一个换行符或一对回车/换行符处理为单个空格;如果返回结果最后是换行符(和回车符),make 将把它们去掉。由函数 shell 调用的命令,一旦函数调用展开,就立即执行。在大多数情况下,当 Makefile 文件读入时,函数 shell 调用的命令就已执行。例外情况是在规则命令行中该函数的调用,因为这种情况下只有在命令运行时函数才能扩展,其他调用函数 shell 的情况与此类似。

这里有一些使用函数 shell 的例子:

```
contents := $(shell cat foo)
```

将文件 foo 的内容设置为变量 contents 的值，是用空格（而不是换行符）分离每一行。

```
files := $(shell echo *.c)
```

将“*.c”的扩展设置为变量 files 的值。

在 Makefile 中还有许多内定的函数，这里就不一一介绍了。

5. Makefile 中的条件语句

Makefile 的条件语句可导致根据变量的值执行或忽略 Makefile 文件中一部分脚本。条件语句可将一个变量与其他变量的值相比较，或将一个变量与一字符串常量相比较。条件语句用于控制 make 实际看见的 Makefile 文件部分，不能用于在执行时控制 shell 命令。

对于没有 else 指令的条件语句的语法为：

```
conditional-directive
text-if-true
endif
```

“text-if-true”可以是任何文本行。在条件为真时，它被认为是 Makefile 文件的一部分；如果条件为假，将被忽略。完整条件语句的语法为：

```
conditional-directive
text-if-true
else
text-if-false
endif
```

如果条件为真，则使用 text-if-true 作为 Makefile 文件的一部分；如果条件为假，则使用 text-if-false 作为 Makefile 文件的一部分。text-if-true 和 text-if-false 可以是任意多行的文本。

关于 conditional-directive 的语法，对于简单条件语句和复杂条件语句完全一样，主要有 4 种不同的指令用于测试不同的条件。

测试相同条件的 conditional-directive 语法如下：

```
ifeq (arg1, arg2)
ifeq 'arg1' 'arg2'
ifeq "arg1" "arg2"
ifeq "arg1" 'arg2'
ifeq 'arg1' "arg2"
```

扩展参数 arg1、arg2 中的所有变量引用，并且比较它们。如果它们完全一致，则使用 text-if-true；否则使用 text-if-false（如果存在）作为 Makefile 文件的一部分。需要注意的一个问题

更多高清PDF图书 www.gqpdf.com

是，如果要测试一个变量是否有非空值，可能由于变量的值包含空格而被认为不是空值，并可能由此造成混乱。因此，可使用 strip 函数消去空格，从而避免空格作为非空值的干扰。例如：

```
ifeq ($(strip $(foo)),)
    text-if-empty
endif
```

即使“$(foo)”中含有空格，也使用 text-if-empty 作为 Makefile 文件的一部分。

测试不同条件的 conditional-directive 语法如下：

```
ifneq (arg1, arg2)
ifneq 'arg1' 'arg2'
ifneq "arg1" "arg2"
ifneq "arg1" 'arg2'
ifneq 'arg1' "arg2"
```

扩展参数 arg1、arg2 中的所有变量引用，并且比较它们。如果它们不同，则使用 text-if-true；否则使用 text-if-false（如果存在）作为 Makefile 文件的一部分。

测试变量有定义的 conditional-directive 语法如下：

```
ifdef variable-name
    text-if-true
  else
    text-if-false
```

如果变量 variable-name 是非空值，则 text-if-true 有效；否则 text-if-false 有效（如果存在）。若变量从未被定义过，则变量是空值。注意，ifdef 仅测试变量是否有值，它不能扩展到看变量是否有非空值。因此，使用 ifdef 测试所有定义过的变量都返回真。

测试变量无定义的 conditional-directive 语法如下：

```
ifndef variable-name
    text-if-true
else
    text-if-false
```

如果变量 variable-name 是空值，则 text-if-true 有效；否则 text-if-false 有效（如果存在）。

在条件语句的指令行前面允许有多余的空格，它们在处理时被忽略，但不允许有 Tab（如果一行以 Tab 开始，那么该行将被认为是规则的命令行）。除此之外，空格和 Tab 可插入到行的任何地方，当然，指令名和参数中间除外。以“#”开始的注释可在行的结尾。

在条件语句中另外两个有影响的指令是 else 和 endif。这两个指令以一个单词的形式出

现，没有任何参数。在指令行前面允许有多余的空格，空格和 Tab 可插入到行的中间，以“#”开始的注释可在行的结尾。

条件语句影响 make 使用的 Makefile 文件。如果条件为真，则 make 读入 text-if-true 包含的行；如果条件为假，则 make 读入 text-if-false 包含的行（如果存在）。需要注意的是，由于当读入 Makefile 文件时，make 需要计算条件的值，因此不能在测试条件时使用 Makefile 内定的一些变量，因为它们是命令执行时才被定义的。

2.5.3 μClinux-dist2.4 中的 Makefile 分析

对基于 μClinux-2.4.x 内核的 μClinux-dist 而言，它包含的多个 Makefile 文件决定了 μClinux 是如何编译 μClinux 内核和生成文件系统的。本小节的主要目的是让读者了解 μClinux 通过 Makefile 完成最终的内核、应用程序和文件系统的生成。

下面就以 μClinux 中的 Makefile 为例来介绍 μClinux 中是如何组织、编译源文件的。在取得 μClinux 内核后，就可以编译了。μClinux 中是靠 Makefile 来组织并控制编译、链接和生成最终的目标文件的。而当用户取得内核源代码时，就可以执行如下的操作且可以编译出内核文件。

一般的步骤如下面的程序所示：

```
$ make * * * _config  这里 * * * 是表示其中一个开发板
$ make oldconfig
$ make dep
$ make
```

上面的这 4 步是使用系统提供的默认情况。如果用户想定制自己的内核，则一般使用如下步骤：

```
$ make menuconfig
$ make dep
$ make
```

比如当使用 LPC2200 开发板时，就可使用如下的命令来配置内核：

```
$ make lpc2200_config
$ make oldconfig
$ make dep
$ make
```

下面就以 LPC2200 为例来介绍是如何将源文件编译成内核的。

1. make lpc2200_config

会执行 arch/armnommu/Makefile 文件中的% _config 部分（参见程序清单 2.9）。

更多高清PDF图书 www.gqpdf.com

程序清单 **2.9**　**arch/armnommu/Makefile** 片断

```
#
# Configuration targets.  Use these to select a
# configuration for your architecture
% _config:
@ ( \\
CFG = $ (@ :_config = ); \\
if \[ -f arch/armnommu/def-configs/$ $ CFG \]; then \\
  $ (RM) arch/armnommu/defconfig; \\
  cp arch/armnommu/def-configs/$ $ CFG arch/armnommu/defconfig; \\
  echo " * * * Default configuration for $ $ CFG installed"; \\
  echo " * * * Next, you may run 'make oldconfig'"; \\
else \\
  echo " $ $ CFG does not exist"; \\
fi; \\
)
```

从程序清单 2.9 可以看出，make lpc2200_config 做了如下的工作。

① 把 make 输入的命令替换出适当的字符串。比如这里就是将 lpc2200_config 替换成 lpc2200，这时 CFG 就是 lpc2200。

② 根据 CFG 在 arch/armnommu/def-configs 找到与 CFG 匹配的文件。这就是用户想要的默认配置文件。比如这里是将 lpc2200 放在 arch/armnommu/def-configs 文件夹下面。

③ 将系统默认的配置文件 copy 到 arch/armnommu/目录下面：

```
cp arch/armnommu/def-configs/$ $ CFG arch/armnommu/defconfig
```

这里就是

```
cp arch/armnommu/def-configs/lpc2200 arch/armnommu/defconfig
```

2. make oldconfig

当执行 make oldconfig 时，就会执行顶层 Makefile 中如程序清单 2.10 所示部分。

程序清单 **2.10**　**Makefile** 片断 **1**

```
symlinks:
        rm -f include/asm
        ( cd include ; ln -sf asm-$ (ARCH) asm)
        @ if \[ ! -d include/linux/modules \]; then \\
```

```
                mkdir include/linux/modules; \\
        fi
oldconfig: symlinks
        $(CONFIG_SHELL) scripts/Configure -d arch/$(ARCH)/config.in
```

从上面可以看出，oldconfig 依赖于 symlinks，而 symlinks 则是根据 ARCH 在 linux/include 中建立 asm 的符号链接的，ARCH 是 linux/Makefile 中定义的变量。lpc 设置为 armnommu。

在建立了符号链接后，oldconfig 就会调用如下命令：

```
$(CONFIG_SHELL) scripts/Configure -d arch/$(ARCH)/config.in
```

linux/scripts/Configure 是一个 bash 脚本，主要作用是根据传入的 config. in 产生. config 和 include/linux/autoconf. h 文件。

3. make dep

make dep 这一步就比较重要了，主要是产生应用程序 scripts/mkdep，然后使用 scripts/mkdep 在每个内核目录下所用的 c、S、h 文件创建依赖关系，这里涉及的目录是 kernel drivers mmnommu fs net ipc lib crypto arch/armnommu/mach-lpc arch/armnommu/kernel arch/armnommu/mm arch/armnommu/lib arch/armnommu/nwfpe。创建的依赖文件是“. depend”，而在 linux/Rules. make 中就将“. depend”包含在 Makefile 文件中。make dep 这一步执行的代码参考程序清单 2. 11。

程序清单 **2. 11**　**Makefile** 片断 **2**

```
dep-files: scripts/mkdep archdep include/linux/version.h
        rm -f .depend .hdepend
        $(MAKE) $(patsubst %,_sfdep_%,$(SUBDIRS)) _FASTDEP_ALL_SUB_DIRS="
$(SUBDIRS)"
        find $(FINDHPATH) \\( -name SCCS -o -name .svn \\) -prune -o -follow -name \
\* .h ! -name modversions.h -print |xargs scripts/mkdep --  > .hdepend
        scripts/mkdep --init/* .c > .depend

depend dep: dep-files
```

从上面可以看出，dep 依赖于 dep-files，而 dep 的命令行为空。dep-files 依赖 3 个目标 scripts/mkdep、archdep 和 include/linux/version. h。其中 scripts/mkdep 就是上面提到的产生依赖关系的 helper function；archdep 则是为每一个 CPU 进行特定的处理目标；include/linux/version. h 是产生该系统的版本号。

上面 dep-files 对应的命令行中第二条命令展开就是：

更多高清PDF图书 www.gqpdf.com

```
make _sfdep_kernel _sfdep_drivers _sfdep_mmnommu _sfdep_fs _sfdep_net _sfdep_ipc
_sfdep_lib _sfdep_crypto _sfdep_arch/armnommu/mach-lpc _sfdep_arch/armnommu/
kernel _sfdep_arch/armnommu/mm _sfdep_arch/armnommu/lib _sfdep_arch/armnommu/
nwfpe _FASTDEP_ALL_SUB_DIRS = "kernel drivers mmnommu fs net ipc lib crypto arch/
armnommu/mach-lpc arch/armnommu/kernel arch/armnommu/mm arch/armnommu/lib arch/
armnommu/nwfpe"
```

对每个目录都有一个_sfdep_目标。该目标定义在 linux/Rules. make 文件中，这样，用这种方式就可对每个目录产生依赖文件了。

```
fastdep: dummy
        $ (TOPDIR) /scripts/mkdep $ (CFLAGS) $ (EXTRA_CFLAGS_nostdinc) -- $ (wild-
card * .\[chS \]) > .depend
ifdef ALL_SUB_DIRS
        $ (MAKE) $ (patsubst % ,_sfdep_% , $ (ALL_SUB_DIRS)) _FASTDEP_ALL_SUB_DIRS
 = " $ (ALL_SUB_DIRS) "
endif

ifdef _FASTDEP_ALL_SUB_DIRS
 $ (patsubst % ,_sfdep_% , $ (_FASTDEP_ALL_SUB_DIRS)):
        $ (MAKE) -C $ (patsubst _sfdep_% ,% , $ @ ) fastdep
endif
```

将上面 ifdef _FASTDEP_ALL_SUB_DIRS 展开，就可得到每个目录的规则：这里_FASTDEP_ALL_SUB_DIRS 就是上面 make 传入的参数。

```
    _FASTDEP_ALL_SUB_DIRS = "kernel drivers mmnommu fs net ipc lib crypto arch/
armnommu/mach-lpc arch/armnommu/kernel arch/armnommu/mm arch/armnommu/lib arch/
armnommu/nwfpe"
```

$ (patsubst %，_sfdep_%，$ (_FASTDEP_ALL_SUB_DIRS))：就是将_FASTDEP_ALL_SUB_DIRS 字符串进行模式匹配，在_FASTDEP_ALL_SUB_DIRS 每项前面添加_sfdep_。这样每个目录就有一个 target，例如：

```
_sfdep_kernel:
    make -C kernel fastdep
```

$ @ 是 make 定义的符号，表示每一个目标，这里就是_sfdep_kernel。显然，上面就是将_sfdep_ 前缀去掉，然后进入 kernel 目录 make fastdep。

而 fastdep 目标相关的命令行如下：

高清PDF gqpdf.com

```
$(TOPDIR)/scripts/mkdep $(CFLAGS) $(EXTRA_CFLAGS_nostdinc) -- $(wildcard *.\
[chS\]) > .depend
```

就是对 kernel 目录下面所有 c、S、h 文件通过工具 scripts/mkdep 产生依赖文件“.depend”。

这里 scripts/mkdep 用法是：

```
Usage: mkdep cflags --file
```

当然，如果在目录下面还有子目录，同样可按这种方式处理。

4. make

make 就要执行实际的编译链接工作了（参见程序清单 2.12）。

程序清单 **2.12　Makefile** 片断 **3**

```
all:    do-it-all
do-it-all:    Version $(LINUX)
Version: dummy
        @ rm -f include/linux/compile.h
```

其中 LINUX = linux。

从上面可以看出，all 是 Makefile 的第一个目标，也就是 make 执行的缺省目标，这个目标进而会依赖于 linux（参见程序清单 2.13）；而 linux 则是 make 中最主要的一个目标了，其中 linuxsubdirs 目标又是系统中所有核心文件目标。

程序清单 **2.13　Makefile** 片断 **3**

更多高清PDF图书　www.gqpdf.com

```
$(LINUX): include/linux/version.h $(CONFIGURATION) init/main.o init/version.o
init/do_mounts.o linuxsubdirs
        $(LD) $(LINKFLAGS) $(HEAD) init/main.o init/version.o init/do_mounts.
o \\
                --start-group \\
                $(CORE_FILES) \\
                $(DRIVERS) \\
                $(NETWORKS) \\
                $(LIBS) \\
                --end-group \\
                -o $(LINUX)
        $(NM) $(LINUX) |grep -v '\\(compiled\\) \\|\\(\\.o $$ \\) \\|\\( \[aUw\] \\)
\\|\\(\\.\\.ng $$ \\) \\|\\(LASH\[RL\]DI \\)' |sort > System.map

linuxsubdirs: $(patsubst %, _dir_%, $(SUBDIRS))

$(patsubst %, _dir_%, $(SUBDIRS)) : dummy include/linux/version.h include/con-
fig/MARKER
        $(MAKE) CFLAGS="$(CFLAGS) $(CFLAGS_KERNEL)" -C $(patsubst _dir_%,
% , $@)
```

上面两个规则中首先是给 SUBDIRS 中每一项加前缀_dir_，而每一个前缀的目标在紧接着一行定义了；然后执行其中命令时又把其中的_dir_去除掉。

下面把它们都展开来说明上面的规则：

```
SUBDIRS         =kernel drivers mm fs net ipc lib crypto
linuxsubdirs: _dir_kernel _dir_drivers _dir_mm _dir_fs _dir_net _dir_ipc _dir_lib
_dir_crypto
_dir_kernel _dir_drivers _dir_mm _dir_fs _dir_net _dir_ipc _dir_lib _dir_crypto:
dummy include/linux/version.h include/config/MARKER
    $(MAKE) CFLAGS="$(CFLAGS) $(CFLAGS_KERNEL)" -C $(patsubst _dir_%, % , $
@)
```

其中以一个 kernel 为例：

```
_dir_kernel: dummy include/linux/version.h include/config/MARKER
```

$(MAKE) CFLAGS="$(CFLAGS) $(CFLAGS_KERNEL)" -C kernel，也就是对于目录中的每一项，都是进入到该目录，然后执行 make 即可。对于每个目录下面有一个 Makefile 文

件，其中包含了 Rules. make。程序清单 2. 14 是 kernel/Makefile。

程序清单 **2. 14　kernel/Makefile**

```
O_TARGET := kernel.o

export-objs = signal.o sys.o kmod.o context.o ksyms.o pm.o exec_domain.o printk.o
dma.o

obj-y     = sched.o fork.o exec_domain.o panic.o printk.o \\
          module.o exit.o itimer.o info.o time.o softirq.o resource.o \\
          sysctl.o acct.o capability.o ptrace.o timer.o user.o \\
          signal.o sys.o kmod.o context.o

ifndef CONFIG_ARM
obj-y    += dma.o
endif

OX_OBJS  += signal.o sys.o kmod.o context.o

obj-$(CONFIG_UID16) += uid16.o
obj-$(CONFIG_MODULES) += ksyms.o
obj-$(CONFIG_PM) += pm.o

include $(TOPDIR)/Rules.make
```

从上面 Makefile 可以看出，最后生成总的目标文件是“O_TARGET := kernel. o”，定义了目标文件是 export-objs 和 obj-y。

而 rules. make 就可根据 O_TARGET、export-objs 和 obj-y 等产生依赖关系了。

当所有依赖关系处理完成后，就可生成内核文件了。以下就是最后生成 linux 的命令：

更多高清PDF图书　www.gqpdf.com

```
arm-elf-ld -p -X -T arch/armnommu/vmlinux.lds arch/armnommu/kernel/head-armv.o
arch/armnommu/kernel/init_task.o init/main.o init/version.o init/do_mounts.o \\
        --start-group \\
        arch/armnommu/kernel/kernel.o
        arch/armnommu/mm/mm.o
        arch/armnommu/mach-atmel/atmel.o
        kernel/kernel.o mmnommu/mmnommu.o
        fs/fs.o ipc/ipc.o \\
        drivers/char/char.o
        drivers/block/block.o
        drivers/misc/misc.o
        drivers/net/net.o
        drivers/media/media.o \\
        net/network.o \\
        arch/armnommu/lib/lib.a
        --end-group \\
        -o linux
arm-elf-nm linux |grep -v '\\(compiled\\) \\|\\(\\.o $ \\) \\|\\( \[aUw\] \\) \\|\\(\\.\
\.ng $ \\) \\|\\(LASH\[RL\]DI \\)' |sort > System.map
```

2.6 GDB 调试器

2.6.1 GDB 简介

GDB(GNU DeBug)是 GNU 开源组织发布的一个 UNIX 下的程序调试工具,也是一个用来调试 C 和 C++语言程序的功能比较强大的调试器。一般来说,GDB 主要有以下 4 方面的功能:

- 启动程序,可按照自定义的要求调试运行程序。
- 让被调试的程序在指定的调试的断点处停住(断点可以是条件表达式)。
- 当程序被停住时,可以检查此时程序中所发生的事情。
- 动态地改变程序的执行环境。

2.6.2 GDB 调试程序示例

下面以一个简单的例子来介绍 GDB 的一些基本的命令及其使用。调试示例源程序见程序清单 2.15。

程序清单 **2.15**　测试程序 **4**

```
1 #include <stdio.h>
2
3 int func(int n)
4 {
5     int sum=0,i;
6     for(i=0; i<n; i++)
7     {
8     sum+=i;
9     }
10    return sum;
11 }
12
13
14 main()
15 {
16    int i;
17    long result=0;
18    for(i=1; i<=100; i++)
19    {
20    result +=i;
21    }
22
23    printf("result\[1-100\]=%d\\n", result);
24    printf("result\[1-250\]=%d\\n", func(250));
25 }
```

编译生成执行文件(Linux 下):

```
work/test > gcc -g tst.c -o tst
```

使用 GDB 调试:

更多高清PDF图书　www.gqpdf.com

```
work/test > gdb tst  <-------启动 GDB
GNU gdb 5.1.1
Copyright 2002 Free Software Foundation, Inc.
GDB is free software, covered by the GNU General Public License, and you are
welcome to change it and/or distribute copies of it under certain conditions.
Type "show copying" to see the conditions.
There is absolutely no warranty for GDB. Type "show warranty" for details.
This GDB was configured as "i386-suse-linux"...
(gdb) list      <------------开始列出源码
1       #include <stdio.h>
2
3       int func(int n)
4       {
5               int sum=0,i;
6               for(i=0; i<n; i++)
7               {
8                       sum+=i;
9               }
10              return sum;
(gdb)       <------------ 直接回车表示,重复上一次命令
11      }
12
13
14      main()
15      {
16              int i;
17              long result=0;
18              for(i=1; i<=100; i++)
19              {
20                      result +=i;
(gdb) break 16      <------------设置断点,在源程序第 16 行处
Breakpoint 1 at 0x8048496: file tst.c, line 16.
(gdb) break func    <------------ 设置断点,在函数 func()入口处
Breakpoint 2 at 0x8048456: file tst.c, line 5.
(gdb) info break    <------------ 查看断点信息
Num Type           Disp Enb Address    What
1   breakpoint     keep y   0x08048496 in main at tst.c:16
```

高清PDF gqpdf.com

```
2    breakpoint     keep y    0x08048456 in func at tst.c:5
(gdb) r              <-------------- 运行程序,run 命令简写
Starting program: /home/work/test/tst

Breakpoint 1, main () at tst.c:17      <------- 在断点处停住
17              long result =0;
(gdb) n           <-------------- 单条语句执行,next 命令简写
18              for(i =1; i < =100; i ++)
(gdb) n
20                      result += i;
(gdb) n
18              for(i =1; i < =100; i ++)
(gdb) n
20                      result += i;
(gdb) c           <-------------- 继续运行程序,continue 命令简写
Continuing.
result\[1-100\] =5050        <-------程序输出

Breakpoint 2, func (n =250) at tst.c:5
5               int sum =0,i;
(gdb) n
6               for(i =1; i < =n; i ++)
(gdb) p i          <-------------- 打印变量 i 的值,print 命令简写
$ 1 =134513808
(gdb) n
8                       sum += i;
(gdb) n
6               for(i =1; i < =n; i ++)
(gdb) p sum
$ 2 =1
(gdb) n
8                       sum += i;
(gdb) p i
$ 3 =2
(gdb) n
6               for(i =1; i < =n; i ++)
(gdb) p sum
```


更多高清PDF图书 www.gqpdf.com

```
$ 4 =3
(gdb) bt          <-------------- 查看函数堆栈
#0   func (n =250) at tst.c:5
#1   0x080484e4 in main () at tst.c:24
#2   0x400409ed in __libc_start_main () from /lib/libc.so.6
(gdb) finish      <-------------- 退出函数
Run till exit from #0   func (n =250) at tst.c:5
0x080484e4 in main () at tst.c:24
24              printf("result \[1-250 \] = % d \\n", func(250) );
Value returned is $ 6 =31375
(gdb) c       <-------------- 继续运行
Continuing.
result \[1-250 \] =31375      <-------程序输出

Program exited with code 027. <------程序退出,调试结束
(gdb) q       <-------------- 退出 GDB
work/test >
```

2.6.3 gdbserver 介绍

gdbserver 是 GDB 中所带的远程调试工具,它一般运行在目标机上,通过串口或网络与主机上的 GDB 通信,利用主机对目标机上的应用程序进行调试。下面举例介绍 gdbserver 的使用。

假设要调试的可执行文件为 foo,使用 ttyS1 作为调试时主机和目标机的通信设备。首先在主机上运行:

```
#gdb foo
```

出现 GDB 的提示符,然后输入“target remote /dev/ttyS1”如下:

```
(gdb) target remote /dev/ttyS1
Remote debugging using /dev/ttyS1
```

然后把要调试的可执行文件 foo 和 gdbserver 放到目标机的同一目录下,在当前目录下运行如下命令:

```
./gdbserver foo /dev/ttyS1
```

这时,可以看到主机端的 GDB 有建立链接已成功的信息,然后就可在主机上执行一些 GDB 命令,且对目标机上的可执行文件进行调试了。

需要注意的是,在主机上调试用到的可执行文件需要包含调试信息,供 GDB 使用。

2.7 GNU Tools 的制作和安装

本节主要讲解如何安装针对 ARM 平台的交叉编译环境，并建立开发环境。GNU Tools 支持多种体系结构及处理器，因此芯片厂商可根据自己芯片的特性来移植适合自己处理器的交叉编译环境。对于 ARM 芯片有多种交叉编译环境，其中比较著名的就是 arm-elf-tools 交叉编译环境和 arm-linux 交叉编译环境。下面分别介绍如何从源码和二进制文件直接安装 arm-elf-tools 交叉编译环境。在此推荐大家安装使用二进制的交叉编译器。

2.7.1 从源码制作交叉编译环境

第一步，首先在/source 目录下准备如下文件：

binutils-2.10.tar.bz2	二进制文件的工具包。
binutils-2.10-full.patch	工具包的补丁。
gcc-2.95.3.tar.bz2	GCC 编译器。
gcc-2.95.3-full.patch	GCC 的补丁。
gcc-2.95.3-arm-pic.patch	GCC 的补丁。
gcc-2.95.3-arm-pic.patch2	GCC 的补丁。
gcc-2.95.3-arm-mlib.patch	GCC 的补丁。
gcc-2.95.3-sigset.patch	GCC 的补丁。
gcc-2.95.3-m68k-zext.patch	GCC 的补丁。
genromfs-0.5.1.tar.gz	生成 romfs 的工具。
STLport-4.5.3.tar.gz	STL 相关补丁。
STLport-4.5.3.patch	STL 相关补丁。
elf2flt-20030314.tar.gz	用来把 elf 格式的文件转换为 flat 格式文件的工具。
build-uclinux-tools.sh	编译交叉编译环境所用的脚本。

第二步，从源码开始来编译自己的交叉工具链。把/source 目录下所有与编译环境相关的文件拷贝到同一目录下，并进入此目录。以 root 用户的身份执行以下命令：

```
# ./build-uclinux-tools.sh build 2>&1 |tee errs
```

由于篇幅关系，在此无法介绍关于从源码来安装交叉编译环境的过程的更多细节，可通过查看 build-uclinux-tools.sh 脚本文件，或访问 http://www.uclinux.org 来获得更多信息。

2.7.2 安装二进制的交叉工具链 arm-elf 工具

直接在命令行下运行 arm-elf-tools-20040427.sh，如果提示权限错误，则需要给此文件加可执行权限，这里要把 arm-elf-tools-20040427.sh 拷贝到根目录下面，以便脚本运行后，交叉编译

更多高清PDF图书 www.gqpdf.com

器可安装到正确的位置。

第一步，运行下面的命令，拷贝 arm-elf-tools-20040427.sh 到根目录：

```
# cp /source/arm-elf-tools-20040427.sh   /
```

第二步，运行下面的命令，该 arm-elf-tools-20040427.sh 文件具有可执行权限：

```
# chmod 755 arm-elf-tools-20040427.sh
```

第三步，运行该脚本：

```
# /arm-elf-tools-20040427.sh
```

最后查看一下/usr/local/bin 目录是否存在以 arm-elf-开头的编译环境文件。如果存在，则说明交叉编译环境安装成功了。

2.8 使用 GNU Tools 编译 μClinux 内核和应用程序

2.8.1 使用 GNU Tools 编译 μClinux 内核

内核是 Linux 操作系统的核心，很多书籍和网上的文档已经详细讨论了内核在用户系统上的安装、使用和内部的源程序，这里就不重复讨论这些问题了。如果对这些问题有兴趣，可参看 O'Reilly 出版社出版的 *Running Linux*、*Linux Device Drivers* 和 *Understanding the Linux Kernel*，这些书包括了内核的安装、使用和对内核各个源程序的分析。通过互联网，也可下载并查看 *Linux Documentation Project*（简称 LDP）中的 *Linux Kernel HOWTO* 文档。

下面以编译一个可在 SkyEye 模拟平台上运行的 μClinux 内核、文件系统和应用程序为例，说明 μClinux 的配置、编译和安装。需要注意，在安装过程中，有些操作需要超级用户权限才能执行，因此以 root 用户登录系统来编译安装。

第一步，安装 arm-elf-tools 交叉编译环境，如 2.7.2 小节所述。

第二步，配置、编译和安装 μClinux。

以 uClinux-dist-20040408.tar.gz 源码软件包为例，首先将 uClinux-dist-20040408.tar.gz 解开。例如，把这个源码软件包解开到/usr/src/uclinux-dist/目录下，然后在/usr/src/uclinux-dist/目录下执行如下命令：

① 解压缩和解包“tar zxf uClinux-dist-20040408.tar.gz”。

② 进入目录“cd uClinux-dist”。

③ 内核配置。在图形方式下，可用命令“make xconfig”；在命令行方式下，可用命令“make menuconfig”。

“make xconfig”的过程如图 2.1 和图 2.2 所示。

图 2.1 “make xconfig”的过程 1

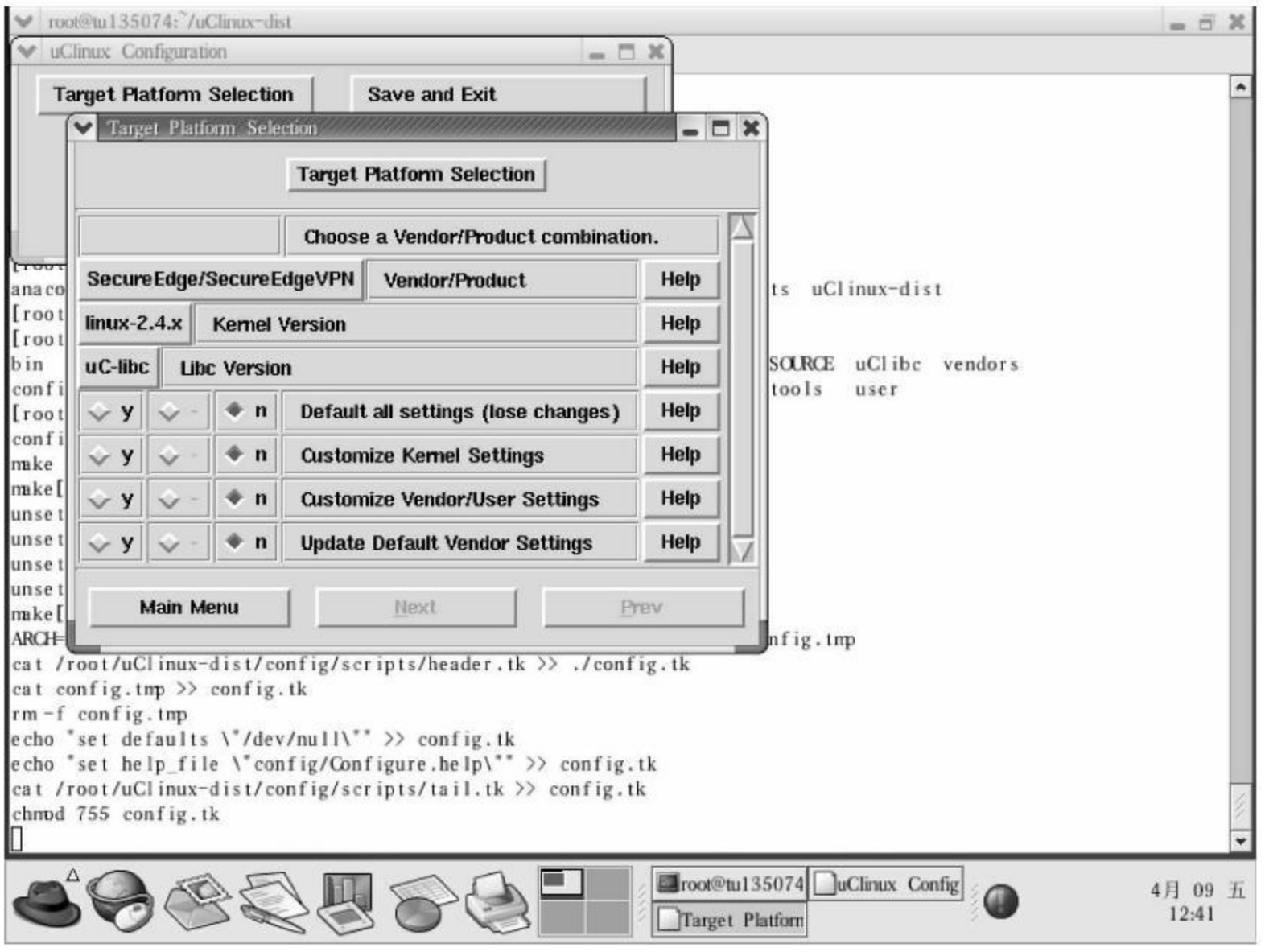

图 2.2 “make xconfig”的过程 2

更多高清PDF图书 www.gqpdf.com

选择 Target Platform Selection 后的界面见图 2.3。

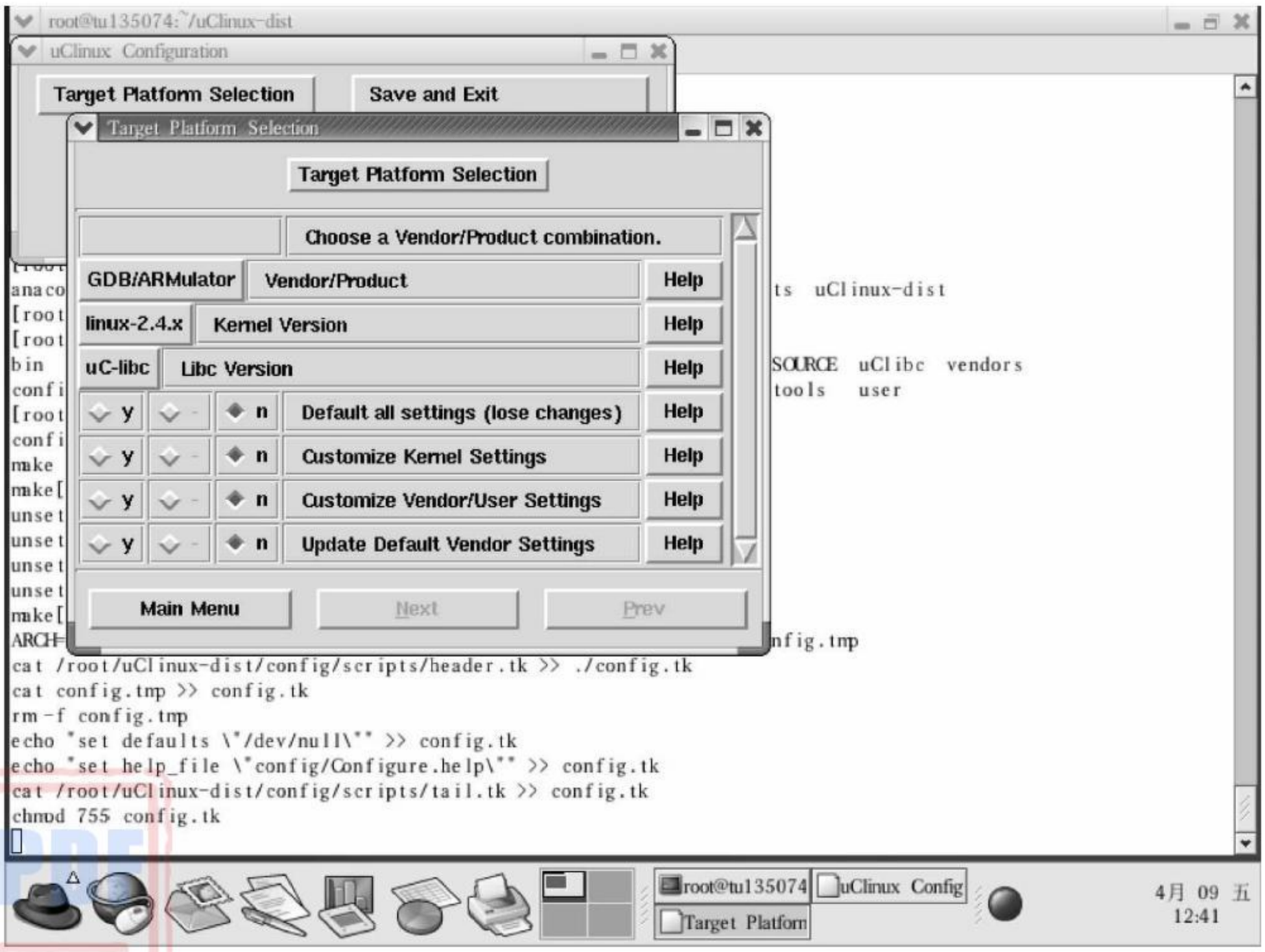

图 2.3　对内核进行配置的界面

在 Vendor/Product 中选择 GDB/ARMulator，Kernel Version 中选择 linux-2.4.x，Libc Version 中选择 μC-libc，然后选择 Save and Exit。

上面所做的内核配置信息都保存在配置文件中：“.config”（执行“ls -a”命令，可在当前目录下看到该文件）。“.config”文件内容见程序清单 2.16。

程序清单 **2.16**　**.config** 文件

```
#
# Automatically generated make config: don't edit
# Target Platform Selection
# Choose a Vendor/Product combination.
#
# CONFIG_DEFAULTS_3COM_PALMIIIX is not set
.....
CONFIG_DEFAULTS_GDB_ARMULATOR=y          //注：系统目标机
```

```
.....
CONFIG_DEFAULTS_KERNEL_2_4 = y                          /*注：核心版本 2.4
# CONFIG_DEFAULTS_UCLIBC is not set
CONFIG_DEFAULTS_OLDUCLIBC = y                           /*注：所用库文件
# CONFIG_DEFAULTS_OVERRIDE is not set
# CONFIG_DEFAULTS_KERNEL is not set
# CONFIG_DEFAULTS_VENDOR is not set
# CONFIG_DEFAULTS_VENDOR_UPDATE is not set
CONFIG_VENDOR = GDB
CONFIG_PRODUCT = ARMulator
CONFIG_LINUXDIR = linux-2.4.x                           /*注：核心源码所在目录
CONFIG_LIBCDIR = libc                                   /*注：库所在目录
CONFIG_LANGUAGE = zh_CN.GB18030:zh_CN.GB2312:zh_CN/*注：语言
.....
```

④ 内核和文件系统的编译，执行以下命令：

```
make dep     /*注：创建内核依赖关系。对于内核目录树上的每一个子目录，在创建内核依赖关系的
               过程中都会产生一个隐藏的“.depend”文件。这个文件包含了所有这个目录的文件
               需要用到的头文件列表
make         /*注：编译整个 μClinux 内核和包含应用程序的文件系统。
```

这时在/usr/src/uclinux-dist/linux-2.4.x 目录下生成一个带调试信息的文件名为 linux 的文件，该文件是 elf 格式的内核执行文件；同时在/usr/src/uclinux-dist/images/下生成文件系统映像文件 romfs.img 等。

⑤ 测试。首先生成配置文件，在 uclinux-dist 目录下建立 skyeye 的配置文件 skyeye.conf，其内容见程序清单 2.17。

程序清单 **2.17　skyeye.conf**

```
cpu: arm7tdmi
mach: XXXX
mem_bank: map = M, type = RW, addr = 0x00000000, size = 0x00004000
mem_bank: map = M, type = RW, addr = 0x01000000, size = 0x00400000
mem_bank: map = M, type = R, addr = 0x01400000, size = 0x00400000, file = images/
romfs.img
mem_bank: map = M, type = RW, addr = 0x02000000, size = 0x00400000
mem_bank: map = M, type = RW, addr = 0x02400000, size = 0x00008000
mem_bank: map = M, type = RW, addr = 0x04000000, size = 0x00400000
mem_bank: map = I, type = RW, addr = 0xf0000000, size = 0x10000000
```


更多高清PDF图书 www.gqpdf.com

这时就可用 skyeye 来调试运行 kernel 了。在/usr/src/uclinux-dist 执行如下命令：

```
skyeye linux-2.4.x/linux
(skyeye)target sim
(skyeye)load
(skyeye)run
```

执行 run 后，会出现如图 2.4 所示的界面。

图 2.4 μClinux 运行在 skyeye 上

如果出现了类似图 2.4 的运行结果，则表示配置、编译 μClinux 内核和文件系统成功。

注意：要在 skyeye.conf 所在目录下执行 skyeye。

2.8.2 使用 GNU Tools 在 μClinux 中编译应用程序

下面以一个“hello word”程序（参见程序清单 2.18）来说明如何编写一个应用程序并且是用开发环境进行编译。

高清PDF gqpdf.com

程序清单 **2.18**　**“hello word”**程序

```
#include <stdio.h>

 int main()
{
  printf("hello world\\n");

  return 0;
}
```

在与 hello.c 同一个目录下编写一个 Makefile 文件(程序清单 2.19)。

程序清单 **2.19**　**Makefile** 文件

高清PDF藏 gqpdf.com
更多高清PDF图书　www.gqpdf.com

```
1
2 cwd = /home/yangjian/tmp/lpc-uclinux-bin
3
4 all : hello.o
5         arm-elf-gcc   -Os   -g    -Dlinux -D__linux__ -Dunix -D__uClinux__ -DEMBED
 \\
6                       -I $(cwd)/uClinux-dist/lib/uClibc/include   \\
7                       -fno-builtin -nostartfiles \\
8                       -D__PIC__ -fpic -msingle-pic-base \\
9                       -Wl,-elf2flt  $ (cwd)/uClinux-dist/lib/uClibc/lib/crt0.o
 \\
10                      $(cwd)/uClinux-dist/lib/uClibc/lib/rti.o    \\
11                      $(cwd)/uClinux-dist/lib/uClibc/lib/crtn.o   \\
12                      -L $(cwd)/uClinux-dist/lib/uClibc/.  \\
13                      -L $(cwd)/uClinux-dist/lib/uClibc/lib \\
14                      -o hello hello.o -lc
15
16 hello.o : hello.c
17        arm-elf-gcc   -Os    -g    -Dlinux -D__linux__ -Dunix -D__uClinux__ -DEMBED
 \\
18                      -I $(cwd)/uClinux-dist/lib/uClibc/include   \\
19                      -fno-builtin -nostartfiles   \\
20                      -D__PIC__ -fpic -msingle-pic-base    \\
21                      -c -o hello.o hello.c
22
23 clean :
24        $(RM) -rf hello hello.o hello.gdb
```

上面就是一个典型的 Makefile 文件，下面简单讲解。这个 Makefile 的 first target 是 all，而 all 又依赖于 hello.o，进而依赖于 hello.c 文件 c。

cwd 这是在 Makefile 中定义的变量，表示 μClinux 的路径，在作者这台机器上就是 cwd = /home/yangjian/tmp/lpc-uclinux-bin。读者可根据自己的 uClinux-dist 的位置做相应的改动，以保证后面的路径正确。

15 ~ 19 行就是编译这个 hello.c 文件，这里的编译选项中：

-g 表示加上调试功能。

-D 表示宏定义。

-I 表示 GCC 的搜索路径，也就是头文件的路径目录。在这里由于使用了 uClibc 库，所以

高清 PDF gqpdf.com

加上它的头文件路径名称。如果是其他库，则只需做相应的改动即可。

-fno-builtin 表示不要试图优化带有__builtin_属性的函数。

-nostartfiles 表示不要使用标准的启动函数。注意，这里的启动函数是在 crt0.o 里面。

-c 表示只是编译。

链接的命令是 5 ~ 13 行，其中：

-fpic 表示使用产生位置无关的代码，应用 μClinux 不支持 MMU，所以没办法使用虚拟内存。这就要保证链接的代码在 RAM 中任何位置都可以运行。

-Wl 打开所有的警告。

-elf2flt 表示把 elf 格式的文件转换成 flat 格式的文件。在没有 MMU 的机器上，大多运行 flat 格式的文件。

最后在 Makefile 的同级目录中运行 make 命令，会发现当前的 hello.c 已被编译为 hello.o。

本章小结

本章主要介绍了在嵌入式 Linux 开发过程中使用到的 GNU 工具。由于嵌入式系统的灵活性和多样性，在开发中可能会碰到各种各样的环境和系统，所以掌握这些工具是非常必要的。

在本章的后面还介绍了如何编译 μClinux 内核，这在下一章的移植工作中将会不断地提到。

高清PDF藏 gqpdf.com
更多高清PDF图书 www.gqpdf.com

第3章　移植 μClinux 到 LPC2200 实例

本章将以一个移植实例具体讲解芯片移植要做的一些工作。总体上说，移植的工作就是要修改 μClinux 底层和硬件相关的部分，让 μClinux 可在指定的硬件平台上运行。这需要移植工程师不但对这个硬件平台比较熟悉，而且必须掌握 μClinux 的代码结构和运行机理。

要移植的芯片型号为 PHILIPS 公司的 LPC2200 的 CPU，它的体系架构属于 ARM7TDMI-S，支持实时钟模拟和调试。关于 LPC2200 芯片的更多信息，可在 PHILIPS 公司的官方网站上找到它的手册。

3.1　选择开发环境

1. 选择编译器

在 μClinux 移植中使用的编译器为 GCC。因为 μClinux 中使用了很多 GCC 特有的语法，所以在编译器上只能选择 GCC。

2. 选择调试器

对于内核开发人员来说，调试是比较艰巨的任务。在内核移植工作中，使用的调试器为模拟器 SkyEye。目前 SkyEye 支持 LPC2200 处理器，其用户界面直接绑定在 GDB 的用户界面，支持 GDB 的基本语法，这给在 LPC2200 的移植工作中调试代码带来了很多方便。

3.2　移植内核

在这里做的是芯片级的移植。当前 μClinux 已支持了无 MMU 的 ARM 体系结构，因此只需添加一些与 LPC2200 处理器相关的代码即可。

尽管 μClinux 中已支持了相应的体系架构，但由于具体处理器设计要求不同，因此各种基于 ARM7 处理器核的芯片的存储系统不同，片内外设不同，具有的中断源也不同，大部分 ARM7 处理器的中断系统也大不相同。可以说，对于 ARM 平台的芯片级移植的很多工作都集中在对底层中断系统的修改上。

下面主要列举移植需要添加的代码。可分为 4 部分：体系架构和机型相关的代码；中断相关的代码；其他代码；修改 Makefile 和配置菜单。每当介绍一部分时，都会简单介绍 μClinux

中相关的一些其他代码，但受本书篇幅所限，不可能描述得很详细。如读者有疑惑，可自行参阅其他 Linux 源代码分析的资料。

3.2.1 增加体系架构和机型相关的代码

在这部分，会添加启动代码和一些体系结构相关的数据结构。主要分为如下 3 部分。

1. 添加机型号

给要移植的 LPC2200 处理器随机选择了一个机型的序号，在 include/asm-armnommu/mach-types.h 中添加 LPC 开发板体系结构号。但 mach-types.h 是一个编译时自动生成的文件，手动修改的内容在下次编译时会被覆盖。每次编译时，arch/armnommu/tools/get-mach-types 会读取 arch/armnommu/tools/mach-types 文件，然后产生/include/asm-armnommu/mach-types.h 文件，因此需要修改的应该是 arch/armnommu/mach-types 文件，方法是在文件最后加上一行：

```
lpc             ARCH_LPC            LPC         91
```

这里随便选择了一个没有使用过的号码。

2. 添加描述机型的数据结构

添加文件 arch/armnommu/mach-lpc/arch.c。在该文件里实现了一个描述机型的数据结构 machine_desc，然后用 MACHINE_START、MAINTAINER、BOOT_MEM、FIXUP、INITIRQ 和 MACHINE_END 这些宏填充到 machine_desc 中，其中 fixup_lpc() 函数用于初始化内存的数据结构 meminfo，会在启动时被调用。arch.c 文件的内容见程序清单 3.1。

程序清单 **3.1** **arch/armnommu/mach-lpc/arch.c**

```
#include <linux/tty.h>
#include <linux/delay.h>
#include <linux/pm.h>
#include <linux/init.h>

#include <asm/elf.h>
#include <asm/setup.h>
#include <asm/mach-types.h>
#include <asm/mach/arch.h>

extern void genarch_init_irq(void);

void fixup_lpc(struct machine_desc *desc, struct param_struct *params, char **
cmdline,
```


更多高清PDF图书 www.gqpdf.com

```
struct meminfo * mi)
{
        mi->bank\[0\].start = 0x81000000;
        mi->bank\[0\].size = 8 * 1024 * 1024;
        mi->bank\[0\].node = 0;
        mi->nr_banks = 1;

#ifdef CONFIG_BLK_DEV_INITRD
        setup_ramdisk(1, 0, 0, CONFIG_BLK_DEV_RAM_SIZE);
        setup_initrd(__phys_to_virt(0x81700000), 1024 * 1024);
        ROOT_DEV = MKDEV(RAMDISK_MAJOR, 0);
#endif
}

MACHINE_START(LPC, "zlg-arm-linux")
        MAINTAINER("tsinghua")
        BOOT_MEM(0x81000000, 0xe0000000, 0xe0000000)
        FIXUP(fixup_lpc)
        INITIRQ(genarch_init_irq)
MACHINE_END
```

3. 添加 LPC2200 的启动代码

在 linux-2.4.x/arch/kernel/head_armv.S 中添加启动代码，读出前面添加的处理器序号和机型序号。这些代码基本上对于所有 ARM 处理器都是类似的，在这个文件中添加一段类似于其他 ARM 处理器的启动代码，完成针对 LPC2200 的 processor_id 和__machine_arch_type 赋值。processor_id（=0xfefefefe）和__machine_arch_type（=91）的值前面已选好，内容见程序清单 3.2。

程序清单 **3.2　arch/kernel/head_armv.S**

```
#if defined(CONFIG_ARCH_LPC)
    adr         r5, LC0
    ldmia       r5, {r5, r6, r8, r9, sp}            @ Setup stack

/* Copy data section to their new home.* /

/* Clear BSS * /
     mov        r4, #0
```

```
1:    cmp       r5, r8
      strcc     r4, \[r5 \], #4
      bcc       1b

 /* Pretend we know what our processor code is (for arm_id) */
      ldr       r2, LPC_PROCESSOR_TYPE
      str       r2, \[r6 \]
      mov       r2, #MACH_TYPE_LPC
      str       r2, \[r9 \]

      mov       fp, #0
      b         start_kernel

LC0: .long__bss_start
      .long processor_id
      .long _end
      .long   __machine_arch_type
      .long init_task_union + 8192
LPC_PROCESSOR_TYPE:     .long 0xfefefefe
#endif
```

3.2.2 添加中断系统

1. LPC2200 的中断系统简介

关于 LPC2200 的中断系统更详细的资料可参看其使用手册，这里仅作简单介绍。LPC2200 的中断系统提供了 32 个中断源，可把它们分为 3 类：快速中断、向量化的中断和非向量化的中断。下面介绍其中主要的寄存器。

- VICRawInt：中断状态寄存器，记录了系统发生的中断。
- VICVectAddr0 ~ 15：记录了中断向量 0 ~ 15 这 16 个向量中断处理函数的地址。
- VICVectAddr：记录了发生中断处理函数的地址。在发生向量中断时，处理器会把发生中断的中断处理函数（即把上面 16 个寄存器中相应寄存器的内容）拷贝到该寄存器。

2. 内核中中断系统的实现代码

ARM 平台共有 7 种异常，中断是其中的一种。当硬件产生中断异常之后，CPU 会跳转到相应的异常处理函数去处理中断异常，也就是 0x18 地址，在这个地址上 Linux 存放了一条跳转语句：

```
b    __real_stubs_start + (vector_IRQ - __stubs_start)
```


更多高清PDF图书 www.gqpdf.com

通过这条语句，内核会跳转到 vector_IRQ 中去执行，在 vector_IRQ 中进行了保存中断的堆栈及返回位置，并且进行了模式的切换；然后跳入__irq_svc 或__irq_usr 程序中去执行，执行完后，返回被中断的执行继续执行。这样就完成了整个中断的处理。

3. 添加中断初始化的代码

添加 include/asm-armnommu/arch-lpc/irq. h，在这里实现了 init_irq（）函数，此函数对 μClinux 系统中的中断数据结构 irq_desc\[\]数组进行了初始化，挂载了一些平台相关的中断操作函数。这个函数会在 μClinux 系统初始化时被调用。文件内容见程序清单 3.3。

程序清单 **3.3　include/asm-armnommu/arch-lpc/irq. h**

```
/*
irq.h:
* /

#ifndef __ASM_ARCH_IRQ_H__
#define __ASM_ARCH_IRQ_H__

#include <asm/hardware.h>
#include <asm/io.h>
#include <asm/mach/irq.h>
#include <asm/arch/irqs.h>

#define fixup_irq(x) (x)

extern void lpc_mask_irq(unsigned int irq);
extern void lpc_unmask_irq(unsigned int irq);
extern void lpc_mask_ack_irq(unsigned int irq);
extern void lpc_init_vic(void);

static __inline__ void irq_init_irq(void)
{
    int irq;
    lpc_init_vic();
    for (irq=0; irq < NR_IRQS; irq++)
    {
        if (! VALID_IRQ(irq)) continue;
```

高清藏 PDF gqpdf.com

```
        irq_desc\[irq\].valid =1;
        irq_desc\[irq\].probe_ok =1;
        irq_desc\[irq\].mask_ack =lpc_mask_ack_irq;
        irq_desc\[irq\].mask =lpc_mask_irq;
        irq_desc\[irq\].unmask =lpc_unmask_irq;
    }
    #endif /* __ASM_ARCH_IRQ_H__ */
```

4. 添加中断的底层函数

添加 arch/armnommu/mach-lpc/irq.c，在这里实现了 3 个中断底层函数：lpc_mask_ack_irq()是用来掩码确认的函数；lpc_mask_irq()是用来屏蔽中断的函数；lpc_unmask_irq()是用来让屏蔽失效的函数。文件内容见程序清单 3.4。

程序清单 **3.4　arch/armnommu/mach-lpc/irq.c**

```
/*
 *  linux/arch/arm/mach-lpc/irq.c
 */
#include <linux/init.h>

#include <asm/mach/irq.h>
#include <asm/hardware.h>
#include <asm/io.h>
#include <asm/irq.h>
#include <asm/system.h>

extern struct irqdesc irq_desc\[\];

void lpc_mask_irq(unsigned int irq)
{
    __arch_putl(1 << irq, VIC_IECR);
}

void lpc_unmask_irq(unsigned int irq)
{
    unsigned long mask =1 << (irq);
    unsigned long ier =__arch_getl(VIC_IER);
```


更多高清PDF图书　www.gqpdf.com

```
    ier = mask |ier;
    __arch_putl(ier, VIC_IER);
}

void lpc_mask_ack_irq(unsigned int irq)
{
    __arch_putl(0x0, VIC_AR);
    lpc_mask_irq(irq);
}

void lpc_init_vic(void)
{
        int irqno;

        /* Disable all interrupts */
        __arch_putl(0xFFFFFFFF, VIC_IECR);

        /* Clear all soft interrupts */
        __arch_putl(0xFFFFFFFF, VIC_SICR);

        /* use IRQ not FIR */
        __arch_putl(0, VIC_ISLR);

        for ( irqno = 0 ; irqno < 16 ; irqno ++ )
        {
            __arch_putl(irqno, VIC_VAR(irqno));     /* index */
            __arch_putl(0x20 |irqno, VIC_VCR(irqno));    /* vect */
        }
        __arch_putl(16, VIC_DVAR);

        /* set protect */
        __arch_putl(1, VIC_PER);

        /* remap IRQ to RAM */
        __arch_putl(0x2, MEMMAP);
}
```

gqpdf.com

5. 添加定义中断号的头文件

添加 include/asm-armnommu/arch-lpc/irqs. h（参见程序清单 3.5），在这里定义了系统中所有用到的外设中断号。

程序清单 **3.5　include/asm-armnommu/arch-lpc/irqs. h**

```
#ifndef __ASM_ARCH_IRQS_H__
#define __ASM_ARCH_IRQS_H__

#define NR_IRQS          32
#define VALID_IRQ(i)     (i >= 0 && i < NR_IRQS)

#define IRQ_WD           0
#define IRQ_TC0          4
#define IRQ_TC1          5
#define IRQ_UART0        6
#define IRQ_UART1        7
#define IRQ_PWM          8
#define IRQ_I2C          9
#define IRQ_SPI0         10
#define IRQ_SPI1         11
#define IRQ_PLL          12
#define IRQ_RTC          13
#define IRQ_EXT0         14
#define IRQ_EXT1         15
#define IRQ_EXT2         16
#define IRQ_EXT3         17
#define IRQ_AD           18

#endif /* __ASM_ARCH_IRQS_H__ */
```

6. 实现 get_irqnr_and_base 宏

对于移植来说，entry-armv. S 文件改动将会很大。要修改__trap_init（）函数，还要添加 disable_fiq、get_irqnr_and_base 和 irq_prio_table 宏。其中 disable_fiq 和 irq_prio_table 没有用处，不需要增加代码，主要是实现 get_irqnr_and_base 宏，内容参考程序清单 3.6，它在 arch/armnommu/kernel/entry_armv. S 第一部分适当位置（比如 ATMEL 的定义后面的 807 号）添加。

更多高清PDF图书　www.gqpdf.com

程序清单 **3.6**　宏 **get_irqnr_and_base**

```
#elif defined(CONFIG_ARCH_LPC)
                .macro disable_fiq
                .endm
                .macro get_irqnr_and_base, irqnr, irqstat, base, tmp
                ldr     r4, =VIC_ISR
                ldr     r4, \[r4 \]
                mov     \\irqstat, r4

                mov     \\irqnr, #0
                tst     r4, #0x00000001
                BNE     .+30 *4 *3 +8

                mov     \\irqnr, #1
                tst     r4, #0x00000002
                BNE     .+29 *4 *3 +8

                mov     \\irqnr, #2
                tst     r4, #0x00000004
                BNE     .+28 *4 *3 +8

                mov     \\irqnr, #3
                tst     r4, #0x00000008
                BNE     .+27 *4 *3 +8

                mov     \\irqnr, #4
                tst     r4, #0x00000010
                BNE     .+26 *4 *3 +8

                mov     \\irqnr, #5
                tst     r4, #0x00000020
                BNE     .+25 *4 *3 +8

                mov     \\irqnr, #6
                tst     r4, #0x00000040
                BNE     .+24 *4 *3 +8
```

高清藏 PDF gqpdf.com

```
mov     \\irqnr, #7
tst     r4, #0x00000080
BNE     .+23*4*3+8

mov     \\irqnr, #8
tst     r4, #0x00000100
BNE     .+22*4*3+8

mov     \\irqnr, #9
tst     r4, #0x00000200
BNE     .+21*4*3+8

mov     \\irqnr, #10
tst     r4, #0x00000400
BNE     .+20*4*3+8

mov     \\irqnr, #11
tst     r4, #0x00000800
BNE     .+19*4*3+8

mov     \\irqnr, #12
tst     r4, #0x00001000
BNE     .+18*4*3+8

mov     \\irqnr, #13
tst     r4, #0x00002000
BNE     .+17*4*3+8

mov     \\irqnr, #14
tst     r4, #0x00004000
BNE     .+16*4*3+8

mov     \\irqnr, #15
tst     r4, #0x00008000
BNE     .+15*4*3+8

mov     \\irqnr, #16
```


更多高清PDF图书 www.gqpdf.com

```
                tst     r4, #0x00010000
                BNE     .+14*4*3+8

                mov     \\irqnr, #17
                tst     r4, #0x00020000
                BNE     .+13*4*3+8

                mov     \\irqnr, #18
                tst     r4, #0x00040000
                BNE     .+12*4*3+8

                mov     \\irqnr, #19
                tst     r4, #0x00080000
                BNE     .+11*4*3+8

                mov     \\irqnr, #20
                tst     r4, #0x00100000
                BNE     .+10*4*3+8

                mov     \\irqnr, #21
                tst     r4, #0x00200000
                BNE     .+9*4*3+8

                mov     \\irqnr, #22
                tst     r4, #0x00400000
                BNE     .+8*4*3+8

                mov     \\irqnr, #23
                tst     r4, #0x00800000
                BNE     .+7*4*3+8

                mov     \\irqnr, #24
                tst     r4, #0x01000000
                BNE     .+6*4*3+8

                mov     \\irqnr, #25
                tst     r4, #0x02000000
```

高清PDF gqpdf.com

```
BNE     .+5*4*3+8

mov     \\irqnr, #26
tst     r4, #0x04000000
BNE     .+4*4*3+8

mov     \\irqnr, #27
tst     r4, #0x08000000
BNE     .+3*4*3+8

mov     \\irqnr, #28
tst     r4, #0x10000000
BNE     .+2*4*3+8

mov     \\irqnr, #29
tst     r4, #0x20000000
BNE     .+1*4*3+8

mov     \\irqnr, #30
tst     r4, #0x40000000
BNE     .+0*4*3+8

mov     \\irqnr, #31

.endm
.macro irq_prio_table
.endm   .macro    irq_prio_table
```

disable_fiq 和 irq_prio_table 宏没有用处，但 get_irqnr_and_base 这个宏很关键，其作用主要是得到发生中断的中断号，它会在__irq_svc()或__usr_svc()函数中被调用。

7. 添加异常处理函数的位置

为了定位异常处理函数的地址，需要添加 RAM_BASE 变量如程序清单 3.7 所示，因此在 include/asm-armnommu/arch-lpc/hardware.h 地址定义了 RAM_BASE 变量，它会在 trap_init()函数中被用到。

程序清单 **3.7**　添加异常处理函数的位置

```
#if defined(CONFIG_ARCH_LPC)
```


更多高清PDF图书　www.gqpdf.com

```
.equ __real_stubs_start, 0x200 + RAM_BASE
```

3.2.3 增加其他代码

1. 添加 dma.h

添加 include/asm-armnommu/arch-lpc/dma.h 文件，内容参考程序清单 3.8。

程序清单 3.8 include/asm-armnommu/arch-lpc/dma.h

```
/*
 * asm/arch-armnommu/arch-lpc/dma.h
 */

#ifndef __ASM_ARCH_DMA_H
#define __ASM_ARCH_DMA_H

#define MAX_DMA_ADDRESS              0xffffffff
#define MAX_DMA_CHANNELS             0
#define arch_dma_init(dma_chan)

#endif /* _ASM_ARCH_DMA_H */
```

2. 添加 memory.h

添加 include/asm-armnommu/arch-lpc/memory.h 文件，内容参考程序清单 3.9。

程序清单 3.9 include/asm-armnommu/arch-lpc/memory.h 文件内容

```
#define TASK_SIZE              (0x01a00000UL)
#define TASK_SIZE_26           TASK_SIZE

#define PHYS_OFFSET            (DRAM_BASE)
#define PAGE_OFFSET            PHYS_OFFSET
#define END_MEM                (DRAM_BASE + DRAM_SIZE)

#define __virt_to_phys(vpage)  ((unsigned long) (vpage))
#define __phys_to_virt(ppage)  ((void *) (ppage))
#define __virt_to_bus(vpage)   ((unsigned long) (vpage))
#define __bus_to_virt(ppage)   ((void *) (ppage))
```

3. 添加 processor.h

添加 include/asm-armnommu/arch-lpc/processor.h 文件，这里面实现了两个函数 arch_

idle() 和 arch_reset()：一个是供处理器在空闲时调用的函数；一个是重启函数（参见程序清单 3.10）。

程序清单 **3.10　include/asm-armnommu/arch-lpc/processor.h**

```
static inline void arch_idle(void)
{
    while (! current->need_resched && ! hlt_counter)
      cpu_do_idle(IDLE_WAIT_FAST);
}

extern inline void arch_reset(char mode)
{
  /* tbd */
}
```

4. 添加 hardware.h

添加 include/asm-armnommu/arch-lpc/hardware.h 文件，这里面定义了大部分外设寄存器的地址，供其他文件使用，其中包括时钟、串口及中断控制器等（参见程序清单 3.11）。由于该文件过长，在此省略了一些代码。

程序清单 **3.11　include/asm-armnommu/arch-lpc/hardware.h**

```
/* 0 = TC0, 1 = TC1 */
#define KERNEL_TIMER 0

#define LPC_TC_BASE 0xe0004000
#define HARD_RESET_NOW()
/*
 * add by lyh, use inner-ram for IRQ */
#define RAM_BASE 0x0
//#define RAM_BASE 0x40000000

/* clocks */
#define Fosc                11059200
#define Fcclk               (Fosc * 4)
#define Fcco                (Fcclk * 4)
#define Fpclk               (Fcclk /4)
```


更多高清PDF图书　www.gqpdf.com

```
/*
 *   added by ksh,2004-06-18,defined interrupt register
 */
#define VIC_BASE        0xfffff000
#define VIC_ISR         (VIC_BASE+0x000)
#define VIC_FSR         (VIC_BASE+0x004)
#define VIC_RISR        (VIC_BASE+0x008)
#define VIC_ISLR        (VIC_BASE+0x00c)     //interrupt select register
#define VIC_IER         (VIC_BASE+0x010)
#define VIC_IECR        (VIC_BASE+0x014)
#define VIC_SIR         (VIC_BASE+0x018)
#define VIC_SICR        (VIC_BASE+0x01c)
#define VIC_PER         (VIC_BASE+0x020)
#define VIC_AR          (VIC_BASE+0x030)
#define VIC_DVAR        (VIC_BASE+0x034)
#define VIC_VAR(i)      (VIC_BASE+0x100+i*4)
#define VIC_VCR(i)      (VIC_BASE+0x200+i*4)

/* EXTERNAL MEMORY CONTROLLER (EMC) */
/* 外部总线控制器 */
#define BCFG0           0xFFE00000          /* lpc22xx only */
#define BCFG1           0xFFE00004          /* lpc22xx only */
#define BCFG2           0xFFE00008          /* lpc22xx only */
#define BCFG3           0xFFE0000C          /* lpc22xx only */

/* External Interrupts */
/* 外部中断控制寄存器 */
#define EXTINT          0xE01FC140
#define EXTWAKE         0xE01FC144
#define EXTMODE         0xE01FC148          /* no in lpc210x */
#define EXTPOLAR        0xE01FC14C          /* no in lpc210x */

/* SMemory mapping control. */
/* 内存 remap 控制寄存器 */
#define MEMMAP          0xE01FC040

/* Phase Locked Loop (PLL) */
```

```
/* PLL 控制寄存器 */
#define PLLCON          0xE01FC080
#define PLLCFG          0xE01FC084
#define PLLSTAT         0xE01FC088
#define PLLFEED         0xE01FC08C

/* Power Control */
/* 功率控制寄存器 */
#define PCON            0xE01FC0C0
#define PCONP           0xE01FC0C4

/* VPB Divider */
/* VLSI 外设总线(VPB)分频寄存器 */
#define VPBDIV          0xE01FC100

/* Memory Accelerator Module (MAM) */
/* 存储器加速模块 */
#define MAMCR           0xE01FC000
#define MAMTIM          0xE01FC004

/* Vectored Interrupt Controller (VIC) */
/* 向量中断控制器(VIC)的特殊寄存器 */
#define VICIRQStatus    0xFFFFF000
#define VICFIQStatus    0xFFFFF004
#define VICRawIntr      0xFFFFF008
#define VICIntSelect    0xFFFFF00C
#define VICIntEnable    0xFFFFF010
#define VICIntEnClr     0xFFFFF014
#define VICSoftInt      0xFFFFF018
#define VICSoftIntClear 0xFFFFF01C
#define VICProtection   0xFFFFF020
#define VICVectAddr     0xFFFFF030
#define VICDefVectAddr  0xFFFFF034
#define VICVectAddr0    0xFFFFF100
#define VICVectAddr1    0xFFFFF104
#define VICVectAddr2    0xFFFFF108
#define VICVectAddr3    0xFFFFF10C
```


更多高清PDF图书 www.gqpdf.com

```
#define VICVectAddr4      0xFFFFF110
#define VICVectAddr5      0xFFFFF114
#define VICVectAddr6      0xFFFFF118
#define VICVectAddr7      0xFFFFF11C
#define VICVectAddr8      0xFFFFF120
#define VICVectAddr9      0xFFFFF124
#define VICVectAddr10     0xFFFFF128
#define VICVectAddr11     0xFFFFF12C
#define VICVectAddr12     0xFFFFF130
#define VICVectAddr13     0xFFFFF134
#define VICVectAddr14     0xFFFFF138
#define VICVectAddr15     0xFFFFF13C
#define VICVectCntl0      0xFFFFF200
#define VICVectCntl1      0xFFFFF204
#define VICVectCntl2      0xFFFFF208
#define VICVectCntl3      0xFFFFF20C
#define VICVectCntl4      0xFFFFF210
#define VICVectCntl5      0xFFFFF214
#define VICVectCntl6      0xFFFFF218
#define VICVectCntl7      0xFFFFF21C
#define VICVectCntl8      0xFFFFF220
#define VICVectCntl9      0xFFFFF224
#define VICVectCntl10     0xFFFFF228
#define VICVectCntl11     0xFFFFF22C
#define VICVectCntl12     0xFFFFF230
#define VICVectCntl13     0xFFFFF234
#define VICVectCntl14     0xFFFFF238
#define VICVectCntl15     0xFFFFF23C

/* Pin Connect Block */
/* 引脚连接模块控制寄存器 */
#define PINSEL0           0xE002C000
#define PINSEL1           0xE002C004
#define PINSEL2           0xE002C014      /* no in lpc210x */

/* General Purpose Input/Output (GPIO) */
/* 通用并行 I/O 口的特殊寄存器 */
```

```
#define IOPIN          0xE0028000     /* lpc210x only */
#define IOSET          0xE0028004     /* lpc210x only */
#define IODIR          0xE0028008     /* lpc210x only */
#define IOCLR          0xE002800C     /* lpc210x only */

#define IO0PIN         0xE0028000     /* no in lpc210x */
#define IO0SET         0xE0028004     /* no in lpc210x */
#define IO0DIR         0xE0028008     /* no in lpc210x */
#define IO0CLR         0xE002800C     /* no in lpc210x */

#define IO1PIN         0xE0028010     /* no in lpc210x */
#define IO1SET         0xE0028014     /* no in lpc210x */
#define IO1DIR         0xE0028018     /* no in lpc210x */
#define IO1CLR         0xE002801C     /* no in lpc210x */

#define IO2PIN         0xE0028020     /* lpc22xx only */
#define IO2SET         0xE0028024     /* lpc22xx only */
#define IO2DIR         0xE0028028     /* lpc22xx only */
#define IO2CLR         0xE002802C     /* lpc22xx only */

#define IO3PIN         0xE0028030     /* lpc22xx only */
#define IO3SET         0xE0028034     /* lpc22xx only */
#define IO3DIR         0xE0028038     /* lpc22xx only */
#define IO3CLR         0xE002803C     /* lpc22xx only */

/* Universal Asynchronous Receiver Transmitter 0 (UART0) */
/* 通用异步串行口 0(UART0)的特殊寄存器 */
#define U0RBR          0xE000C000
#define U0THR          0xE000C000
#define U0IER          0xE000C004
#define U0IIR          0xE000C008
#define U0FCR          0xE000C008
#define U0LCR          0xE000C00C
#define U0LSR          0xE000C014
#define U0SCR          0xE000C01C
#define U0DLL          0xE000C000
#define U0DLM          0xE000C004
```


更多高清PDF图书 www.gqpdf.com

```
/* Universal Asynchronous Receiver Transmitter 1 (UART1) */
/* 通用异步串行口 1(UART1)的特殊寄存器 */
#define U1RBR           0xE0010000
#define U1THR           0xE0010000
#define U1IER           0xE0010004
#define U1IIR           0xE0010008
#define U1FCR           0xE0010008
#define U1LCR           0xE001000C
#define U1MCR           0xE0010010
#define U1LSR           0xE0010014
#define U1MSR           0xE0010018
#define U1SCR           0xE001001C
#define U1DLL           0xE0010000
#define U1DLM           0xE0010004

/* I2C (8/16 bit data bus) */
/* 芯片间总线(I2C)的特殊寄存器 */
#define I2CONSET        0xE001C000
#define I2STAT          0xE001C004
#define I2DAT           0xE001C008
#define I2ADR           0xE001C00C
#define I2SCLH          0xE001C010
#define I2SCLL          0xE001C014
#define I2CONCLR        0xE001C018

/* SPI (Serial Peripheral Interface) */
/* SPI 总线接口的特殊寄存器 */
/* only for lpc210x */
#define SPI_SPCR        0xE0020000
#define SPI_SPSR        0xE0020004
#define SPI_SPDR        0xE0020008
#define SPI_SPCCR       0xE002000C
#define SPI_SPINT       0xE002001C

#define S0PCR           0xE0020000   /* no in lpc210x */
#define S0PSR           0xE0020004   /* no in lpc210x */
#define S0PDR           0xE0020008   /* no in lpc210x */
```

高PDF gqpdf.com

```
#define S0PCCR          0xE002000C    /*no in lpc210x*/
#define S0PINT          0xE002001C    /*no in lpc210x*/

#define S1PCR           0xE0030000    /*no in lpc210x*/
#define S1PSR           0xE0030004    /*no in lpc210x*/
#define S1PDR           0xE0030008    /*no in lpc210x*/
#define S1PCCR          0xE003000C    /*no in lpc210x*/
#define S1PINT          0xE003001C    /*no in lpc210x*/

/*CAN CONTROLLERS AND ACCEPTANCE FILTER*/
/*CAN 控制器和接收滤波器*/
(省略代码…)
/*Timer 0*/
/*定时器0 的特殊寄存器*/
#define T0IR            0xE0004000
#define T0TCR           0xE0004004
#define T0TC            0xE0004008
#define T0PR            0xE000400C
#define T0PC            0xE0004010
#define T0MCR           0xE0004014
#define T0MR0           0xE0004018
#define T0MR1           0xE000401C
#define T0MR2           0xE0004020
#define T0MR3           0xE0004024
#define T0CCR           0xE0004028
#define T0CR0           0xE000402C
#define T0CR1           0xE0004030
#define T0CR2           0xE0004034
#define T0CR3           0xE0004038
#define T0EMR           0xE000403C

/*Timer 1*/
/*定时器1 的特殊寄存器*/
(省略代码…)
/*Pulse Width Modulator (PWM)*/
/*脉宽调制器的特殊寄存器*/
(省略代码…)
```


更多高清PDF图书 www.gqpdf.com

```
/* A/D CONVERTER */
/* A/D 转换器 */
#define ADCR            0xE0034000      /* no in lpc210x */
#define ADDR            0xE0034004      /* no in lpc210x */

/* Real Time Clock */
/* 实时时钟的特殊寄存器 */
#define ILR             0xE0024000
#define CTC             0xE0024004
#define CCR             0xE0024008
#define CIIR            0xE002400C
#define AMR             0xE0024010
#define CTIME0          0xE0024014
#define CTIME1          0xE0024018
#define CTIME2          0xE002401C
#define SEC             0xE0024020
#define MIN             0xE0024024
#define HOUR            0xE0024028
#define DOM             0xE002402C
#define DOW             0xE0024030
#define DOY             0xE0024034
#define MONTH           0xE0024038
#define YEAR            0xE002403C
#define ALSEC           0xE0024060
#define ALMIN           0xE0024064
#define ALHOUR          0xE0024068
#define ALDOM           0xE002406C
#define ALDOW           0xE0024070
#define ALDOY           0xE0024074
#define ALMON           0xE0024078
#define ALYEAR          0xE002407C
#define PREINT          0xE0024080
#define PREFRAC         0xE0024084

/* Watchdog */
/* 看门狗的特殊寄存器 */
#define WDMOD           0xE0000000
```

高清PDF藏 gqpdf.com

```
#define WDTC                0xE0000004
#define WDFEED              0xE0000008
#define WDTV                0xE000000C

/* Define firmware Functions */
/* 定义固件函数 */
#define rm_init_entry()                 ((void (*)())(0x7fffff91))()
#define rm_undef_handler()              ((void (*)())(0x7fffffa0))()
#define rm_prefetchabort_handler()      ((void (*)())(0x7fffffb0))()
#define rm_dataabort_handler()          ((void (*)())(0x7fffffc0))()
#define rm_irqhandler()                 ((void (*)())(0x7fffffd0))()
#define rm_irqhandler2()                ((void (*)())(0x7fffffe0))()
#define iap_entry(a, b)                 ((void (*)())(0x7ffffff1))(a, b)

#ifndef __ASSEMBLER__
struct lpc_timers
{
    unsigned  long ir;                  //
    unsigned  long tcr;                 //
    unsigned  long tc;                  //
    unsigned  long pr;                  //
    unsigned  long pc;                  //
    unsigned  long mcr;                 //
    unsigned  long mr0;                 //
    unsigned  long mr1;                 //
    unsigned  long mr2;                 //
    unsigned  long mr3;                 //
    unsigned  long ccr;                 //
    unsigned  long cr0;                 //
    unsigned  long cr1;                 //
    unsigned  long cr2;                 //
    unsigned  long cr3;                 //
    unsigned  long emr;                 //
};
#endif
```


更多高清PDF图书 www.gqpdf.com

```
/* TC control register */
#define TC_SYNC             (1)

/* TC mode register */
#define TC2XC2S(x)          (x & 0x3)
#define TC1XC1S(x)          (x << 2 & 0xc)
#define TC0XC0S(x)          (x << 4 & 0x30)
#define TCNXCNS(timer,v)    ((v) << (timer << 1))

/* TC channel control */
#define TC_CLKEN            (1)
#define TC_CLKDIS           (1 << 1)
#define TC_SWTRG            (1 << 2)

/* TC interrupts enable/disable/mask and status registers */
#define TC_MTIOB            (1 << 18)
#define TC_MTIOA            (1 << 17)
#define TC_CLKSTA           (1 << 16)

#define TC_ETRGS            (1 << 7)
#define TC_LDRBS            (1 << 6)
#define TC_LDRAS            (1 << 5)
#define TC_CPCS             (1 << 4)
#define TC_CPBS             (1 << 3)
#define TC_CPAS             (1 << 2)
#define TC_LOVRS            (1 << 1)
#define TC_COVFS            (1)

/*
 *     USART registers
 */

#define LPC_UART_CNT 2
#define LPC_UART0_BASE      0xe000c000
#define LPC_UART1_BASE      0xe001C000

/* US control register */
```

高清PDF gqpdf.com

```
#define US_SENDA            (1 < <12)
#define US_STTO             (1 < <11)
#define US_STPBRK           (1 < <10)
#define US_STTBRK           (1 < <9)
#define US_RSTSTA           (1 < <8)
#define US_TXDIS            (1 < <7)
#define US_TXEN             (1 < <6)
#define US_RXDIS            (1 < <5)
#define US_RXEN             (1 < <4)
#define US_RSTTX            (1 < <3)
#define US_RSTRX            (1 < <2)

/* US mode register */
#define US_CLK0             (1 < <18)
#define US_MODE9            (1 < <17)
#define US_CHMODE(x)        (x < <14 & 0xc000)
#define US_NBSTOP(x)        (x < <12 & 0x3000)
#define US_PAR(x)           (x < <9 & 0xe00)
#define US_SYNC             (1 < <8)
#define US_CHRL(x)          (x < <6 & 0xc0)
#define US_USCLKS(x)        (x < <4 & 0x30)

/* US interrupts enable/disable/mask and status register */
#define US_DMSI             (1 < <10)
#define US_TXEMPTY          (1 < <9)
#define US_TIMEOUT          (1 < <8)
#define US_PARE             (1 < <7)
#define US_FRAME            (1 < <6)
#define US_OVRE             (1 < <5)
#define US_ENDTX            (1 < <4)
#define US_ENDRX            (1 < <3)
#define US_RXBRK            (1 < <2)
#define US_TXRDY            (1 < <1)
#define US_RXRDY            (1)

#define US_ALL_INTS (US_DMSI |US_TXEMPTY |US_TIMEOUT |US_PARE |US_FRAME |US_OVRE |US
_ENDTX |US_ENDRX |US_RXBRK |US_TXRDY |US_RXRDY)
```


更多高清PDF图书 www.gqpdf.com

3.2.4 修改 Makefile 和配置菜单

关于 μClinux 的配置，涉及如下一些文件 config. in、Makefile、Rules. make、. config、autoconf. h、. depend 和 . hdepend。下面逐一解释它们的用途。

1. 修改 config. in 文件

首先从 config. in 开始。编译内核之前，首先要运行“make menuconfig”（或类似命令）。config. in 就是用来定义这个配置菜单的。最重要的 config. in 在 arch/armnommu/目录下，不妨打开看一看。config. in 的语法很简单，详细说明放在 Documentation/kbuild/config-language. txt 中。

修改 arch/armnommu/config. in，添加程序清单 3.12 中带下划线的行。相关宏的含义参见 Documentation/Configure. help。

程序清单 **3.12　config. in** 内容

```
mainmenu_option next_comment
comment 'System Type'
choice 'ARM system type' \\
    "TI-DSC21       CONFIG_ARCH_DSC21 \\
     TI-C5471       CONFIG_ARCH_C5471 \\
     Conexant       CONFIG_ARCH_CNXT \\
     NET+ARM        CONFIG_ARCH_NETARM \\
     Triscend-A7S   CONFIG_ARCH_TA7S \\
     Triscend-A7V   CONFIG_ARCH_TA7V \\
     SWARM          CONFIG_ARCH_SWARM \\
     Samsung        CONFIG_ARCH_SAMSUNG \\
     LPC            CONFIG_ARCH_LPC \\
     Atmel          CONFIG_ARCH_ATMEL" TI-DSC21
     .....
if \[ "$CONFIG_ARCH_SWARM" = "y" \]; then
#  define_bool CONFIG_CPU_ARM610 y
   define_bool CONFIG_CPU_32 y
   define_bool CONFIG_CPU_26 n
   define_bool CONFIG_CPU_32v3 y
   define_bool CONFIG_CPU_ARM7V3 y
   define_bool CONFIG_NO_PGT_CACHE y
   define_bool CONFIG_CPU_WITH_MCR_INSTRUCTION y
   define_hex FLASH_MEM_BASE 0x00000000
```

高清PDF藏 gqpdf.com

```
    define_hex FLASH_SIZE 0x00100000
fi
if \[ "$ CONFIG_ARCH_LPC" = "y" \]; then
    define_bool CONFIG_NO_PGT_CACHE y
    define_bool CONFIG_CPU_ARM710 y
    define_bool CONFIG_CPU_32 y
    define_bool CONFIG_CPU_32v4 y
    define_bool CONFIG_CPU_WITH_CACHE n
    define_bool CONFIG_CPU_WITH_MCR_INSTRUCTION n
fi
if \[ "$ CONFIG_ARCH_ATMEL" = "y" \]; then
    define_bool CONFIG_NO_PGT_CACHE y
    define_bool CONFIG_CPU_ARM710 y
    define_bool CONFIG_CPU_32 y
    define_bool CONFIG_CPU_32v4 y
    define_bool CONFIG_CPU_WITH_CACHE n
    define_bool CONFIG_CPU_WITH_MCR_INSTRUCTION n
    define_bool CONFIG_SERIAL_ATMEL y
    .....
if \[ "$ CONFIG_ARCH_EBSA110" = "y" -o \\
    "$ CONFIG_ARCH_SA1100" = "y" -o \\
    "$ CONFIG_ARCH_CLPS7500" = "y" -o \\
    "$ CONFIG_ARCH_PERSONAL_SERVER" = "y" -o \\
    "$ CONFIG_ARCH_CATS" = "y" -o \\
    "$ CONFIG_ARCH_ATMEL" = "y" -o \\
    "$ COFNIG_ARCH_LPC"   = "y" -o \\
    "$ CONFIG_ARCH_NETARM" = "y" -o \\
    "$ CONFIG_ARCH_SAMSUNG" = "y" -o \\
    "$ CONFIG_ARCH_TA7S" = "y" -o \\
    "$ CONFIG_ARCH_TA7V" = "y" \]; then
    bool 'Compiled-in Kernel Boot Parameter' CONFIG_CMDLINE_BOOL
    #if \[ "$ CONFIG_CMDLINE_BOOL" = "y" \]; then
    #    string 'Default kernel command string' CONFIG_CMDLINE ""
    #fi
fi
```

2. 修改 Makefile 文件

添加 mach-lpc 下的 Makefile 文件，文件内容见程序清单 3.13。

更多高清PDF图书 www.gqpdf.com

程序清单 **3.13　mach-lpc\\Makefile** 文件内容

```
USE_STANDARD_AS_RULE := true
O_TARGET     := lpc.o
# Object file lists.

obj-y        := $(patsubst %.c, %.o, $(wildcard *.c))
obj-m        :=
obj-n        :=
obj-         :=

export-objs  :=
include $(TOPDIR)/Rules.make
```

修改 linux-2.4.x/Makefile 文件，内容见程序清单 3.14。

程序清单 **3.14　linux-2.4.x/Makefile** 文件内容

```
ARCH := armnommu
CROSS_COMPILE = arm-elf-
```

修改 arch/armnommu/Makefile 文件（参见程序清单 3.15），添加程序清单 3.15 中带下划线的行。

程序清单 **3.15　arch/armnommu/Makefile** 文件内容

```
LINKFLAGS    := -p -X -T arch/armnommu/vmlinux.lds
GZFLAGS      := -9
CFLAGS       += -g  -fno-common -pipe -fno-builtin -D__linux__

.....

ifeq ($(CONFIG_ARCH_ATMEL),y)
TEXTADDR     =0x1000000
MACHINE      =atmel
endif

ifeq ($(CONFIG_ARCH_LPC),y)
#TEXTADDR    =0x80008000
TEXTADDR     =0x80020000
```

高清PDF gqpdf.com

```
MACHINE         =lpc
endif
```

3. autoconf.h 和 config.h

很多源文件都包含一个特殊的头文件 <linux/config.h>，该文件的内容很简单，只有一行：

```
#include <linux/autoconf.h>
```

linux/autoconf.h 文件是在编译时产生的，其中包含了各种配置选项（宏），有 500 多行。通过这种方式，将配置选项加入到各个源文件中。除了上述方式产生的宏定义外，还有一个宏“__KERNEL__”和“__LINUX__”是通过命令行的方式定义的。

3.3 移植驱动程序

对于最基本的系统，串口和时钟是必不可少的。串口起到了输入/输出的作用，而时钟则是整个操作系统的心跳。这里主要讲解时钟驱动程序和串口驱动程序的移植两部分内容。

3.3.1 时钟驱动程序的移植

时钟驱动程序最主要的功能是定时产生一个中断，两个时钟中断之间的时间间隔是确定的，这样就能为一些基于时钟驱动或者周期运行的任务提供服务。这个时间间隔可通过硬件来设定。

1. 添加 include/asm-armnommu/arch-lpc/time.h 文件

include/asm-armnommu/arch-lpc/time.h 文件内容见程序清单 3.16。

程序清单 **3.16** **include/asm-armnommu/arch-lpc/time.h** 文件内容

```
/*
linux/include/asm-armnommu/arch-lpc/time.h
*
*/

#if (KERNEL_TIMER == 0)
#   define KERNEL_TIMER_IRQ_NUM IRQ_TC0
#elif (KERNEL_TIMER == 1)
#   define KERNEL_TIMER_IRQ_NUM IRQ_TC1

#else
```


更多高清PDF图书 www.gqpdf.com

```
#error Wierd --KERNEL_TIMER is not defined or something....
#endif

static unsigned long lpc_gettimeoffset (void)
{
    /* volatile struct lpc_timers * tt = (struct lpc_timers *) (LPC_TC_BASE); */
    return 0;
}

static void lpc_timer_interrupt(int irq, void *dev_id, struct pt_regs *regs)
{
    /*printk("lpc_timer_interrupt\\n"); */
    __arch_putl(0x1, T0IR);
    do_timer(regs);
    do_profile(regs);
}

extern void lpc_unmask_irq(int);

extern inline void setup_timer (void)
{
    //init timer
    __arch_putl(0x2, T0TCR);
    __arch_putl(0xffffffff, T0IR);
    __arch_putl(0x0, T0PR);
    __arch_putl(0x3, T0MCR);
    __arch_putl(Fpclk/HZ, T0MR0);
    __arch_putl(0x1, T0TCR);

     lpc_unmask_irq(KERNEL_TIMER_IRQ_NUM);

     gettimeoffset = lpc_gettimeoffset;
     timer_irq.handler = lpc_timer_interrupt;
     setup_arm_irq(KERNEL_TIMER_IRQ_NUM, &timer_irq);
}
```

在 Linux 内核启动时，会调用函数 setup_timer()，把 KERNEL_TIMER_IRQ_NUM 这个中断号和 timer_irq 时钟相关的函数挂载起来，这样当发生时钟中断时，就会调用 lpc_timer_interrupt

高清PDF gqpdf.com

时钟的中断处理函数来进行处理。

2. 添加 include/asm-armnommu/arch-lpc/timex.h 文件

include/asm-armnommu/arch-lpc/timex.h 文件内容见程序清单 3.17。

程序清单 **3.17　include/asm-armnommu/arch-lpc/timex.h** 文件内容

```
/*
 * timex.h:
 */
#ifndef __ASM_ARCH_TIMEX_H__
#define __ASM_ARCH_TIMEX_H__

#define CLOCK_TICK_RATE       1000/8

#endif /* __ASM_ARCH_TIMEX_H__ */
```

3.3.2　串口驱动程序的移植

LPC2200 处理器内置了两个串口 UART0 和 UART1。其主要特性有：带有 16 字节的接收和发送 FIFO；寄存器结构符合 550 工业标准；接收 FIFO 可在 1、4、8、14 字节时分别触发；内置波特率发生器。

目前，在 μClinux 内核中已提供了支持标准串口的驱动程序，只需添加一个宏来定义 LPC2200 串口的 I/O 基地址，使用中断号等与硬件相关的一些信息，串口的驱动程序就完成了。当然，这得益于 Linux 内核良好的代码可重用性和可移植性。

在 include/asm-armnommu/arch-lpc/目录下，添加 serial.h 文件，文件内容参考程序清单 3.18。

程序清单 **3.18　serial.h** 文件内容

```
#define RS_TABLE_SIZE    2
#define BASE_BAUD        115200
#define STD_COM_FLAGS    (ASYNC_BOOT_AUTOCONF | ASYNC_SKIP_TEST)

#define STD_SERIAL_PORT_DEFNS                   \\
    {                                           \\
      type: PORT_16550A,                        \\
      xmit_fifo_size: 16,                       \\
      baud_base: BASE_BAUD,                     \\
      irq: IRQ_UART0,                           \\
      flags: STD_COM_FLAGS,                     \\
```


更多高清PDF图书　www.gqpdf.com

```
    iomem_base: (u8 *)LPC_UART0_BASE,        \\
    io_type: SERIAL_IO_MEM,                  \\
    iomem_reg_shift: 2                       \\
  }, /* ttyS0 */                            \\
  {                                          \\
    type: PORT_16550A,                       \\
    xmit_fifo_size: 16,                      \\
    baud_base: BASE_BAUD,                    \\
    irq: IRQ_UART1,                          \\
    flags: STD_COM_FLAGS,                    \\
    iomem_base: (u8 *)LPC_UART1_BASE,        \\
    io_type: SERIAL_IO_MEM,                  \\
    iomem_reg_shift: 2                       \\
  } /* ttyS1 */

unsigned int baudrate_div(unsigned int baudrate)
{
    return ((Fpclk /16) /baudrate);
}
```

其中 LPC_UART0_BASE 为 LPC2200 串口 0 的基地址，IRQ_UART0 为串口 0 的中断号。这样在定义这个宏后，移植的 μClinux 内核就可向串口输出信息了。

本章小结

本章讲述了如何移植 μClinux 到 LPC2200 CPU，主要涉及如何选择编译器、调试器以及在移植 μClinux 内核中需要添加或修改的代码、配置文件和其他相关文件。在内核移植过程中，往往会碰到各种各样的错误，很难调试。这时一定要耐心地去看代码，弄懂原理，而不仅仅是瞎猜，凭运气去解决问题。如果碰到困难，要多问身边了解相关工作的人；也可通过在网上搜索答案，或者通过 Linux 社区的一些相关邮件列表进行咨询。

高清PDF藏 gqpdf.com

第4章 μClinux 设备驱动程序概述

4.1 设备驱动程序基础知识

设备驱动程序是介于硬件和 Linux 内核之间的软件接口，是一种低级的、专用于某一硬件的软件组件。Linux（μClinux）有两种方式使用设备驱动程序：直接编译到内核中；在运行时加载（也就是内核模块）。

直接将硬件驱动程序写入内核的优点在于，用户可随时对它进行调用而无须安装，但是这样大大增加了内核占用的空间。将硬件驱动程序编写成一种可加载的内核模块，虽然会因为寻找驱动模块而增加系统资源的占用和运行时间，但与庞大的内核所消耗的资源相比显得微不足道。将硬件驱动程序编写成一种可加载的内核模块，还可为软件开发提供许多便利。当用户需要对某一硬件驱动程序进行开发或纠错时，用户可动态地卸载旧的版本并加载新的版本；但如果用户的驱动程序已写入内核，那么必须对内核进行重新编写，并且每次对修改后的程序进行测试时，都必须重新启动系统。另外，将硬件驱动程序视为可加载的内核模块进行开发和配置，用户就可将硬件驱动程序作为一种独立的系统进行升级，而不必对内核进行改动。

本书不介绍第一种情况，因为这种情况灵活性差，并且其源代码必须遵循 GPL 协议。若采用第二种方法，则可遵守 GPL 协议，也可不遵守。

4.1.1 驱动程序分类

在 Linux（μClinux）中一般把设备分为 3 类：字符设备、块设备和网络接口。在 3 类设备中，Linux（μClinux）看待设备的方式有所区别，每种方式是为了实现不同的任务。相应地，设备驱动程序也分为 4 类（尽管可编写“混合”驱动程序，但一般不会这样做）。下面分别介绍这 4 类设备。

1. 字符设备

字符设备是指存取时没有缓存的设备。可像文件一样访问字符设备，字符设备驱动程序负责实现这些行为。这样的驱动程序通常会实现 open、close、read 和 write 系统调用。系统控制台和并口就是字符设备的例子，它们可很好地用概念“流”来描述。通过文件系统节点可访问字符设备，例如/dev/tty1 和/dev/lp1。字符设备和普通文件系统间的惟一区别是，普通文件

更多高清PDF图书 www.gqpdf.com

允许在其上来回读/写，而大多数字符设备仅仅是数据通道，只能顺序读/写。当然，也存在这样的字符设备，看起来像一个数据区，可来回读取其中的数据。

2. 块设备

块设备是文件系统的宿主，如磁盘。在大多数 Unix 系统中，只能将块设备看作多个块进行访问。一个块设备通常是 1 KB 数据。Linux（μClinux）允许像字符设备那样读取块设备——允许一次传输任意数目的字节。结果是，块设备与字符设备只在内核内部的管理上有所区别，因此也就是在内核/驱动程序间的软件接口上有所区别。正如字符设备一样，每个块设备也通过文件系统节点来读/写数据，它们之间的不同对用户来说是透明的。块设备驱动程序和内核的接口与字符设备驱动程序的接口是一样的，也通过一个传统的面向块的接口与内核通信，但这个接口对用户来说是不可见的。

3. 网络接口设备

任何网络事务处理都是通过接口（可与其他宿主交换数据的设备）实现的。通常，接口是一个硬件设备，但也可像 loopback（回路）接口一样是软件工具。网络接口是由内核网络子系统驱动的，它负责发送和接收数据包，而且无须了解每次事务是如何映射到实际被发送的数据包的。尽管 telnet 和 ftp 连接都是面向流的，它们使用同样的设备进行传输，但设备并没有看到任何流，仅看到数据报。

由于不是面向流的设备，所以网络接口不能像/dev/tty1 那样简单地映射到文件系统的节点上。Linux（μClinux）调用这些接口的方式是给它们分配一个独立的名字（如 eth0）。这样的名字在文件系统中并没有对应项。内核和网络设备驱动程序之间的通信与字符设备驱动程序和块设备驱动程序与内核间的通信是完全不同的。内核不再调用 read 和 write，它调用与数据包传送相关的函数。

4. 其他设备

Linux（μClinux）还有一些设备（如 Linux 中还有一类设备驱动程序模块：SCSI 设备驱动程序）不属于以上 3 类。对于大多数读者来说，不太会编写这类设备驱动程序，因此，本书不做详细介绍。

4.1.2 加载和删除驱动程序

当内核模块（驱动程序）编译成功后，可使用命令“insmod 模块文件名称”加载驱动程序。当内核模块不再需要时，可使用“rmmod 模块名称”卸载驱动程序。**注意**：这两条命令参数是不同的。“模块文件名称”是可修改的，而“模块名称”是编译时由源代码确定的。用户还可使用命令“lsmod”查看当前加载了哪些驱动程序。

注意：这些命令必须在 root 权限下使用。

下面列出与内核模块有关的命令：

lsmode	列出已被内核调入的模块。
insmode	将某个 module 插入到内核中。
rmmod	将某个 module 从内核中卸载。
depmod	生成依赖文件，告诉将来的 insmod 要从何处调入 modules。这个依赖文件就是/lib/modules/\[您的 kernel 版本\]/modules. dep。
Kerneld	负责自动将模块调入内核和把模块从内核中卸载。

4.2 最简单的内核模块

4.2.1 内核模块的概念

众所周知，Linux 把存储器分为“内核空间”和“用户空间”。操作系统内核和驱动程序在内核空间中运行，可以访问内核空间，也可通过系统调用访问用户空间。而用户程序在用户空间中运行，只能访问自己的用户空间（针对多个用户共享的存储器空间，逻辑上也是访问自己的用户空间），对内核空间的访问具有严格的限制。μClinux 由 Linux 派生而来，虽然内核不能真正地防止用户程序对内核空间的访问，但这样的划分还是有的。

不知何时，Linux 出现了 modules 这种东西。modules 的字面意思就是模块，在此指的是 kernel modules（内核模块）。简单地说，一个模块提供了一个功能，这些模块是可按需要随时装入内核空间和从内核空间卸载的。因此，内核模块是为了给内核动态增减功能而设计的，并不仅限于驱动程序。但在一个操作系统中，驱动程序是变化最大的部分，所以往往把驱动程序编译成内核模块。在本书后续章节中，除非特别说明，对“内核模块”和“驱动程序”这两个概念不加以区分。

4.2.2 编写内核模块

因为内核模块需要加载到内核空间，所以其程序的编写与一般应用程序不同。程序清单 4.1 是一个简单的内核模块源代码，尽管其代码不一定是最简单的，但它也相当简单，且兼容性很好。

程序清单 **4.1** **test. c**

```
#ifndef __KERNEL__
    #define __KERNEL__                          (1)
#endif
#ifndef MODULE
    #define MODULE                              (2)
```

（1）程序在内核空间运行

（2）这是一个内核模块

更多高清PDF图书 www.gqpdf.com

```
#endif

#include <linux/module.h>                                          (3)
#include <linux/sched.h>                                           (4)
#include <linux/kernel.h>       /*printk() */                      (5)
#include <linux/init.h>                                            (6)

int  test_init(void);                                              (7)
void test_cleanup(void);                                           (8)

module_init(test_init);                                            (9)

module_exit(test_cleanup);                                         (10)

int  test_init(void)                                               (11)
{
    printk(KERN_ERR "test: init OK\\n");                           (12)
    return 0;                                                      (13)
}

void test_cleanup(void)                                            (14)
{
    printk(KERN_ERR "test: remove OK\\n");                         (15)
}
```

关键语句：注册加载时执行的函数

关键语句：注册卸载时执行的函数

程序明显地分为 4 部分：

第 1 部分包括程序清单 4.1(1)和(2)。这是关键的部分，用来声明程序为内核模块。

注意：这部分必须在包含系统头文件前定义，这样程序才能使用内核的内部函数。

第 2 部分包括程序清单 4.1(3)～(6)。这部分包含必需的系统头文件。程序清单 4.1 中仅包含了内核模块必须包含的部分。在编写特定的内核模块时，可能需要添加更多的系统头文件。

第 3 部分包括程序清单 4.1(7)～(10)。这是关键的部分，用于声明内核模块的初始化函数和清除函数，这部分在 4.2.3 小节和 4.2.4 小节中详细介绍。

第 4 部分包括程序清单 4.1(11)～(15)。这部分是内核模块的初始化函数和卸载函数的具体实现。

4.2.3 关于内核模块初始化(加载)函数

当用户输入命令“insmod 模块文件名称”(或用其他方式)加载内核模块时,系统会检测此模块是否能被加载,如果能被加载,内核件调用模块的初始化函数。在 Linux(μClinux)2.4 中,内核模块的初始化函数名为 init_module()。但如果驱动程序需要编译进内核,则初始化函数不能与内核的其他部分(包括其他内核模块)的函数同名。这样,如果程序还可能编译到内核中时就比较麻烦。不过,在 <linux/init.h> 这个头文件中定义了宏 module_init(),用来屏蔽两者的差别。事实上,使用这个宏可使驱动程序向上兼容 Linux(μClinux)2.6,而兼容 Linux(μClinux)2.0 也会比较方便。程序清单 4.1(9)使用 module_init()注册驱动程序的初始化函数,宏 module_init()的参数就是初始化函数的名称。

内核模块的初始化函数原型参考程序清单 4.1(11),它用来初始化驱动程序本身,其返回值为负数时表示出错,负数的绝对值为错误编码,其在 <linux/errno.h> 中定义。

4.2.4 关于内核模块清除(卸载)函数

当内核模块使用完毕,用户可输入命令“rmmod 模块名称”(或用其他方式)卸载内核模块。此时,系统会检测此模块是否能被卸载,如果能被卸载,内核件调用模块的清除函数。在 Linux(μClinux)2.4 中,清除函数的函数名称为 cleanup_module()。但如果驱动程序需要编译进内核,则清除函数不能与内核的其他部分(包括其他内核模块)的函数同名。这样,如果程序还可能编译到内核中时就比较麻烦。不过,在 <linux/init.h> 这个头文件中定义了宏 module_exit(),用来屏蔽两者的差别。事实上,使用这个宏可使驱动程序向上兼容 Linux(μClinux)2.6,而兼容 Linux(μClinux)2.0 也会比较方便。程序清单 4.1(10)使用 module_exit()注册驱动程序的清除函数,宏 module_exit()的参数为驱动程序的清除函数。

内核模块的清除函数原型参考程序清单 4.1(14),它是内核模块退出前最后执行的代码,没有返回值。

4.2.5 关于 printk()函数

代码中还出现 printk()(程序清单 4.1(12)和(15))这个函数,它是给内核(包括内核模块)输出信息用的。函数 printk()是由 Linux(μClinux)内核定义的,功能与函数 printf()相似,其使用方法也极其相似,在一般的使用中,它们仅第一个参数有所区别。函数 printk()第一个参数的开始几个字符用于指定所谓的“日志级别(loglevel)”,Linux(μClinux)可配置为仅让高于指定级别的内容在文本控制台上显示。在 <linux/kernel.h> 中通过宏定义了几个级别,它们被展开为相应的字符串,详细情况见表 4.1,它们按照级别从高到低排序,即 KERN_EMERG 级别最高,KERN_DEBUG 级别最低。

更多高清PDF图书 www.gqpdf.com

表 4.1 printk () 的日志级别

日志级别	说 明
KERN_EMERG	用于紧急事件，它们一般是系统崩溃前的提示信息
KERN_ALERT	用于需要立即采取动作的场合
KERN_CRIT	临界状态，通常设计验证的硬件或软件操作失败
KERN_ERR	用于报告错误状态。设备驱动程序通常会使用它报告来自硬件的问题
KERN_WARNING	就可能出现的问题提出警告。这些问题通常不会对系统造成严重破坏
KERN_NOTICE	有必要提示的正常情况。许多与安全相关的情况用这个级别汇报
KERN_INFO	提示性信息。很多驱动程序在启动时用这个级别打印相关信息
KERN_DEBUG	用于调试的信息

GCC 还有一个扩展功能，即使用空格可把两个字符串常量连接成一个字符串。例如：

```
"abd" "def"
```

就被认为是：

```
"abddef"
```

同样，也可把更多的字符串连接起来。因此，调用 printk () 会写成程序清单 4.1 (12)、(15) 那样，读者自己把字符串连接起来，即可知道函数 printk () 的实际参数。

4.3 编写 Makefile 文件

4.3.1 Makefile 文件整体介绍

GCC 没有通用且优秀的集成开发环境，一般编译程序都是通过 GNU make 实用程序来管理的，而 GNU make 实用程序通过规则文件 (Makefile 文件) 来管理工程。GNU make 实用程序十分灵活，其 Makefile 文件也可编写得很复杂，关于这方面就可写一章。由于本章的目的不是介绍 make 实用程序，所以这里仅在分析 Makefile 文件时介绍相关部分，需要具体了解 make 程序的使用请参考 2.5 节及相关文档。其实大多数编译器都提供 make 工具，其使用方法也类似，不过很多商业编译器具有自己完善的集成开发环境，多数情况下不需要使用 make 工具。

本内核模块的 Makefile 文件内容见程序清单 4.2。有了这个文件，把它与源代码存于同一目录，在这个目录下执行命令“make”，即可编译出相应的目标代码。

程序清单 4.2　Makefile 文件内容

```
EXEC = test                                                                    (1)
OBJS = test.o                                                                  (2)
SRC = test.c                                                                   (3)

INCLUDE = /work/uClinux-dist/linux-2.4.x/include                               (4)
CC = arm-elf-gcc                                                               (5)
LD = arm-elf-ld                                                                (6)
MODCFLAGS = -D__KERNEL__ -I $(INCLUDE) -O2 -fno-strict-aliasing -fno-common -pipe
-fno-
builtin -D__linux__ -g -DNO_MM -mapcs-32 -march = armv4 -mtune = arm7tdmi -mshort-
load-
bytes -msoft-float -nostdinc -iwithprefix include                              (7)
LDFLAGS = -m armelf -r                                                         (8)

all: $(EXEC)                                                                   (9)

$(EXEC): $(OBJS)                                                               (10)
    $(LD) $(LDFLAGS) -o $@  $(OBJS)                                            (11)

%.o:%.c                                                                        (12)
    $(CC) $(MODCFLAGS) -mapcs -c $< -o $@                                      (13)

clean:                                                                         (14)
    -rm -f $(EXEC) *.elf *.gdb *.o                                             (15)
```

Makefile 文件中的内容大致可分成宏定义区和基于依赖关系的指令区两大部分。宏定义区主要定义了一些变量，它们可在 Makefile 文件的任何地方被引用。同时，GCC make 实用工具行预先定义了一些变量。**注意**：变量名必须为大写字母。Makefile 文件中的指令区主要定义了一些规则，这些规则用于解释如何及何时创建或处理特定的文件，一般这些特定的文件是规则的目标。GCC make 需要向调用命令对依赖关系进行检查，然后才能创建或更新目标。一个规则也可用于解释如何及何时执行一个动作。下面分别介绍这个驱动程序的这两部分。

4.3.2　Makefile 文件中的变量

Makefile 文件中的变量部分包含程序清单 4.2(1) ~ (8)，它们又可分为以下 5 部分。

更多高清PDF图书　www.gqpdf.com

1. 目标文件及源代码管理

这包含程序清单 4.2(1) ~ (3)。如果需要使用这个 Makefile 文件编译其他设备驱动程序,一般只需修改该部分。

程序清单 4.2(1)用于设置变量 EXEC,这是目标文件名,用户编译的驱动程序生成的被加载文件就是这个名称(程序清单 4.2 为 test)。程序清单 4.2(2)用于设置变量 OBJS,它用于保存编译中间文件的列表。把所有源文件(*.c)的后缀名改为.o 即可。

程序清单 4.2(2)所示为一个文件 test .o,它们由 test .c 编译得到。**注意:** 这个版本的 Makefile 要求编译中间文件的主文件名必须与源文件的主文件名一致,否则不能通过编译。将来可能修改这个 Makefile 文件,允许准建文件的主文件名与目标文件名不一致。

程序清单 4.2(3)用于设置变量 SRC,它用于保存编译源代码文件的列表。在该版本的 Makefile 文件中,这个变量没有使用,是预留给以后使用的。

2. 设置驱动所需要的头文件目录

这包含程序清单 4.2(4),用于设置变量 INCLUDE,实质是设置 μClinux 头文件所在的目录。本例中这个目录为/work/uClinux-dist/linux-2.4.x/include,表示 μClinux 源代码所在的目录为/work/uClinux-dist/。如果 μClinux 源代码所在的目录不是这个目录,则需要修改这一行。

3. 设置编译器

这包含程序清单 4.2(5)、(6)。对于 GCC 支持的每一种 CPU 体系结构,GCC 都有工具包对应。对于 ARM7TDMI 来说,不同的组织为了自己不同的需要,制作了不同的工具包来支持。不过,编译 μClinux 内核、μClinux 的驱动程序和 μClinux 的应用程序一般使用 arm-elf-gcc 这个工具包。在这个工具包中,一般通过实用程序 arm-elf-gcc(在 x86 系统中为 gcc)调用编译器,而链接器为 arm-elf-ld(在 x86 系统中为 ld)。程序清单 4.2(5)和程序清单 4.2(6)就是分别设置这两个实用程序的名称的。如果使用不同体系结构的 CPU,则需要修改这两个变量来指示出这两个实用程序的名称。

4. 设置编译参数

这包含程序清单 4.2(7)。这个编译参数很长,不过大多与体系结构相关。如果不清楚,可参见参考文献[1]。如果是使用 arm_elf_gcc 编译,体系结构为 ARM7TDMI,则无须改变。如果两者有一个不是,则需要改变,至于如何改变请参见参考文献[1]。不过“-DMODULE:”、“-D__KERNEL__”和“-I $(INCLUDE)”不能改变,这是编译模块必需的。

5. 设置连接参数

这包含程序清单 4.2(8)。其中-m 用于指出输出文件的格式,这里为基于 ARM 的 elf 格式(在 x86 系统中为 elf_i386),而-r 告诉链接器生成的目标文件是可重定位的。

4.3.3 Makefile 文件中的规则

对于使用者来说,这些规则无须改变,介绍这些规则主要是方便读者理解编译的过程。

GNU make 的一个主要参数就是指定规则名称。当指定了规则时，GNU make 从指定的规则开始执行。Makefile 文件中规则的格式如下：

```
目标/文件:依赖文件列表;命令
```

或更常用的：

```
目标/文件:依赖文件列表
(TAB)命令
```

第二种格式命令可以多条，每条占一行。**注意**：第二种格式中命令这一行第一个字符为 TAB 而不是空格，后面紧接命令。这是初学者容易出现的问题。一个文件/目标依赖于一个或多个文件（依赖文件），而每个依赖文件和它自己的依赖文件又显式或隐含地构成新的规则。这样，目标对于一个目标文件就构成一个规则树，GNU make 通过这个规则树编译需要编译的文件，即直接或间接被修改过的文件。一条规则的执行过程如下：

① 依次执行“依赖文件列表”每个文件对应的规则。

② 如果“依赖文件列表”中任意一个文件的更改日期比“目标/文件”更改时间更晚，则顺序执行本规则的各个命令，这一般导致本规则的“目标/文件”被更新。

如果文件不存在，则认为这个文件是最老的。

1. all 规则

这包含程序清单 4.2(9)。这是 Makefile 文件第一个规则。GCC make 工具把第一个规则作为默认规则。当键入 make 命令时，GCC make 工具从这条规则开始执行。因为一般不存在“all”这个文件，所以这个规则一般会执行。但因为这个规则下没有命令，所以只会依次执行这个规则的依赖文件列表中每个文件对应的规则，本例中为 $(EXEC)（即 zlg_ffs）这条规则。

2. .o 文件到目标文件的规则（$(EXEC)规则）

这包含程序清单 4.2(10)、(11)。程序清单 4.2(10)这条规则展开后的内容如下：

```
zlg_ffs: zlg_ffs_uClinx.o ZLG_FFS.o
```

如果执行了所有 $(OBJS)（本例中为 zlg_ffs_uClinx.o 规则和 ZLG_FFS.o 规则）规则后，$(OBJS)（本例中为 zlg_ffs_uClinx.o 规则和 ZLG_FFS.o 规则）中任意一个有变化，则执行链接操作（程序清单 4.2(11)）。

3. .c 文件到 .o 文件的规则

这包含程序清单 4.2(12)、(13)。这条规则与普通规则不一样，使用了“%”（表示匹配任何字符串）和“$”（表明这是预定义的内部变量）这样的符号。事实上，其作用也不太一样：它用来改变 GNU make 默认的规则。GNU make 具有默认 *.o 规则：它依赖于具有相同主文件名但扩展名为.c 文件。它也有默认的命令：通过默认的 GCC 工具使用默认的参数编译对

更多高清PDF图书 www.gqpdf.com

应的 C 语言源文件。“%.o:%.c”表明不改变这种依赖关系，而下面的命令就改变了。其中“$ <”表示当前规则中的依赖文件列表中的第一个依赖文件，“$@”表示当前规则的目的文件名。

4. clean 规则

这包含程序清单 4.2(14)、(15)。这个规则为清除编译的目标文件和中间文件。输入“make clean”，即可执行这个动作。

这个 Makefile 文件其实还不够完善，它没有给出各 C 语言源文件的规则，也就是没有给定 C 语言源文件之间的依赖关系及 C 语言源文件与头文件间的依赖关系。这些规则只需给出依赖关系，而不必给出命令。虽然没有给出这些规则，但对于简单的驱动程序，该 Makefile 文件已够用。

本章小结

本章首先介绍了 Linux(μClinux)设备驱动程序相关知识，包括简单的使用；然后主要介绍了一个最简单的设备驱动程序，包括源码分析和 Makefile 文件的编写和分析。这些是 Linux(μClinux)设备驱动程序最基础的公共知识，是必须掌握的知识。本章介绍的知识是后续章节的基础。

高清 PDF 藏 gqpdf.com

高清PDF
gqpdf.com

第5章 字符设备驱动程序初步

5.1 几个概念

字符设备是指存取时没有缓存的设备。典型的字符设备包括鼠标、键盘、串行口等。在一个实际的电脑系统中，往往是字符设备占多数。本章将介绍一个可编译和运行的字符设备驱动程序，但它没有任何实际功能。它的所有功能函数都几乎为空，事实上应称之为字符设备驱动程序框架。另外，本章还介绍了字符设备的使用方法，这个使用方法是针对程序员而不是针对普通使用者的。普通使用者应通过程序员开发的软件来使用相关的设备。

5.1.1 关于主设备号和从设备号

Linux（μClinux）使用设备号来标识字符设备。设备号分为主设备号和从设备号。主设备号标识设备对应的驱动程序。每个在内核中活动的字符设备驱动程序，包括编译进内核的和后来动态加载的，都有惟一的主设备号。内核利用主设备号将设备与相应的驱动程序对应起来。次设备号只由设备驱动程序使用，内核的其他部分不使用它，仅将它传递给驱动程序。驱动程序可按照自己的方法使用从设备号。通常情况下，字符驱动程序使用从设备号区分自己管理的不同物理设备，所以，由主设备号和从设备号可确定当前内核支持的某个具体的物理设备。

向系统增加一个驱动程序意味着要赋予它一个主设备号。一旦设备注册到内核表中，则无论何时操作与设备驱动程序的主设备号匹配的设备文件，内核都会调用驱动程序中的正确函数。

注意：块设备也有设备号，它与字符设备的设备号相互独立。

5.1.2 关于设备节点、节点文件和设备文件

设备节点、节点文件、设备文件及近似的其他名词实际上是指同一样东西。Linux（μClinux）几乎把所有东西都当作文件，字符设备也一样。但设备与文件毕竟不一样，于是Linux（μClinux）中就出现了一种特殊的文件，它们存储在文件系统中，但仅占用文件目录项而不涉及存储数据。事实上，它们仅仅记录了其所属的设备类别、主设备号和从设备号等设备相

更多高清PDF图书 www.gqpdf.com

关信息。这些文件称作设备文件、节点文件或设备节点。

当一个程序打开一个设备文件时，内核就可获取对应设备的设备类型、主设备号和从设备号等信息，内核也就知道了程序需要操作使用哪个设备驱动程序。程序随后对这个文件的操作都会调用相应的驱动程序的函数，同时把从设备号传递给驱动程序。

因此，对设备文件的操作就是对其所代表的物理设备的操作。

5.2 最简单的字符设备驱动程序

5.2.1 驱动程序代码

这个驱动程序的代码见程序清单5.1。

程序清单 **5.1** **test.c**

```
#ifndef __KERNEL__
    #define __KERNEL__                                                    (1)
#endif
#ifndef MODULE
    #define MODULE                                                        (2)
#endif

#include <linux/module.h>                                                 (3)
#include <linux/sched.h>                                                  (4)
#include <linux/kernel.h>       /*printk()*/                              (5)
#include <linux/init.h>                                                   (6)

static loff_t test_llseek(struct file *filp, loff_t off, int whence);    (7)
static ssize_t test_read(struct file *filp, char *buf, size_t count, loff_t *f_
pos);                                                                     (8)
static ssize_t test_write(struct file *filp, const char *buf, size_t count,
                                    loff_t *f_pos);                       (9)
static int test_open(struct inode *inode, struct file *filp);            (10)
static int test_release(struct inode *inode, struct file *filp);         (11)
static int test_ioctl(struct inode *inode, struct file *filp, unsigned int cmd,
                                    unsigned long param);                 (12)
int  test_init(void);                                                    (13)
```

```
void test_cleanup(void);                                                    (14)

#define MAJOR_NR 125                                                        (15)

#define DEVICE_NAME "test"                          /* name for messaging */ (16)

module_init(test_init);                                                     (17)
module_exit(test_cleanup);                                                  (18)

static struct file_operations test_fops =           /* driver info    */ (19)
{
    owner:      THIS_MODULE,                                                (20)
    llseek:     test_llseek,                                                (21)
    read:       test_read,                                                  (22)
    write:      test_write,                                                 (23)
    ioctl:      test_ioctl,                                                 (24)
    open:       test_open,                                                  (25)
    release:    test_release,                                               (26)
};

static loff_t test_llseek(struct file *filp, loff_t off, int whence)       (27)
{
    return 0;                                                               (28)
}

static ssize_t test_read(struct file *filp, char *buf, size_t count,
                         loff_t *f_pos)                                     (29)
{
    return 0;                                                               (30)
}

static ssize_t test_write(struct file *filp, const char *buf, size_t count,
                          loff_t *f_pos)                                    (31)
{
    return 0;                                                               (32)
}
```

这就是主设备号

字符设备必需的结构体

更多高清PDF图书 www.gqpdf.com

```
static int  test_open(struct inode *inode, struct file *filp)          (33)
{
    MOD_INC_USE_COUNT;                                                 (34)
    return 0;       /* success */                                      (35)
}

static int  test_release(struct inode *inode, struct file *filp)       (36)
{
    MOD_DEC_USE_COUNT;                                                 (37)
    return(0);                                                         (38)
}

static int test_ioctl(struct inode *inode, struct file *filp,
                      unsigned int cmd, unsigned long arg)             (39)
{
    return 0;                                                          (40)
}

int  test_init(void)                                                   (41)
{
    int  result;

    result = register_chrdev(MAJOR_NR,  DEVICE_NAME,  &test_fops);     (42)
    if (result < 0)                                                    (43)
    {
        printk(KERN_ERR DEVICE_NAME ": Unable to get major % d\\n", MAJOR_NR );
                                                                       (44)
        return result;                                                 (45)
    }

    printk(KERN_ERR DEVICE_NAME ": init OK\\n");                       (46)
    return(0);                                                         (47)
}

void test_cleanup(void)                                                (48)
{
    unregister_chrdev(MAJOR_NR, DEVICE_NAME);                          (49)
}
```

配对使用的关键代码

特别注意：一切变化从这里开始

两者要配对

高清PDF藏 gqpdf.com

与 4.2 节介绍的内核模块一样，这个驱动程序不一定是最简单的，但它也相当简单，且兼容性很好。这个代码是从程序清单 4.1 的基础上发展而来的，是在程序清单 4.1 上增加了字符设备所必需的代码得到的。

程序清单 5.1 看起来比较长，但实际上去掉宏定义、包含文件及函数声明外并不算多。事实上，核心部分仅有两个函数（初始化函数和清除函数）和一个数据结构变量（test_fops）。因为这个代码由程序清单 4.1 发展而来，所以这里不详细介绍其与程序清单 4.1 相同的部分。下面的小节将着重介绍增加的部分。

5.2.2 驱动程序的初始化函数

一切变化从初始化函数开始。

Linux（μClinux）在加载内核模块时会调用驱动程序的初始化函数。字符设备驱动程序初始化函数的示意代码见程序清单 5.2。驱动程序的初始化函数用来初始化驱动程序本身，其返回值为负数时表示出错，负数的绝对值为错误编码，错误编码在 < linux/errno. h > 中定义。程序清单 5.1（41）~（47）就是驱动程序初始化函数的例子。其实，这个函数最重要的是调用系统提供的函数 register_chrdev（）（程序清单 5.1（42））来注册字符设备驱动程序，其他代码都是错误处理和显示处理。相对于程序清单 4.1，调用这个函数其实也是所有变化的根源。函数 register_chrdev（）的原型见程序清单 5.3。它有 3 个参数：第一个为驱动程序的主设备号；第二个参数为驱动程序名称（字符串）；第三个参数为结构体 file_operations 指针，这个结构体保存驱动程序操作字符设备的方法。这个结构体后面还会介绍，这里不做过多说明。

程序清单 **5.2** 字符设备初始化函数的示意代码

```
int  init_module (void)
{
    初始化模块自身(包括请求一些资源);
    return  register_chrdev(MAJOR_NR,  DEVICE_NAME,  &test_fops);
}
```

程序清单 **5.3** **register_chrdev**（）的原型

```
int register_chrdev(unsigned int, const char *, struct file_operations *);
```

5.2.3 驱动程序的清除函数

Linux（μClinux）在卸载内核模块时会调用驱动程序的清除函数。驱动程序清除函数的示意代码见程序清单 5.4。驱动程序的清除函数主要用来释放自身获得的资源。清除函数是一个没有参数和返回值的函数。程序清单 5.1（48）、（49）就是清除函数的例子，它仅调用系统

更多高清PDF图书 www.gqpdf.com

提供的函数 unregister_chrdev () 通知内核对应的字符设备已被卸载(程序清单 5.1(49))。函数 unregister_chrdev () 是与函数 register_chrdev () 配对的函数。如果驱动程序在初始化或工作时向内核申请了其他资源(如内存),在卸载时如果还没有被释放,也需要在这个函数中释放。**特别注意**:释放的顺序应当按申请的相反顺序执行。函数 unregister_chrdev () 的原型见程序清单 5.5。它有两个参数:第一个参数为驱动程序主设备号;第二个参数为驱动程序的名称(字符串)。

程序清单 **5.4** 字符设备清除函数的示意代码

```
void cleanup_module (void)
{
    关闭模块自身(包括释放资源);
    unregister_chrdev(MAJOR_NR,  DEVICE_NAME);
}
```

程序清单 **5.5** **unregister_chrdev** () 的原型

```
int unregister_chrdev(unsigned int, const char *);
```

5.2.4 保存设备信息的结构体

在块设备驱动程序初始化时需要调用函数 register_chrdev (),其中第三个参数为指向结构体 file_operations 的指针,它存储了字符设备的操作方法。结构体 file_operations 的定义见程序清单 5.6。程序清单 5.6 仅列出部分常用的成员。对于普通的字符设备驱动程序,这些成员已足够,其他成员仅在特殊情况下使用,感兴趣的读者可自己阅读源码。

程序清单 **5.6** 字符设备驱动程序操作方法

```
struct file_operations {
    struct module *owner;
    loff_t (*llseek) (struct file *, loff_t, int);
    ssize_t (*read) (struct file *, char *, size_t, loff_t *);
    ssize_t (*write) (struct file *, const char *, size_t, loff_t *);
    int (*ioctl) (struct inode *, struct file *, unsigned int, unsigned long);
    int (*open) (struct inode *, struct file *);
    int (*release) (struct inode *, struct file *);
    .....
};
```

程序清单 5.1(19) ~ (26) 定义了该结构体变量并进行了初始化。程序清单 5.1(19) ~

(26)较易看懂：仅对结构体指定的成员初始化，未指定的成员用默认值初始化。这是 GCC 对标准 C 的扩展，好处是结构体成员增加时无须更改初始化代码。

下面分别介绍每个成员的意义。

1. open()

在 Linux(μClinux)和/或应用程序使用设备前会调用驱动程序的这个方法。open()方法最重要的是调用宏 MOD_INC_USE_COUNT(程序清单 5.1(34))，这个宏主要用来增加驱动程序使用计数器，以避免不正确的卸载驱动程序。open()方法一般还用于(在第一次使用驱动程序支持的设备时)对驱动程序支持的设备进行初始化。一般情况下 open()方法的示意代码见程序清单 5.7。这个代码是针对一个驱动管理多个设备的情况(常见)；如果一个驱动仅管理一个设备，就不需要这么复杂了，仅调用宏 MOD_INC_USE_COUNT 即可，而设备初始化可在驱动程序初始化时进行。

程序清单 **5.7** **open()** 方法示意代码

```
static int  open(struct inode *inode, struct file *filp)
{
    从设备号 = MINOR(inode->i_rdev);                                    (1)
    通过从设备号判断需要操作的设备;                                     (2)
    if (需要操作的设备的使用计数器 == 0)                                (3)
    {
        初始化设备(包括申请资源);                                       (4)
        给 filp->private_data 赋值，以标识物理设备                      (5)
    }
    需要操作的设备的使用计数器 ++;                                      (6)

    MOD_INC_USE_COUNT;          必需的语句                              (7)
    return 0;           /* success */                                  (8)
}
```

open()方法的第一个参数 inode 是打开文件所对应的 i 节点。对于驱动程序来说，这个参数的主要作用是获取从设备号，其方法见程序清单 5.7(1)。对于这个参数的其他成员，驱动程序几乎都不会用上。

open()方法的第二个参数 filp 代表一个打开的文件，但程序清单 5.7 仅使用了它的 private_data 成员，以便将来读/写操作时识别物理设备。事实上，open()方法可根据需要访问其中的成员。关于结构体 file 的具体介绍参见 5.2.5 小节。

2. release()

当 Linux(μClinux)和/或应用程序不再使用设备时会调用驱动程序的这个方法。release()

更多高清PDF图书 www.gqpdf.com

方法最重要的是调用宏 MOD_DEC_USE_COUNT(程序清单 5.1(37)),这个宏与宏 MOD_INC_USE_COUNT 对应,主要用来减少驱动程序使用计数器。release()方法一般还用于(在驱动程序支持的设备使用完毕时)对驱动程序支持的设备进行关闭操作。一般情况下 release()方法的示意代码见程序清单5.8。这个代码是针对一个驱动管理多个设备的情况(常见);如果一个驱动仅管理一个设备,就不需要这么复杂了,仅调用宏 MOD_DEC_USE_COUNT 即可,而关闭设备可在卸载驱动程序时进行。

release()方法的第一个参数 inode 是打开文件所对应的 i 节点。对于驱动程序来说,这个参数的主要作用是获取从设备号,其方法见程序清单 5.8(1)。对于这个参数的其他成员,驱动程序几乎都不会用上。

release()方法的第二个参数 filp 代表一个打开的文件,在程序清单 5.8 中没有使用。事实上,release()方法可根据需要访问其中的成员。关于结构体 file 的具体介绍参见 5.2.5 小节。

程序清单 **5.8** **release()方法示意代码**

```
static int  release(struct inode *inode, struct file *filp)
{
    从设备号=MINOR(inode->i_rdev);                                    (1)
    通过从设备号判断需要操作的设备;                                    (2)
    MOD_DEC_USE_COUNT;                                                (3)
                                          必需的语句
    需要操作的设备的使用计数器--;                                      (4)
    if(需要操作的设备的使用计数器==0)                                  (5)
    {
        关闭设备处理(包括释放 revalidate()方法获取的资源;             (6)
    }

    return 0;                                                         (7)
}
```

3. read()

Linux(μClinux)会调用驱动程序的 read()方法从字符设备中读取数据。read()方法的示意代码见程序清单 5.9。read()方法的第一个参数 filp 代表一个打开的文件,其具体信息见 5.2.5 小节;第二个参数 buf 为一个指针,从设备读取的数据将要保存到它指向的内存空间;第三个参数 count 指明将要读取的数据个数,count 实际上是一个 32 位无符号整型变量,在 ARM7TDMI 中 size_t 与 unsigned int 等价;第四个参数 f_pos 为一个指针,它指向 file->f_pos。loff_t 实质是 64 位有符号类型,在 ARM7TDMI 中与 long long 等价。

程序清单 **5.9**　**read**()方法示意代码

```
static ssize_t read(struct file *filp, char *buf, size_t count, loff_t *f_pos)
{
    if (! access_ok(VERIFY_WRITE, (void *)buf, count))                    (1)
    {
        return -EFAULT;                                                   (2)
    }
    通过 filp->private_data 确定实际操作的物理设备;                        (3)
    从设备的 *f_pos 位置读取 count 个字节数;                               (4)
    使用 copy_to_user()或 put_user()拷贝数据到 buf 指向的空间;             (5)
    更新 *f_ops;                                                          (6)
    return 读到的数据总数;                                                (7)
}
```

read()方法的返回值为 ssize_t 类型，实质是 32 位整型，在 ARM7TDMI 中 ssize_t 与 int 等价。对于 read()方法的返回值，分成以下几种情况：

- 如果数据全部传输完成，则返回值为 count。
- 如果只有部分数据成功传送，则返回值为已传输的数据个数（比 count 小）。这种情况下，应用程序一般会重新读数据。例如，如果用 fread()函数读数据，则这个库函数会不断地调用系统调用，直至所请求的数据传输完成。
- 如果已到达了文件尾，则返回值为 0。
- 如果发生错误，造成没有任何数据被传输，则返回负值，值就是错误编码，错误编码的绝对值在 <linux/errno.h> 中定义。如果已读了部分数据，返回值为第二种情况。

在程序清单 5.9 中出现了几个函数 access_ok()、copy_to_user()和 put_user()，它们是为数据传输服务的，详细情况参考 5.2.6 小节。

4. write()

Linux（μClinux）会调用驱动程序的 write()方法写多个字节数据到字符设备中。write()方法的示意代码见程序清单 5.10。write()方法的第一个参数 filp 代表一个打开的文件，其具体信息见 5.2.5 小节；第二个参数 buf 为一个指针，要写到设备中的数据就存储在它指向的内存空间；第三个参数 count 指明将要写入的数据个数，count 实际上是一个 32 位无符号整型变量，在 ARM7TDMI 中 size_t 与 unsigned int 等价；第四个参数 f_pos 为一个指针，它指向 file->f_pos。loff_t 实质是 64 位有符号类型，在 ARM7TDMI 中与 long long 等价。

程序清单 **5.10**　**write**()方法示意代码

```
static ssize_t write(struct file *filp, const char *buf, size_t count, loff_t *
f_pos)
```


更多高清PDF图书　www.gqpdf.com

```
{
    if (! access_ok(VERIFY_READ, (void *)buf, count))                    (1)
    {
        return -EFAULT;                                                  (2)
}
    通过 filp->private_data 确定实际操作的物理设备;                      (3)
    使用 copy_from_user()或 get_user()从 buf 指向的空间获取数据;          (4)
    从设备的 *f_pos 位置写入获取的数据;                                  (5)
    更新 *f_ops;                                                         (6)
    return 写入的数据总数;                                               (7)
}
```

write()方法的返回值为 ssize_t 类型，实质是 32 位整型，在 ARM7TDMI 中 ssize_t 与 int 等价。对于 write()方法的返回值，分成以下几种情况：

- 如果数据全部传输完成，则返回值为 count。
- 如果只有部分数据成功传送，则返回值为已传输的数据个数（比 count 小）。这种情况下，应用程序一般会重新读数据。例如，如果用 fwrite()函数读数据，则这个库函数会不断地调用系统调用，直至所请求的数据传输完成。
- 如果已到达了文件尾，则返回值为 0。
- 如果发生错误，造成没有任何数据被传输，则返回负值，值就是错误编码，错误编码的绝对值在 <linux/errno.h> 中定义。

在程序清单 5.10 中出现了几个函数 access_ok()、copy_from_user()和 get_user()，它们是为数据传输服务的，详细情况参考 5.2.6 小节。

5. llseek()

Linux（μClinux）使用驱动程序的 llseek()方法来修改文件（也就是设备）的读/写位置，实质上是改变 filp->f_pos 的值。llseek()方法的示意代码见程序清单 5.11。llseek()方法有三个参数：第一个参数就是多次见到的 file 指针，详细情况见 5.2.5 小节；第二个参数 off 为需要移动的偏移量，其实质是 64 位有符号数，在 ARM7TDMI 中与 long long 等价；第三个参数 whence 为移动的方式，0 为从文件头移动，1 为从当前位置移动，2 为从文件尾开始移动。程序清单 5.11 比较简单，这里不再说明。

程序清单 **5.11　llseek()方法示意代码**

```
static loff_t llseek(struct file *filp, loff_t off, int whence)
{
    loff_t newpos;
```

```
通过 filp->private_data 确定实际操作的物理设备;                    (1)
获取文件长度;                                                    (2)
switch(whence)                                                   (3)
{
    case 0: /* SEEK_SET */                                       (4)
        newpos = off;                                            (5)
        break;                                                   (6)
    case 1: /* SEEK_CUR */                                       (7)
        newpos = filp->f_pos + off;                              (8)
        break;                                                   (9)
    case 2: /* SEEK_END */                                       (10)
        newpos = 文件长度 + off;                                  (11)
        break;                                                   (12)
    default: /* can't happen */                                 (13)
        return -EINVAL;                                          (14)
}
if (newpos<0)                                                    (15)
{
    return -EINVAL;                                              (16)
}
filp->f_pos = newpos;                                            (17)
return newpos;                                                   (18)
}
```

6. ioctl()

对于一个具体的设备来说，它可能有一些独特的操作，这些操作不容易通过 read() 方法和 write() 方法实现。对于这些独特的操作，大多数都通过 ioctl() 方法实现。虽然 ioctl() 方法本身不复杂，但涉及的知识比较多，将在 5.2.8 小节详细介绍。

7. owner

用来表明驱动程序的所有者，设置为 THIS_MODULE 即可（程序清单 5.1(20)）。

注意：在一个特定的驱动程序中，除 open() 方法和 release() 方法外，其他方法不是必须实现的。如果不需要实现某个方法，则不初始化它即可。

5.2.5 关于 file 结构

在 5.2.4 小节介绍的字符设备的方法中多次看到指向 file 结构的指针，本小节将详细介绍它。file 结构在 <linux/fs.h> 中定义，在 Linux（μClinux）2.4.x 中，其定义见程序清单 5.12。

更多高清PDF图书 www.gqpdf.com

程序清单 **5.12**　**file** 结构的定义

```
struct file {
    struct list_head        f_list;
    struct dentry           * f_dentry;
    struct vfsmount         * f_vfsmnt;
    struct file_operations  * f_op;
    atomic_t                f_count;
    unsigned int            f_flags;
    mode_t                  f_mode;
    loff_t                  f_pos;
    unsigned long           f_reada, f_ramax, f_raend, f_ralen, f_rawin;
    struct fown_struct      f_owner;
    unsigned int            f_uid, f_gid;
    int                     f_error;

    unsigned long           f_version;

    /* needed for tty driver, and maybe others */
    void                    *private_data;

    /* preallocated helper kiobuf to speedup O_DIRECT */
    struct kiobuf           * f_iobuf;
    long                    f_iobuf_lock;
};
```

file 结构是设备驱动程序所使用的一个最重要的数据结构。注意，file 与用户程序中的 FILE 没有任何关联。FILE 在 C 库中定义且从不出现在内核代码中，而 struct file 是一个内核结构，从不出现在用户程序中。

file 结构代表一个“打开的文件”。它由内核在 open() 时创建且在 close() 前作为参数传递给操作在设备上的函数。在文件关闭后，内核释放这个数据结构。一个“打开的文件”与由 struct inode 表示的“磁盘文件”有所不同。

在内核源码中，指向 struct file 的指针通常称为 file 或 filp（文件指针）。为了不与这个结构相混淆，笔者将一直称指针为 filp。filp 是一个指针（同样，它也是设备方法的参数之一），而 file 是结构体本身。

file 结构中的大部分成员都用内核操作，驱动程序无须关心。下面将介绍驱动程序可能用到的成员。

gqpdf.com

1. f_mode;

文件模式由 FMODE_READ 和 FMODE_WRITE 标识。驱动程序的 ioctl() 方法可能需要查看这个域来检查读/写权限。因为内核在调用驱动程序的 read() 方法和 write() 方法前已检查了权限,所以 read() 方法和 write() 方法无须检查读/写权限。例如,在驱动程序还不知道的情况下,一个不允许的写操作就已被内核拒绝了。在 ARM7TDMI 中 mode_t 为 16 位无符号整数,等价于 unsigned short。

2. f_ops;

当前读/写位置。loff_t 是一个 64 位整数(就是 long long)。如果驱动程序需要知道这个值,可直接读取这个字段。如果定义了 lseek() 方法,则应更新 f_pos 的值。当传输数据时,read() 和 write() 也应更新该值。

3. f_flags;

文件标志,如 O_RDONLY、O_NONBLOCK 和 O_SYNC。为了支持非阻塞型操作,驱动程序需要检查这个标志,而其他标志很少用到。注意,检查读/写权限应查看 f_mode 而不是f_flags。所有这些标志都定义在 <linux/fcntl.h> 中。

4. f_inode;

打开文件所对应的 i 节点(内核的一个概念,不必深究)。inode 指针是内核传递给所有文件操作的第一个参数,所以驱动程序一般不需要访问 file 结构的这个字段。在某些特殊情况下只能访问 struct file 时,驱动程序可通过这个字段找到相应的 i 节点。对于驱动程序来说,这个成员的主要作用是获取从设备号,其方法参考程序清单 5.7(1)。对于这个成员的其他成员,驱动程序几乎都不会用到。

5. f_op;

与文件对应的操作。内核在完成 open() 时对这个指针赋值,以后需要分派操作时就读这些数据。filp->f_op 中的值从不保存供以后引用;也就是说,可在需要时修改文件所对应的操作,下一次再操作那个打开文件的相应操作时就会调用新方法。例如,主设备号为 1 的设备(/dev/null,/dev/zero 等)的 open() 代码根据要打开的次设备号替换 filp->f_op 中的操作。这种技巧有助于在不增加系统调用负担的情况下方便识别主设备号相同的设备。能替换文件操作的能力在面向对象编程技术中称为“方法重载”。

6. private_data;

系统调用 open() 在调用驱动程序的 open() 方法前将这个指针置为 NULL。驱动程序可将这个字段用于任意目的或者简单忽略这个字段。驱动程序可用这个字段指向已分配的数据,但一定要在内核释放 file 结构前的 release() 方法中清除它。

实际的结构里还有其他一些字段,但它们对于驱动程序并不是特别有用。由于驱动程序从不填写 file 结构,它们只是简单地访问其他地方创建的结构,因此可大胆地忽略这些字段。

更多高清PDF图书 www.gqpdf.com

5.2.6 内核空间与用户空间交换数据

通常情况下，应用程序通过内核接口访问驱动程序，这是驱动程序的主要使用方式。因此，驱动程序需要与应用程序交换数据。

Linux 把存储器分为“内核空间”和“用户空间”。操作系统内核和驱动程序在内核空间中运行，而用户程序在用户空间中运行，两者访问各自的内存空间都没有需要特殊注意的地方。用户程序不能访问内核空间，但操作系统内核和驱动程序也不能使用指针或 memcpy 等常规方法与用户空间传输数据。造成这种状况有多种原因，其主要原因是 Linux 系统使用了虚拟内存机制。使用了虚拟内存机制后，用户空间的内存可能被换出，当内核使用用户空间指针时，对应的页面可能不在内存中了。另外，有一些系统的用户空间与内核空间采用完全不同的映射机制，用户空间的指针根本不能在内核空间中使用。

对于 μClinux 来说，虽然没有这些问题，但为了兼容和可靠，还是使用内核提供的函数。

Linux（μClinux）2.4.x 提供多个函数和宏用于内核空间与用户空间之间拷贝数据，主要有以下几个：access_ok()、copy_to_user()、copy_from_user()、put_user() 和 get_user()。下面逐一介绍。

1. access _ok()

函数 access_ok() 用于检查程序是否能以指定的方式访问指定地址指定长度的内存。函数 access_ok() 的原型见程序清单 5.13。函数 access_ok() 的第一个参数 type 为访问方式，可选 VERIFY_READ 和 VERIFY_WRITE。VERIFY_READ 表示需要对指定地址的内存进行读操作，而 VERIFY_WRITE 表示需要对指定地址的内存进行写操作。函数 access_ok() 的第二个参数 addr 指定要操作的地址。函数 access_ok() 的第三个参数 size 指定需要读或写多长的数据（以字节为单位）。如果函数 access_ok() 返回真（1），则表示可对指定的内存范围进行相应的操作；否则函数 access_ok() 返回假（0），表示不可以。

程序清单 **5.13** **access_ok()** 的原型

```
int access_ok(int type, unsigned long addr, unsigned long size);
```

2. copy _to _user() 和 copy _from _user()

这两个函数的原型见程序清单 5.14，它们用于在用户空间和内核空间之间进行数据拷贝。从程序清单 5.14 可看出，这两个函数很相似，它们具有相同的参数和返回值。事实上，除了数据的拷贝方向不一样外，它们是一致的。函数 copy_to_user() 是把数据从内核空间拷贝到用户空间，而函数 copy_from_user() 是把数据从用户空间拷贝到内核空间。

函数 copy_to_user() 和 copy_from_user() 的第一个参数 to 为数据的目标地址，即数据将要拷贝到哪里。函数 copy_to_user() 和 copy_from_user() 的第二个参数 from 为数据的源地址，即数据从哪里拷贝。函数 copy_to_user() 和 copy_from_user() 的第三个参数 len 为需要拷贝的数

gqpdf.com

据个数，以字节为单位。

这个函数内部调用 access_ok()。返回值是未能传送的字节数。这样，如果发生错误，返回值必然大于 0，在这种情况下，驱动程序应返回 - EFAULT，因为错误是由错误的内存访问引起的。

程序清单 **5.14**　**copy_to_user()** 和 **copy_from_user()** 函数原型

```
unsigned long copy_to_user(void *to, const void *from, unsigned long len);
unsigned long copy_from_user(void *to, const void *from, unsigned long len);
```

3. __copy _to _user() 和__copy _from _user()

这两个函数是 copy_to_user() 和 copy_from_user() 的变体。__copy_to_user() 与 copy_to_user() 功能和使用方法一样，仅仅是不调用 access_ok()。__copy_from_user() 与 copy_from_user() 功能和使用方法一样，仅仅是不调用 access_ok()。

4. put _user() 和 get _user()

这实际上是两个宏。当程序拷贝的数据为基本数据类型（1、2 或 4 字节，以及 64 位平台上的 8 字节）时，使用它们拷贝数据比较快。put_user() 和 get_user() 仅数据拷贝的方向不同：put_user() 把数据由内核空间拷贝到用户空间；get_user() 把数据由用户空间拷贝到内核空间。put_user() 和 get_user() 的调用原型见程序清单 5.15。

put_user() 和 get_user() 的第一个参数为内核空间的变量，而第二个参数为一个指针。因为在宏表达式中不进行类型检查，所以可传递给 put_user() 和 get_user() 任何类型的数据指针，不过它应是一个用户空间地址。数据传输的尺寸依赖于 ptr 参数的类型，这是在编译时通过特殊的 GCC 伪函数实现的，这里没有必要介绍。结果是，如果 ptr 是一个字符指针，就传递 1 字节；依次类推，分别有 2、4 和 8 字节。

put_user() 和 get_user() 会调用 access_ok()。如果操作成功，则 put_user() 和 get_user() 返回 0；否则返回 - EFAULT。

程序清单 **5.15**　**get_user()** 和 **put_user()** 的原型

```
int get_user(lvalue, address)
int put_user(expression, address);
```

5. __put _user() 和__get _user()

__put_user() 和__get_user() 是 put_user() 和 get_user() 的变体。__put_user() 与 put_user() 功能和使用方法一样，仅仅是不调用 access_ok()。__get_user() 与 get_user() 功能和使用方法一样，仅仅是不调用 access_ok()。

5.2.7　内核空间与 I/O 空间交换数据

不同的体系结构对 I/O 的处理不一样。例如，对于 x86 体系结构来说，I/O 是与内存完全

更多高清PDF图书　www.gqpdf.com

不同的东西，它使用专用指令访问，也具有专门的地址空间；对于 ARM 体系结构来说，是不区分 I/O 和内存的，它们在统一空间内，使用同样的指令访问。Linux（μClinux）内核为了屏蔽这些差别，提供了一组宏给驱动程序访问 I/O，见表 5.1。

表 5.1　访问 I/O 的宏

函数名	原　型	功　能	参数说明	返回值
outb	outb(v,p)	写一个 8 位数据到 8 位 I/O 端口	v：写入 I/O 的数据 p：I/O 端口地址	无
outw	outw(v,p)	写一个 16 位数据到 16 位 I/O 端口		
outl	outl(v,p)	写一个 32 位数据到 32 位 I/O 端口		
inb	inb(p)	从 8 位 I/O 端口读一个 8 位数据	p：I/O 端口地址	读到的数据
inw	inw(p)	从 16 位 I/O 端口读一个 16 位数据		
inl	inl(p)	从 32 位 I/O 端口读一个 32 位数据		
outsb	outsb(p,d,l)	写多个 8 位数据到 8 位 I/O 端口	p：I/O 端口地址 d：指针，指向要发送的数据 l：要写的数据个数	无
outsw	outsw(p,d,l)	写多个 16 位数据到 16 位 I/O 端口		
outsl	outsl(p,d,l)	写多个 32 位数据到 32 位 I/O 端口		
insb	insb(p,d,l)	从 8 位 I/O 端口读多个 8 位数据	p：I/O 端口地址 d：指针，指向数据保存位置 l：要读的数据个数	无
insw	insw(p,d,l)	从 16 位 I/O 端口读多个 16 位数据		
insl	insl(p,d,l)	从 32 位 I/O 端口读多个 32 位数据		

另外，在 LPC2200 的移植代码的 hardware.h 文件中已定义了各寄存器的地址（注意：仅仅是一个地址、一个数而已），这样表 5.1 中宏的 p 参数便可直接使用寄存器名称。

5.2.8　ioctl() 方法的编写

1. 为什么需要 ioctl() 方法

在现实世界中，电脑的外设多种多样，它们各自有独特的操作方法，不可能全部通过 read() 方法和 write() 方法实现，而每种独特的操作方法使用独立的方法实现又难以使用。为了解决这个难题，Linux（μClinux）使用一个通用方法 ioctl() 来处理。用户程序可调用适当的内核的 ioctl() 系统调用（参考 5.3 节）间接地调用这个方法。

2. ioctl() 方法的示意代码

ioctl() 方法的示意代码见程序清单 5.16。ioctl() 方法的第一个参数 inode 为设备节点，在 5.2.4 小节 open() 方法和 release() 方法有介绍。ioctl() 方法的第二个参数 filp 代表打开的文件，详情见 5.2.5 小节。ioctl() 方法的第三个参数 cmd 为驱动程序的特殊命令编号，它不能任意选择。一般来说，一个系统中所有驱动程序的特殊命令编号不能重复。关于特殊命令编号在本小节的“选择 ioctl 命令”部分介绍。ioctl() 方法的第四个参数 arg 看起来是一个长整型

参数，但实质上却不一定。驱动程序利用它来接收剩余的参数。因此，如果有多个参数，arg 往往会强制转换成一个指针。关于 arg 的使用请参考本小节介绍的“使用 ioctl 参数”部分。

当成功地完成所需的操作时，ioctl () 方法应返回 0；否则返回一个负数，一般为 - ENOTTY。对于非法的命令，尽管可返回 - EINVAL 更贴切，也并不会有问题，但标准的返回值为 - ENOTTY。

ioctl () 方法的示意代码比较简单，且程序清单 5.16 (3)、(5) 所提到的宏在本小节介绍的“选择 ioctl 命令”部分有说明，这里不详细介绍。

程序清单 **5.16　ioctl** () 方法示意代码

更多高清PDF图书　www.gqpdf.com

```
static int ioctl(struct inode * inode, struct file * filp, unsigned int cmd, un-
signed long arg)
{
    从设备号 = MINOR(inode->i_rdev);                                    (1)
    通过从设备号判断需要操作的设备;                                    (2)

    if (_IOC_TYPE(cmd) != XXX_IOC_MAGIC)                                (3)
    {
        return -ENOTTY;                                                 (4)
    }
    if (_IOC_NR(cmd) >= XXX_MAXNR)                                      (5)
    {
        return -ENOTTY;                                                 (6)
    }

    switch(cmd)                                                         (7)
    {
        case A 命令:                                                    (8)
            处理 A 命令;                                                (9)
            break;                                                      (10)
        case B 命令:                                                    (11)
            处理 B 命令;                                                (12)
            break;                                                      (13)
        ......                                                          (14)
        default:                                                        (15)
            return -ENOTTY;                                             (16)
            break;                                                      (17)
    }
```

驱动程序幻数，XXX应当改成驱动程序的名称

驱动程序特殊命令数，XXX应当改成驱动程序的名称

```
    return 0;                                                    (18)
}
```

3. 选择 ioctl()命令

在编写 ioctl()代码前，需要选择对应不同命令的命令号。遗憾的是，简单地从 1 开始选择号码是不能凑效的。

为了防止对错误的设备使用正确的命令，命令号应在系统范围内是惟一的。这种失配并不是不难发生，程序可能发现自己正在对像 FIFO 和 kmouse 这类非串口输入流修改波特率。如果每一个 ioctl()命令都是惟一的，应用程序就会获得一个 EINVAL 错误，而不是无意间成功地完成了操作。

为了达到惟一性的目的，每一个命令号都应由多个位字段组成。定义号码的方法使用了 4 个位字段，它们有如下意义（下面所介绍的新符号都定义在 <linux/ioctl.h> 中）。

（1）type（类型）

幻数（magic number）。选择一个系统中没有使用的号码，并在整个驱动程序中使用这个号码。这个字段有 8 位宽（_IOC_TYPEBITS）。在源代码中 Documentation\\目录下 magic-number.txt 文件记录当前已使用的幻数。

（2）number（号码）

序（顺序的）数。它也是 8 位宽（_IOC_NRBITS）。

（3）direction（方向）

如果该命令有数据传输，则它定义数据传输的方向。可使用的值有：_IOC_NONE（没有数据传输）、_IOC_READ、_IOC_WRITE 和_IOC_READ | _IOC_WRITE（双向传输数据）。数据传输是从应用程序的角度看的：IOC_READ 意味着从设备中读数据，即驱动程序必须向用户空间写数据。注意，该字段是一个位屏蔽码，因此可用逻辑 AND 操作从中分解出_IOC_READ 和_IOC_WRITE。

（4）size（尺寸）

所涉及的数据大小。这个字段的宽度与体系结构有关，当前的范围为 8～14 位不等。可在宏_IOC_SIZEBITS 中找到某种体系结构的具体数值。不过，如果想要驱动程序可移植，则只能认为最大尺寸可达 255 字节。系统并不强制使用这个字段。如果驱动程序需要更大尺度的数据传输，则可忽略这个字段。

包含在 <linux/ioctl.h> 中的头文件 <asm/ioctl.h> 定义了可用来构造命令号码的宏：_IO(type,nr)、_IOR(type,nr,size)、_IOW(type,nr,size)和 IOWR(type,nr,size)。每个宏都对应一种可能的数据传输方向，其他字段通过参数传递。头文件还定义了解码宏_IOC_DIR(nr)、_IOC_TYPE(nr)、_IOC_NR(nr)和_IOC_SIZE(nr)，下面分别介绍。

_IO(type,nr)：定义一个没有数据传输的命令编号。type 为幻数，nr 为命令编号。例如：

更多高清PDF图书 www.gqpdf.com

```
#define ADC_SET_CLKDIV      _IO(ADC_IOC_MAGIC,    0)
```

_IOR(type,nr,size)：定义一个读数据的命令编号。type 为幻数，nr 为命令编号，而 sizeof(size) 应为数据大小(字节单位)。例如：

```
#define GPIO_READ_PORT      _IOR(GPIO_IOC_MAGIC,  8, u32)
```

_IOW(type,nr,size)：定义一个写数据的命令编号。type 为幻数，nr 为命令编号，而 sizeof(size) 应为数据大小(字节单位)。例如：

```
#define SPI_SET_CLKDIV      _IO(SPI_IOC_MAGIC,    0, u32)
```

_IOW(type,nr,size)：定义一个既读数据又写数据的命令编号。type 为幻数，nr 为命令编号，而 sizeof(size)应为数据大小(字节单位)。例如：

```
#define SPI_WR_DATA         _IOWR(SPI_IOC_MAGIC,  7, u8)
```

_IOC_DIR(nr)：获得命令编号中的数据传输方向(direction)。这个宏的参数就是命令编号。

_IOC_TYPE(nr)：获得命令编号中的幻数(type)。这个宏的参数就是命令编号。

_IOC_NR(nr)：获得命令编号中的号码(number)。这个宏的参数就是命令编号。

_IOC_SIZE(nr)：获得命令编号中的尺寸(size)。这个宏的参数就是命令编号。

驱动程序应选取惟一幻数并通过这个幻数构造自己的命令。另外，为方便应用程序使用这些命令，驱动程序应提供一个头文件给应用程序使用，这个头文件应定义了这些命令。

4. 使用 ioctl() 参数

ioctl() 方法的第四个参数 arg 是一个额外的参数，Linux(μClinux)2.4.x 没有使用它，将来也不大可能使用它。它由内核的 ioctl 系统调用直接传递过来。如果它是一个整数就非常简单了：可直接使用它；如果它是一个指针，就必须注意一些问题，这些问题实际上就是用户空间与核和空间相互拷贝数据的问题，详情参考 5.2.6 小节。

5.2.9 一些宏定义

相对于程序清单 4.1，程序清单 5.1 仅增加了两个宏 MAJOR_NR 和 DEVICE_NAME(程序清单 5.1(15)、(16))。宏 MAJOR_NR 用来定义驱动程序的主设备号，而 DEVICE_NAME 用来定义驱动程序的名称。

5.2.10 关于头文件

本节介绍的驱动程序使用了最少的系统头文件，而真正的驱动程序一般需要包含更多的系统头文件。当不知道一个宏或函数在哪个系统头文件中声明时，一般是直接参考源代码(可利用搜索工具来查找)。

高清PDF gqpdf.com

5.2.11 其他说明

编译字符设备驱动程序主要编写 Makefile 文件，具体参考 4.3 节。

本节仅介绍了字符设备驱动程序的基本部分，很多细节还没有介绍，其中重要且本书不会详细介绍的部分有：动态使用主设备号、支持带参数加载等。如果想了解这些方面，可参见参考文献[2]。

另外，linux2.6 的字符设备驱动程序的接口有变化，估计主要影响模块的初始化代码、open() 和 release() 方法。

5.3 使用字符设备驱动程序

本节主要讲述程序员如何使用字符设备驱动程序。普通用户不会直接使用驱动程序，一般使用应用程序，而编写应用程序是程序员的责任。

5.3.1 建立设备文件

使用驱动程序的目的是为了使用其支持的设备。在 Linux（μClinux）中，几乎把所用东西都当作文件，设备也不例外。每个设备都有对应的特殊文件（成为设备文件或节点文件，详情请参考 5.1.2 小节。一般情况下，系统中应存在所需要使用的设备文件。设备文件一般存储在/dev 目录下。

如果没有对应的设备文件，则需要自己建立。建立设备文件的命令是 mknod。对于字符设备，一般使用以下命令：

```
mknod   文件名称(可包含路径)  c   \[主设备号\]   \[从设备号\]
```

输入“mknod --help”可得到 mknod 的详细帮助。

5.3.2 加载驱动程序

本书介绍的驱动程序都编译成内核模块，这就必须把驱动程序加载到内核中才能被使用。加载内核模块的命令为 insmod，一般使用方法如下：

```
insmod   内核模块文件名(可包含路径)
```

输入“insmod --help”可得到 mknod 的详细帮助。

注意：执行此命令必须有 root 权限。

5.3.3 接口函数

在 Linux（μClinux）中，几乎把所用东西都当作文件，设备也不例外。因此，（通过驱动程

更多高清PDF图书 www.gqpdf.com

序)操作设备也是使用操作文件的函数。在 Linux(μClinux)中,操作文件的函数有两组:

- 标准的 C 语言库函数。
- 内核提供给应用程序接口函数(对应系统调用)。

对于一般的驱动程序来说,使用标准的 C 语言库函数并不合适,这是因为:

- 不能调用 ioctl 系统调用,进而不能使用驱动程序的 ioctl 方法。
- 不能确定调用驱动程序的 read()、write()方法时传输的参数和次数。而尽管传输中数据一样多,但调用 read()或 write()方法的参数和次数不同,其结果也可能不同。当然,标准的流设备不存在这个问题。

如果上面两个问题对程序没有影响,也可使用标准的 C 语言库函数访问设备。介绍标准 C 语言库函数的资料非常多,这里就不介绍了。下面重点介绍内核提供给应用程序的接口函数。

1. open()(打开文件)

在使用文件前必须打开文件,对于字符设备来说,就是打开对应的字符设备文件。打开文件通过调用函数 open()实现。函数 open()的详细信息见表 5.2。

表 5.2 open()函数详细说明

包含文件	#include <sys/types.h> #include <sys/stat.h> #include <fcntl.h>
函数原型	int open(const char * pathname, int flags); int open(const char * pathname,int flags, mode_t mode);
函数参数	pathname: 指向欲打开的文件路径字符串 flags: 文件打开模式 O_RDONLY 以只读方式打开文件 O_WRONLY 以只写方式打开文件 O_RDWR 以可读/写方式打开文件 上述 3 种模式是互斥的,不可同时使用,但可与下列的模式利用 OR(|)运算符组合 O_CREAT 若欲打开的文件不存在,则自动建立该文件 O_EXCL 如果 O_CREAT 也被设置,则此指令会去检查文件是否存在。若文件不存在,则建立该文件;否则将导致打开文件错误。此外,若 O_CREAT 与 O_EXCL 同时设置,并且欲打开的文件为符号连接,则会令打开文件失败 O_NOCTTY 如果欲打开的文件为终端机设备,则不会将该终端机当成进程控制终端机 O_TRUNC 若文件存在且以可写的方式打开,则此模式会令文件长度清 0,而原来存于该文件的资料也会消失 O_APPEND 当读/写文件时会从文件尾开始移动,也就是所写入的数据会以附加的方式加入到文件后面

续表 5.2

函数参数	O_NONBLOCK　以不可阻断的方式打开文件，也就是无论有无数据读取或等待，都会立即返回进程之中 O_NDELAY　同 O_NONBLOCK 的内容 O_SYNC　以同步的方式打开文件 O_NOFOLLOW　如果参数 pathname 所指的文件为符号连接，则会令打开文件失败 O_DIRECTORY　如果参数 pathname 所指的文件并非目录，则会令打开文件失败。此为 Linux2.2 以后特有的旗标，以避免一些系统安全问题。参数 mode 则有下列数种组合，只有在建立新文件时才会生效，此外真正建文件时的权限会受到 umask 值所影响，因此该文件权限应为（mode-umaks） S_IRWXU：00700 权限，代表该文件所有者具有可读/写及可执行的权限 S_IRUSR 或 S_IREAD：00400 权限，代表该文件所有者具有可读取的权限 S_IWUSR 或 S_IWRITE：00200 权限，代表该文件所有者具有可写入的权限 S_IXUSR 或 S_IEXEC：00100 权限，代表该文件所有者具有可执行的权限 S_IRWXG：00070 权限，代表该文件用户组具有可读/写及可执行的权限。 S_IRGRP：00040 权限，代表该文件用户组具有可读的权限 S_IWGRP：00020 权限，代表该文件用户组具有可写入的权限 S_IXGRP：00010 权限，代表该文件用户组具有可执行的权限 S_IRWXO：00007 权限，代表其他用户具有可读/写及可执行的权限 S_IROTH：00004 权限，代表其他用户具有可读的权限 S_IWOTH：00002 权限，代表其他用户具有可写入的权限 S_IXOTH：00001 权限，代表其他用户具有可执行的权限
函数返回值	正数：对应的文件句柄 -1：打开文件失败，errno 会存放错误代码
错误代码	EEXIST：参数 pathname 所指的文件已存在，却使用了 O_CREAT 和 O_EXCL 旗标 EACCESS：参数 pathname 所指的文件不符合所要求测试的权限 EROFS：欲测试写入权限的文件存在于只读文件系统内 EFAULT：参数 pathname 指针超出可存取内存空间 EINVAL：参数 mode 不正确 ENAMETOOLONG：参数 pathname 太长 ENOTDIR：参数 pathname 不是目录 ENOMEM：核心内存不足 ELOOP：参数 pathname 有过多符号连接问题 EIO：I/O 存取错误
特殊说明和注意点	使用 access() 作用户认证方面的判断要特别小心，例如在 access() 后再做 open() 空文件，可能会造成系统安全上的问题

更多高清PDF图书 www.gqpdf.com

续表 5.2

范　例	#include < unistd. h >
	#include < sys/types. h >
	#include < sys/stat. h >
	#include < fcntl. h >
	
	int main (void)
	{
	
	int fd, size;
	char s\[\] = "Linux Programmer! \\n", buffer\[80\];
	
	fd = open ("/tmp/temp", O_WRONLY | O_CREAT);
	write (fd, s, sizeof (s));
	close (fd);
	fd = open ("/tmp/temp", O_RDONLY);
	size = read (fd, buffer, sizeof (buffer));
	close (fd);
	printf ("%s", buffer);
	return 0;
	}

2. close ()（关闭文件）

文件使用完毕需要关闭，否则可能造成文件损坏。操作设备也需要关闭文件操作，这样内核才能正确地跟踪驱动程序的使用。关闭文件通过调用函数 close () 实现。函数 close () 的具体信息见表 5.3。

表 5.3　close () 函数的详细说明

包含文件	#include < unistd. h >
函数原型	int close (int fd);
函数参数	fd：文件句柄 (open () 返回值)
函数返回值	0：文件顺利关闭 - 1：发生错误，errno 会存放错误代码
错误代码	EBADF 参数 fd 非有效的文件描述词或该文件已关闭
特殊说明和注意点	虽然在进程结束时，系统会自动关闭已打开的文件，但仍建议自行关闭文件，并确实检查返回值
范　例	见 open () 函数说明

3. read()（由已打开的文件读取数据）

应用程序可调用函数 read() 从文件读取数据。对于设备来说，这个函数最终调用驱动程序的 read() 方法。此外，文件读/写位置会随读取到的字节移动。一般来说，调用一次 read() 函数对应执行一次驱动程序的 read() 方法。read() 函数的详细说明见表 5.4。

表 5.4　read() 函数的详细说明

包含文件	#include < unistd.h >
函数原型	ssize_t read(int fd, void * buf, size_t count);
函数参数	fd：文件句柄（open() 返回值） buf：存储将要读取的数据缓冲区 count：读取的数据数目（以字节为单位）
函数返回值	正数：实际读取的数据数目 0：表示已到达文件尾或无可读取的数据 -1：发生错误
错误代码	EINTR：此调用被信号所中断 EAGAIN：当使用不可阻断 I/O 时（O_NONBLOCK），若无数据可读取，则返回此值 EBADF：参数 fd 非有效的文件句柄，或该文件已关闭
特殊说明和注意点	如果顺利，read() 会返回实际读到的字节数，最好能将返回值与参数 count 比较。若返回的字节数比要求读取的字节数少，则有可能读到了文件尾、从管道（pipe）或终端机读取，或者是 read() 被信号中断了读取动作。当有错误发生时，则返回 -1，错误代码存入 errno 中，而文件读/写位置则无法预期。若参数 count 为 0，则 read() 不会有作用并返回 0
范　例	见 open() 函数说明

4. write()（将数据写入已打开的文件内）

应用程序可调用函数 write() 从文件读取数据。对于设备来说，这个函数最终调用驱动程序的 write() 方法。此外，文件读/写位置会随写入的字节移动。一般来说，调用一次 write() 函数对应执行一次驱动程序的 write() 方法。write() 函数的详细说明见表 5.5。

5. lseek()（移动文件读/写位置）

每一个已打开的文件都有一个读/写位置。函数 read() 和 write() 都是从文件的当前位置读/写数据，当应用程序需要从其他位置读/写数据时，需要通过调用函数 lseek() 改变文件的当前位置。对于设备来说，这个函数最终调用驱动程序的 lseek() 方法。一般来说，调用一次 lseek() 函数对应执行一次驱动程序的 lseek() 方法。lseek() 函数的详细说明见表 5.6。

更多高清PDF图书　www.gqpdf.com

表 5.5　write() 函数的详细说明

包含文件	#include < unistd. h >
函数原型	ssize_t write (int fd, void * buf , size_t count) ;
函数参数	fd：文件句柄 (open () 返回值) buf：存储将要写入文件的数据的缓冲区 count：写入的数据数目 (以字节为单位)
函数返回值	正数：实际读取的数据数目 0：表示已到达文件尾或无可读取的数据 - 1：发生错误
错误代码	EINTR：此调用被信号所中断 EAGAIN：当使用不可阻断 I/O 时 (O_NONBLOCK)，若无数据可读取，则返回此值 EBADF：参数 fd 非有效的文件句柄，或该文件已关闭
特殊说明和注意点	如果顺利，write () 会返回实际读到的字节数，最好能将返回值与参数 count 比较。若返回的字节数比要求读取的字节数少，则有可能读到了文件尾、从管道 (pipe) 或终端机读取，或者是 write () 被信号中断了读取动作。当有错误发生时，返回 - 1，错误代码存入 ermo 中，而文件读/写位置则无法预期。若参数 count 为 0，则 write () 不会有作用并返回 0
范　例	见 open () 函数说明

表 5.6　lseek() 函数的详细说明

包含文件	#include < sys/types. h > #include < unistd. h >
函数原型	off_t lseek (int fildes, off_t offset , int whence) ;
函数参数	fd：文件句柄 (open () 返回值) offset：根据参数 whence 来移动读/写位置的位移数 whence：移动的方式 SEEK_SET　参数 offset 即为新的读/写位置 SEEK_CUR　以目前的读/写位置往后增加 offset 个位移量 SEEK_END　将读/写位置指向文件尾后再增加 offset 个位移量 当 whence 值为 SEEK_CUR 或 SEEK_END 时，参数 offset 允许负值的出现
函数返回值	非负数：目前的读/写位置，也就是距离文件开头多少个字节 1：发生错误，errno 会存放错误代码
特殊说明和注意点	Linux 系统不允许 lseek () 对 tty 装置作用，此项动作会令 lseek () 返回 ESPIPE

6. ioctl()(输入/输出控制)

这个函数是针对设备的,普通文件没有必要调用该函数。一般情况下,程序正确地调用函数 ioctl()时会导致对某个驱动程序 ioctl()方法的调用。ioctl()函数的第 2 个参数会传递给 ioctl()方法的第 3 个参数,ioctl()函数的第 3 个参数会传递给 ioctl()方法的第 4 个参数。ioctl()函数的原型可参考表 5.7。ioctl()函数的原型名义上参数数目可变,但实际上并不可变。ioctl()的第 3 个参数事实上只是一个可选类型的参数,这里用省略号只是为了在编译时防止编译器进行类型检查。第 3 个参数的具体情况与要完成的控制命令(第 2 个参数)有关。某些命令不需要参数(实际上给了一个垃圾参数),某些命令需要一个整数作参数,而某些命令需要一个指针作参数。使用指针通常可用来向 ioctl()传递任意数目的数据,设备可从用户空间接收任意大小的数据。ioctl()函数的详细说明见表 5.7。

表 5.7　ioctl()函数的详细说明

包含文件	#include <sys/ ioctl.h>
函数原型	int ioctl (int fd, unsigned long int cmd, ...)
函数参数	fd: 文件句柄(open()返回值) cmd: 驱动程序特殊命令 ...: 由驱动程序解释
函数返回值	0: 成功 其他: 错误码

5.3.4　示例代码

应用程序的示例代码见程序清单 5.17。这里用的是内核提供给应用程序接口函数。因为使用标准 C 库函数的资料很多,所以这里不做介绍。程序清单 5.17 很简单,这里不再说明。

程序清单 **5.17**　应用程序示例代码

```
#include <fcntl.h>                                                    (1)
#include "驱动程序提供的.h"                                            (2)

int main(void)
{
    int fd;

    fd = open("设备文件名(包含路径)", 打开模式);                        (3)
    if(fd == -1)                                                      (4)
    {
```


更多高清PDF图书　www.gqpdf.com

```
        printf("\\nDon't open this file! \\n");                    (5)
        return -1;                                                (6)
    }
/*根据需要调用 read()、write()和 ioctl() */                        (7)
    close(fd);                                                    (8)
    return 0;                                                     (9)
}
```

5.3.5 编写 Makefile 文件

应用程序编译方法与驱动程序实际是类似的，仅 Makefile 文件不同，具体信息可参考 4.3 节。应用程序的 Makefile 文件的例子见程序清单 5.18。程序清单 5.18 其实与程序清单 4.2 所示的 Makefile 文件类似，使用方法也类似，只是编译参数不一样。此 Makefile 的使用要点在程序清单 5.18 已注明，其他请参考 4.3 节。

程序清单 **5.18** 应用程序的 **Makefile** 文件

```
EXEC = test                                                          (1)
OBJS = test.o                                                        (2)
SRC = test.c                                                         (3)

CC = arm-elf-gcc                                                     (4)

BASEPATH = /work/uClinux-dist                                        (5)
LIBPATH = $(BASEPATH)/lib                                            (6)
LLIBPATH = $(LIBPATH)/uClibc/lib                                     (7)
INCLUDEPATH = $(BASEPATH)/linux-2.4.x/include                        (8)

LDFLAGS = -Os -g -Dlinux -D__linux__ -Dunix -D__uClinux__ -DEMBED     (9)
LDLIBS = -I $(LIBPATH)/uClibc/include -I $(LIBPATH)/libm -I $(LIBPATH)/libcrypt_
old -I $
(BASEPATH) -fno-builtin -nostartfiles -D__PIC__ -fpic -msingle-pic-base -I $(IN-
CLUDEPATH)                                                           (10)
LDLIBS_EXEC = -Wl, -elf2flt $(LLIBPATH)/crt0.o $(LLIBPATH)/crti.o $(LLIBPATH)/
crtn.o -L $(LIBPATH)/uClibc/. -L $(LLIBPATH) -L $(LIBPATH)/libm -L $(LIBPATH)/libnet
-L $(LIBPATH)/libdes -L $(LIBPATH)/libaes -L $(LIBPATH)/libpcap -L $(LIBPATH)/
libcrypt_
old -L $(LIBPATH)/libssl -L $(LIBPATH)/zlib -lc                       (11)
LDLIBS_OBJS = -c                                                     (12)
```

与程序清单4.2一样的修改方法

修改成自己的路径

关键的连接参数，指定可执行文件格式

高PDF 清藏 gqpdf.com

更多高清PDF图书 www.gqpdf.com

```
all: $(EXEC)                                                              (13)

$(EXEC): $(OBJS)                                                          (14)
    $(CC) $(LDFLAGS) $(LDLIBS) $(LDLIBS_EXEC) -o $@  $(OBJS)              (15)

%.o:%.c                                                                   (16)
    $(CC) $(LDFLAGS) $(LDLIBS) $(LDLIBS_OBJS) -c $< -o $@                 (17)

clean:                                                                    (18)
    -rm -f $(EXEC) *.elf *.gdb *.o                                        (19)
```

本章小结

本章首先介绍了 Linux（μClinux）字符设备驱动程序的相关概念；然后主要介绍了一个最简单的字符设备驱动程序，主要包括源码分析和编译；最后介绍程序员如何使用字符设备。这些是 Linux（μClinux）字符设备驱动程序最基础的公共知识，也是编写字符设备必须掌握的知识。

相对于第 1 章，本章主要增加了以下知识：

- 字符驱动的相关函数和数据结构（参考 5.2.2、5.2.3 和 5.2.5 小节）。
- 内核空间与用户空间交换数据（参考 5.2.6 小节）。
- 内核空间与 I/O 空间交换数据（参考 5.2.7 小节）。
- 程序员使用字符设备的方法（参考 5.3 节）。

本章介绍的知识是第 6 ~ 10 章的基础。

高PDF 清藏 gqpdf.com

第6章　GPIO（字符设备）驱动程序编写范例

LPC2200 系列 ARM 微控制器具有 4 个 GPIO 端口 P0、P1、P2 和 P3。GPIO 口为 32 位，但并不是所有位都引到芯片引脚上。LPC2200 的所有 GPIO 均为双向口（但 LPC2000 系列有一些不是），每一位都可以独立设置为输入或输出。大部分 GPIO 都是推挽输出，部分 GPIO 为开漏输出。部分 GPIO 内部有弱上拉。关于 GPIO 的详细说明请参见参考文献[3]。

本章介绍的驱动程序就是驱动 LPC2200 系列的 GPIO 口，可以完全控制 GPIO。

6.1　在应用程序中使用 GPIO 驱动程序

关于字符设备驱动程序的一般使用方法请参考 5.3 节，本节仅介绍使用 GPIO 驱动程序需要特殊注意的地方。

6.1.1　建立设备文件

驱动程序默认的主设备号为 126，从设备号用来标识操作的 GPIO 端口。其中，从设备号 0 表示 P0；从设备号 1 表示 P1；从设备号 2 表示 P2；从设备号 3 表示 P3。因此，需要建立 4 个设备文件，其主设备号为 126，从设备号为 0 ~ 3，文件名可定为 P0、P1、P2 和 P3。建立设备文件的命令请参考程序清单 6.1。

程序清单 **6.1**　建立设备文件

```
mknod /dev/P0 c 126 0
mknod /dev/P1 c 126 1
mknod /dev/P2 c 126 2
mknod /dev/P3 c 126 3
```

6.1.2　可使用的函数

因为本驱动程序仅实现了 open()、release() 和 ioctl() 方法，所以本驱动程序不能使用标准 C 的库函数来访问，只能使用内核提供给应用程序的接口函数来访问，并且仅部分内核接

更多高清PDF图书　www.gqpdf.com

口函数可使用，其具体情况见表 6.1，具体使用方法见 5.3.3 小节。

表 6.1　可使用的内核接口函数

函数名称	函数功能	函数原型	备　注
open ()	打开文件	int open (const char * pathname, int flags);	在使用 GPIO 前调用，由此函数确定操作的是 P0、P1、P2 还是 P3
close ()	关闭文件	int close (int fd);	在使用完 GPIO 后调用
ioctl ()	I/O 控制	int ioctl (int fd, unsigned long int cmd, ...);	实际控制 GPIO

6.1.3　提供给应用程序的头文件

函数 ioctl () 的第二个参数 cmd 为驱动程序的特殊命令，它的值应由驱动程序提供。本驱动程序提供了一个头文件 gpio. h 定义这些命令。文件 gpio. h 的内容见程序清单 6.2。其中，程序清单 6.2 (5) ~ (16) 定义了 12 个宏，对应 12 条命令。而程序清单 6.2 (1) 定义了 I/O 口低电平的表示方法；程序清单 6.2 (2) 定义了 I/O 口高电平的表示方法。GPIO_PIN_ERR (程序清单 6.2 (3)) 是函数的一个返回值，表示读 I/O 出错。关于各具体命令的说明参见表 6.2。

程序清单 **6.2　gpio. h**

```
#ifndef __GPIO_H
#define __GPIO_H
#include <linux/ioctl.h>

#define GPIO_PIN_LOW          0                                    (1)
#define GPIO_PIN_HIGH         1                                    (2)
#define GPIO_PIN_ERR          (~0)                                 (3)
#define GPIO_IOC_MAGIC        0xd0                                 (4)

#define GPIO_SET_PIN          _IO(GPIO_IOC_MAGIC,    0)            (5)
#define GPIO_SET_ALL_PIN      _IO(GPIO_IOC_MAGIC,    1)            (6)
#define GPIO_CLR_PIN          _IO(GPIO_IOC_MAGIC,    2)            (7)
#define GPIO_CLR_ALL_PIN      _IO(GPIO_IOC_MAGIC,    3)            (8)

#define GPIO_SET_PIN_OUT      _IO(GPIO_IOC_MAGIC,    4)            (9)
#define GPIO_SET_PIN_IN       _IO(GPIO_IOC_MAGIC,    5)           (10)
#define GPIO_SET_MULTI_PIN_OUT _IO(GPIO_IOC_MAGIC,   6)           (11)
#define GPIO_SET_MULTI_PIN_IN  _IO(GPIO_IOC_MAGIC,   7)           (12)
```

高清 PDF 藏 gaopdf.com

```
#define GPIO_READ_PORT          _IOWR(GPIO_IOC_MAGIC,  8,   uint32)  (13)
#define GPIO_READ_ALL_PORT      _IOWR(GPIO_IOC_MAGIC,  9,   uint32)  (14)
#define GPIO_READ_PIN           _IOWR(GPIO_IOC_MAGIC,  10,  uint32)
                                                                     (15)
#define GPIO_READ_ALL_PIN       _IOWR(GPIO_IOC_MAGIC,  11,  uint32)
                                                                     (16)

#define GPIO_MAXNR              12                                   (17)
#endif
```

表 6.2　GPIO 命令

命　令	功　能	备　注
GPIO_SET_PIN	让指定 I/O 引脚输出高电平	参考 6.1.5 小节
GPIO_SET_ALL_PIN	让本 I/O 端口引脚的任意子集输出高电平	
GPIO_CLR_PIN	让指定的 I/O 引脚输出低电平	
GPIO_CLR_ALL_PIN	让本 I/O 端口引脚的任意子集输出低电平	
GPIO_SET_PIN_OUT	把指定 I/O 引脚设置为输出状态	参考 6.1.4 小节
GPIO_SET_PIN_IN	把指定 I/O 引脚设置为输入状态	
GPIO_SET_MULTI_PIN_OUT	把本 I/O 端口引脚的任意子集设置为输出状态	
GPIO_SET_MULTI_PIN_IN	把本 I/O 端口引脚的任意子集设置为输入状态	
GPIO_READ_PORT	读指定 I/O 输出电平	参考 6.1.6 小节
GPIO_READ_ALL_PORT	读本端口所有 I/O 输出电平	
GPIO_READ_PIN	读指定 I/O 引脚上的电平	参考 6.1.7 小节
GPIO_READ_ALL_PIN	读本端口所有 I/O 引脚上的电平	

6.1.4　设置 GPIO 口数据传输方向

GPIO 都有输入和输出两种状态，这部分包含 4 条命令，分别把指定的 I/O 设置为输入或输出状态。

注意：设置为输出的引脚不能接到电源、地或其他输出引脚（包括其他 IC 的输出引脚）上。

1. GPIO_SET_PIN_OUT

- 功能：设置本端口某个 I/O 引脚为输出状态。
- ioctl()函数的第 3 个参数：I/O 引脚索引。整型参数，用于指定 I/O 引脚，0 对应 Px.0，1 对应 Px.1，以此类推。

更多高清PDF图书　www.gqpdf.com

● 使用示例：程序清单 6.3 为将 P0.14 设置为输出。

程序清单 **6.3　GPIO_SET_PIN_OUT** 命令使用示例

```
int fd;
......
fd = open("/dev/P0", O_RDONLY);
......
ioctl(fd, GPIO_SET_PIN_OUT, 14);
......
```

2. GPIO_SET_PIN_IN

● 功能：设置本端口某个 I/O 引脚为输入状态。

● ioctl() 函数的第 3 个参数：I/O 引脚索引。整型参数，用于指定 I/O 引脚，0 对应 Px.0，1 对应 Px.1，以此类推。此参数必须为 0 ~ 31，且指定的 I/O 引脚必须物理存在。

● 使用示例：程序清单 6.4 为将 P0.14 设置为输入。

程序清单 **6.4　GPIO_SET_PIN_IN** 命令使用示例

```
int fd;
......
fd = open("/dev/P0", O_RDONLY);
......
ioctl(fd, GPIO_SET_PIN_IN, 14);
......
```

3. GPIO_SET_MULTI_PIN_OUT

● 功能：设置本端口任意子集引脚为输出状态，未指定的引脚状态不变。

● ioctl() 函数的第 3 个参数：端口引脚位图。32 位整型参数，用于指定端口引脚子集。本参数第 0 位对应 Px.0，第 1 位对应 Px.1，以此类推。当某位为 1 时，表示对应的引脚设置为输出，为 0 的位对应的引脚状态不变。

● 使用示例：程序清单 6.5 为将 P0.14 和 P0.25 设置为输出。

程序清单 **6.5　GPIO_SET_MULTI_PIN_OUT** 命令使用示例

```
int fd;
......
fd = open("/dev/P0", O_RDONLY);
......
ioctl(fd, GPIO_SET_MULTI_PIN_OUT, (1 < < 14) | (1 < < 25));
......
```

gqpdf.com

4. GPIO_SET_MULTI_PIN_IN

- 功能：设置本端口任意子集引脚为输入状态，未指定的引脚状态不变。
- ioctl()函数的第3个参数：端口引脚位图。32位整型参数，用于指定端口引脚子集。本参数第0位对应Px.0，第1位对应Px.1，以此类推。当某位为1时，表示对应的引脚设置为输入，为0的位对应的引脚状态不变。
- 使用示例：程序清单6.6为将P0.14和P0.25设置为输入。

程序清单**6.6　GPIO_SET_MULTI_PIN_IN**命令使用示例

```
int fd;
......
fd = open("/dev/P0", O_RDONLY);
......
ioctl(fd, GPIO_SET_MULTI_PIN_IN, (1 < < 14) | (1 < < 25));
......
```

6.1.5　让GPIO口输出指定电平

GPIO的输出只有两种电平：高电平和低电平。高电平在程序中用GPIO_PIN_HIGH表示，低电平用GPIO_PIN_LOW表示。本驱动提供4条命令，分别把指定的I/O输出电平。

注意：仅当对应的I/O引脚被设置为输出时，才会把电平输出到引脚上。

1. GPIO_SET_PIN

- 功能：设置本端口某个I/O引脚为高电平。
- ioctl()函数的第3个参数：I/O引脚索引。整型参数，用于指定I/O引脚，0对应Px.0，1对应Px.1，以此类推。
- 使用示例：程序清单6.7为将P0.14设置为高电平。

程序清单**6.7　GPIO_SET_PIN**命令使用示例

```
int fd;
......
fd = open("/dev/P0", O_RDONLY);
......
ioctl(fd, GPIO_SET_PIN, 14);
......
```

2. GPIO_SET_ALL_PIN

- 功能：设置本端口任意子集引脚为高电平，未指定的引脚状态不变。

更多高清PDF图书　www.gqpdf.com

- ioctl()函数的第3个参数：端口引脚位图。32位整型参数，用于指定端口引脚子集。本参数第0位对应Px.0，第1位对应Px.1，以此类推。当某位为1时，表示对应的引脚设置为输出，为0的位对应的引脚状态不变。
- 使用示例：程序清单6.8为将P0.14和P0.25设置为高电平。

程序清单**6.8　GPIO_SET_ALL_PIN**命令使用示例

```
int fd;
......
fd = open("/dev/P0", O_RDONLY);
......
ioctl(fd, GPIO_SET_ALL_PIN, (1 < < 14) |(1 < < 25));
......
```

3. GPIO_CLR_PIN

- 功能：设置本端口某个I/O引脚为低电平。
- ioctl()函数的第3个参数：I/O引脚索引。整型参数，用于指定I/O引脚，0对应Px.0，1对应Px.1，以此类推。
- 使用示例：程序清单6.9为将P0.14设置为低电平。

程序清单**6.9　GPIO_CLR_PIN**命令使用示例

```
int fd;
......
fd = open("/dev/P0", O_RDONLY);
......
ioctl(fd, GPIO_CLR_PIN, 14);
......
```

4. GPIO_CLR_ALL_PIN

- 功能：设置本端口任意子集引脚为低电平，未指定的引脚状态不变。
- ioctl()函数的第3个参数：端口引脚位图。32位整型参数，用于指定端口引脚子集。本参数第0位对应Px.0，第1位对应Px.1，以此类推。当某位为1时，表示对应的引脚设置为低电平，为0的位对应的引脚状态不变。
- 使用示例：程序清单6.10为将P0.14和P0.25设置为低电平。

程序清单**6.10　GPIO_CLR_ALL_PIN**命令使用示例

```
int fd;
......
```

高清藏 PDF gqpdf.com

```
fd = open("/dev/P0", O_RDONLY);
......
ioctl(fd, GPIO_CLR_ALL_PIN, (1 < < 14) |(1 < < 25));
......
```

6.1.6 读 GPIO 口输出电平

GPIO 的输出电平与实际引脚上的电平不一定相同，不一致的原因可能是：对应引脚被设置为输入状态，此时的输出不表现在引脚上；引脚不是推挽输出，在 LPC2000 系列上部分引脚未开漏输出，此时引脚不可能输出高电平；外部电路强制引脚为某个电平，如引脚接地了，这是应当避免的情况。本驱动的 GPIO_READ_PORT 命令和 GPIO_READ_ALL_PORT 命令就是读输出电平的，它不理会 GPIO 引脚上的电平。

1. GPIO_READ_PORT

- 功能：读本端口某个 I/O 的输出电平。
- ioctl() 函数的第 3 个参数：一个 32 位整型指针，用于传递参数和返回结果。在调用 ioctl() 函数前，此指针指向的变量为 I/O 引脚索引，用于指定 I/O 引脚，0 对应 Px.0，1 对应 Px.1，以此类推。ioctl() 函数的返回值也存于这个变量中，GPIO_PIN_LOW 表示低电平，GPIO_PIN_HIGH 表示高电平，GPIO_PIN_ERR 表示出错。
- 使用示例：程序清单 6.11 为读取 P0.14 的输出电平。

程序清单 **6.11　GPIO_READ_PORT** 命令使用示例

```
int fd;
u32 pin;
......
fd = open("/dev/P0", O_RDONLY);
......
pin = 14;
ioctl(fd, GPIO_READ_PORT, &pin);
......
```

2. GPIO_READ_ALL_PORT

- 功能：读出本端口任意子集引脚的输出电平。
- ioctl() 函数的第 3 个参数：一个 32 位整型指针，用于返回结果。调用 ioctl() 函数的返回值存于这个指针指向的变量中。返回值第 0 位对应 Px.0，第 1 位对应 Px.1，以此类推。当某位为 1 时，表示对应的引脚输出电平为高，为 0 的位对应的输出电平为低。
- 使用示例：程序清单 6.12 为读取 P0 的输出电平。

更多高清PDF图书 www.gqpdf.com

程序清单 **6.12**　**GPIO_READ_ALL_PORT** 命令使用示例

```
int fd;
u32 pin;
......
fd = open("/dev/P0", O_RDONLY);
......
ioctl(fd, GPIO_READ_ALL_PORT, &pin);
......
```

6.1.7　读 GPIO 口引脚上的电平

GPIO 的输出电平与实际引脚上的电平不一定相同，其原因见 6.2.5 小节。本驱动的 GPIO_READ_PIN 命令和 GPIO_READ_ALL_PIN 命令就是读取引脚电平的。

1. GPIO_READ_PIN

- 功能：读本端口某个 I/O 的输出电平。
- ioctl() 函数的第 3 个参数：一个 32 位整型指针，用于传递参数和返回结果。在调用 ioctl() 函数前，此指针指向的变量为 I/O 引脚索引，用于指定 I/O 引脚，0 对应 Px.0，1 对应 Px.1，以此类推。ioctl() 函数的返回值也存于这个变量中，GPIO_PIN_LOW 表示低电平，GPIO_PIN_HIGH 表示高电平，GPIO_PIN_ERR 表示出错。
- 使用示例：程序清单 6.13 为读取 P0.14 的输出电平。

程序清单 **6.13**　**GPIO_READ_PIN** 命令使用示例

```
int fd;
u32 pin;
......
fd = open("/dev/P0", O_RDONLY);
......
pin = 14;
ioctl(fd, GPIO_READ_PIN, &pin);
......
```

2. GPIO_READ_ALL_PIN

- 功能：读本端口引脚的电平，未指定的引脚状态不变。
- ioctl() 函数的第 3 个参数：一个 32 位整型指针，用于返回结果。调用 ioctl() 函数的返回值存于这个指针指向的变量中。返回值第 0 位对应 Px.0，第 1 位对应 Px.1，以此类推。当某位为 1 时，表示对应的引脚输出电平为高，为 0 的位对应的输出电平为低。

高清PDF

● 使用示例：程序清单 6.14 为读取 P0 的引脚电平。

程序清单 **6.14　GPIO_READ_ALL_PIN** 命令使用示例

```
int fd;
u32 pin;
......
fd = open("/dev/P0", O_RDONLY);
......
ioctl(fd, GPIO_READ_ALL_PIN, &pin);
......
```

6.2　编　译

当建立相关的开发环境和获取驱动程序的源代码后，只需建立（或修改）Makefile 文件，通过 make 命令即可编译驱动程序。关于建立开发环境不是本书的任务，请读者自行参考相关文档。本书介绍的源代码可在广州致远电子有限公司出品的相关产品中获得，或根据本书自行输入。下面介绍本驱动所用的 Makefile 文件（见程序清单 6.15）。

程序清单 **6.15　Makefile** 文件

```
EXEC = gpio
OBJS = gpio.o
SRC = gpio.c

INCLUDE = /work/uClinux-dist/linux-2.4.x/include

CC = arm-elf-gcc
LD = arm-elf-ld
MODCFLAGS = -D__KERNEL__ -I $(INCLUDE) -Wall -O2 -fno-strict-aliasing -fno-common -
pipe -fno-builtin -D__linux__ -g -DNO_MM -mapcs-32 -march = armv4 -mtune = arm7tdmi -
mshort-load-bytes -msoft-float -nostdinc -iwithprefix include
LDFLAGS = -m armelf -r

all: $(EXEC)

$(EXEC): $(OBJS)
    $(LD) $(LDFLAGS) -o $@  $(OBJS)
```

与程序清单4.2不同的部分

根据实际情况修改的部分

更多高清PDF图书　www.gqpdf.com

```
% .o:% .c
    $(CC) $(MODCFLAGS) -mapcs -c $< -o $@

clean:
    -rm -f $(EXEC) *.elf *.gdb *.o
```

可以看出，程序清单6.15与程序清单4.2十分相似，仅前3句不同。实际上，程序清单6.15是根据程序清单4.2修改而来。因此，关于Makefile的详细介绍请参考4.3节。

6.3 源码分析

事实上，第5章已介绍了编写字符驱动程序的一般知识，而本驱动程序也是在那个驱动程序的基础上修改而来。因此，应先阅读第5章，否则不易理解本节内容。

6.3.1 初始化函数

本驱动程序的初始化函数见程序清单6.16。通过对比程序清单5.2可知，程序清单6.16仅比程序清单5.2多程序清单6.16(5)、(6)两句，它用于动态分配主设备号。众所周知，主设备号是正整数，在Linux(μClinux)2.4.x中为1～127。但函数register_chrdev()却可以接受0为主设备号，此时，如果函数register_chrdev()操作成功，则其会返回一个系统未使用的主设备号给本驱动程序使用。此时，驱动程序使用者可通过查看/proc/devices文件了解驱动程序实际使用的主设备号。

程序清单**6.16** **GPIO**驱动程序初始化函数

```
int gpio_init(void)
{
    int  result;

    result = register_chrdev(MAJOR_NR,  DEVICE_NAME,  &gpio_fops);          (1)
    if (result < 0)                                                          (2)
    {
        printk(KERN_ERR DEVICE_NAME ": Unable to get major % d\\n", MAJOR_NR );
                                                                             (3)
        return(result);                                                      (4)
    }

    if (MAJOR_NR == 0)                                                       (5)
    {
```

```
        MAJOR_NR = result; /* dynamic */                                    (6)
    }

    printk(KERN_INFO DEVICE_NAME ": init OK\\n");                           (7)
    return 0;                                                               (8)
}
```

由程序清单 6.16(6)可看出，宏 MAJOR_NR 与第 5 章介绍的不一样。与主设备号相关的其他代码见程序清单 6.17。注意，这些代码并不是紧挨着，甚至不一定在同一个文件中，程序清单 6.17 仅仅是把它们按照逻辑顺序组织起来。其中需要注意的仅仅是程序清单 6.17(3)。程序中有这一句后，insmod 命令就可改变变量 major 的初始值。关于这方面，本书不打算详细介绍，感兴趣的读者可参见参考文献[2]。

程序清单 **6.17**　主设备号相关代码

```
#define GPIO_MAJOR_NR          126                                          (1)
#define MAJOR_NR major                                                      (2)
ODULE_PARM(major, "i");                                                     (3)
static int major = GPIO_MAJOR_NR;                                           (4)
```

6.3.2　清除函数

本驱动程序的清除函数见程序清单 6.18，它与程序清单 5.4 一样，这里不再介绍。

程序清单 **6.18**　**GPIO** 驱动程序清除函数

```
void gpio_cleanup(void)
{
    unregister_chrdev(MAJOR_NR, DEVICE_NAME);
}
```

6.3.3　open()方法

本驱动程序的清除函数见程序清单 6.19，它完全按照程序清单 5.7 编写，且只有必需的代码，这里不再介绍。

程序清单 **6.19**　**GPIO** 驱动程序 **open**()方法

```
static int gpio_open(struct inode *inode, struct file *filp)
{
    MOD_INC_USE_COUNT;
```


更多高清PDF图书　www.gqpdf.com

```
    return 0;           /* success */
}
```

6.3.4 release()方法

本驱动程序的清除函数见程序清单6.20，它完全按照程序清单5.8编写，且只有必需的代码，这里不再介绍。

程序清单 **6.20** **GPIO** 驱动程序 **release()** 方法

```
static int gpio_release(struct inode * inode, struct file * filp)
{
    MOD_DEC_USE_COUNT;
    return 0;
}
```

6.3.5 ioctl()方法

ioctl()方法会访问I/O端口，关于访问I/O端口的方法请参考5.2.7小节。

本驱动的ioctl()方法的代码见程序清单6.21。其实，程序清单6.21完全是按照程序清单5.16编写的，其公共部分增加的代码（程序清单6.21(8)）是计算P0、P1、P2或P3的寄存器基址。LPC2200系列的4个32位GPIO除寄存器基址不同外，其他对于编程来说是完全相同的，而4个GPIO寄存器基址的间隔相同，所以很容易计算出指定GPIO的寄存器基址，而事先计算出的寄存器基址可简化代码的编写。其他部分就是分别实现每一条特定的命令。关于命令的说明请参考6.1节，下面将分类介绍其实现。

程序清单 **6.21** **GPIO** 驱动程序 **ioctl()** 方法

```
static int gpio_ioctl(struct inode * inode, struct file * filp,
                      unsigned int cmd, unsigned long arg)
{
    int num;
    volatile u32 * Reg;
    u32 temp1, temp2;

    num = MINOR(inode->i_rdev);                                    (1)
    if (num > = MAX_PORT)                                          (2)
{
        return -ENODEV;                                            (3)
```

```
    }
    if (_IOC_TYPE(cmd) != GPIO_IOC_MAGIC)                                    (4)
    {
        return -ENOTTY;                                                      (5)
    }

    if (_IOC_NR(cmd) >= GPIO_MAXNR)                                          (6)
    {
        return -ENOTTY;                                                      (7)
    }

    Reg = (volatile u32 *)(GPIO_ADDR_BASE + GPIO_PORT_ADD * num);            (8)
    switch(cmd)                                                              (9)
    {
        case GPIO_SET_PIN:                                                   (10)
            if (arg < 32)                                                    (11)
            {
                Reg[GPIO_SET_OFFSET] = 1u << arg;                            (12)
            }
            break;                                                           (13)
        case GPIO_SET_ALL_PIN:                                               (14)
            Reg[GPIO_SET_OFFSET] = arg;                                      (15)
            break;                                                           (16)
        case GPIO_CLR_PIN:                                                   (17)
            if (arg < 32)                                                    (18)
            {
                Reg[GPIO_CLR_OFFSET] = 1u << arg;                            (19)
            }
            break;                                                           (20)
        case GPIO_CLR_ALL_PIN:                                               (21)
            Reg[GPIO_CLR_OFFSET] = arg;                                      (22)
            break;                                                           (23)
        case GPIO_SET_PIN_OUT:                                               (24)
            if (arg < 32)                                                    (25)
            {
                Reg[GPIO_DIR_OFFSET] |= 1u << arg;                           (26)
```


更多高清PDF图书 www.gqpdf.com

```
        }
        break;                                                          (27)
    case GPIO_SET_PIN_IN:                                               (28)
        if (arg < 32)                                                   (29)
        {
            Reg\[GPIO_DIR_OFFSET\] &= ~(1u << arg);                     (30)
        }
        break;                                                          (31)
    case GPIO_SET_MULTI_PIN_OUT:                                        (32)
        Reg\[GPIO_DIR_OFFSET\] |= arg;                                  (33)
        break;                                                          (34)
    case GPIO_SET_MULTI_PIN_IN:                                         (35)
        Reg\[GPIO_DIR_OFFSET\] &= ~arg;                                 (36)
        break;                                                          (37)
    case GPIO_READ_PORT:                                                (38)
        if (! access_ok(VERIFY_READ, (void *)arg, _IOC_SIZE(cmd)))      (39)
        {
            return -EFAULT;                                             (40)
        }
        if (! access_ok(VERIFY_WRITE, (void *)arg, _IOC_SIZE(cmd)))     (41)
        {
            return -EFAULT;                                             (42)
        }
        temp1 = GPIO_PIN_ERR;                                           (43)
        get_user(temp2, (u32 *)arg);                                    (44)
        if (temp2 < 32)                                                 (45)
        {
            temp1 = Reg\[GPIO_SET_OFFSET\] & (1u << temp2);             (46)
            if (temp1 != 0)                                             (47)
            {
                temp1 = GPIO_PIN_HIGH;                                  (48)
            }
            else
            {
                temp1 = GPIO_PIN_LOW;                                   (49)
            }
        }
```

```
            put_user(temp1, (u32 *)arg);                                    (50)
            break;                                                          (51)
        case GPIO_READ_ALL_PORT:                                            (52)
            if (! access_ok(VERIFY_READ, (void *)arg, _IOC_SIZE(cmd)))      (53)
            {
                return -EFAULT;                                             (54)
            }
            if (! access_ok(VERIFY_WRITE, (void *)arg, _IOC_SIZE(cmd)))     (55)
            {
                return -EFAULT;                                             (56)
            }
            temp1 = Reg\[GPIO_SET_OFFSET\];                                 (57)
            put_user(temp1, (u32 *)arg);                                    (58)
            break;                                                          (59)
        case GPIO_READ_PIN:                                                 (60)
            if (! access_ok(VERIFY_READ, (void *)arg, _IOC_SIZE(cmd)))      (61)
            {
                return -EFAULT;                                             (62)
            }
            if (! access_ok(VERIFY_WRITE, (void *)arg, _IOC_SIZE(cmd)))     (63)
            {
                return -EFAULT;                                             (64)
            }
            temp1 = GPIO_PIN_ERR;                                           (65)
            get_user(temp2, (u32 *)arg);                                    (66)
            if (temp2 < 32)                                                 (67)
            {
                temp1 = Reg\[GPIO_PIN_OFFSET\] & (1u < < temp2);            (68)
                if (temp1 != 0)                                             (69)
                {
                    temp1 = GPIO_PIN_HIGH;                                  (70)
                }
                else
                {
                    temp1 = GPIO_PIN_LOW;                                   (71)
                }
            }
            put_user(temp1, (u32 *)arg);                                    (72)
```


更多高清PDF图书 www.gqpdf.com

```
            break;                                                          (73)
        case GPIO_READ_ALL_PIN:                                             (74)
            if (! access_ok(VERIFY_READ, (void *)arg, _IOC_SIZE(cmd)))      (75)
            {
                return -EFAULT;                                             (76)
            }
            if (! access_ok(VERIFY_WRITE, (void *)arg, _IOC_SIZE(cmd)))     (77)
            {
                return -EFAULT;                                             (78)
            }
            temp1 = Reg\[GPIO_PIN_OFFSET\];                                 (79)
            put_user(temp1, (u32 *)arg);                                    (80)
            break;                                                          (81)
        default:                                                            (82)
            return -ENOTTY;                                                 (83)
            break;                                                          (84)
    }
    return 0;                                                               (85)
}
```

1. GPIO_SET_PIN 和 GPIO_CLR_PIN

除了操作的寄存器不同外，这 2 条命令的实现是一样的。其中 GPIO_SET_PIN 操作寄存器 IO0SET、IO1SET、IO2SET 和 IO3SET 中之一，而 GPIO_CLR_PIN 操作 IO0CLR、IO1CLR、IO2CLR 和 IO3CLR 中之一。至于具体操作哪个寄存器，就由次设备号决定。这里以 GPIO_SET_PIN 为例介绍这些命令。GPIO_SET_PIN 命令的实现代码见程序清单 6.22。这几条命令的第 3 个参数（arg）为一个整数，指明操作的是哪一位 I/O 口。因为一个 GPIO 只有 32 位，所以 arg 大于或等于 32 时不操作（程序清单 6.22（1））。对于 LPC2200 来说，这些寄存器还有一个特点：第 0 位对应 GPIO 的第 0 位，第 1 位对应 GPIO 的第 1 位，以此类推。同时，对这些寄存器写一个数，其中为 1 的位有效，为 0 的位被忽略。因此，写寄存器的语句为程序清单 6.22（2），其中方括号中的宏是与事先定义的寄存器的相对基址偏移量。

程序清单 **6.22　GPIO_SET_PIN** 实现代码

```
case GPIO_SET_PIN:
    if (arg < 32)                                                           (1)
    {
        Reg\[GPIO_SET_OFFSET\] =1u < < arg;                                 (2)
    }
    break;                                                                  (3)
```

高清PDF gqpdf.com

2. GPIO_SET_PIN_OUT 和 GPIO_SET_PIN_IN

命令 GPIO_SET_PIN_OUT 和 GPIO_SET_PIN_IN 的实现代码见程序清单 6.23。这两条命令操作的是同一个寄存器 IO0DIR、IO1DIR、IO2DIR 和 IO3DIR 中之一。至于具体操作哪个寄存器，就由次设备号决定。命令 GPIO_SET_PIN_OUT 将这些寄存器的某一位置 1，而命令 GPIO_SET_PIN_IN 将这些寄存器的某一位置 0。这些寄存器与上面提到的 IOxSET 和 IOxCLR（x 代表 0、1、2 或 3）不同：无论是 1 的位还是 0 的位都是有效的位。不过，它们各位的含义是一样的。而且，这 2 条命令的第 3 个参数也一样。所以，这些代码与程序清单 6.18 很相似，这里不再分析。

程序清单 **6.23　GPIO_SET_PIN_OUT 和 GPIO_SET_PIN_IN**

```
case GPIO_SET_PIN_OUT:
    if (arg < 32)
    {
        Reg\[GPIO_DIR_OFFSET\] |=1u < < arg;
    }
    break;
case GPIO_SET_PIN_IN:
    if (arg < 32)
    {
        Reg\[GPIO_DIR_OFFSET\] & = ~(1u < < arg);
    }
    break;
```

3. GPIO_SET_ALL_PIN 和 GPIO_CLR_ALL_PIN

GPIO_SET_ALL_PIN 命令操作的寄存器与 GPIO_SET_PIN 操作的寄存器是一样的，而 GPIO_CLR_ALL_PIN 操作的寄存器与 GPIO_CLR_PIN 操作的寄存器是一样的。与 GPIO_SET_PIN 和 GPIO_CLR_PIN 命令不同，这两条命令的第 3 个参数（arg）直接是寄存器的值，因此这两条命令的实现代码更简单（参见程序清单 6.24）：仅一条语句。这里不再分析这部分代码。

程序清单 **6.24　GPIO_SET_ALL_PIN 和 GPIO_CLR_ALL_PIN**

```
case GPIO_SET_ALL_PIN:                                    (1)
    Reg\[GPIO_SET_OFFSET\] = arg;                         (2)
    break;                                                (3)
case GPIO_CLR_ALL_PIN:                                    (4)
    Reg\[GPIO_CLR_OFFSET\] = arg;                         (5)
    break;                                                (6)
```


更多高清PDF图书　www.gqpdf.com

4. GPIO_SET_MULTI_PIN_OUT 和 GPIO_SET_MULTI_PIN_IN

GPIO_SET_MULTI_PIN_OUT、GPIO_SET_MULTI_PIN_IN、GPIO_SET_PIN_OUT 和 GPIO_SET_PIN_IN 四条命令操作相同的寄存器，不过 GPIO_SET_PIN_OUT 和 GPIO_SET_PIN_IN 对单个位置 1 或清 0，而 GPIO_SET_MULTI_PIN_OUT 和 GPIO_SET_MULTI_PIN_IN 是对多个位置 1 或清 0。根据这个寄存器的特性，这两条命令需要按位“与”或按位“或”操作，代码见程序清单 6.25。

程序清单 **6.25　GPIO_SET_MULTI_PIN_OUT 和 GPIO_SET_MULTI_PIN_IN**

```
case GPIO_SET_MULTI_PIN_OUT:
    Reg\[GPIO_DIR_OFFSET\] |= arg;
    break;
case GPIO_SET_MULTI_PIN_IN:
    Reg\[GPIO_DIR_OFFSET\] &= ~arg;
    break;
```

5. GPIO_READ_PORT 和 GPIO_READ_PIN

除了操作的寄存器不同外，这 2 条命令的实现是一样的。其中 GPIO_READ_PORT 操作寄存器 IO0SET、IO1SET、IO2SET 和 IO3SET 中之一，而 GPIO_READ_PIN 操作 IO0PIN、IO1PIN、IO2PIN 和 IO3PIN 中之一。至于具体操作哪个寄存器，就由次设备号决定。这两个命令的实现代码见程序清单 6.26，这个代码已把出错处理代码删除，仅留下操作的核心代码。程序清单 6.26 会在内核空间与用户空间之间交换数据。关于内核空间与用户空间之间交换数据在 5.2.6 小节有介绍，读者可参考这一小节。这几条命令的第 3 个参数（arg）为一个整数，指明操作的是哪一位 I/O 口。对于 LPC2200 来说，这些寄存器有一个特点：第 0 位对应 GPIO 的第 0 位，第 1 位对应 GPIO 的第 1 位，以此类推。如果某一位为 0，则代表对应位为高电平；否则为低电平。经过简化的代码还是比较简单的，阅读过 5.2.6 小节后应当很容易读懂。

程序清单 **6.26　GPIO_READ_PORT 和 GPIO_READ_PIN**

```
case GPIO_READ_PORT:
    get_user(temp2, (u32 *)arg);                                    (1)
    temp1 = Reg\[GPIO_SET_OFFSET\] & (1u << temp2);                 (2)
    if (temp1 !=0)                                                  (3)
    {
        temp1 =GPIO_PIN_HIGH;                                       (4)
    }
    else
    {
```

```
        temp1 = GPIO_PIN_LOW;                                          (5)
    }
    put_user(temp1, (u32 *)arg);                                       (6)
    break;                                                             (7)
case GPIO_READ_PIN:                                                    (8)
    get_user(temp2, (u32 *)arg);                                       (9)
    temp1 = Reg\[GPIO_PIN_OFFSET\] & (1u << temp2);                    (10)
    if (temp1 != 0)                                                    (11)
    {
        temp1 = GPIO_PIN_HIGH;                                         (12)
    }
    else
    {
        temp1 = GPIO_PIN_LOW;                                          (13)
    }
    put_user(temp1, (u32 *)arg);                                       (14)
    break;                                                             (15)
```

6. GPIO_READ_ALL_PORT 和 GPIO_READ_ALL_PIN

GPIO_READ_ALL_PORT 命令与 GPIO_READ_PORT 命令操作的是同一个寄存器，只不过 GPIO_READ_PORT 返回指定位的内容，而 GPIO_READ_ALL_PORT 返回整个寄存器。GPIO_READ_ALL_PIN 命令和 GPIO_READ_PIN 命令之间具有同样的关系。因此这两条命令的实现代码（参见程序清单 6.27）比 GPIO_READ_PORT 和 GPIO_READ_PIN 命令简单。程序清单 6.27 同样经过简化，删掉了出错处理代码。

程序清单 **6.27　GPIO_READ_ALL_PORT 和 GPIO_READ_ALL_PIN**

```
case GPIO_READ_ALL_PORT:                                               (1)
    temp1 = Reg\[GPIO_SET_OFFSET\];                                    (2)
    put_user(temp1, (u32 *)arg);                                       (3)
    break;                                                             (4)
case GPIO_READ_ALL_PIN:                                                (5)
    temp1 = Reg\[GPIO_PIN_OFFSET\];                                    (6)
    put_user(temp1, (u32 *)arg);                                       (7)
    break;                                                             (8)
```

6.3.6　其他说明

1. 未实现的方法

本驱动未实现 write()、read() 和 llseek() 方法。

更多高清PDF图书　www.gqpdf.com

2. 修改版权及作者声明

在内核模块中包含了本内核模块使用的许可协议。内核模块默认是遵循 GPL 许可协议，但若内核模块的作者不能或不愿遵循 GPL，则可不遵循 GPL 许可协议，这通过宏 MODULE_LICENSE() 实现。这个宏的参数为一个字符串，对于 Linux（μClinux）2.4.x 来说，这个字符串可从表 6.3 中选择。

表 6.3　可选的许可协议

字符串（不包含引号）	许可协议	备　注
"GPL"	GNU 公共许可协议 v2 及以上版本	
"GPL v2"	GNU 公共许可协议 v2 版本	
"GPL and additional rights"	GNU 公共许可协议 v2 版本，但具有更多权力	
"Dual BSD/GPL"	BSD 和 GPL 双重许可协议	
"Dual MPL/GPL"	MPL 和 GPL 双重许可协议	
"Proprietary"	私有协议	非自由协议，协议由作者确定

类似地，Linux（μClinux）提供宏 MODULE_DESCRIPTION()、MODULE_SUPPRTED_DEVICE() 和 MODULE_AUTHOR() 分别用于声明内核模块的描述、支持的器件和作者。这些宏的参数均为一个任意的字符串。

关于本驱动的这部分代码见程序清单 6.28，代码比较简单，这里不再说明。

程序清单 **6.28**　版权声明等

```
MODULE_LICENSE("Proprietary");
MODULE_DESCRIPTION("Guangzou ZLG-MCU Development Co.,LTD.\\ngraduate school \\
nhttp://www.zlgmcu.com");
MODULE_SUPPORTED_DEVICE("uClinux2.4.x LPC2200 GPIO");
MODULE_AUTHOR("chenmingji");
```

本章小结

本章以 LPC2200 系列的 GPIO 为例介绍了实际的 Linux（μClinux）字符设备驱动程序的相关知识。首先介绍该驱动程序的使用方法，然后对源码进行分析。

相对于第 5 章，本章主要增加了以下知识：

- 让 insmod 命令可改变主设备号的代码（参考 6.3.1 小节）。
- 修改版权及作者声明（参考 6.3.6 小节）。

高清PDF藏 gqpdf.com

第7章　PWM（字符设备）驱动程序编写范例

LPC2200 系列 ARM 微控制器具有 PWM（脉宽调制器）功能。LPC2200 的 PWM 具有以下特性：

- 带可编程 32 位预分频器的 32 位定时器/计数器。
- 7 个匹配寄存器，可实现 6 个单边沿控制或 3 个双边沿控制 PWM 输出，或这两种类型的混合输出。
- 支持单边沿控制和双边沿控制的 PWM 输出。
- 脉冲周期和宽度可以是任何的定时器计数值。这样可实现灵活的分辨率和重复速率的设定。所有 PWM 输出都以相同的重复率发生。
- 匹配寄存器更新与脉冲输出同步，防止产生错误的脉冲。
- 如果不使能 PWM 模式，则可作为一个标准定时器。

关于 LPC2200 的 PWM 的具体介绍请参见参考文献[3]。

本章介绍的驱动程序就是驱动 LPC2200 系列的 PWM，但固定设置为单边沿输出。可控制任意一路 PWM 输出占空比，

7.1　在应用程序中使用 PWM 驱动程序

关于字符设备驱动程序的一般使用方法请参考 5.3 节，本节仅介绍使用 PWM 驱动程序需要特殊注意的地方。

7.1.1　建立设备文件

PWM 驱动程序默认的主设备号为 125，没有使用从设备号。因此，需要建立 1 个设备文件，其主设备号为 125，从设备号可任意选择，文件名可定为 PWM。建立设备文件的命令请参考程序清单 7.1。

更多高清PDF图书　www.gqpdf.com

程序清单 **7.1**　建立设备文件

```
mknod /dev/PWM c 125 0
```

7.1.2　可使用的函数

因为本驱动程序仅实现了 open()、release() 和 ioctl() 方法，所以本驱动程序不能使用标准 C 的库函数来访问，只能使用内核提供给应用程序的接口函数来访问。并且，仅部分内核接口函数都可使用，其具体情况见表 7.1，其具体使用方法见 5.3.3 小节。

表 7.1　可使用的内核接口函数

函数名称	函数功能	函数原型	备　注
open()	打开文件	int open(const char * pathname, int flags);	在使用 PWM 前调用
close()	关闭文件	int close(int fd);	在使用完 GPIO 后调用
ioctl()	I/O 控制	int ioctl (int fd, unsigned long int cmd, ...);	实际控制 GPIO

7.1.3　提供给应用程序的头文件

函数 ioctl() 的第二个参数 cmd 为驱动程序的特殊命令，其值应由驱动程序提供。本驱动程序提供了一个头文件 pwm.h 定义这些命令。文件 pwm.h 的内容见程序清单 7.2。其中，程序清单 7.2（2）~（21）定义了 20 个宏，对应 20 条命令。关于各具体命令的说明，请参考表 7.2。

程序清单 **7.2**　**pwm.h**

```
#ifndef __PWM_H
#define __PWM_H
#include <linux/ioctl.h>

#define PWM_IOC_MAGIC          0xd1                                    (1)

#define PWM_SET_CYC            _IO(PWM_IOC_MAGIC,    0)                (2)
#define PWM_GET_RUN_TIME       _IOWR(PWM_IOC_MAGIC,  1, uint32)        (3)

#define PWM_1_ENABLE           _IO(PWM_IOC_MAGIC,    2)                (4)
#define PWM_1_DISABLE          _IO(PWM_IOC_MAGIC,    3)                (5)
#define PWM_1_SET_DUTY         _IO(PWM_IOC_MAGIC,    4)                (6)
```

高清PDF藏 gqpdf.com

```
#define PWM_2_ENABLE          _IO(PWM_IOC_MAGIC,   5)          (7)
#define PWM_2_DISABLE         _IO(PWM_IOC_MAGIC,   6)          (8)
#define PWM_2_SET_DUTY        _IO(PWM_IOC_MAGIC,   7)          (9)

#define PWM_3_ENABLE          _IO(PWM_IOC_MAGIC,   8)          (10)
#define PWM_3_DISABLE         _IO(PWM_IOC_MAGIC,   9)          (11)
#define PWM_3_SET_DUTY        _IO(PWM_IOC_MAGIC,   10)         (12)

#define PWM_4_ENABLE          _IO(PWM_IOC_MAGIC,   11)         (13)
#define PWM_4_DISABLE         _IO(PWM_IOC_MAGIC,   12)         (13)
#define PWM_4_SET_DUTY        _IO(PWM_IOC_MAGIC,   13)         (15)

#define PWM_5_ENABLE          _IO(PWM_IOC_MAGIC,   14)         (16)
#define PWM_5_DISABLE         _IO(PWM_IOC_MAGIC,   15)         (17)
#define PWM_5_SET_DUTY        _IO(PWM_IOC_MAGIC,   16)         (18)

#define PWM_6_ENABLE          _IO(PWM_IOC_MAGIC,   17)         (19)
#define PWM_6_DISABLE         _IO(PWM_IOC_MAGIC,   18)         (20)
#define PWM_6_SET_DUTY        _IO(PWM_IOC_MAGIC,   19)         (21)

#define PWM_MAXNR              20                              (22)
#endif
```

表 7.2　pwm 命令

命　令	功　能	备　注
PWM_SET_CYC	设置 PWM 频率	参考 7.1.4 小节
PWM_GET_RUN_TIME	获得 PWM 运行时间	
PWM_1_ENABLE	允许第 1 路 PWM 输出	参考 7.1.5 小节
PWM_2_ENABLE	允许第 2 路 PWM 输出	
PWM_3_ENABLE	允许第 3 路 PWM 输出	
PWM_4_ENABLE	允许第 4 路 PWM 输出	
PWM_5_ENABLE	允许第 5 路 PWM 输出	
PWM_6_ENABLE	允许第 6 路 PWM 输出	

更多高清PDF图书　www.gqpdf.com

续表 7.2

命　令	功　能	备　注
PWM_1_DISABLE	禁止第 1 路 PWM 输出	参考 7.1.6 小节
PWM_2_DISABLE	禁止第 2 路 PWM 输出	
PWM_3_DISABLE	禁止第 3 路 PWM 输出	
PWM_4_DISABLE	禁止第 4 路 PWM 输出	
PWM_5_DISABLE	禁止第 5 路 PWM 输出	
PWM_6_DISABLE	禁止第 6 路 PWM 输出	
PWM_1_SET_DUTY	设置第 1 路 PWM 高脉冲宽度	参考 7.1.7 小节
PWM_2_SET_DUTY	设置第 2 路 PWM 高脉冲宽度	
PWM_3_SET_DUTY	设置第 3 路 PWM 高脉冲宽度	
PWM_4_SET_DUTY	设置第 4 路 PWM 高脉冲宽度	
PWM_5_SET_DUTY	设置第 5 路 PWM 高脉冲宽度	
PWM_6_SET_DUTY	设置第 6 路 PWM 高脉冲宽度	

7.1.4　PWM 总体控制

LPC2200 的所有 PWM 具有同样的频率，在使用 PWM 前必须设置这个频率。同时，本驱动还提供一条命令获取 PWM 的运行时间，它以上面提到的频率的倒数为单位。下面分别介绍。

1. PWM_SET_CYC

- 功能：设置 PWM 频率。实际上是以 VPB 总线周期（参见参考文献[3]）为单位设置 PWM 的周期。不过，频率是周期的倒数，所以也就设置了频率。
  注意：当第一次打开设备文件时，PWM 频率设置为可能的最高值。
- ioctl()函数的第 3 个参数：32 位整型参数，以 VPB 总线周期为单位的周期时间。
- 使用示例：当 VPB 总线频率为 11 059 200 Hz 时，程序清单 7.3 将 PWM 频率设置为 1 000 Hz。

程序清单 7.3　PWM_SET_CYC 命令使用示例

```
int fd;
......
fd = open("/dev/PWM", O_RDONLY);
......
ioctl(fd, PWM_SET_CYC, (11059200 + 500) /1000);
......
```

2. PWM_GET_RUN_TIME

- 功能：获得 PWM 的运行时间。实际返回的是 PWM 周期的计数值。因此，如果改变了 PWM 频率，将造成不能准确地计算运行时间。因此一般用它来测量相对时间。
- ioctl()函数的第 3 个参数：一个 32 位整型指针，周期数保存到此指针指向的变量中。
- 使用示例：见程序清单 7.4。

程序清单 **7.4　PWM_GET_RUN_TIME** 命令使用示例

```
int fd;
u32 sum;
......
fd = open("/dev/PWM", O_RDONLY);
......
ioctl(fd, PWM_GET_RUN_TIME, &sum);
......
```

7.1.5　允许某路 PWM 输出

LPC2200 系列 ARM 微控制器有 6 路 PWM 输出，但在打开设备文件时它们都是禁止输出的，需要用相应的命令让它们输出。这些命令就是 PWM_1_ENABLE、PWM_2_ENABLE、PWM_3_ENABLE、PWM_4_ENABLE、PWM_5_ENABLE 和 PWM_6_ENABLE，命令 PWM_1_ENABLE 允许第 1 路 PWM 输出，命令 PWM_2_ENABLE 允许第 2 路 PWM 输出，以此类推。这些命令使用很简单，它们不需要 ioctl()函数的第 3 个参数。其使用示例见程序清单 7.5。

程序清单 **7.5　PWM_x_ENABLE** 命令使用示例

```
int fd;
u32 sum;
......
fd = open("/dev/PWM", O_RDONLY);
......
ioctl(fd, PWM_6_ENABLE);
......
```

7.1.6　禁止某路 PWM 输出

当某路 PWM 不需要输出时可禁止它，这通过命令 PWM_1_ DISABLE、PWM_2_ DISABLE、PWM_3_ DISABLE、PWM_4_ DISABLE、PWM_5_ DISABLE 和 PWM_6_ DISABLE 实现。命令 PWM_1_ DISABLE 禁止第 1 路 PWM 输出，命令 PWM_2_ DISABLE 禁止第 2 路 PWM 输出，以

更多高清PDF图书　www.gqpdf.com

此类推。这些命令使用很简单，它们不需要 ioctl() 函数的第 3 个参数。其使用示例见程序清单 7.6。

程序清单 **7.6　PWM_x_DISABLE** 命令使用示例

```
int fd;
u32 sum;
......
fd = open("/dev/PWM", O_RDONLY);
......
ioctl(fd, PWM_6_DISABLE);
.....
```

7.1.7　设置某路 PWM 高电平宽度

PWM 最重要的功能是控制 PWM 输出的频率和占空比。PWM 的输出频率可通过命令 PWM_SET_CYC 实现，而另一组命令可分别设置每一路 PWM 的高电平时间。知道了频率（也就知道了周期）和高电平时间，就可计算占空比。这一组命令为 PWM_1_SET_DUTY、PWM_2_SET_DUTY、PWM_3_SET_DUTY、PWM_4_SET_DUTY、PWM_5_SET_DUTY 和 PWM_6_SET_DUTY，其中命令 PWM_1_SET_DUTY 对应第 1 路，命令 PWM_2_SET_DUTY 对应第 2 路，以此类推。下面详细介绍。

- 功能：设置 PWM 高电平时间。实际上是以 VPB 总线周期（参见参考文献[3]）为单位设置 PWM 的高电平时间。
- ioctl() 函数的第 3 个参数：32 位整型参数，以 VPB 总线周期为单位的高电平时间。
- 使用示例：当 VPB 总线频率为 11 059 200 Hz 时，程序清单 7.7 将第 6 路 PWM 的占空比设置为 50%。

程序清单 **7.7　PWM_x_SET_DUTY** 命令使用示例

```
int fd;
u32 sum;
......
fd = open("/dev/PWM", O_RDONLY);
......
ioctl(fd, PWM_SET_CYC, (11059200 + 500) /1000);
ioctl(fd, PWM_6_SET_DUTY, (11059200 + 500) /2000);
......
```

7.2 编　译

当建立相关的开发环境和获取驱动程序的源代码后，只需建立（或修改）Makefile 文件，通过 make 命令即可编译驱动程序。关于建立开发环境不是本书的任务，请读者自行参考相关文档。本书介绍的源代码可从广州致远电子有限公司出品的相关产品中获得，或根据本书自行输入。下面介绍本驱动所用的 Makefile 文件（参见程序清单 7.8）。

程序清单 **7.8**　**Makefile** 文件

```
EXEC = pwm
OBJS = pwm.o
SRC = pwm.c

INCLUDE = /work/uClinux-dist/linux-2.4.x/include

CC = arm-elf-gcc
LD = arm-elf-ld
MODCFLAGS = -D__KERNEL__ -I $(INCLUDE) -Wall -O2 -fno-strict-aliasing -fno-common -
pipe -fno-builtin -D__linux__ -g -DNO_MM -mapcs-32 -march = armv4 -mtune = arm7tdmi -
mshort-load-bytes -msoft-float -nostdinc -iwithprefix include
LDFLAGS = -m armelf -r

all: $(EXEC)

$(EXEC): $(OBJS)
    $(LD) $(LDFLAGS) -o $@  $(OBJS)

%.o:%.c
    $(CC) $(MODCFLAGS) -mapcs -c $< -o $@

clean:
    -rm -f $(EXEC) *.elf *.gdb *.o
```

与程序清单4.2不同的部分

根据实际情况修改的部分

可以看出，程序清单 7.8 与程序清单 4.2 十分相似，仅头 3 句不同。实际上，程序清单 7.8 是根据程序清单 4.2 修改而来的。因此，关于 Makefile 的详细介绍请参考 4.3 节。

更多高清PDF图书　www.gqpdf.com

7.3 源码分析

事实上，第5章已介绍了编写字符驱动程序一般知识，而本驱动程序也是在那个驱动程序的基础上修改而来。因此，应先阅读第5章，否则不易理解本节介绍的内容。

另外，本驱动使用了中断，所以本节先介绍 Linux（μClinux）中断服务程序的编写方法。

7.3.1 为驱动程序增加中断服务程序

Linux（μClinux）内核具有中断管理功能，因此中断服务程序必须按照其规范使用。Linux（μClinux）在使用中断服务程序之前，需要向内核注册中断服务程序；不再使用中断服务程序时，需要注销中断服务程序。通常，中断服务程序分成上半部分和下半部分。其中上半部分处理紧急的任务，而对时间要求不严的任务由下半部分处理。本驱动所用的中断服务程序没有用到下半部分，本章不对下半部分进行讨论。关于 Linux（μClinux）中断服务程序更详尽的信息请参见参考文献[2]和[4]。

1. 注册中断服务程序（申请中断通道）

中断信号线是宝贵且非常有限的资源。内核维护了一个类似于 I/O 端口注册表的中断信号线的注册表。一个内核模块可申请一个中断通道，并且处理完后还可释放它。注册中断服务程序（申请中断通道）通过调用函数 request_irq（）实现。中断处理程序可在驱动程序初始化或设备第一次打开时安装，但在设备第一次打开时安装更合理一些。函数 request_irq（）的原型见程序清单7.9。

程序清单 **7.9** 函数 **request_irq**（）原型

```
int request_irq(unsigned int irq, void (*handler)(int, void*, struct pt_regs
*),
                unsigned long flags, const char *device, void *dev_id);
```

通常申请中断的函数返回值为0时表示成功，或者返回一个负的错误码。函数返回 -EBUSY，表示另一个设备驱动程序已使用了要申请的中断信号线（不常见）。函数参数定义如下：

unsigned int irq

该参数为中断号。有时从 Linux（μClinux）中断号到硬件中断号的映射并不是一对一的。这里，传递给内核函数的参数是 Linux（μClinux）的中断号而不是硬件中断号。但对于 LPC2200 来说，这两者是对应的，具体可参考文件 <linux-2.4.x/include/asm-armnommu/arch-lpc/irqs.h>。

void（*handler）（int, void *, struct pt_regs *）

指向要安装的中断处理函数的指针。

unsigned long flags

这是一个与中断管理有关的各种选项的字节掩码。

const char * device

传递给内核的字符串，在/proc/interrupts 中用于显示中断的拥有者。

void * dev_id

传递给中断服务程序的第 2 个参数。事实上，这个指针用于共享的中断信号线。内核用它区分统一中断通道的各中断服务程序。设备驱动程序可自由地使用 dev_id。

在 flags 中可设置的位有：

SA_INTERRUPT

如果设置该位，则指示这是一个“快速”中断处理程序；如果清除该位，则它就是一个“慢速”中断处理程序。但 2.4.x 已不再区分快速中断处理程序和慢速中断处理程序。

SA_SHIRQ

该位表明中断可在设备间共享。但除了外部中断，LPC2200 的所有中断都是独占的。

SA_SAMPLE_RANDOM

该位表明产生的中断对/dev/random 和/dev/urandom 设备要使用的熵池（entropy pool）有贡献。读这些设备返回真正的随机数，它们用来帮助应用软件选取用于加密的安全钥匙。这些随机数是从一个熵池中取得的，各种随机事件都会对系统的熵池（无序度）有贡献。如果中断是随机产生的，则应设置这个标志位；如果中断是可预测的，则不应设置这个标志位。

2. 注销中断服务程序（归还中断通道）

注销中断服务程序（归还中断通道）通过调用函数 free_irq()实现。前面提到，通过中断通道号和 dev_id 可确定一个中断服务程序，所以注销中断服务程序（归还中断通道）的函数需要这两个参数。注销中断服务程序的函数见程序清单 7.10。函数 free_irq()的所有参数前面已经介绍，这里不再说明。

程序清单 **7.10**　函数 **free_irq**()原型

```
void free_irq(unsigned int irq, void *dev_id);
```

3. 简单的中断服务程序的编写

简单的中断服务程序的示意代码见程序清单 7.11。因为内核已经做了很多工作，所以中断服务程序很简单，必需的工作仅仅是清除中断源（程序清单 7.11(1)），它与具体的设备有关。下面着重介绍中断服务程序的参数。

unsigned int irq

更多高清PDF图书 www.gqpdf.com

该参数为中断号。一个中断服务程序需要处理多个中断时它很有用。

void * dev_id

实际上是传递 request_irq () 的最后一个参数。中断服务程序可自由地使用它。

struct pt_regs * regs

很少使用。它存放着处理器进入中断代码前一个处理器上下文的快照。

程序清单 **7.11**　简单的中断服务程序示意代码

```
void test_irq_handle(int irq, void * dev_id, struct pt_regs * regs)
{
    清除中断源;                                          (1)
    其他必需的工作;                                      (2)
}
```

（“清除中断源;”旁标注：必需的代码）

7.3.2　关于临界区

有中断就有临界区。Linux（μClinux）内核有多种方式控制临界区，这主要有“锁”、“信号量”和“开关中断”，它们每一个都是一簇函数。限于篇幅，在此不能一一介绍，感兴趣的读者可参见参考文献[4]。本小节仅介绍在嵌入式系统最常用的开关中断的两个函数，其他仅在用到时才进行说明。

μClinux 一般不会在对称多 CPU 上运行（μClinux 本来就是面向低端的，多 CPU 不太现实），所以只需使用开关本地中断的函数，而且这些函数最好可嵌套调用。符合这些条件的开关中断函数为 local_irq_save () 和 local_irq_restore ()。其实，local_irq_save () 和 local_irq_restore () 是宏而不是函数，它们需要配对使用。其使用示意代码见程序清单 7.12，仔细阅读注释，应该知道其使用方法。

程序清单 **7.12**　**local_irq_save** () 和 **local_irq_restore** () 使用示例

```
unsigned long flag;                                      (1)
......
local_irq_save(flag);                                    (2)
......                                                   (3)
local_irq_restore(flag);                                 (4)
......                                                   (5)
```

（“unsigned long flag;”旁标注：辅助变量，不能用于其他目的）

（“local_irq_save(flag);”旁标注：这里中断肯定是关闭的）

（“local_irq_restore(flag);”旁标注：如果(1)句处中断是关闭的，则此时中断是关闭的；否则是打开的）

高清PDF藏 gqpdf.com

注意：在程序清单 7.12(3)、(5)之间 flag 变量不能被使用。如果在此期间还需要调用 local_irq_save()和 local_irq_restore()，则需要使用其他变量。

7.3.3 初始化函数

本驱动程序的初始化函数代码见程序清单 7.13。实际上，程序清单 7.13 与程序清单 6.16 是一样的，关于这部分代码请参考 6.3.1 小节。

程序清单 **7.13** **PWM** 驱动程序初始化函数代码

```
int pwm_init(void)
{
    int  result;

    result = register_chrdev(MAJOR_NR,  DEVICE_NAME,  &pwm_fops);
    if (result < 0)
    {
        printk(KERN_ERR DEVICE_NAME ": Unable to get major % d\\n", MAJOR_NR );
        return(result);
    }

    if (MAJOR_NR == 0)
    {
        MAJOR_NR = result; /* dynamic */
    }
    printk(KERN_INFO DEVICE_NAME ": init OK\\n");
    return(0);
}
```

7.3.4 清除函数

本驱动程序的清除函数见程序清单 7.14，它与程序清单 5.4 一样，这里不再详细介绍。

程序清单 **7.14** **PWM** 驱动程序清除函数

```
void pwm_cleanup(void)
{
    unregister_chrdev(MAJOR_NR, DEVICE_NAME);
}
```


更多高清PDF图书 www.gqpdf.com

7.3.5 open()方法及中断处理

open()方法会访问 I/O 端口。关于访问 I/O 端口的方法请参考 5.2.7 小节。

open()方法的代码请参考程序清单 7.15。其实这个方法是按照程序清单 5.7 编写的，只是因为本驱动没有使用从设备号，程序清单 5.7(1)、(2)没有对应语句而已。下面重点介绍程序清单 5.7(4)、(5)对应的语句。实际上在程序清单 7.15 中没有与程序清单 5.7(5)对应的语句，这是因为本驱动不需要使用 file 结构体的 private_data 成员。

程序清单 **7.15** **PWM** 驱动程序 **open**()方法

```
static int pwm_open(struct inode *inode, struct file *filp)
{
    unsigned long flag;

    if (usage == 0)                                                    (1)
    {
        request_irq(IRQ_PWM, pwm_irq_handle, SA_INTERRUPT, "my" DEVICE_NAME,
NULL);                                                                 (2)
        local_irq_save(flag);                                          (3)
        PinSel0Save = inl(PINSEL0) & 0xfc00f;                          (4)
        PinSel1Save = inl(PINSEL1) & (0x03 << 10);                     (5)
        outl(0x0f | (0x07 << 8), PWMIR);                               (6)
        outl(0x00, PWMPR);                                             (7)
        outl(10000, PWMMR0);                                           (8)
        outl(0x03, PWMMCR);                                            (9)
        outl(3 | (1 << 3), PWMTCR);                                    (10)
        outl(1 | (1 << 3), PWMTCR);                                    (11)
        outl(inl(PWMLER) |1, PWMLER);                                  (12)
        RunTime = 0;                                                   (13)
        local_irq_restore(flag);                                       (14)
    }
    usage ++;                                                          (15)
    MOD_INC_USE_COUNT;                                                 (16)
    return 0;            /* success */                                 (17)
}
```

usage是使用计数器，它是全局变量

PinSel0Save 和 PinSel1Save 用于恢复硬件连接模块的对应部分

RunTime是全局变量，在中断服务程序中使用

程序清单 5.7(4)对应于程序清单 7.15(3) ~ (14)。程序清单 7.15(3)为向内核注册一个中

断服务程序 pwm_irq_handle()。关于这方面的信息请参考7.3.1 小节。因为接着就实际操作硬件了，也就是进入临界区了，需要先关中断（程序清单 7.15 (3)），后开中断（程序清单 7.15 (14)）。关于临界区方面的信息请参考 7.3.2 小节。本书不打算详细说明程序清单 7.15 (6) ~ (12)，它对 LPC2200 的 PWM 进行了基本的初始化。这部分完全与硬件相关，且无技巧，本书没有必要花大量篇幅介绍它。如果想了解它们的含义请参见参考文献[3]。下面介绍程序清单7.15 (4)、(5)。LPC2200 的 I/O 引脚大多数是复用的，应用程序可通过引脚连接模块来选择其作为某一个特定的用途。LPC2200 的 PWM 输出也是与其他功能复用的。程序清单 7.15 (4)、(5) 是用来保存 PWM 所用到的引脚（PWM 被打开前）的连接配置信息，这样，当 PWM 被完全关闭时可恢复这些引脚的配置。于是，这些引脚就可分时复用了。加上这个功能可提高程序的健壮性。

程序清单 7.15 (3) 向内核注册一个中断服务程序，其代码见程序清单 7.16。中断服务程序是按照程序清单 7.11 编写的，其中程序清单 7.16 (1) 对应于程序清单 7.11 (1)，它用于清除 PWM 中断源。其后的程序清单 7.16 (2) 是给中断次数计数的。因为中断的周期是确定的，所以这个计数值代表了时间的相对值。

程序清单 **7.16**　**PWM** 驱动程序的中断服务程序

```
static void pwm_irq_handle(int irq, void *dev_id, struct pt_regs *regs)
{
    outl(0x0f | (0x07 << 8), PWMIR);                                    (1)
    RunTime++;                                                          (2)
}
```

7.3.6　release() 方法

release() 方法会访问 I/O 端口。关于访问 I/O 端口的方法请参考 5.2.7 小节。

release() 方法的代码请参考程序清单 7.17。其实这个方法是按照程序清单 5.8 编写的，只是因为本驱动没有使用从设备号，程序清单 5.8 (1)、(2) 没有对应语句而已。下面重点介绍与程序清单 5.8 (6) 对应的语句。

程序清单 **7.17**　**release()** 方法代码

更多高清PDF图书　www.gqpdf.com

```
static int pwm_release(struct inode * inode, struct file * filp)
{
    unsigned long flag;

    MOD_DEC_USE_COUNT;                                          (1)
    usage--;                                                    (2)
if (usage == 0)                                                 (3)
```

usage是使用计数器，它是全局变量

高清PDF藏 gqpdf.com

```
    {
        local_irq_save(flag);                                              (4)
        outl(0, PWMTCR);                                                   (5)
        outl(0, PWMPCR);                                                   (6)
        outl(PinSel0Save | (inl(PINSEL0) & (~0xfc00f)), PINSEL0);          (7)
        outl(PinSel1Save | (inl(PINSEL1) & (~(0x3 << 10))), PINSEL1);      (8)
        local_irq_restore(flag);                                           (9)
        free_irq(IRQ_PWM, NULL);                                           (10)
    }
    return 0;                                                              (11)
}
```

程序清单 5.8(6)对应于程序清单 7.17(4)~(10)。因为接着就实际操作硬件了,也就是进入临界区了,需要先关中断(程序清单 7.17(4)),后开中断(程序清单 7.17(9))。关于临界区方面的信息请参考 7.3.2 小节。本书不打算详细说明程序清单 7.17(5)、(6),它对 LPC2200 的 PWM 进行关闭处理。这部分完全与硬件相关,且无技巧,本书没有必要花大量篇幅介绍它。如果想了解它们的含义请参见参考文献[3]。下面介绍程序清单 7.17(7)、(8)。它就是程序清单 7.15(4)、(5)的逆操作,而程序清单 7.17(10)就是注销已注册的中断服务程序。

注意: open()方法和 release()方法的操作顺序是先使用的资源后注销。

7.3.7 ioctl()方法

ioctl()方法会访问 I/O 端口。关于访问 I/O 端口的方法请参考 5.2.7 小节。

ioctl()方法的代码见程序清单 7.18。程序清单 7.18 虽然有 100 多条语句,但还是按照程序清单 5.16 编写的。而且,因为本驱动没有使用从设备号,所以没有与程序清单 5.16(1)、(2)对应的语句。程序之所以如此长,是因为 6 路 PWM 各自有独特的地方,不易把相同功能的代码合并起来。

程序清单 **7.18** **ioctl()方法代码**

```
static int pwm_ioctl(struct inode *inode, struct file *filp,
                     unsigned int cmd, unsigned long arg)
{
    if (_IOC_TYPE(cmd) != PWM_IOC_MAGIC)                                   (1)
    {
        return -ENOTTY;                                                    (2)
    }
    if (_IOC_NR(cmd) >= PWM_MAXNR)                                         (3)
    {
```


更多高清PDF图书 www.gqpdf.com

```
        return -ENOTTY;                                                    (4)
    }

    switch(cmd)                                                            (5)
    {
        case PWM_SET_CYC:                                                  (6)
            outl(arg ,PWMMR0);                                             (7)
            outl(inl(PWMLER) |1, PWMLER);                                  (8)
            break;                                                         (9)
        case PWM_GET_RUN_TIME:                                             (10)
            if (! access_ok(VERIFY_WRITE, (void *)arg, _IOC_SIZE(cmd)))    (11)
            {
                return -EFAULT;                                            (12)
            }
            put_user(RunTime, (u32 *)arg);                                 (13)
            break;                                                         (14)

        case PWM_1_ENABLE:                                                 (15)
            outl(inl(PWMLER) |(1 < < 9), PWMPCR);                          (16)
            outl(0x02 |(inl(PINSEL0) & ~0x03), PINSEL0);                   (17)
            break;                                                         (18)
        case PWM_1_DISABLE:                                                (19)
            outl(inl(PWMPCR) & ~(1 < < 9), PWMPCR);                        (20)
            outl((PinSel0Save & 0x03) |(inl(PINSEL0) & ~0x03), PINSEL0);
                                                                           (21)
            break;                                                         (22)
        case PWM_1_SET_DUTY:                                               (23)
            if (arg < inl(PWMMR0))                                         (24)
            {
                outl(arg, PWMMR1);                                         (25)
                outl(inl(PWMLER) |(1 < < 1), PWMLER);                      (26)
            }
            break;                                                         (27)

        case PWM_2_ENABLE:                                                 (28)
            outl(inl(PWMPCR) & ~(1 < < 2), PWMPCR);                        (29)
            outl(inl(PWMPCR) |(1 < < 10), PWMPCR);                         (30)
            outl((0x02 < < 14) |(inl(PINSEL0) & (~(0x03 < < 14))), PINSEL0);
                                                                           (31)
```

```
            break;                                                          (32)
        case PWM_2_DISABLE:                                                 (33)
            outl(inl(PWMPCR) & ~(1 < < 10), PWMPCR);                        (34)
            outl((PinSel0Save & (0x03 < < 14)) | (inl(PINSEL0) & (~(0x03 < <
                14))),
                PINSEL0);                                                   (35)
            break;                                                          (36)
        case PWM_2_SET_DUTY:                                                (37)
            if (arg < inl(PWMMR0))                                          (38)
            {
                outl(arg, PWMMR2);                                          (39)
                outl(inl(PWMLER) | (1 < < 2), PWMLER);                      (40)
            }
            break;                                                          (41)

        case PWM_3_ENABLE:                                                  (42)
            outl(inl(PWMPCR) & ~(1 < < 3), PWMPCR);                         (43)
            outl(inl(PWMPCR) | (1 < < 11), PWMPCR);                         (44)
            outl((0x02 < < 2) | (inl(PINSEL0) & (~(0x03 < < 2))), PINSEL0);
                                                                            (45)
            break;                                                          (46)
        case PWM_3_DISABLE:                                                 (47)
            outl(inl(PWMPCR) & ~(1 < < 11), PWMPCR);                        (48)
            outl((PinSel0Save & (0x03 < < 2)) | (inl(PINSEL0) & (~(0x03 < <
                2))),
                PINSEL0);                                                   (49)
            break;                                                          (50)
        case PWM_3_SET_DUTY:                                                (51)
            if (arg < inl(PWMMR0))                                          (52)
            {
                outl(arg, PWMMR3);                                          (53)
                outl(inl(PWMLER) | (1 < < 3), PWMLER);                      (54)
            }
            break;                                                          (55)

        case PWM_4_ENABLE:                                                  (56)
            outl(inl(PWMPCR) & ~(1 < < 4), PWMPCR);                         (57)
            outl(inl(PWMPCR) | (1 < < 12), PWMPCR);                         (58)
            outl((0x02 < < 16) | (inl(PINSEL0) & (~(0x03 < < 16))), PINSEL0);
                                                                            (59)
            break;                                                          (60)
```


更多高清PDF图书 www.gqpdf.com

```
        case PWM_4_DISABLE:                                                    (61)
            outl(inl(PWMPCR) & ~(1 < < 12), PWMPCR);                           (62)
            outl((PinSel0Save & (0x03 < < 16)) | (inl(PINSEL0) & (~(0x03 < <
                 16))),
                 PINSEL0);                                                     (63)
            break;                                                             (64)
        case PWM_4_SET_DUTY:                                                   (65)
            if (arg < inl(PWMMR0))                                             (66)
            {
                outl(arg, PWMMR4);                                             (67)
                outl(inl(PWMLER) | (1 < < 4), PWMLER);                         (68)
            }
            break;                                                             (69)

        case PWM_5_ENABLE:                                                     (70)
            outl(inl(PWMPCR) & ~(1 < < 5), PWMPCR);                            (71)
            outl(inl(PWMPCR) | (1 < < 13), PWMPCR);                            (72)
            outl((0x01 < < 10) | (inl(PINSEL1) & (~(0x03 < < 10))), PINSEL1);
                                                                               (73)
            break;                                                             (74)
        case PWM_5_DISABLE:                                                    (75)
            outl(inl(PWMPCR) & ~(1 < < 13), PWMPCR);                           (76)
            outl((PinSel1Save & (0x03 < < 10)) | (inl(PINSEL1) & (~(0x03 < <
            10))),
            PINSEL1);                                                          (77)
            break;                                                             (78)
        case PWM_5_SET_DUTY:                                                   (79)
            if (arg < inl(PWMMR0))                                             (80)
            {
                outl(arg, PWMMR5);                                             (81)
                outl(inl(PWMLER) | (1 < < 5), PWMLER);                         (82)
            }
            break;                                                             (83)

        case PWM_6_ENABLE:                                                     (84)
            outl(inl(PWMPCR) & ~(1 < < 6), PWMPCR);                            (85)
            outl(inl(PWMPCR) | (1 < < 14), PWMPCR);                            (86)
            outl((0x02 < < 18) | (inl(PINSEL0) & (~(0x03 < < 18))), PINSEL0);
                                                                               (87)
            break;                                                             (88)
        case PWM_6_DISABLE:                                                    (89)
```

gqpdf.com

```
            outl(inl(PWMPCR) & ~(1 < < 14), PWMPCR);                    (90)
            outl((PinSel0Save & (0x03 < < 18)) | (inl(PINSEL0) & (~(0x03 < <
                 18))),
                 PINSEL0);                                              (91)
            break;                                                      (92)
        case PWM_6_SET_DUTY:                                            (93)
            if (arg < inl(PWMMR0))                                      (94)
            {
                outl(arg, PWMMR6);                                      (95)
                outl(inl(PWMLER) | (1 < < 6), PWMLER);                  (96)
            }
            break;                                                      (97)

        default:                                                        (98)
            return -ENOTTY;                                             (99)
            break;                                                     (100)
    }
    return 0;                                                          (101)
}
```

因为操作每路 PWM 的命令是一致的，仅由于硬件的限制不好把代码合并，本书并不着重讲解硬件，所以本节除了讲解整体命令代码外，仅讲解关于第 1 路 PWM 的命令。

1. 整体控制命令

本驱动有两条整体控制命令 PWM_SET_CYC 和 PWM_GET_RUN_TIME。PWM_SET_CYC 命令用于设置 PWM 周期，而 PWM_GET_RUN_TIME 用于返回 PWM 中断次数。

（1）PWM_SET_CYC 命令

PWM_SET_CYC 命令代码见程序清单 7.19。这实际上是对 PWM 寄存器的操作，都是硬件的要求，详细信息可参见参考文献[3]。

程序清单 **7.19　PWM_SET_CYC** 命令代码

```
    case PWM_SET_CYC:
        outl(arg ,PWMMR0);                                              (1)
        outl(inl(PWMLER) |1, PWMLER);                                   (2)
        break;
```

（2）PWM_GET_RUN_TIME 命令

PWM_GET_RUN_TIME 命令代码见程序清单 7.20。PWM_GET_RUN_TIME 命令需要传输数据到用户空间，这部分的知识可参考 5.2.6 小节。程序清单 7.20（1）、（2）实际上仅是错误处理的，仅程序清单 7.20（3）具有实际意义，它把数据传输到用户空间。

更多高清PDF图书　www.gqpdf.com

程序清单 **7.20　PWM_GET_RUN_TIME** 命令代码

```
case PWM_GET_RUN_TIME:
    if (! access_ok(VERIFY_WRITE, (void *)arg, _IOC_SIZE(cmd)))          (1)
    {
        return -EFAULT;                                                  (2)
    }
    put_user(RunTime, (u32 *)arg);                                       (3)
    break;                                                               (4)
```

2. 某一路 PWM 控制命令

每一路 PWM 控制命令都是一样的，这里仅介绍第 1 路 PWM 的控制命令。第 1 路 PWM 具有 3 条控制命令 PWM_1_ENABLE、PWM_1_DISABLE 和 PWM_1_SET_DUTY。PWM_1_ENABLE 命令用于允许第 1 路 PWM 输出，注意：是输出而不是工作。当打开 PWM 时，所有 6 路 PWM 均已工作，只是引脚上没有信号输出而已。PWM_1_DISABLE 命令是禁止第 1 路 PWM 输出。PWM_1_SET_DUTY 命令是设置第 1 路 PWM 输出高电平时间。

（1）PWM_1_ENABLE 命令

PWM_1_ENABLE 命令代码见程序清单 7.21。程序清单 7.21（1）是使能第 1 路 PWM 输出。程序清单 7.21（2）是把相应的引脚配置为第 1 路 PWM 输出功能。这些均是硬件上的要求，详情请参见参考文献[3]。

程序清单 **7.21　PWM_1_ENABLE** 命令代码

```
case PWM_1_ENABLE:
    outl(inl(PWMPCR) | (1 < < 9), PWMPCR);                               (1)
    outl(0x02 | (inl(PINSEL0) & ~0x03), PINSEL0);                        (2)
    break;                                                               (3)
```

（2）PWM_1_DISABLE 命令

PWM_1_DISABLE 命令代码见程序清单 7.22。程序清单 7.22（1）是禁止第 1 路 PWM 输出。程序清单 7.22（2）是恢复相应的引脚的配置，这个配置是在 open（）方法（参考 7.3.5 小节）中保存的。这些均是硬件上的要求，详情请参见参考文献[3]。

程序清单 **7.22　PWM_1_DISABLE** 命令代码

```
case PWM_1_DISABLE:
    outl(inl(PWMPCR) & ~(1 < < 9), PWMPCR);                              (1)
    outl((PinSel0Save & 0x03) | (inl(PINSEL0) & ~0x03), PINSEL0);        (2)
    break;
```

（3）PWM_1_SET_DUTY 命令

PWM_1_SET_DUTY 命令代码见程序清单 7.23。程序清单 7.23（1）检查参数是否有效，设置的时间大于 PWM 周期是没有意义的，此时就不接受这个参数。然后就把相应的值写入硬件（程序清单 7.23（2）、（3）），这些是硬件上的要求，详情请参见参考文献[3]。

程序清单 **7.23　PWM_1_SET_DUTY** 命令代码

```
case PWM_1_SET_DUTY:
    if (arg < inl(PWMMR0))                                          (1)
    {
        outl(arg, PWMMR1);                                          (2)
        outl(inl(PWMLER) | (1 < < 1), PWMLER);                      (3)
    }
    break;
```

7.3.8　其他说明

1. 未实现的方法

本驱动程序未实现 write（）、read（）和 llseek（）方法。

2. 修改版权及作者声明

这部分比较简单，请参考 6.3.6 小节。

本章小结

本章以 LPC2200 系列的 PWM 为例介绍了实际的 Linux（μClinux）字符设备驱动程序的相关知识。首先介绍这个驱动的使用方法，然后对源码进行分析。

相对于第 5 章，本章主要增加了以下知识：

- 驱动程序的编写方法（参考 7.3.1 小节）。
- 关于临界区的保护方法（参考 7.3.2 小节）。

更多高清PDF图书　www.gqpdf.com

第8章　A/D（字符设备）驱动程序编写范例

LPC2200 系列 ARM 微控制器具有 1 个 10 位逐次比较 A/D 转换器，其特性如下：

- 10 位逐次比较模/数转换器；
- 8 个（LPC2210/2212/2214）引脚复用为 A/D 输入引脚；
- 测量范围为 0 ~3.3 V；
- 10 位转换时间大于或等于 2.44 μs；
- 1 路或多路输入的 Burst 转换模式；
- 可选择由输入引脚的跳变或定时器的匹配信号触发转换；
- 具有掉电模式。

关于这个 A/D 转换器的详细说明请参见参考文献[3]。

本章介绍的驱动程序就是操作 LPC2200 系列的 A/D 转换器，它具有多种工作模式，本驱动仅让它处于程序启动转换模式。本驱动程序可设置 A/D 转换器的转换速率和位数。

8.1　在应用程序中使用 A/D 驱动程序

关于字符设备驱动程序的一般使用方法请参考 5.3 节，本节仅介绍使用 A/D 驱动程序需要特殊注意的地方。

8.1.1　建立设备文件

驱动程序默认的主设备号为 124，从设备号用来标识 A/D 的输入通道号，0 代表通道 0，1 代表通道 1，以此类推，但本驱动仅支持前 4 路。程序清单 8.1 为建立 A/D 通道设备文件的命令。

程序清单 **8.1**　建立设备文件

```
mknod /dev/ADC0.0 c 124 0
mknod /dev/ADC0.1 c 124 1
```

```
mknod /dev/ADC0.2 c 124 2
mknod /dev/ADC0.3 c 124 3
```

8.1.2 可使用的函数

虽然本驱动实现了 read()方法(但没有实现 write()方法),但它仍然不能使用标准 C 的库函数来访问,这是因为每次 read()调用是不可分割的。对于 A/D 来说,每次读的数据个数是(相对)确定的(不是 1 字节就是 2 字节),所以只能使用内核提供给应用程序的接口函数来访问,并且仅部分内核接口函数可使用,其具体情况见表 8.1,具体使用方法见 5.3.3 小节。

表 8.1 可使用的内核接口函数

函数名称	函数功能	函数原型	备 注
open()	打开文件	int open(const char * pathname, int flags);	在使用 A/D 前调用由此函数确定操作的通道
close()	关闭文件	int close(int fd);	在使用完 A/D 后调用
ioctl()	I/O 控制	int ioctl(int fd, unsigned long int cmd, ...);	实际控制 A/D
read()	读数据	ssize_t read(int fd, void * buf, size_t count);	读取 A/D 转换后的数据

8.1.3 提供给应用程序的头文件

函数 ioctl()的第 2 个参数 cmd 为驱动程序的特殊命令,其值应由驱动程序提供。本驱动程序提供了一个头文件 adc.h 定义这些命令。文件 adc.h 的内容见程序清单 8.2。其中,程序清单 8.2(2)、(3)定义了 2 个宏,对应 2 条命令。关于各具体命令的说明参见表 8.2。

程序清单 **8.2 adc.h**

```
#ifndef __ADC_H
#define __ADC_H
#include <linux/ioctl.h>

#define ADC_IOC_MAGIC       0xd2                                    (1)

#define ADC_SET_CLKDIV       _IO(ADC_IOC_MAGIC,    0)               (2)
#define ADC_SET_BITS         _IO(ADC_IOC_MAGIC,    1)               (3)

#define ADC_MAXNR           2                                       (4)
#endif
```


更多高清PDF图书 www.gqpdf.com

表 8.2　adc 命令

命　令	功　能	备　注
ADC_SET_CLKDIV	设置 A/D 工作频率	参考 8.1.4 小节
ADC_SET_BITS	设置 A/D 转换精度	参考 8.1.5 小节

8.1.4　设置 A/D 工作频率

逐次比较 A/D 转换器需要一个时钟才能工作，LPC2200 的 A/D 转换器也不例外。对于 LPC2200 的 A/D 转换器来说，这个时钟的频率（最高为 4.5 MHz）和转换位数共同决定 A/D 转换速率。

注意：LPC2200 的 A/D 转换器的所有通道使用同一个工作频率。

在本驱动中，使用命令 ADC_SET_CLKDIV 设置 A/D 的工作频率，实际是设置相对 VBP 频率的分频值（参见参考文献[3]）。关于 ADC_SET_CLKDIV 的具体说明如下。

- 功能：设置 A/D 工作频率（相对 VBP 频率的）分频值。
- ioctl() 函数的第 3 个参数：32 位整型参数，为需要设置的分频值减 1。
- 使用示例：当 VPB 总线频率为 11 059 200 Hz 时，程序清单 8.3 将 A/D 的工作频率设置为接近 4.5 MHz。

程序清单 **8.3**　设置 **A/D** 工作频率范例

```
int fd;
......
fd = open("/dev/ADC0.0", O_RDONLY);                                    (1)
......
ioctl(fd, ADC_SET_CLKDIV, (11059200 + 4500000 -1) /4500000 - 1);       (2)
......
```

8.1.5　设置 A/D 转换精度

LPC2200 的 A/D 转换精度可为 3 ~ 10 位，驱动程序也提供这方面的支持。使用 ADC_SET_BITS 即可设置转换精度。

注意：LPC2200 的 A/D 转换器的所有通道使用相同的转换位数。

关于 ADC_SET_BITS 的具体说明如下。

- 功能：设置 A/D 转换精度。
- ioctl() 函数的第 3 个参数：转换精度，也就是 A/D 输出位数，32 位整型参数。可能值为 3 ~ 10。

高清 PDF 藏 gqpdf.com

● 使用示例：见程序清单 8.4。

程序清单 **8.4**　写数据到 **I²C** 从器件示例

```
int fd;
......
fd = open("/dev/ADC0.0", O_RDONLY);                    (1)
......
ioctl(fd, ADC_SET_BITS, 10);                           (2)
......
```

8.1.6　读取 A/D 转换结果

读 A/D 转换结果很简单，只要按照函数 read() 的规范调用即可。惟一要注意的是，一次只能读取 1 字节（转换精度小于或等于 8 位）或 2 字节（转换精度大于 8 位）。读 A/D 转换结果示例见程序清单 8.5。

程序清单 **8.5**　读 **A/D** 转换结果示例

```
int fd;
volatile u16 ad_data;
......
fd = open("/dev/ADC0.0", O_RDONLY);                    (1)
......
read(fd, &ad_data, 2);                                 (2)
......
```

8.2　编　译

当建立相关的开发环境和获取驱动程序的源代码后，只需建立（或修改）Makefile 文件，通过 make 命令即可编译驱动程序。建立开发环境不是本书的任务，请自行参考相关文档。本书介绍的源代码可从广州致远电子有限公司出品的相关产品中获得，或根据本书自行输入。下面介绍本驱动所用的 Makefile 文件（见程序清单 8.6）。

程序清单 **8.6**　**Makefile** 文件

```
EXEC = adc
OBJS = adc.o
SRC = adc.c
```

与程序清单4.2不同的部分

更多高清PDF图书　www.gqpdf.com

```
INCLUDE = /work/uClinux-dist/linux-2.4.x/include
CC = arm-elf-gcc
LD = arm-elf-ld
MODCFLAGS = -D__KERNEL__ -I $(INCLUDE) -Wall -O2 -fno-strict-aliasing -fno-common -
pipe -fno-builtin -D__linux__ -g -DNO_MM -mapcs-32 -march = armv4 -mtune = arm7tdmi -
mshort-load-bytes -msoft-float -nostdinc -iwithprefix include
LDFLAGS = -m armelf -r

all: $(EXEC)

$(EXEC): $(OBJS)
    $(LD) $(LDFLAGS) -o $@  $(OBJS)

%.o:%.c
    $(CC) $(MODCFLAGS) -mapcs -c $< -o $@

clean:
    -rm -f $(EXEC) *.elf *.gdb *.o
```

根据实际情况修改的部分

可以看出，程序清单 8.6 与程序清单 4.2 十分相似，仅头 3 句不同。实际上，程序清单 8.6 是根据程序清单 4.2 修改而来的，因此，关于 Makefile 的详细介绍请参考 4.3 节。

8.3 源码分析

事实上，第 5 章已介绍了编写字符驱动程序的一般知识，而本驱动程序也是在那个驱动程序的基础上修改而来的，因此，应先阅读第 5 章，否则不易理解本节内容。

8.3.1 初始化函数

本驱动程序的初始化函数代码如程序清单 8.7 所示，这是按照程序清单 5.2 编写的。请注意方框中的代码，用于对引脚连接状态进行保存及对 ADCR 寄存器进行设置和初始化。

程序清单 **8.7** **A/D** 转换驱动程序初始化代码

```
int adc_init(void)
{
    int  result;
```

```
    result = register_chrdev(MAJOR_NR,  DEVICE_NAME,  &adc_fops);          (1)
    if (result < 0)                                                        (2)
    {
        printk(KERN_ERR DEVICE_NAME ": Unable to get major % d\\n", MAJOR_NR );
                                                                           (3)
        return(result);                                                    (4)
    }

    if (MAJOR_NR == 0)                                                     (5)
    {
        MAJOR_NR = result; /* dynamic */                                   (6)
    }

    PinSel1Save = inl(PINSEL1) & (0xff < < 22);                            (7)

    AdcrSave = (0xff < < 8) | (0x00 < < 17) | (1 < < 21);                  (8)
    outl(0, ADCR);                                                         (9)

    printk(KERN_INFO DEVICE_NAME ": init OK\\n");                          (10)
    return(0);
}
```

程序清单 8.7(7) 保存引脚连接寄存器 PINSEL1 与 A/D 相关的引脚的状态；程序清单 8.7(8) 初始化用于设置 ADCR 寄存器的全局变量 AdcrSave，这个变量在驱动程序代码中多处用到；程序清单 8.7(9) 初始化 A/D 为不工作状态，往 ADCR 寄存器写入 0 与写入 1 等价，请参见参考文献[3]的相关部分。

8.3.2 清除函数

本驱动程序的清除函数代码如程序清单 8.8 所示，这是完全按照程序清单 5.4 编写的，请注意方框中的代码。

程序清单 **8.8** **A/D** 转换驱动程序清除函数代码

更多高清PDF图书 www.gqpdf.com

```
void adc_cleanup(void)
{
    outl(0, ADCR);                                              (1)
    outl(inl(PINSEL1) & (~(0xff << 22)), PINSEL1);              (2)
    outl(inl(PINSEL1) |PinSel1Save, PINSEL1);                   (3)
```

高清藏 gqpdf.com

```
    unregister_chrdev(MAJOR_NR, DEVICE_NAME);                              (4)
}
```

程序清单8.8(1)设置ADCR状态为不工作状态;程序清单8.8(2)~(3)恢复PINSEL1与A/D相关引脚的连接配置。

8.3.3 open()方法

open()方法会访问I/O端口。关于访问I/O端口的方法请参考5.2.7小节。

A/D转换驱动open()方法的函数代码如程序清单8.9所示,这是按照程序清单5.7编写的。

程序清单**8.9** **A/D**转换驱动**open()**方法

```
static int adc_open(struct inode *inode, struct file *filp)
{
    unsigned long flag;                                                    (1)
    int num, temp;                                                         (2)

    num = MINOR(inode->i_rdev);                                            (3)
    if (num >= MAX_ADC)                                                    (4)
    {
        return -ENODEV;                                                    (5)
    }

    if (adc_usage\[num\] == 0)                                             (6)
    {
        local_irq_save(flag);                                              (7)
        temp = num + num;                                                  (8)
        outl(inl(PINSEL1) & (~(0x03 << (temp + 22))), PINSEL1);            (9)
        outl(inl(PINSEL1) | (0x01 << (temp + 22)), PINSEL1);               (10)
        filp->private_data = (void *)(num);                                (11)
        local_irq_restore(flag);                                           (12)
    }
    adc_usage\[num\]++;                                                    (13)
    MOD_INC_USE_COUNT;                                                     (14)
    return 0;          /* success */                                       (15)
}
```


更多高清PDF图书 www.gqpdf.com

程序清单 8.9(3) ~ (5)用于获取设备节点的从设备号,并判断设备节点是否在允许的范围之内。MAX_ADC 在 config 文件中定义,表示该驱动支持的 A/D 转换通道数,目前配置为 4。程序清单 8.9(6) ~ (13)用于判断各 A/D 通道的被使用情况,并进行相应的处理。adc_usage\[num\]是一个全局变量,用于记录各 A/D 通道被打开的次数,若某通道计数器为 0,则表示还未被使用,需要进行 A/D 引脚的连接设置(程序清单 8.9(7) ~ (10))。程序清单 8.9(11)则给 filp->private_data 赋值,标识不同的 ADC 设备,至于 A/D 引脚连接设置的详情请参见参考文献[3]。程序清单 8.9(7) 和程序清单 8.9(12)为关/开中断处理,用于保护临界区,具体信息请参考 7.3.2 小节。

8.3.4 release()方法

A/D 转换 release()方法的函数代码如程序清单 8.10 所示,这是按照程序清单 5.8 编写的。

程序清单 **8.10** A/D 转换驱动程序 **release()**方法

```
static int adc_release(struct inode *inode, struct file *filp)
{
    unsigned long flag;                                              (1)
    int num;                                                         (2)
    u32 temp, temp1;                                                 (3)

    num = MINOR(inode->i_rdev);                                      (4)
    if (num >= MAX_ADC)                                              (5)
    {
        return -ENODEV;                                              (6)
    }
    MOD_DEC_USE_COUNT;                                               (7)
    adc_usage\[num\]--;                                              (8)
    if (adc_usage\[num\] == 0)                                       (9)
    {
        local_irq_save(flag);                                        (10)

        temp = temp1 = inl(PINSEL1);                                 (11)
        temp &= (PinSel1Save & (0x03 << (num * 2 + 22))) |           (12)
                (temp1 & ~((0x03 << (num * 2 + 22))));               (13)

        outl(temp, PINSEL1);                                         (14)
```

高清藏 PDF gqpdf.com

```
        local_irq_restore(flag);                                    (15)
    }
    return(0);
}
```

程序清单 8.10（4）~（6）用于先获取设备的从设备号，并判断从设备号是否在合法的范围之内。MAX_ADC 在 config.h 文件中定义，表示 A/D 的通道数和从设备号的合法范围。程序清单 8.10（8）用于将 A/D 通道的使用计数器递减。程序清单 8.10（10）和（15）为关/开中断，构造临界区，关于临界区请参考 7.3.2 小节。程序清单 8.10（11）~（13）恢复 PINSEL1 中与A/D 转换相关引脚的连接设置。

8.3.5 ioctl()方法

ioctl()方法会访问 I/O 端口。关于访问 I/O 端口的方法请参考 5.2.7 小节。

A/D 转换驱动 ioctl()方法的实现代码如程序清单 8.11 所示，用于实现设置 A/D 转换的时钟和转换精度。

程序清单 **8.11** **A/D** 转换驱动 **ioctl()**方法代码

```
static int adc_ioctl(struct inode *inode, struct file *filp, unsigned int cmd, un-
signed long arg)
{
    u32 temp;                                                       (1)

    if (_IOC_TYPE(cmd) != ADC_IOC_MAGIC)                            (2)
    {
        return -ENOTTY;                                             (3)
    }
    if (_IOC_NR(cmd) >= ADC_MAXNR)                                  (4)
    {
        return -ENOTTY;                                             (5)
    }

    switch(cmd)                                                     (6)
    {
        case ADC_SET_CLKDIV:                                        (7)
            temp = AdcrSave;                                        (8)
            temp &= (~(0xff << 8));                                 (9)
            temp |= ((arg & 0xff) << 8);                            (10)
```


更多高清PDF图书 www.gqpdf.com

```
            AdcrSave = temp;                                    (11)
            break;                                              (12)

        case ADC_SET_BITS:                                      (13)
            arg &= 0x0f;                                        (14)
            if (arg < 3)                                        (15)
            {
                arg = 3;                                        (16)
            }
            if (arg > 10)                                       (17)
            {
                arg = 10;                                       (18)
            }
            arg++;                                              (19)
            arg = 11 - arg;                                     (20)
            temp = AdcrSave;                                    (21)
            temp &= ~(0x07 << 17);                              (22)
            temp |= (arg << 17);                                (23)
            AdcrSave = temp;                                    (24)
            break;                                              (25)

        default:
            return -ENOTTY;
            break;
    }
    return 0;
}
```

程序清单 8.11(2) ~ (3)用于获取命令的幻数,并判断幻数是否正确。程序清单 8.11(4) ~ (5)用于获取命令的编号,并判断编号是否正确。程序清单 8.11(6)用于根据命令进行分支。程序清单8.11(7) ~ (12) 用于设定 A/D 转换时钟。程序清单 8.11(13) ~ (25)用于设定 A/D 转换精度,精度设定的有效范围为 3 ~ 10。至于如何设置 A/D 寄存器,请参见参考文献[3]的相关部分。

8.3.6 read() 方法

A/D 转换驱动程序 read() 方法的函数代码如程序清单 8.12 所示,这是按照程序清单 5.9 编写的。A/D 转换驱动 read() 方法用于读取 A/D 转换结果。根据设定的 A/D 转换精度的不

同，该程序最多支持一次读取 2 字节数据。

程序清单 **8.12**　**A/D** 转换驱动 **read**（）方法

高清PDF gqpdf.com

更多高清PDF图书　www.gqpdf.com

```
static ssize_t adc_read(struct file * filp, char * buf, size_t count, loff_t * f_
pos)
{
    unsigned long flag;                                                    (1)
    u32 temp;                                                              (2)
    u16 temp1;                                                             (3)
    u8  temp2;                                                             (4)

    if (count > 2)                                                         (5)
    {
        return -EFBIG;                                                     (6)
    }

    if (! access_ok(VERIFY_WRITE, (void *)buf, count))                     (7)
    {
        return -EFAULT;                                                    (8)
    }

    temp = (int)filp->private_data;                                        (9)

    local_irq_save(flag);                                                  (10)
    outl(AdcrSave | (1u << temp) | (1u << 24), ADCR);                      (11)
    while (1)                                                              (12)
    {
        temp = inl(ADDR);                                                  (13)
        if ((temp & (1u << 31)) !=0)                                       (14)
        {
            break;                                                         (15)
        }
    }
    local_irq_restore(flag);                                               (16)

    temp1 = temp & 0xffff;                                                 (17)
    if (count ==1)                                                         (18)
    {
```

高清PDF gqpdf.com

```
        temp2 = temp1 > > 8;                                    (19)
        put_user(temp2, (u8 *)buf);                             (20)
    }
    if (count == 2)                                             (21)
    {
        put_user(temp1, (u16 *)buf);                            (22)
    }
    return count;                                               (23)
}
```

程序清单 8.12（5）~（6）用于判断需要读取的字节数是否合法，允许依次读取 1 或 2 字节。程序清单 8.12（7）~（8）用于检查存放读取结果的缓冲区是否可写。程序清单 8.12（9）为获取设备标志，标识将要操作的设备。程序清单 8.12（10）和（16）为关/开中断处理，构造临界区，关于临界区请参考 7.3.2 小节。程序清单 8.12（11）用于切换 A/D 转换通道，启动 A/D 转换。程序清单 8.12（13）~（15）为等待 A/D 转换完成。程序清单 8.12（17）为保留转换结果（ADCR 低 16 位）。程序清单 8.12（18）~（23）用于根据需要读取的字节数进行返回。

8.3.7 其他说明

1. 未实现的方法

本驱动程序未实现 write（）和 llseek（）方法。

2. 修改版权及作者声明

这部分比较简单，请参考 6.3.6 小节。

本章小结

本章以 LPC2200 系列的 ADC 为例介绍了实际的 Linux（μClinux）字符设备驱动程序的相关知识。首先介绍该驱动程序的使用方法，然后对源码进行分析。

更多高清PDF图书 www.gqpdf.com

第9章 SPI(字符设备)驱动程序编写范例

LPC2200 系列 ARM 微控制器具有 2 个独立的 SPI 总线接口,其特性如下:

- 具有 2 个完全独立的 SPI 控制器;
- 遵循同步串行接口(SPI)规范;
- 全双工数据通信;
- 可配置为 SPI 主机或从机;
- 最大数据位速率为外设时钟 Fpclk 的 1/8。

关于 SPI 总线接口的详细说明请参见参考文献[3]。

本章介绍的驱动程序就是驱动 LPC2200 系列的 SPI 总线接口,仅可配置为主机,不可配置为从机。驱动程序可设置总线频率和一些时序。可通过这个总线接口控制 SPI 从器件。

9.1 在应用程序中使用 SPI 驱动程序

关于字符设备驱动程序的一般使用方法请参考 5.3 节,本节仅介绍使用 SPI 总线驱动程序需要特殊注意的地方。

注意:本驱动程序仅操作 SPI 总线,让应用程序与相应的器件通信而已,对具体器件的操作还需要看器件手册。

9.1.1 建立设备文件

驱动程序默认的主设备号为 123,从设备号用来标识使用的是哪个 SPI 接口,0 代表 SPI0,1 代表 SPI1。程序清单 9.1 为建立设备文件的命令。

程序清单 **9.1** 建立设备文件

```
mknod /dev/SPI0 c 123 0
mknod /dev/SPI1 c 123 1
```

高清藏 gqpdf.com

9.1.2 使用 SPI 的特殊注意点

SPI 器件的使用有一个很大的特点：从器件的片选信号不由本身提供，而由其他手段提供。如果一个 SPI 接口上只接一个从器件，则将从器件的片选接地，此时 SPI 驱动程序的使用没有特殊的地方；如果一个 SPI 接口接多个从器件，则 SPI 驱动的使用就有特殊的地方。程序清单 9.2 就是使用 SPI 的示意代码。关于程序中相关的操作在后面几个小节会详细介绍。程序清单 9.2 写得比较清楚，这里不做过多描述。

程序清单 **9.2**　使用 **SPI** 的伪代码

```
int main()
{
    int spi;

    spi = open("对应设备文件", O_RDWR);                    (1)
    配置 SPI 总线;                                         (2)
    ioctl(spi, SPI_START);                                 (3)
    允许从器件 1 片选;                                     (4)
    读/写从器件 1;                                         (5)
    禁止从器件 1 片选;                                     (6)
    ioctl(spi, SPI_END);                                   (7)

    ioctl(spi, SPI_START);                                 (8)
    允许从器件 2 片选;                                     (9)
    读/写从器件 2;                                         (10)
    禁止从器件 2 片选;                                     (11)
    ioctl(spi, SPI_END);                                   (12)

    .......
    close(spi);                                            (13)
    return 0;                                              (14)
}
```

使用任何一个SPI从器件必须按照这个顺序操作

这里可以有多条操作SPI从器件的语句

9.1.3 可使用的函数

本驱动可使用标准的 C 语言库函数来访问，前面提示一个 SPI 接口仅接一个从设备，同时从设备为标准的流设备。鉴于使用标准 C 语言库函数的条件比较苛刻，同时标准 C 语言库函数的资料很多，这里不再说明。下面介绍可访问本驱动的内核接口函数，其具体情况见

更多高清PDF图书 www.gqpdf.com

表 9.1，具体使用方法见 5.3.3 小节。

表 9.1　可使用的内核接口函数

函数名称	函数功能	函数原型	备　注
open ()	打开文件	int open (const char * pathname, int flags);	在使用 SPI 从器件前调用，由此函数确定使用哪个 SPI 接口
close ()	关闭文件	int close (int fd);	在使用完 SPI 后调用
ioctl ()	I/O 控制	int ioctl (int fd, unsigned long int cmd, ...);	实际控制 SPI
read ()	读数据	ssize_t read (int fd, void * buf , size_t count);	从 SPI 从器件中读数据
write ()	写数据	ssize_t write (int fd, void * buf , size_t count);	把数据写入 SPI 从器件中

9.1.4　提供给应用程序的头文件

函数 ioctl () 的第 2 个参数 cmd 为驱动程序的特殊命令，其值应由驱动程序提供。本驱动程序提供了一个头文件 spi. h 定义这些命令。文件 spi. h 的内容见程序清单 9.3。其中，程序清单 (2) ~ (11) 定义了 10 个宏，对应 10 条命令。关于各具体命令的说明，请参考表 9.2。

程序清单 **9.3　spi. h**

```
#ifndef __SPI_H
#define __SPI_H
#include <linux/ioctl.h>

#define SPI_IOC_MAGIC          0xd3                                    (1)

#define SPI_SET_CLKDIV         _IO(SPI_IOC_MAGIC,    0)                (2)
#define SPI_SET_CPHA_FIRST     _IO(SPI_IOC_MAGIC,    1)                (3)
#define SPI_SET_CPHA_SECOND    _IO(SPI_IOC_MAGIC,    2)                (4)
#define SPI_SET_CPOL_LOW       _IO(SPI_IOC_MAGIC,    3)                (5)
#define SPI_SET_CPOL_HIGHT     _IO(SPI_IOC_MAGIC,    4)                (6)
#define SPI_SET_LSBF_BIT0      _IO(SPI_IOC_MAGIC,    5)                (7)
#define SPI_SET_LSBF_BIT7      _IO(SPI_IOC_MAGIC,    6)                (8)
#define SPI_WR_DATA            _IOWR(SPI_IOC_MAGIC,  7, uint8)         (9)
#define SPI_START              _IO(SPI_IOC_MAGIC,    8)                (10)
#define SPI_END                _IO(SPI_IOC_MAGIC,    9)                (11)

#define SPI_MAXNR              10 /* max command */                    (12)
#endif
```

表 9.2　SPI 命令

命　令	功　能	备　注
SPI_SET_CLKDIV	设置 SPI 时钟分频值	参考 9.1.5 小节
SPI_SET_CPHA_FIRST	数据在第一个时钟输出	参考 9.1.6 小节
SPI_SET_CPHA_SECOND	数据在第二个时钟输出	
SPI_SET_CPOL_LOW	时钟低电平有效	参考 9.1.7 小节
SPI_SET_CPOL_HIGHT	时钟高电平有效	
SPI_SET_LSBF_BIT0	先传送数据最低位	参考 9.1.8 小节
SPI_SET_LSBF_BIT7	先传送数据最高位	
SPI_START	开始 SPI 操作	参考 9.1.9 小节
SPI_END	结束 SPI 操作	
SPI_WR_DATA	发送同时接收一字节数据	参考 9.1.10 小节

9.1.5　设置 SPI 分频值

在使用 SPI 总线接口前必须设置总线频率。LPC2200 系列 ARM 微控制器的 SPI 时钟基于 VPB 时钟（参见参考文献[3]），VPB 时钟经过一个分频器提供给 SPI 接口。这个分频器的分频值是可设定的，但必须是大于 8 的偶数。分频器为 8 位。ioctl () 方法的 SPI_SET_CLKDIV 命令都是设置这个分频值，它具有同样的限制。关于 SPI_SET_CLKDIV 的具体说明如下：

- 功能：设置 SPI 接口相对 VBP 频率的分频值。
- ioctl () 函数的第 3 个参数：32 位整型参数，为需要设置的分频值。其必须为大于或等于 8 且小于 255 的偶数。
- 使用示例：参见程序清单 9.4。

程序清单 **9.4**　设置 **SPI** 时钟分频值范例

```
int fd;
......
fd = open("/dev/SPI1", O_RDWR);                                    (1)
......
ioctl(fd, SPI_SET_CLKDI, 24);                                      (2)
......
```

9.1.6　SPI 时钟相位控制

时钟相位控制决定 SPI 传输时数据和时钟的关系，并控制从机传输的起始和结束。ioctl () 方法的 SPI_SET_CPHA_FIRST 命令和 SPI_SET_CPHA_SECOND 命令用于设置 SPI 时钟相位，

更多高清PDF图书　www.gqpdf.com

下面具体介绍这两个命令。

1. SPI_SET_CPHA_FIRST

- 功能：将 SPI 时钟相位设置为数据在 SCK 的第一个时钟沿采样；传输从 SSEL 信号激活时开始，并在 SSEL 信号无效时结束。
- ioctl() 函数的第 3 个参数：没有使用。
- 使用示例：见程序清单 9.5。

程序清单 **9.5　SPI_SET_CPHA_FIRST** 命令使用范例

```
int fd;
......
fd = open("/dev/SPI1", O_RDWR);                                    (1)
......
ioctl(fd, SPI_SET_CPHA_FIRST);                                     (2)
......
```

2. SPI_SET_CPHA_SECOND

- 功能：将 SPI 时钟相位设置为数据在 SCK 的第二个时钟沿采样；当 SSEL 信号激活时，传输从第一个时钟沿开始并在最后一个采样时钟沿结束。
- ioctl() 函数的第 3 个参数：没有使用。
- 使用示例：见程序清单 9.6。

程序清单 **9.6　SPI_SET_CPHA_SECOND** 命令使用范例

```
int fd;
......
fd = open("/dev/SPI1", O_RDWR);                                    (1)
......
ioctl(fd, SPI_SET_CPHA_SECOND);                                    (2)
......
```

9.1.7　SPI 时钟极性控制

时钟极性控制决定 SPI 时钟的有效电平。ioctl() 方法的 SPI_SET_CPOL_LOW 命令和 SPI_SET_CPOL_HIGHT 命令用于设置 SPI 时钟极性，下面具体介绍这两个命令。

1. SPI_SET_CPOL_LOW

- 功能：将 SPI 时钟极性设置为低有效。
- ioctl() 函数的第 3 个参数：没有使用。

● 使用示例：见程序清单 9.7。

程序清单 **9.7　SPI_SET_CPOL_LOW** 命令使用范例

```
int fd;
......
fd = open("/dev/SPI1", O_RDWR);                                    (1)
......
ioctl(fd, SPI_SET_CPOL_LOW);                                       (2)
......
```

2. SPI_SET_CPOL_HIGHT

● 功能：将 SPI 时钟极性设置为高有效。

● ioctl() 函数的第 3 个参数：没有使用。

● 使用示例：见程序清单 9.8。

程序清单 **9.8　SPI_SET_CPOL_HIGHT** 命令使用范例

```
int fd;
......
fd = open("/dev/SPI1", O_RDWR);                                    (1)
......
ioctl(fd, SPI_SET_CPOL_HIGHT);                                     (2)
......
```

9.1.8　SPI 数据传输顺序控制

当 SPI 接口传输数据时，可先传输一个字节的最低位，也可传输一个字节的最高位。LPC2200 的 SPI 接口对两种传输方式都支持。ioctl() 方法的 SPI_SET_LSBF_BIT0 命令和 SPI_SET_LSBF_BIT7 命令就是用于设置 SPI 的传输顺序，下面具体介绍这两个命令。

1. SPI_SET_LSBF_BIT0

● 功能：将 SPI 传输顺序设置为先传输一字节的最低位。

● ioctl() 函数的第 3 个参数：没有使用。

● 使用示例：见程序清单 9.9。

程序清单 **9.9　SPI_SET_LSBF_BIT0** 命令使用范例

```
int fd;
......
fd = open("/dev/SPI1", O_RDWR);                                    (1)
```


更多高清PDF图书　www.gqpdf.com

```
.....
ioctl(fd, SPI_SET_LSBF_BIT0);                                    (2)
.....
```

2. SPI_SET_LSBF_BIT7

- 功能：将 SPI 传输顺序设置为先传输一个字节的最高位。
- ioctl()函数的第 3 个参数：没有使用。
- 使用示例：见程序清单 9.10。

程序清单 **9.10　SPI_SET_LSBF_BIT7** 命令使用范例

```
int fd;
.....
fd = open("/dev/SPI1", O_RDWR);                                  (1)
.....
ioctl(fd, SPI_SET_LSBF_BIT7);                                    (2)
.....
```

9.1.9　开始和结束 SPI 操作

当多个 SPI 从器件共享一个 SPI 接口时，需要避免多个器件同时被选中，这就需要互斥使用总线。ioctl()方法的 SPI_START 命令和 SPI_END 命令就是为这个需求设计的。当应用程序发送 SPI_START 命令时，驱动程序会挂起应用程序进程直至获得总线，随后驱动程序会锁定总线。当 SPI_START 命令返回后，应用程序可以使能自己操作的 SPI 总线，然后进行一系列的操作。当应用程序使用完总线后，应先禁止自己操作的 SPI 从器件，然后发送 SPI_END 命令释放总线。当所有应用程序都遵循这个规则时，SPI 从器件可靠地被访问。这两个命令很简单，其使用示例见程序清单 9.2。

9.1.10　发送同时接收一字节数据

LPC2200 的 SPI 接口与操作的从器件实际上组成一个 16 位的环状移位寄存器，也就是说，在发送数据的同时也接收了数据。但使用普通的接口函数读/写是分开的。当然，大多数情况下数据传输是单向的。如果需要使用 SPI 的这个特性，可使用 ioctl()方法的 SPI_WR_DATA 命令实现。关于 SPI_WR_DATA 命令的具体说明如下：

- 功能：发送一字节数据的同时接收一字节数据。
- ioctl()函数的第 3 个参数：一个 8 位整型指针，调用 ioctl()前其指向的变量应保存将要发送的数据，ioctl()返回后它指向的变量保存读到的数据。
- 使用示例：见程序清单 9.11。

程序清单 **9.11**　发送同时接收一字节数据

```
int fd;
uint8 data
......
fd = open("/dev/SPI1", O_RDWR);                                    (1)
......
data = 4;                                                          (2)
ioctl(fd, SPI_WR_DATA, &data);                                     (3)
printf("data = % u", data);                                        (4)
......
```

发送的数据

现在data为读到的数据

9.1.11　写数据到 SPI 从器件

写数据到 SPI 从器件很简单，按照函数 write() 的规范调用即可。写数据到 SPI 从器件的例子见程序清单 9.12。

注意：此操作将丢弃从 SPI 从器件读到的数据。

程序清单 **9.12**　写数据到 **SPI** 从器件的例子

```
int fd;
uint8 temp\[2\];
......
fd = open("/dev/SPI1", O_RDWR);                                    (1)
......
write(fd, temp, 1);                                                (2)
......
```

9.1.12　从 SPI 从器件读数据

从 SPI 从器件读数据很简单，按照函数 read() 的规范调用即可。从 SPI 从器件读数据的例子见程序清单 9.13。

注意：此操作将给 SPI 从器件写 0。

程序清单 **9.13**　从 **SPI** 从器件读数据的例子

```
int fd;
uint8 temp\[2\];
......
fd = open("/dev/SPI1", O_RDWR);                                    (1)
```


更多高清PDF图书　www.gqpdf.com

```
......
read (fd, temp, 2);                                                    (2)
......
```

9.2 编　译

当建立了相关的开发环境和获取了驱动程序的源代码后，只需建立（或修改）Makefile 文件，通过 make 命令即可编译驱动程序。建立开发环境不是本书的任务，请自行参考相关文档。本书介绍的源代码可从广州致远电子有限公司出品的相关产品中获得，或根据本书自行输入。下面介绍本驱动程序所用的 Makefile 文件（见程序清单 9.14）。

程序清单 **9.14**　**Makefile** 文件

```
EXEC = spi
OBJS = spi.o
SRC = spi.c

INCLUDE = /work/uClinux-dist/linux-2.4.x/include

CC = arm-elf-gcc
LD = arm-elf-ld
MODCFLAGS = -D__KERNEL__ -I $(INCLUDE) -Wall -O2 -fno-strict-aliasing -fno-common -
pipe -fno-builtin -D__linux__ -g -DNO_MM -mapcs-32 -march = armv4 -mtune = arm7tdmi -
mshort-load-bytes -msoft-float -nostdinc -iwithprefix include
LDFLAGS = -m armelf -r

all: $(EXEC)

$(EXEC): $(OBJS)
    $(LD) $(LDFLAGS) -o $@  $(OBJS)

% .o:% .c
    $(CC) $(MODCFLAGS) -mapcs -c $ < -o $@


clean:
    -rm -f $(EXEC) *.elf *.gdb *.o
```

与程序清单4.2不同的部分

根据实际情况修改的部分

可以看出，程序清单 9.14 与程序清单 4.2 十分相似，仅头 3 句不同。实际上，程序清单 9.14 是根据程序清单 4.2 修改而来的。因此，关于 Makefile 的详细介绍请参考 4.3 节。

9.3 源码分析

事实上，第 5 章已介绍了编写字符驱动程序的一般知识，而本驱动程序也是在那个驱动程序的基础上修改而来的，因此，应先阅读第 5 章，否则不易理解本节内容。

9.3.1 初始化函数

SPI 驱动程序初始化函数如程序清单 9.15 所示，这是按照程序清单 5.2 编写的。实际上，程序清单 9.15 与程序清单 6.16 是一样的，关于这部分代码请参考 6.3.1 小节。

程序清单 **9.15** **SPI** 驱动程序的初始化函数代码

```
int spi_init(void)
{
    int  result;

    result = register_chrdev(MAJOR_NR,  DEVICE_NAME,  &spi_fops);
    if (result < 0)
    {
        printk(KERN_ERR DEVICE_NAME ": Unable to get major % d\\n", MAJOR_NR );
        return(result);
    }

    if (MAJOR_NR == 0)
    {
        MAJOR_NR = result; /* dynamic */
    }

    printk(KERN_INFO DEVICE_NAME ": init OK\\n");
    return(0);
}
```

9.3.2 清除函数

SPI 驱动程序的清除函数代码如程序清单 9.16 所示。它与程序清单 5.4 一样，这里不再详细介绍。

程序清单 **9.16** **SPI** 驱动程序的清除函数代码

```
void spi_cleanup(void)
```


更多高清PDF图书 www.gqpdf.com

```
{
    unregister_chrdev(MAJOR_NR, DEVICE_NAME);
}
```

9.3.3 open()方法

在详细介绍具体实现代码之前，先介绍一个数据结构 Spi_Dev，如程序清单 9.17 所示。这个数据结构保存了 SPI 设备的一些信息，如基地址、使用计数器和使用信号量。

程序清单 **9.17** **Spi_Dev** 数据结构

```
typedef struct Spi_Dev
{
    u32                BaseAddr;
    unsigned int        usage;      /* device using count */
    struct semaphore    sem;        /* mutual exclusion semaphore */
}   Spi_Dev;
```

这个数据结构并不是必需的，只是为了方便而设定的；当然，也可通过其他方式实现。使用信号量是为了防止出现同一总线上多个 SPI 器件同时使用 SPI 总线的情况。SPI 总线同一时刻只能由一个器件独占 SPI 总线，这是由 SPI 总线的特点决定的。但 SPI 总线不能决定向哪个器件进行访问，具体访问哪个器件只能由器件的片选决定，被选中的器件应独占 SPI 总线。如果此时不幸将另外器件的片选选中，则会出现多个器件同时使用 SPI 总线的情况。使用信号量则可有效地解决这个问题。当然，如果只准备在 SPI 总线上接一个 SPI 器件，则 SPI 驱动程序可以不使用信号量。

open()方法会访问 I/O 端口。关于访问 I/O 端口的方法请参考 5.2.7 小节。

SPI 驱动程序 open()方法的实现代码如程序清单 9.18 所示。它完全是按照程序清单 5.7 编写的。

程序清单 **9.18** **SPI** 驱动程序 **open()** 方法的实现代码

```
static int spi_open(struct inode *inode, struct file *filp)
{
    unsigned long flag;                                    (1)
    int num;                                               (2)
    u32 temp;                                              (3)

    num = MINOR(inode->i_rdev);                            (4)
    if (num >= MAX_SPI)                                    (5)
    {
```

```
        return -ENODEV;                                                    (6)
    }

    if (SpiDevices\[num\].usage ==0)                                       (7)
    {
        local_irq_save(flag);                                              (8)
        if (num ==0)                                                       (9)
        {
            temp =inl(PINSEL0);                                            (10)
            PinSel0Save =temp & (0xff < < 8);                              (11)
            temp &= ~(0xff < < 8);                                         (12)
            temp |=(0x55 < < 8);                                           (13)
            outl(temp, PINSEL0);                                           (14)
        }
        if (num ==1)                                                       (15)
        {
            temp =inl(PINSEL1);                                            (16)
            PinSel1Save =temp & (0xff < < 2);                              (17)
            temp &= ~(0xff < < 2);                                         (18)
            temp |=(0xaa < < 2);                                           (19)
            outl(temp, PINSEL1);                                           (20)
        }

        sema_init(&(SpiDevices\[num\].sem), 1);                            (21)
        SpiDevices\[num\].BaseAddr =num * (S1PCR -S0PCR) + S0PCR;          (22)
        outl(8, SpiDevices\[num\].BaseAddr + S0PCCR -S0PCR);               (23)
        inl(SpiDevices\[num\].BaseAddr + S0PSR -S0PCR);                    (24)
        outl(1 < < 5, SpiDevices\[num\].BaseAddr + S0PCR -S0PCR);          (25)
        filp ->private_data =&SpiDevices\[num\];                           (26)
        local_irq_restore(flag);                                           (27)
        }
        SpiDevices\[num\].usage ++;                                        (28)
        MOD_INC_USE_COUNT;                                                 (29)
        return 0;          /* success */                                   (30)
    }
```

目前的 SPI 驱动程序支持 LPC2000 系列芯片的 2 路标准 SPI（SPI0 和 SPI1）。程序先判断从设备号是否在支持的范围之内（程序清单 9.18（4）～（6））。MAX_SPI 是定义在 config.h 文

更多高清PDF图书 www.gqpdf.com

件中的宏，用于指示驱动支持 SPI 的路数。然后根据当前 SPI 设备的使用计数器的值进行不同的操作。如果使用计数器为 0，则表示第一次使用，需要进行初始化（程序清单 9.18（8）～（27））。程序清单 9.18（8）和（27）为关/开中断处理，用于保护临界区，关于临界区请参考 7.3.2 小节。程序清单 9.18（9）～（14）为对 SPI0 进行初始化；程序清单 9.18（15）～（20）为对 SPI1 进行初始化。对 2 路 SPI 初始化时，都对 SPI 相关引脚的连接配置进行了保存程序清单 9.18（11）和（17）。程序清单 9.18（21）为信号量初始化，详情请参考函数 sema_init（）的函数原型（见程序清单 9.19）及其使用方法。程序清单 9.18（22）～（25）设置 SPI 通道的基地址，关于 SPI 基地址的详情请参见参考文献[3]相关部分或 LPC2000 芯片用户手册。程序清单 9.18（26）为 filp->private_data 赋值，标识不同的 SPI 设备。程序清单 9.18（28）增加 SPI 使用计数器。程序清单 9.18（29）～（30）是每个驱动程序 open（）方法都必需的，不再赘述。

程序清单 **9.19**　**sema_init**（）函数原型

```
void sema_init (struct semaphore *sem, int val)
```

函数 sema_init（）的原型以指定的计数值初始化信号量。信号量一般被初始化为可用资源的数量。

注意：与信号量有关的函数定义在 <asm/semaphore.h> 中。

9.3.4　release（）方法

release（）方法会访问 I/O 端口。关于访问 I/O 端口的方法请参考 5.2.7 小节。

SPI 驱动程序 release（）方法的函数代码如程序清单 9.20 所示。程序按照程序清单 5.8 编写。

程序清单 **9.20**　**SPI** 驱动程序 **release**（）方法的函数代码

```
static int spi_release(struct inode *inode, struct file *filp)
{
    unsigned long flag;                                          (1)
    int num;                                                     (2)
    u32 temp;                                                    (3)

    num = MINOR(inode->i_rdev);                                  (4)
    if (num >= MAX_SPI)                                          (5)
    {
        return -ENODEV;                                          (6)
    }
    MOD_DEC_USE_COUNT;                                           (7)
    SpiDevices\[num\].usage--;                                   (8)
```

```
if (SpiDevices\[num\].usage == 0)                                    (9)
{
    local_irq_save(flag);                                            (10)
    if (num == 0)                                                    (11)
    {
        temp = inl(PINSEL0);                                         (12)
        temp &= ~(0xff << 8);                                        (13)
        temp |= PinSel0Save;                                         (14)
        outl(temp, PINSEL0);                                         (15)
    }
    if (num == 1)                                                    (16)
    {
        temp = inl(PINSEL1);                                         (17)
        temp &= ~(0xff << 2);                                        (18)
        temp |= PinSel1Save;                                         (19)
        outl(temp, PINSEL1);                                         (20)
    }
    local_irq_restore(flag);                                         (21)
    }
    return(0);                                                       (22)
}
```

程序先获取设备的从设备号，并判断是否在合法的范围内（程序清单 9.20（4）~（6））。程序清单 9.20（7）和（8）分别将模块使用计数器和 SPI 设备使用计数器递减。如果 SPI 设备不再使用（使用计数器为 0），则对相应的 SPI 设备进行释放（程序清单 9.20（9）~（21））。程序清单 9.20（10）和（21）为关/开中断，用于保护临界区，关于临界区请参考 7.3.2 小节。程序清单 9.20（10）~（15）为释放 SPI0 设备，同时恢复与 SPI0 相关引脚的连接配置。程序清单 9.20（16）~（20）为释放 SPI1 设备，同时恢复与 SPI1 相关引脚的连接配置。

9.3.5 ioctl() 方法

ioctl() 方法会访问 I/O 端口。关于访问 I/O 端口的方法请参考 5.2.7 小节。

SPI 驱动程序 ioctl() 方法的实现代码如程序清单 9.21 所示。ioctl() 方法实现对 SPI 总线的大部分操作，如启动和停止 SPI 总线，设置 SPI 时钟分频、电平和相位以及 SPI 数据传输顺序等。

程序清单 **9.21** **SPI** 驱动程序 **ioctl()** 方法的实现代码

更多高清PDF图书 www.gqpdf.com

```
static int spi_ioctl(struct inode *inode, struct file *filp, unsigned int cmd, un-
signed long arg)
{
```

高清PDF藏 gqpdf.com

```
int num;                                                    (1)
u32 temp;                                                   (2)
Spi_Dev *dep;                                               (3)

num = MINOR(inode->i_rdev);                                 (4)
if (num >= MAX_SPI)                                         (5)
{
    return -ENODEV;                                         (6)
}

if (_IOC_TYPE(cmd) != SPI_IOC_MAGIC)                        (7)
{
    return -ENOTTY;                                         (8)
}
if (_IOC_NR(cmd) >= SPI_MAXNR)                              (9)
{
    return -ENOTTY;                                         (10)
}

dep = filp->private_data;                                   (11)

switch(cmd)                                                 (12)
{
    case SPI_SET_CLKDIV:                                    (13)
        if (arg < 8)                                        (14)
        {
            arg = 8;                                        (15)
        }
        arg++;                                              (16)
        arg &= ~1;                                          (17)
        outl(arg, dep->BaseAddr + S0PCCR - S0PCR);          (18)
        break;                                              (19)

    case SPI_SET_CPHA_FIRST:                                (20)
        temp = inl(dep->BaseAddr + S0PCR - S0PCR);          (21)
        temp &= ~(1 << 3);                                  (22)
        outl(temp, dep->BaseAddr + S0PCR - S0PCR);          (23)
```


更多高清PDF图书 www.gqpdf.com

```
            break;                                                          (24)

        case SPI_SET_CPHA_SECOND:                                           (25)
            temp = inl(dep->BaseAddr + S0PCR - S0PCR);                      (26)
            temp |= (1 << 3);                                               (27)
            outl(temp, dep->BaseAddr + S0PCR - S0PCR);                      (28)
            break;                                                          (29)

        case SPI_SET_CPOL_LOW:                                              (30)
            temp = inl(dep->BaseAddr + S0PCR - S0PCR);                      (31)
            temp |= (1 << 4);                                               (32)
            outl(temp, dep->BaseAddr + S0PCR - S0PCR);                      (33)
            break;                                                          (34)

        case SPI_SET_CPOL_HIGHT:                                            (35)
            temp = inl(dep->BaseAddr + S0PCR - S0PCR);                      (36)
            temp &= ~(1 << 4);                                              (37)
            outl(temp, dep->BaseAddr + S0PCR - S0PCR);                      (38)
            break;                                                          (39)

        case SPI_SET_LSBF_BIT0:                                             (40)
            temp = inl(dep->BaseAddr + S0PCR - S0PCR);                      (41)
            temp |= (1 << 6);                                               (42)
            outl(temp, dep->BaseAddr + S0PCR - S0PCR);                      (43)
            break;                                                          (44)

        case SPI_SET_LSBF_BIT7:                                             (45)
            temp = inl(dep->BaseAddr + S0PCR - S0PCR);                      (46)
            temp &= ~(1 << 6);                                              (47)
            outl(temp, dep->BaseAddr + S0PCR - S0PCR);                      (48)
            break;                                                          (49)

        case SPI_WR_DATA:                                                   (50)
            if (! access_ok(VERIFY_READ, (void *)arg, _IOC_SIZE(cmd)))      (51)
            {
                return -EFAULT;                                             (52)
            }
```

```
if (! access_ok(VERIFY_WRITE, (void *)arg, _IOC_SIZE(cmd)))
{
    return -EFAULT;
}
get_user(temp, (u8 *)arg);

outl(temp, dep->BaseAddr + S0PDR - S0PCR);
while (1)
{
    temp = inl(dep->BaseAddr + S0PSR - S0PCR);
    if ((temp & (1 << 7)) != 0)
    {
        break;
    }
    if (temp != 0)
    {
        return -EIO;
    }
    temp = inl(dep->BaseAddr + S0PDR - S0PCR);

    put_user(temp, (u8 *)arg);
    break;

case SPI_START:
    if (down_interruptible(&dep->sem))
    {
        return -ERESTARTSYS;
    }
    break;

case SPI_END:
    up(&dep->sem);
    break;

default:
    return -ENOTTY;
```


更多高清PDF图书 www.gqpdf.com

```
            break;                                                    (75)
        }

        return 0;
    }
```

程序先获取从设备号并判断是否合法(程序清单 9.21(4) ~ (6)),然后判断命令的幻数和命令编号是否正确(程序清单 9.21(7) ~ (10))。参数都合法后,通过获取 filp->private_data 的值,确定将要操作的 SPI 设备(程序清单 9.21(11)),接续行对命令进行分支处理(程序清单 9.21(13) ~ (75)),所有命令中除写命令 SPI_WR_DATA 外,都比较简单,参见参考文献[3]相关部分,很容易理解。下面着重介绍 SPI_WR_DATA 命令。

根据 SPI 总线特性,当主机往从机发送一字节数据的同时,从机也同时向主机返回一字节数据。ioctl() 方法的 SPI_WR_DATA 命令实现了这个特性。程序先判断用于接收返回数据的缓冲区是否合法(程序清单 9.21(51) ~ (54));然后通过 get_user() 把数据由用户空间拷贝到内核空间(程序清单 9.21(55)),关于 get_user() 的用法请参考 5.2.6 小节。程序清单 9.21(56) ~ (63) 进行数据写入和接收,得到从机返回的数据后,通过 put_user() 将结果拷贝到用户空间(程序清单 9.21(64)),put_user() 的用法请参考 5.2.6 小节。

另外,程序清单 9.21(67) 和 (71) 分别使用了两个操作信号量的函数,其函数原型如程序清单 9.22 所示。

程序清单 **9.22　down_interruptible() 和 up() 函数原型**

```
int down_interruptible(struct semaphore * sem)
void up(struct semaphore * sem)
```

down_interruptible() 试图获得指定的信号量,如果信号量已被使用,则进入可中断睡眠状态;如果成功获取信号量,则返回 0;如果被中断了,则返回 - EINTR。它们都有惟一的指针指向信号量结构体变量。

up() 发送指定的信号量,如果睡眠队列不空,则唤醒其中一个任务。

注意:与信号量有关的函数定义在 <asm/semaphore.h> 中。

程序清单 9.21(67) 中,启动 SPI 总线,在启动 SPI 总线的同时试图获取信号量。只有获得了信号量,SPI 总线才启动成功,保证享有 SPI 总线独占。程序清单 9.21(71) 停止 SPI 总线,同时释放信号量,取消对 SPI 总线的占有权。

9.3.6　write() 方法

SPI 驱动程序 write() 方法的实现代码如程序清单 9.23 所示。该程序是按照程序清单 5.10 编写的。

程序清单 **9.23**　**SPI** 驱动程序 **write**（）方法代码

高清PDF
gqpdf.com

更多高清PDF图书　www.gqpdf.com

```
static ssize_t spi_write(struct file *filp, const char *buf, size_t count, loff_
t *f_pos)
{
    size_t i;                                                          (1)
    unsigned int temp;                                                 (2)
    Spi_Dev *dep;                                                      (3)

    if (! access_ok(VERIFY_READ, (void *)buf, count))                  (4)
    {
        return -EFAULT;                                                (5)
    }

    dep = filp->private_data;                                          (6)

    i = count;                                                         (7)
    do                                                                 (8)
    {
        get_user(temp, (u8 *)buf);                                     (9)
        outl(temp, dep->BaseAddr + S0PDR - S0PCR);                     (10)
        while (1)                                                      (11)
        {
            temp = inl(dep->BaseAddr + S0PSR - S0PCR);                 (12)
            if ((temp & (1 << 7)) != 0)                                (13)
            {
                break;                                                 (14)
            }
            if (temp != 0)                                             (15)
            {
                goto end;                                              (16)
            }
            inl(dep->BaseAddr + S0PDR - S0PCR);                        (17)
            buf ++;                                                    (18)
        } while (--i != 0);                                            (19)

end:                                                                   (20)
```

高清藏 gqpdf.com

```
    return count -i;                                                    (21)
}
```

程序先检查发送缓冲区的合法性（程序清单 9.23（4）~（5）），然后通过 filp->private_data 确定将要操作的 SPI 设备（程序清单 9.23（6））。程序清单 9.23（9）通过 get_user（）将要写入的数据拷贝到内核空间；程序清单 9.23（10）则把 get_user（）拷贝到内核空间的数据写到相应 SPI 的数据寄存器。程序清单 9.23（13）~（16）查询 SPI 总线状态，并进行处理。程序清单 9.23（17）则读取 SPI 返回的数据，然后缓冲区指针递增（程序清单 9.23（18）），直到写完需要写入的个数（程序清单 9.23（19）），最后返回已写入数据的个数（程序清单 9.23（21））。

9.3.7 read（）方法

SPI 总线驱动程序 read（）方法的实现代码如程序清单 9.24 所示。该程序是按照程序清单 5.9 编写的。

程序清单 **9.24** **SPI** 总线驱动程序 **read**（）方法的实现代码

更多高清PDF图书 www.gqpdf.com

```
static ssize_t spi_read(struct file *filp, char *buf, size_t count, loff_t *f_
pos)
{
    size_t i;                                                          (1)
    unsigned int temp;                                                 (2)
    Spi_Dev *dep;                                                      (3)

    if (! access_ok(VERIFY_WRITE, (void *)buf, _IOC_SIZE(cmd)))        (4)
    {
        return -EFAULT;                                                (5)
    }

    dep = filp->private_data;                                          (6)

    i = count;                                                         (7)
    do                                                                 (8)
    {
        outl(0, dep->BaseAddr + S0PDR - S0PCR);                        (9)
        while (1)                                                      (10)
        {
            temp = inl(dep->BaseAddr + S0PSR - S0PCR);                 (11)
            if ((temp & (1 << 7)) !=0)                                 (12)
            {
```

高清藏 PDF gqpdf.com

```
            break;                                                    (13)
        }
        if (temp != 0)                                                (14)
        {
            goto end;                                                 (15)
        }
        temp = inl(dep -> BaseAddr + S0PDR - S0PCR);                  (16)
        put_user(temp, (u8 *)buf);                                    (17)
        buf ++;                                                       (18)
    } while (--i != 0);                                               (19)

end:                                                                  (20)
    return count - i;                                                 (21)
}
```

程序先检查接收缓冲区的合法性（程序清单 9.24（4）~（5）），然后通过 filp->private_data 确定将要操作的 SPI 设备（程序清单 9.24（6））。程序清单 9.24（9）往 SPI 数据寄存器写入一字节数据 0（根据 SPI 总线特性，在读取数据之前必须写入一字节数据），然后等待 SPI 数据写入完毕（程序清单 9.24（10）~（15））。程序清单 9.24（16）读取 SPI 返回的数据，程序清单 9.24（17）通过 put_user（）将读到的数据送到用户空间的接收缓冲区。然后接收缓冲区指针递增，个数递减，直到读取完毕（程序清单 9.24（18）、（19）），最后返回读取的个数（程序清单 9.24（21））。

9.3.8 其他说明

1. 未实现的方法

本驱动程序未实现 llseek（）方法。

2. 修改版权及作者声明

这部分比较简单，请参考 6.3.6 小节。

本章小结

本章以 LPC2200 系列的 SPI 为例介绍了实际的 Linux（μClinux）字符设备驱动程序的相关知识。首先介绍该驱动程序的使用方法，然后对源码进行分析。

相对于第 5 章，本章主要增加了信号量的使用方法（参考 9.3.5 小节）。

更多高清PDF图书 www.gqpdf.com

第10章 I^2C（字符设备）驱动程序编写范例

LPC2200 系列 ARM 微控制器具有 1 个标准的 I^2C 总线接口，其特性如下：

- 标准的 I^2C 总线接口；
- 可配置为主机、从机或主/从机；
- 可编程时钟可实现通用速率控制；
- 主/从机之间双向数据传输；
- 多主机总线（无中央主机）；
- 同时发送的主机之间进行仲裁，避免了总线数据的冲突。

关于这个 I^2C 总线接口的详细说明请参见参考文献[3]。

本章介绍的驱动程序就是驱动 LPC2200 系列的 I^2C 总线接口，仅可配置为主机，不可配置为从机。驱动程序可设置总线频率，可通过这个总线接口控制 I^2C 从器件。

10.1 在应用程序中使用 I^2C 总线驱动程序

关于字符设备驱动程序的一般使用方法请参考 5.3 节，本节仅介绍使用 I^2C 总线驱动程序需要特殊注意的地方。

注意：本驱动程序仅操作 I^2C 总线让应用程序与相应的器件通信而已，对具体器件的操作还需要看器件手册。

10.1.1 建立设备文件

驱动程序默认的主设备号为 122，从设备号用来标识 I^2C 从器件的地址。例如，cat1025 的地址为 0xA0，则其设备号为 160；zlg7290 的地址为 0x70，则其设备号为 112；lm75 的地址为 0x90，则其设备号为 144。程序清单 10.1 为建立这几个器件的设备文件的命令。

程序清单 **10.1** 建立设备文件

```
mknod /dev/cat1025 c 122 160
```

```
mknod /dev/zlg7290 c 122 112
mknod /dev/lm75 c 122 144
```

10.1.2 可使用的函数

虽然本驱动程序实现了 read() 和 write() 方法，但本驱动程序不能使用标准 C 的库函数来访问，这是因为每次 read() 和 write() 调用是不可分割的。对于 I^2C 从器件来说，把 10 字节数据一次写入和分两次写入结果可能大不相同，所以只能使用内核提供给应用程序的接口函数来访问，并且仅部分内核接口函数可使用，其具体情况见表 10.1，具体使用方法见 5.3.3 小节。

表 10.1 可使用的内核接口函数

函数名称	函数功能	函数原型	备 注
open()	打开文件	int open(const char * pathname, int flags);	在使用 I^2C 从器件前调用，由此函数确定操作的从器件地址
close()	关闭文件	int close(int fd);	在使用完 I^2C 后调用
ioctl()	I/O 控制	int ioctl(int fd, unsigned long int cmd, ...);	实际控制 I^2C
read()	读数据	ssize_t read(int fd, void * buf, size_t count);	从 I^2C 从器件中读数据
write()	写数据	ssize_t write(int fd, void * buf, size_t count);	把数据写入 I^2C 从器件中

10.1.3 提供给应用程序的头文件

函数 ioctl() 的第 2 个参数 cmd 为驱动程序的特殊命令，其值应由驱动程序提供。本驱动程序提供了一个头文件 i2c.h 定义这些命令。文件 i2c.h 的内容见程序清单 10.2。其中，程序清单 10.2(2) ~ (3) 定义了 2 个宏，对应 2 条命令。关于各具体命令的说明，请参考表 10.2。

程序清单 **10.2 i2c.h**

更多高清PDF图书 www.gqpdf.com

```
#ifndef __I2C_H
#define __I2C_H
#include <linux/ioctl.h>

#define I2C_IOC_MAGIC          0xd4                                    (1)

#define I2C_SET_CLH            _IO(I2C_IOC_MAGIC,    0)                (2)
#define I2C_SET_CLL            _IO(I2C_IOC_MAGIC,    1)                (3)

#define I2C_MAXNR              2 /* max command */                     (4)
#endif
```

表 10.2　I^2C 命令

命　令	功　能	备　注
I2C_SET_CLH	设置高电平时间	参考 10.1.4 小节
I2C_SET_CLL	设置低电平高电平	

10.1.4　设置 I^2C 总线频率

在使用 I^2C 总线接口前必须设置总线频率。ioctl()方法的两个命令都是为设置总线服务的，其中 I2C_SET_CLH 为设置高电平时间(VBP 总线周期为单位)，I2C_SET_CLL 为设置低电平时间(VBP 总线周期为单位)。若高低电平时间都确定了，则周期便确定了，也就确定了总线频率。设置总线频率的例子见程序清单 10.3，在 VBP 总线(参见参考文献[3])频率为 11 059 200 Hz 时，I^2C 总线频率为 30 000 Hz。

注意：I^2C 总线频率是针对 I^2C 总线接口的。如果同时使用多个 I^2C 从器件，则只需设置一次频率。当系统中(不是应用程序中)所有与基于 I^2C 的设备文件均关闭时，总线恢复默认频率。总线默认频率为可能的最低值。

程序清单 **10.3**　设置 **I^2C** 总线频率范例

```
int fd;
......
fd = open("/dev/zlg7290", O_RDWR);                                     (1)
......
ioctl(fd, I2C_SET_CLH, ((11059200 /30000) + 1) /2);                    (2)
ioctl(fd, I2C_SET_CLL, (11059200 /30000) /2);                          (3)
......
```

10.1.5 写数据到 I²C 从器件

写数据到 I²C 从器件很简单，按照函数 write() 的规范调用即可。写数据到 I²C 从器件的例子见程序清单 10.4。

程序清单 **10.4** 写数据到 **I²C** 从器件的例子

```
int fd;
uint8 temp\[2\];
......
fd = open("/dev/zlg7290", O_RDWR);                    (1)
......
write(fd, temp, 1);                                   (2)
......
```

10.1.6 从 I²C 从器件读数据

从 I²C 从器件读数据很简单，按照函数 read() 的规范调用即可。从 I²C 从器件读数据的例子见程序清单 10.5。

程序清单 **10.5** 从 **I²C** 从器件读数据的例子

```
int fd;
uint8 temp\[2\];
......
fd = open("/dev/zlg7290", O_RDWR);                    (1)
......
read (fd, temp, 2);                                   (2)
......
```

10.2 编 译

当建立相关的开发环境和获取驱动程序的源代码后，只需建立（或修改）Makefile 文件，通过 make 命令即可编译驱动程序。建立开发环境不是本书的任务，请读者自行参考相关文档。本书介绍的源代码可在广州致远电子有限公司出品的相关产品中获得，或根据本书自行输入。下面介绍本驱动所用的 Makefile 文件（见程序清单 10.6）。

程序清单 **10.6** **Makefile** 文件

更多高清PDF图书 www.gqpdf.com

```
EXEC = gpio
OBJS = gpio.o
SRC = gpio.c

INCLUDE = /work/uClinux-dist/linux-2.4.x/include
CC = arm-elf-gcc
LD = arm-elf-ld
MODCFLAGS = -D__KERNEL__ -I $(INCLUDE) -Wall -O2 -fno-strict-aliasing -fno-common -
pipe -fno-builtin -D__linux__ -g -DNO_MM -mapcs-32 -march = armv4 -mtune = arm7tdmi -
mshort-load-bytes -msoft-float -nostdinc -iwithprefix include
LDFLAGS = -m armelf -r

all: $(EXEC)
```

与程序清单4.2不同的部分

根据实际情况修改的部分

高清PDF藏 gqpdf.com

```
$(EXEC): $(OBJS)
    $(LD) $(LDFLAGS) -o $@  $(OBJS)

%.o:%.c
    $(CC) $(MODCFLAGS) -mapcs -c $< -o $@

clean:
    -rm -f $(EXEC) *.elf *.gdb *.o
```

可以看出，程序清单 10.6 与程序清单 4.2 十分相似，仅头 3 句不同。实际上，程序清单 10.6 是根据程序清单 4.2 修改而来的，因此，关于 Makefile 的详细介绍请参考 4.3 节。

10.3 源码分析

事实上，第 5 章已介绍了编写字符驱动程序的一般知识，而本驱动程序也是在那个驱动程序的基础上修改而来的，因此，应先阅读第 5 章，否则不易理解本节内容。

注意：I^2C 总线驱动只操作 I^2C 总线，并不涉及 I^2C 总线上所接 I^2C 器件的特殊操作。但是，使用 I^2C 驱动时，在 I^2C 总线上必须接有 I^2C 器件，驱动程序将所接器件的从地址作为 I^2C 总线驱动的从设备号，而且需要根据 I^2C 器件的数据手册操作相应的 I^2C 器件。

10.3.1 初始化函数

I^2C 驱动程序的初始化代码如程序清单 10.7 所示。这是按照程序清单 5.2 编写的，实际上与程序清单 6.16 是一样的，关于这部分代码请参考 6.3.1 小节。

程序清单 **10.7** **I^2C 驱动程序的初始化代码**

```
int i2c_init(void)
{
    int  result;

    result = register_chrdev(MAJOR_NR,  DEVICE_NAME,  &i2c_fops);
    if (result < 0)
    {
        printk(KERN_ERR DEVICE_NAME ": Unable to get major %d\\n", MAJOR_NR );
        return(result);
    }

    if (MAJOR_NR == 0)
```


更多高清PDF图书 www.gqpdf.com

```
{
    MAJOR_NR = result; /* dynamic */
}

printk(KERN_INFO DEVICE_NAME ": init OK\\n");
return(0);
}
```

10.3.2 清除函数

I^2C 驱动程序的清除函数代码如程序清单 10.8 所示。它与程序清单 5.4 一样，这里不再详细介绍。

程序清单 **10.8** **I^2C** 驱动程序的清除函数代码

```
void i2c_cleanup(void)
{
    unregister_chrdev(MAJOR_NR, DEVICE_NAME);
}
```

10.3.3 open()方法

open()方法会访问 I/O 端口。关于访问 I/O 端口的方法请参考 5.2.7 小节。

I^2C 驱动 open()方法的实现代码如程序清单 10.9 所示。它完全是按照程序清单 5.7 编写的。需要注意的是，LPC2000 系列芯片提供了硬件 I^2C 接口，可采用中断方式处理，为了提高效率，I^2C 驱动程序也基于中断方式。在驱动程序中使用中断请参考 7.3.1 小节。

程序清单 **10.9** **I^2C** 驱动程序的 **open()** 方法代码

```
static int i2c_open(struct inode *inode, struct file *filp)
{
    unsigned long flag;                                          (1)
    unsigned int num;                                            (2)
    u32 temp;                                                    (3)

    num = MINOR(inode->i_rdev);                                  (4)

    if (usage == 0)                                              (5)
    {
        request_irq(IRQ_I2C, i2c_irq_handle, SA_INTERRUPT, "my" DEVICE_NAME,
NULL);                                                           (6)
```

```
        local_irq_save(flag);                                          (7)
        temp = inl(PINSEL0);                                           (8)
        PinSel0Save = temp & (0x0f << 4);                              (9)
        temp &= ~(0xf << 4);                                           (10)
        temp |= (0x05 << 4);                                           (11)
        outl(temp, PINSEL0);                                           (12)

        outl(0x6C, I2CONCLR);          /* 清除控制寄存器 */            (13)
        outl(0xffff, I2SCLH);          /* 设置高电平时间 */            (14)
        outl(0xffff, I2SCLL);          /* 设置低电平时间 */            (15)

        sema_init(&sem, 1);                                            (16)
        sema_init(&irq_sem, 0);                                        (17)
        filp->private_data = (void *)num;                              (18)
        local_irq_restore(flag);                                       (19)
    }
    usage++;                                                           (20)
    MOD_INC_USE_COUNT;                                                 (21)

    return 0;          /* success */                                   (22)
}
```

程序清单 10.9(4) 获取 I^2C 总线上设备的从设备号，该驱动程序需要将 I^2C 器件的从地址作为从设备号。接下来判断 I^2C 总线的使用情况(程序清单 10.9(5))，如果使用计数器为 0，则表示第一次使用 I^2C 总线，需要对 I^2C 总线进行设置(程序清单 10.9(6) ~ (19))。程序清单 10.9(6) 向内核注册 I^2C 中断。程序清单 10.9(7) 和 (19) 为关/开中断，用于保护临界区，关于临界区请参考 7.3.2 小节。程序清单 10.9(8) ~ (15) 先保存与 I^2C 相关的引脚连接设置，然后对 I^2C 的一些寄存器进行初始化。程序清单 10.9(16) 和 (17) 对两个信号量进行初始化，可参考程序清单 9.19。程序清单 10.9(18) 为 private_data 赋值，标识不同的 I^2C 器件(根据驱动程序的要求，实际上传送 I^2C 器件的从地址)。

信号量 sem 实现对 I^2C 总线的保护，在使用 I^2C 总线之前必须先获取信号量，使用完毕必须发送信号量。

信号量 irq_sem 用于实现 I^2C 中断保护，在启动 I^2C 中断操作之前申请，在中断服务程序中被释放。I^2C 中断处理完毕，可成功地获取信号量 irq_sem。通过获取 irq_sem 可判断 I^2C 中断处理是否完成。

更多高清PDF图书 www.gqpdf.com

10.3.4 release()方法

release()方法会访问 I/O 端口。关于访问 I/O 端口的方法请参考 5.2.7 小节。

I^2C 驱动程序 release()方法的函数代码如程序清单 10.10 所示。程序按照程序清单 5.8 编写。对比程序清单 7.17，很容易理解该程序。

程序清单 **10.10** I^2C 驱动程序 **release()** 方法的函数代码

```
static int i2c_release(struct inode *inode, struct file *filp)
{
    unsigned long flag;                                          (1)
    u32 temp;                                                    (2)

    MOD_DEC_USE_COUNT;                                           (3)
    usage--;                                                     (4)
    if (usage == 0)                                              (5)
    {
        local_irq_save(flag);                                    (6)
        temp = inl(PINSEL0);                                     (7)
        temp &= ~(0x0f << 4);                                    (8)
        temp |= PinSel0Save;                                     (9)
        outl(temp, PINSEL0);                                     (10)
        local_irq_restore(flag);                                 (11)
        free_irq(IRQ_I2C, NULL);                                 (12)
    }

    return(0);
}
```

10.3.5 I^2C 中断处理

I^2C 中断处理代码如程序清单 10.11 所示。该程序是按照程序清单 7.11 编写的。关于中断服务程序的更多信息请参考 7.3.1 小节。

该程序会访问 I/O 端口。关于访问 I/O 端口的方法请参考 5.2.7 小节。

该程序涉及用户空间和内核空间的数据交换，请参考 5.2.6 小节。

程序清单 **10.11** I^2C 中断处理代码

```
static void i2c_irq_handle(int irq, void *dev_id, struct pt_regs *regs)
```

```
{
    unsigned int temp;                                              (1)

    temp = inl(I2STAT);                                             (2)
    switch(temp & 0xf8)                                             (3)
    {
        case  0x08:                    /* 已发送起始条件,与 0x10 相同处理 */ (4)
        case  0x10:                    /* 已发送重复起始条件 */          (5)
            outl(I2cAddr, I2DAT);      /* 发送地址 */                   (6)
            outl(0x28, I2CONCLR);      /* 清除标志 */                   (7)
            break;                                                  (8)

        case  0x18:                    /* 已发送 SLA + W,并已接收应答 */ (9)
            get_user(temp, (u8 *)I2cBuf);                           (10)
            outl(temp, I2DAT);                                      (11)
            I2cBuf ++;                                              (12)
            I2cNbyte --;                                            (13)
            outl(0x28, I2CONCLR);      /* 清除标志 */                   (14)
            break;                                                  (15)

        case  0x28:                    /* 已发送 I²C 数据,并接收到应答 */ (16)
            if (I2cNbyte > 0)                                       (17)
            {
                get_user(temp, (u8 *)I2cBuf);                       (18)
                outl(temp, I2DAT);                                  (19)
                outl(0x28, I2CONCLR); /* 清除标志 */                (20)
                I2cBuf ++;                                          (21)
                I2cNbyte --;                                        (22)
            }
            else                                                    (23)
            {
                outl(0x28, I2CONCLR); /* 清除标志 */                (24)
                outl(1 < < 4, I2CONSET);/* 结束总线 */              (25)
                up(&irq_sem);                                       (26)
            }
            break;                                                  (27)
```


更多高清PDF图书 www.gqpdf.com

```
        case  0x20:  /* 已发送 SLA + W;已接收非 ACK, 与 0x48 处理相同 */      (28)
        case  0x30:  /* 已发送 I2DAT 中的数据字节;已接收非 ACK,与 0x48 处理相同 */
                                                                              (29)
        case  0x48:  /* 已发送 SLA + R;已接收非 ACK */                        (30)
            outl(1 < < 4, I2CONSET);        /* 发送停止信号 */               (31)
            outl(0x28, I2CONCLR);           /* 清除标志 */                   (32)
            up(&irq_sem);                                                     (33)
            break;                                                            (34)

        case  0x38:                         /* 在 SLA + R/W 或数据字节中丢失仲裁 */
                                                                              (35)
            outl(0x28, I2CONCLR);           /* 清除标志 */                   (36)
            up(&irq_sem);                                                     (37)
            break;                                                            (38)

        case  0x40:                         /* 已发送 SLA + R;已接收 ACK */  (39)
            if (I2cNbyte < =1)                                                (40)
            {
                outl(1 < < 2, I2CONCLR);    /* 下次发送非应答信号 */         (41)
            }
            else
            {
                outl(1 < < 2, I2CONSET);    /* 下次发送应答信号 */           (42)
            }
            outl(0x28, I2CONCLR);           /* 清除标志 */                   (43)
            break;                                                            (44)

        case  0x50:                         /* 已接收数据字节;已发送 ACK */  (45)
            temp = inl(I2DAT);                                                (46)
            put_user(temp, (u8 *)I2cBuf);                                     (47)

            /* 接收数据 */
            I2cBuf ++;                                                        (48)
            I2cNbyte --;                                                      (49)
            if (I2cNbyte < =1)                                                (50)
            {
                outl(1 < < 2, I2CONCLR); /* 下次发送非应答信号 */             (51)
            }
            outl(0x28, I2CONCLR);           /* 清除标志 */                   (52)
```

高清藏 gqpdf.com

```
            break;                                                          (53)

        case  0x58:                         /* 已接收数据字节;已发送非 ACK */   (54)
            temp = inl(I2DAT);                                              (55)
            put_user(temp, (u8 *)I2cBuf);                                   (56)

            /* 接收数据 */
            I2cNbyte--;                                                     (57)
            outl(1 << 4, I2CONSET);   /* 结束总线 */                        (58)
            outl(0x28, I2CONCLR);     /* 清除标志 */                        (59)
            up(&irq_sem);                                                   (60)
            break;                                                          (61)

        default:                                                            (62)
            outl(0x28, I2CONCLR);     /* 清除标志 */                        (63)
            break;                                                          (64)
    }
}
```

该中断服务程序对 I^2C 状态机的处理,完全按照硬件 I^2C 规范编写,这里不再介绍,请参见参考文献[3]相关部分。

其中用到了信号量 irq_sem(程序清单 10.11(26)、(33)、(37)和(60))。中断服务程序使用 up()方法通知驱动程序其他部分:总线已使用完毕,可进行新的数据传输。关于信号量可参考 9.3.3 和 9.3.5 小节。

10.3.6 ioctl()方法

ioctl()方法会访问 I/O 端口。关于访问 I/O 端口的方法请参考 5.2.7 小节。

I^2C 驱动程序 ioctl()方法的实现代码如程序清单 10.12 所示。ioctl()方法实现对 I^2C 总线速率的设定。

程序清单 **10.12** I^2C 驱动程序 **ioctl()**方法的实现代码

```
static int i2c_ioctl(struct inode *inode, struct file *filp, unsigned int cmd, un-
signed long arg)
{
    if (_IOC_TYPE(cmd) != I2C_IOC_MAGIC)                                    (1)
    {
        return -ENOTTY;                                                     (2)
```


更多高清PDF图书 www.gqpdf.com

```
    }
    if (_IOC_NR(cmd) >= I2C_MAXNR)                                    (3)
    {
        return -ENOTTY;                                               (4)
    }

    switch(cmd)                                                       (5)
    {
        case I2C_SET_CLH:                                             (6)
            if (arg < 4)                                              (7)
            {
                arg=4;                                                (8)
            }
            outl(arg, I2SCLH);              /* 设置高电平时间 */       (9)
            break;                                                    (10)

        case I2C_SET_CLL:                                             (11)
            if (arg < 4)                                              (12)
            {
                arg=4;                                                (13)
            }
            outl(arg, I2SCLL);              /* 设置低电平时间 */       (14)
            break;                                                    (15)

        default:                                                      (16)
            return -ENOTTY;                                           (17)
            break;                                                    (18)
    }

    return 0;                                                         (19)
    }
```

程序先判断命令的幻数是否合法（程序清单 10.12（1）、（2）），然后判断命令编号是否合法（程序清单 10.12（3）、（4））。接下来根据命令进行分支处理（程序清单 10.12（6）～（18））。其中程序清单 10.12（7）、（8）以及程序清单 10.12（12）、（13）将小于 4 的参数设置为 4 是由 I^2C 寄存器决定的，请参见参考文献[3]。

10.3.7 write()方法

I^2C 驱动程序 write()方法的实现代码如程序清单 10.13 所示。该程序是按照程序清单 5.10 编写的。

程序清单 **10.13** **I^2C 驱动程序 write()方法的实现代码**

高清PDF藏 gqpdf.com

更多高清PDF图书 www.gqpdf.com

```
static ssize_t i2c_write(struct file *filp, const char *buf, size_t count, loff_t
*f_pos)
{
    unsigned long flag;                                                    (1)
    unsigned int num;                                                      (2)

    if (! access_ok(VERIFY_READ, (void *)buf, count))                      (3)
    {
        return -EFAULT;                                                    (4)
    }

    num = (unsigned int)filp->private_data;                                (5)

    if (down_interruptible(&sem))                                          (6)
    {
        return -ERESTARTSYS;                                               (7)
    }

    local_irq_save(flag);                                                  (8)
    outl(0x6C, I2CONCLR);                                                  (9)
    outl(0x40, I2CONSET);          /* 使能 I²C */                          (10)
    I2cAddr = num & 0xfe;          /* 存储发送地址 */                      (11)
    I2cNbyte = count;              /* 存储写字节数 */                      (12)
    I2cBuf = (u8 *)buf;            /* 存储写的数据的指针 */                (13)
    outl(0x24, I2CONSET);          /* 设置为主机，并启动总线 */            (14)
    local_irq_restore(flag);                                               (15)

    if (down_interruptible(&irq_sem))                                      (16)
    {
        up(&sem);                                                          (17)
        return -ERESTARTSYS;                                               (18)
    }
```

高PDF 清藏 gqpdf.com

```
        up(&sem);                                                    (19)
        return count - I2cNbyte;                                     (20)
    }
```

程序先判断写缓冲区是否合法(程序清单 10.13(3)、(4))。程序清单 10.13(5)获取设备标识(根据驱动要求,实际获取 I^2C 器件 I^2C 从地址)。程序清单 10.13(6)、(7)获取信号量 sem,获取成功,则独占 I^2C 总线。程序清单 10.13(8)和(15)为关/开中断,用于保护临界区,关于临界区请参考 7.3.2 小节。程序清单 10.13(9)~(14)为设置 I^2C 读操作的一些参数和启动I^2C 总线。程序清单 10.13(16)为获取使用 IRQ 中断的信号量 irq_sem(在 open()方法中,将irq_sem 设置为 0,数据传输完成,中断服务程序会将 irq_sem 设置为 1,即可获取成功):如果获取不成功,则释放使用 I^2C 总线的信号量 sem(程序清单 10.13(17)),并返回错误代码(程序清单 10.13(18));获取成功,则先释放信号量 sem(程序清单 10.13(19)),然后返回写入个数(程序清单 10.13(20))。

有关信号量操作的函数请参考 9.3.3 和 9.3.5 小节。

10.3.8 read()方法

I^2C 总线驱动程序 read()方法的实现代码如程序清单 10.14 所示。该程序是按照程序清单 5.9 编写的。

程序清单 **10.14** I^2C 总线驱动程序 **read**()方法的实现代码

gqpdf.com

更多高清PDF图书 www.gqpdf.com

```
static ssize_t i2c_read(struct file *filp, char *buf, size_t count, loff_t *f_
pos)
{
    unsigned long flag;                                                    (1)
    unsigned int num;                                                      (2)

    if (! access_ok(VERIFY_WRITE, (void *)buf, _IOC_SIZE(cmd)))            (3)
    {
        return -EFAULT;                                                    (4)
    }

    if (down_interruptible(&sem))                                          (5)
    {
        return -ERESTARTSYS;                                               (6)
    }

    num = (unsigned int)filp->private_data;                                (7)
```

高清藏 PDF gqpdf.com

```
    local_irq_save(flag);                                              (8)
    outl(0x6C, I2CONCLR);                                              (9)
    outl(0x40, I2CONSET);              /* 使能 I²C */                  (10)
    I2cAddr = num | 0x01;              /* 存储发送地址 */              (11)
    I2cNbyte = count;                  /* 存储读字节数 */              (12)
    I2cBuf = (u8 *)buf;                /* 存储读到的数据 */            (13)
    outl(0x24, I2CONSET);              /* 设置为主机,并启动总线 */     (14)
    local_irq_restore(flag);                                           (15)

    if (down_interruptible(&irq_sem))                                  (16)
    {
        up(&sem);                                                      (17)
        return -ERESTARTSYS;                                           (18)
    }
    up(&sem);                                                          (19)
    return count - I2cNbyte;                                           (20)
}
```

I^2C 总线驱动程序 read() 方法的实现代码与该驱动程序 write() 方法的实现代码很类似。对比程序清单 10.13，很容易理解，不再赘述。

10.3.9 其他说明

1. 未实现的方法

本驱动程序未实现 llseek() 方法。

2. 修改版权及作者声明

这部分比较简单，请参考 6.3.6 小节。

本章小结

本章以 LPC2200 系列的 PWM 为例介绍了实际的 Linux（μClinux）字符设备驱动程序的相关知识。首先介绍该驱动的使用方法，然后对源码进行分析。

更多高清PDF图书 www.gqpdf.com

第11章　块设备驱动程序初步

11.1　几个概念

块设备是指数据传输以块为单位的(例如软盘和硬盘)设备,这里硬件的块一般被称作“扇区(sector)”。而名词“块”常用来指软件上的概念:驱动程序常常使用1 KB大小的块,即扇区大小为512字节。

块设备是文件系统的宿主,如磁盘。在大多数Unix系统中,只能将块设备看作多个块进行访问。Linux(μClinux)允许像字符设备那样读取块设备——允许一次传输任意数目的字节。结果是,块设备和字符设备只在内核内部的管理上有所区别,因此也就是在内核/驱动程序间的软件接口上有所区别。如同字符设备一样,每个块设备也通过文件系统节点来读/写数据,它们之间的不同对用户来说是透明的。块设备驱动程序和内核的接口与字符设备驱动程序的接口相同,也通过一个传统的面向块的接口与内核通信,但该接口对用户来说是不可见的。

11.1.1　关于主设备号和从设备号

Linux(μClinux)使用设备号来标识块设备。设备号分为主设备号和从设备号。主设备号标识设备对应的驱动程序。每个在内核中活动的块设备驱动程序,包括编译进内核的和后来动态加载的,都有惟一的主设备号。内核利用主设备号将设备与相应的驱动程序对应起来。次设备号只由设备驱动程序使用,内核的其他部分不使用它,仅将它传递给驱动程序。驱动程序可按照自己的方法使用从设备号。通常,驱动程序使用从设备号区分自己管理的不同的物理设备和/或用来区分不同的分区。因此,由主设备号和从设备号可确定当前内核支持的某个具体的物理设备和/或某个具体物理设备的某个分区。

向系统增加一个驱动程序意味着要赋予它一个主设备号。一旦设备注册到内核表中,无论何时操作与设备驱动程序的主设备号匹配的设备文件,内核都会使用驱动程序中的正确函数。

注意:字符设备也有设备号,它与块设备的设备号相互独立。

11.1.2　关于设备节点、节点文件和设备文件

设备节点、节点文件、设备文件及近似的其他名词实际上是指同一样东西。Linux

(μClinux)几乎把所有东西都当作文件,块设备也一样。但设备与文件毕竟不一样,于是 Linux(μClinux)中就出现了一种特殊的文件,它们存储在文件系统中,但仅占用文件目录项而不涉及存储数据。事实上,它们仅仅记录了其所属的设备类别、主设备号和从设备号等设备相关信息。这些文件称作设备文件、节点文件或设备节点。

当一个程序打开一个设备文件时,内核就可获取对应设备的设备类型、主设备号和从设备号等信息,内核也就知道了程序需要操作使用哪个设备驱动程序。程序随后对这个文件的操作都会调用相应的驱动程序的函数,同时把从设备号传递给驱动程序。

因此,对设备文件的操作就是对其所代表的物理设备的操作。

11.2 最简单的块设备驱动程序

11.2.1 驱动程序代码

程序清单 11.1 为一个简单的块设备驱动程序的完整源码。理论上说这个代码还可简化,但再简化就不容易使用了,所以这里仍然被称为最简单的块设备驱动程序(注意是特指块设备驱动程序)。其实,这仅仅是一个可编译、加载和卸载的驱动程序,不能完成任何实际的工作。不过,可把它当作模板,块设备驱动程序就从这里开始编写。

程序清单 **11.1** 简单的块设备驱动程序

```
#ifndef __KERNEL__
    #define __KERNEL__                                                    (1)
#endif
#ifndef MODULE
    #define MODULE                                                        (2)
#endif

#include <linux/module.h>                                                 (3)
#include <linux/sched.h>                                                  (4)
#include <linux/kernel.h>     /* printk() */                              (5)
#include <linux/init.h>                                                   (6)

static int test_open(struct inode *inode, struct file *filp);             (7)
static int test_release(struct inode *inode, struct file *filp);          (8)
static int test_ioctl(struct inode *inode, struct file *filp, unsigned int cmd,
                      unsigned long param);                               (9)
static int check_test_change(kdev_t dev);                                 (10)
```


更多高清PDF图书 www.gqpdf.com

```
static int test_revalidate(kdev_t dev);                                    (11)
int  test_init(void);                                                      (12)
void test_cleanup(void);                                                   (13)

#define MAJOR_NR 125                                                       (14)
#define DEVICE_NAME "mydrive"                    /* name for messaging */ /
                                                                           (15)
#define DEVICE_NR(device) MINOR(device)                                    (16)
#define DEVICE_NO_RANDOM                                                   (17)
#define DEVICE_OFF(d)                                                      (18)
module_init(test_init);                                                    (19)
module_exit(test_cleanup);                                                 (20)

#include <linux/blk.h>                                                     (21)

static struct block_device_operations test_fops =     /* driver info */   (22)
{
    owner:                THIS_MODULE,                                     (23)
    open:                 test_open,                                       (24)
    release:              test_release,                                    (25)
    ioctl:                test_ioctl,                                      (26)
    check_media_change:   check_test_change,                               (27)
    revalidate:           test_revalidate,                                 (28)
};
static int  test_open(struct inode *inode, struct file *filp)              (29)
{
    MOD_INC_USE_COUNT;                                                     (30)
    return 0;   /* success */                                              (31)
}
static int  test_release(struct inode *inode, struct file *filp)           (32)
{
    MOD_DEC_USE_COUNT;                                                     (33)
    return 0;                                                              (34)
}
static int test_ioctl(struct inode *inode, struct file *filp,
                      unsigned int cmd, unsigned long arg)                 (35)
```

这就是主设备号

块设备必需的结构体变量

配对使用，关键代码

```
{
    return 0;                                                    (36)
}
static int check_test_change(kdev_t dev)                         (37)
{
    return 0;                                                    (38)
}
static int test_revalidate(kdev_t dev)                           (39)
{
    return 0;                                                    (40)
}
int  test_init(void)                                             (41)
{
    int  result;

    result = register_blkdev(MAJOR_NR,  DEVICE_NAME,  &test_fops);   (42)
    if (result < 0)                                              (43)
    {
        printk(KERN_ERR DEVICE_NAME ": Unable to get major % d\\n", MAJOR_NR );
                                                                 (44)
        return(result);                                          (45)
    }
    printk(KERN_ERR DEVICE_NAME ": init OK\\n");                 (46)
    return(0);                                                   (47)
}

void test_cleanup(void)                                          (48)
{
    unregister_blkdev(MAJOR_NR, DEVICE_NAME);                    (49)
}
```

特别注意，一切变化从这里开始

两者要配对

程序清单 11.1 看起来比较长，但实际上去掉宏定义、包含文件及函数声明外内容并不多。事实上，核心部分仅有两个函数（初始化函数和清除函数）和一个数据结构变量（test_fops）。

11.2.2 注册初始化函数和清除函数

在 Linux（μClinux）2.4 中，内核模块的初始化函数名为 init_module（），清除函数的函数名

更多高清PDF图书 www.gqpdf.com

为 cleanup_module()；但如果驱动程序需要编译进内核，则初始化函数和清除函数不能与内核的其他部分（包括其他驱动程序）同名。这样，如果驱动程序要编译成内核模块又可能编译到内核中时，就比较麻烦。不过，在 <linux/init.h> 这个头文件中定义了两个宏 module_init() 和 module_exit() 用来屏蔽两者差别。事实上，使用这两个宏可使驱动程序向上兼容 Linux（μClinux）2.6，而兼容 Linux（μClinux）2.0 也会比较方便。程序清单 11.1(19) 使用 module_init() 注册驱动程序的初始化函数，宏 module_init() 的参数就是初始化函数的名称。而程序清单 11.1(20) 使用 module_exit() 注册驱动程序的清除函数，宏 module_exit() 的参数为驱动程序的清除函数。

11.2.3 驱动程序的初始化函数

Linux（μClinux）在加载内核模块时会调用驱动程序初始化函数。驱动程序初始化函数的示意代码见程序清单 11.2。驱动程序的初始化函数用来初始化驱动程序本身，其返回值为负数时表示出错，负数的绝对值为错误编码，它在 <linux/errno.h> 中定义。程序清单 11.1(41) ~ (47) 就是驱动程序初始化函数的例子。其实，这个函数最重要的是调用系统提供的函数 register_blkdev()（程序清单 11.1(42)）来注册块设备驱动程序，其他代码都是错误处理和显示处理。函数 register_blkdev() 的原型见程序清单 11.3，它具有 3 个参数：第 1 个为驱动程序的主设备号；第 2 个参数为驱动程序名称（字符串）；第 3 个参数为结构体 block_device_operations 指针，这个结构体保存驱动程序的操作块设备的方法。这个结构体后面还会介绍，这里不再说明。

程序清单 **11.2** 模块初始化函数的示意代码

```
int  init_module (void)
{
    初始化模块自身(包括请求一些资源);
    return   register_blkdev(MAJOR_NR,  DEVICE_NAME,  &test_fops);
}
```

注册块设备

程序清单 **11.3** 函数 **register_blkdev()** 的原型

```
int register_blkdev(unsigned int, const char *, struct block_device_operations
*);
```

11.2.4 驱动程序的清除函数

Linux（μClinux）在卸载内核模块时会调用驱动程序的清除函数。驱动程序清除函数的示意代码见程序清单 11.4。驱动程序的清除函数主要用来释放自身获得的资源。清除函数是

一个没有参数和返回值的函数。程序清单 11.1(48)、(49)就是清除函数的例子,它仅调用系统提供的函数 unregister_blkdev()通知 Linux(μClinux)本模块已被卸载(程序清单 11.1(49))。如果驱动程序在初始化或工作时向内核申请了其他资源(如内存),在卸载时如果还没有被释放,也需要在这个函数释放。**特别注意**:释放的顺序应以申请的相反顺序执行。函数 unregister_blkdev()的原型见程序清单 11.5。它有 2 个参数:第 1 个参数为驱动程序主设备号;第 2 个参数为驱动程序的名称(字符串)。

程序清单 **11.4**　模块清除函数的示意代码

```
void  cleanup_module (void)
{
    关闭模块自身(包括释放资源);
    unregister_blkdev(MAJOR_NR, DEVICE_NAME);
}
```

删除块设备

程序清单 **11.5**　函数 **unregister_blkdev**() 的原型

```
int unregister_blkdev(unsigned int, const char *);
```

11.2.5　保存设备信息的结构体

在块设备驱动程序初始化时需要调用函数 register_blkdev(),其中第 3 个参数为指向结构体 block_device_operations 的指针,其存储了块设备的操作方法。结构体 block_device_operations 的定义见程序清单 11.6。

程序清单 **11.6**　块设备驱动程序操作方法

```
struct block_device_operations {
    int (*open) (struct inode *, struct file *);
    int (*release) (struct inode *, struct file *);
    int (*ioctl) (struct inode *, struct file *, unsigned, unsigned long);
    int (*check_media_change) (kdev_t);
    int (*revalidate) (kdev_t);
    struct module *owner;
#ifdef MAGIC_ROM_PTR
    int (*romptr) (kdev_t, struct vm_area_struct *);
#endif /* MAGIC_ROM_PTR */
};
```

程序清单 11.1(22)~(28)就是定义了这个结构体变量并进行了初始化。程序清单 11.1

更多高清PDF图书　www.gqpdf.com

(22)~(28)较容易看懂：仅对结构体指定的成员初始化，未指定的成员用默认值初始化。这是 GCC 对标准 C 的扩展，好处是，结构体成员增加时无须更改初始化代码。

下面分别介绍每个成员的意义。

1. open()

在 Linux(μClinux)和/或应用程序使用设备前会调用驱动程序的 open()方法。这个方法最重要的是调用宏 MOD_INC_USE_COUNT(程序清单 11.1(30))，这个宏主要用来增加驱动程序使用计数器，以避免不正确地卸载驱动程序。open()方法一般还用于(在第一次使用驱动程序支持的设备时)对驱动程序支持的设备进行初始化。一般情况下，open()方法的示意代码见程序清单 11.7。这个代码针对一个驱动管理多个设备的情况(常见)。如果一个驱动管理仅管理一个设备，就不需要这么复杂了，仅调用宏 MOD_INC_USE_COUNT 即可，而设备初始化可在驱动程序初始化时进行。

注意：参数 filp 是为了兼容而加入的，在这里没有任何意义。

程序清单 **11.7　open()方法的示意代码**

```
static int  open(struct inode *inode, struct file *filp)
{
    从设备号=MINOR(inode->i_rdev);                                    (1)
    通过从设备号判断需要操作的设备;                                    (2)
    通知内核进入临界区;                                                (3)
    需要操作的设备的使用计数器++;                                      (4)
    if(需要操作的设备的使用计数器==1)                                  (5)
    {
        初始化设备(包括申请资源);                                      (6)
    }
    通知内核退出临界区;                                                (7)

    MOD_INC_USE_COUNT;                                                (8)
    return 0;          /* success */                                  (9)
}
```

2. release()

在 Linux(μClinux)和/或应用程序使用设备后会调用驱动程序的 release()方法。这个方法最重要的是调用宏 MOD_DEC_USE_COUNT(程序清单 11.1(33))，这个宏与宏 MOD_INC_USE_COUNT 对应，主要用来减少驱动程序使用计数器。release()方法一般还用于(驱动程序支持的设备使用完毕时)对驱动程序支持的设备进行关闭操作。一般情况下，release()方法的示意代码见程序清单 11.8。这个代码针对一个驱动管理多个设备的情况(常见)。如果一个驱动管理仅管理一个设备，就不需要这么复杂了，仅调用宏 MOD_DEC_USE_COUNT 即可，

而关闭设备可在卸载驱动程序时进行。

注意：参数 filp 是为了兼容而加入的，在这里没有任何意义。

程序清单 **11.8** **release**() 方法示意代码

```
static int  release(struct inode *inode, struct file *filp)
{
从设备号 =MINOR(inode->i_rdev) ;                                    (1)
通过从设备号判断需要操作的设备;                                     (2)
通知内核进入临界区;                                                 (3)
需要操作的设备的使用计数器--;                                       (4)
if（需要操作的设备的使用计数器 ==0）                                (5)
{
    关闭设备处理(包括释放 revalidate()方法获取的资源;               (6)
}
通知内核退出临界区;                                                 (7)

MOD_DEC_USE_COUNT;                                                  (8)
return 0;                                                           (9)
}
```

3. ioctl()

应用程序和 Linux(μClinux) 内核可(直接或间接)通过驱动程序的 ioctl() 方法来操作块设备。针对块设备，Linux(μClinux) 内核提供了 ioctl() 的默认操作函数 blk_ioctl()，驱动程序的编辑只需书写少量的命令即可。一般地，驱动程序只需实现 BLKGETSIZE(返回设备扇区数目)、BLKRRPART(重读分区表)和 HDIO_GETGEO 命令(返回设备的物理)。当驱动程序不支持分区时，其示意代码见程序清单 11.9。

值得注意的是 HDIO_GETGEO 命令、返回块设备的柱面数和磁头数等信息，这是因为 Linux(μClinux) 最早的块设备是具有这些参数的磁存储设备——硬盘(目前对于 Linux，硬盘也是最重要的块设备)。Linux(μClinux) 使用了这些参数进行一些操作，因此，没有这些参数的块设备就需要“伪造”这些参数了。

众所周知，对于 Linux 应用程序运用于用户空间，而内核(包括类和模块)运行于内核空间，两者之间不能直接访问。ioctl() 可能需要把参数返回给运行于用户空间的程序，需要把存于内核空间变量的数据从内核空间拷贝到存于用户空间的变量。此时，程序先调用函数 access_ok() 检查是否当前进程允许存取某个地址，然后通过调用函数 copy_to_user() 实现。关于用户空间与系统空间传输数据的问题请参考 5.2.6 和 5.2.7 小节。

注意：参数 filp 是为了兼容而加入的，在这里没有任何意义。

更多高清PDF图书 www.gqpdf.com

参数 inode 保存着设备节点的信息，它具有很多成员，但绝大多数对于驱动程序没有用处，这里仅用于获取从设备号，方法是 MINOR（inode - > i_rdev）。

程序清单 **11.9　ioctl**（）方法的示意代码

高清PDF藏
gqpdf.com

```
static int ioctl(struct inode * inode, struct file * filp, unsigned int cmd, un-
signed long arg)
{
    long size;
    struct hd_geometry geo;

    switch(cmd)
    {
        case BLKGETSIZE:
            /* 返回设备扇区数目 */
            if (arg == 0)
            {
                return -EINVAL; /* NULL pointer: not valid */
            }
            if(access_ok(VERIFY_WRITE, arg, sizeof(long)) == 0)
            {
                return -EFAULT;
            }
            size = 设备扇区数目;
            if (copy_to_user((long *) arg, &size, sizeof (long)))
            {
                return -EFAULT;
            }
            return 0;
        case BLKRRPART:
            /* 如果是不支持分区的驱动程序使用以下语句 */
            return -ENOTTY;
            /* 如果是支持分区的驱动程序使用以下语句 */
            return 本驱动的 revalidate(inode->i_rdev);
        case HDIO_GETGEO:
            /* 返回设备物理参数 */
            if (access_ok(VERIFY_WRITE, arg, sizeof(geo)) == 0)
            {
                return -EFAULT;
            }
            从设备号 = MINOR(inode->i_rdev);
```


更多高清PDF图书 www.gqpdf.com

```
            通过从设备号判断需要操作的设备;
            geo.cylinders = 操作的设备的柱面数;
            geo.heads = 操作的设备的磁头数;
            geo.sectors = 操作的设备的每磁道扇区数;
            geo.start = 操作的设备的数据起始扇区索引;
            if (copy_to_user((void *) arg, &geo, sizeof(geo)))
                {
                    return -EFAULT;
                }
                return 0;
            default:
                return blk_ioctl(inode->i_rdev, cmd, arg);
        }
        return -ENOTTY;
    }
```

注意：实际的数据传输并不是在 ioctl() 方法中实现的。

事实上，这个驱动程序中根本没有数据传输的函数。驱动程序需要通过另外的系统调用注册数据传输函数，后续章节中会介绍。

4. check_media_change()

check_media_change() 方法是用来支持可移动设备的，当其返回值为 1 时，表明介质发生了变化；否则返回 0。check_media_change 方法的示意代码见程序清单 11.10。

注意：参数 dev 保存着操作的设备的主次设备号信息，使用 MINOR(dev) 可获取次设备号。

程序清单 **11.10　check_media _change()** 方法的示意代码

```
static int check_media _change(kdev_t dev)
{
    从设备号 = MINOR (dev);
    通过从设备号判断需要操作的设备;
    if(需要操作的设备的介质放生了变化)
    {
        return 1;
    }
    else
    {
        return 0;
    }
}
```

高清PDF gqpdf.com

5. revalidate()

Linux(μClinux)在检测到介质变化时会调用驱动程序的 revalidate()方法。由 revalidate()执行的操作是设备特定的,但通常用于更新设备内部状态,以便反映出新设备。最简单的 revalidate()方法是仅返回 0。revalidate()方法的示意代码见程序清单 11.11。

注意:参数 dev 保存着操作的设备的主次设备号信息,使用 MINOR(dev)可获取次设备号。

程序清单 **11.11　revalidate()方法的示意代码**

```
static int revalidate(kdev_t dev)
{
    return 0;
}
```

6. owner

用来表明驱动程序的所有者,设置为 THIS_MODULE 即可(程序清单 11.1(23))。

7. romptr

几乎没有驱动程序会用到这个方法,这里就不介绍了。

11.2.6　一些宏定义

在这个简单的驱动程序中还定义了几个宏:MAJOR_NR、DEVICE_NAME、DEVICE_NR、DEVICE_NO_RANDOM 和 DEVICE_OFF,详细说明见程序清单 11.12。至于 MAJOR_NR,它必须在“#include <linux/blk.h>”这一句之前,这是因为该头文件使用这个宏为驱动程序做了很多工作。

程序清单 **11.12**　一些宏

```
#define MAJOR_NR 125                          /* 驱动程序的主设备号,必须在包含 blk.h 前定义 */
#define DEVICE_NAME "mydrive"                 /* 驱动程序的名称 */
#define DEVICE_NR(device) MINOR(device)       /* 从 kdev 设备编号获得设备的物理编号 */
#define DEVICE_NO_RANDOM                      /* 对随机数发生器没有贡献 */
#define DEVICE_OFF(d)                         /* 没有用,为兼容而设置 */
```

11.2.7　关于头文件

本节介绍的驱动程序使用了最少的系统头文件,而真正的驱动程序一般需要包含更多的系统头文件。当不知道一个宏或函数在哪个系统头文件中声明时,一般是直接参考源代码(可利用搜索工具来查找)。

更多高清PDF图书 www.gqpdf.com

11.2.8 其他说明

本节仅介绍了块设备驱动程序基本部分，很多细节还没有介绍，其中重要且本书不会详细介绍的部分有：动态使用主设备号、支持分区的块设备、支持带参数加载等。如果想了解这些方面，可参见参考文献[2]。

另外，linux2.6 的块设备驱动程序的接口有变化，估计主要影响模块的初始化代码、open()和 release()方法。

编译块设备驱动程序主要是编写 Makefile 文件，具体请参考 4.3 节。

11.3 使用块设备

块设备是文件的载体。事实上，一般的应用程序不会直接使用块设备（直接使用块设备的应用程序一般是系统实用程序，不需要用户编写）。应用程序所有对普通文件和目录的操作都间接地操作了某个块设备，但应用程序不知道具体操作的是哪个块设备。事实上，块设备（的某个分区）是被映射为文件系统的一个目录，对这个目录中所有文件操作都是对这个块设备（的某个分区）的操作。如果没有其他块设备（的某个分区）被映射为这个目录的子目录，则对这个目录的所有子目录（包括其中的文件）的操作都是对这个块设备（的某个分区）的操作，而一个块设备（的某个分区）被映射到哪个目录是由系统的超级用户指定的。这些知识在介绍 Linux 操作的书籍中均有介绍，可自行深入了解。

因此，使用块设备实际上就是把块设备（的某个分区）映射到指定的目录。

注意：这些命令大多需要 root 权限。

11.3.1 建立设备文件

使用驱动程序的目的是为了使用其支持的设备。在 Linux（μClinux）中，几乎把所有东西都当作文件，设备也不例外。每个设备都有对应的特殊文件（称为设备文件或节点文件，详情请参考 5.1.2 小节。一般情况下，系统中应存在所需要使用的设备文件。设备文件一般存储在/dev 目录下。

如果没有对应的设备文件，则需要自己建立。建立设备文件的命令是 mknod。对于块符设备，一般使用以下命令：

```
mknod   文件名称(可包含路径)  b   \[主设备号\]   \[从设备号\]
```

输入“mknod --help”可得到 mknod 的详细帮助。

11.3.2 加载驱动程序

本书介绍的驱动程序都编译成内核模块，这就必须把驱动程序加载到内核中才能被使用。

加载内核模块的命令为 insmod，一般使用方法如下：

```
insmod   内核模块文件名(可包含路径)
```

输入“insmod --help”可得到 mknod 的详细帮助。

11.3.3 把块设备(的某个分区)映射到指定的目录

把块设备(的某个分区)映射到指定的目录后即可使用，这通过命令 mount 实现。mount 命令的一般使用方法如下：

```
mount   块设备文件名(可包含路径) 映射的目录(可包含路径)
```

输入“mount --help”可得到 mount 的详细帮助。

11.3.4 取消目录映射

为保证数据的完整性，在使用完块设备后最好取消目录映射，这通过命令 umount 实现。umount 命令的一般使用方法如下：

```
umount   映射的目录(可包含路径)
```

输入“umount --help”可得到 umount 的详细帮助。

11.3.5 卸载驱动程序

不再使用驱动程序时可卸载它，这通过 rmmod 命令实现。rmmod 命令的一般使用方法如下：

```
rmmod   模块名
```

输入“rmmod --help”可得到 rmmod 的详细帮助。

本章小结

本章首先介绍了 Linux(μClinux)块设备驱动程序的相关概念；然后主要介绍了一个最简单的块设备驱动程序，主要包括源码分析和编译使用；最后介绍如何使用块设备。这些是 Linux(μClinux)块设备驱动程序最基础的公共知识，也是要编写块设备必须掌握的知识。

相对于第 4 章和第 5 章，本章主要增加了以下知识：

- 块驱动的相关函数和数据结构(参考 11.2.3、11.2.4 和 11.2.5 小节)；
- 使用块符设备的方法(参考 11.3 节)。

本章介绍的知识是第 12 ~ 14 章的基础。

更多高清PDF图书 www.gqpdf.com

第12章　简单的块设备编写范例

12.1　使用范例

12.1.1　范例简介

本公司具有 FAT 兼容的文件系统软件包 ZLG/FFS，其最底层也是硬件驱动，具有扇区读/写功能，详情可参见参考文献[5]。在 ZLG/FS 的驱动程序中，本公司还编写了一个基于 NAND FALSH 的驱动程序 ZLG/FFS。本范例就是 μClinux 与 ZLG/FFS 的接口，通过它，μClinux 可支持 ZLG/FFS 支持的芯片。

注意：这个驱动程序还不支持分区的设备。因此，ZLG/FFS 所驱动的 NAND FLASH 也必须为软盘格式的（即没有分区表）。

12.1.2　编译驱动程序

整个驱动程序包含 2 个 C 语言文件、5 个头文件、1 个 bin 文件和 1 个 Makefile 文件。在编译时除 Makefile 文件外，其他文件无须修改。程序清单 12.1 为 Makefile 文件的内容。其中程序清单 12.1 (4) 为读者需要修改的地方（方框内的代码）。这一条指出系统头文件所在的目录，范例为/work/uClinux-dist/linux-2.4.x/include，表明 μClinux 源代码安装在/work/uClinux-dist/目录下，读者可根据实际情况修改。Makefile 文件其他语句不需要修改，具体介绍见 4.3 节。

程序清单 **12.1**　**Makefile** 文件内容

```
EXEC = zlg_ffs                                              (1)
OBJS = zlg_ffs_uClinx.o ZLG_FFS.o                           (2)
SRC = zlg_ffs_uClinx.c ZLG_FFS.c                            (3)

INCLUDE = /work/uClinux-dist/linux-2.4.x/include            (4)
CC = arm-elf-gcc                                            (5)
LD = arm-elf-ld                                             (6)
```

```
MODCFLAGS = -D__KERNEL__ -I $(INCLUDE) -O2 -fno-strict-aliasing -fno-common -pipe -
fno-builtin -D__linux__ -g -DNO_MM -mapcs-32 -march = armv4 -mtune = arm7tdmi -mshort-
load-bytes -msoft-float -nostdinc -iwithprefix include                    (7)
LDFLAGS = -m armelf -r                                                    (8)

all: $(EXEC)                                                              (9)

$(EXEC): $(OBJS)                                                          (10)
    $(LD) $(LDFLAGS) -o $@  $(OBJS)                                       (11)

% .o:% .c                                                                 (12)
    $(CC) $(MODCFLAGS) -mapcs -c $ < -o $@                                (13)

clean:                                                                    (14)
    -rm -f $(EXEC) *.elf *.gdb *.o                                        (15)
```

修改完 Makefile 文件后，将所有文件放在同一个目录中，在此目录执行程序清单 12.2(1)所示的命令时可编译驱动程序。使用程序清单 12.2(2)所示命令可清除编译结果。执行 make 命令后，会生成 ZLG_FFS.o、zlg_ffs_uClinx.o 和 zlg_ffs 三个文件。其中 ZLG_FFS.o 和 zlg_ffs_uClinx.o 为中间文件，而 zlg_ffs 为生成的驱动程序，是需要的文件。

程序清单 **12.2**　编译命令

```
make                                                                      (1)
make clean                                                                (2)
```

12.1.3　使用驱动程序

生成目标文件 zlg_ffs 后，便可在目标板上使用它了。下面假设目标板为DeviceARM2200，使用 ZLG/BOOT 引导 CPU 执行 μClinux。下面介绍使用步骤，其详情可参见参考文献[7]和[8]的相关章节。

① 移植并编译内核。假设内核映像文件为 linux.bin，加载地址为 0x81008000。

② 制作根目录文件系统映像文件。假设制作的映像文件为 init.img，加载地址为 0x81700000。

③ 将文件 zlg_ffs 拷贝到 μClinux 根文件系统中的/bin 目录下，使用 cp 命令即可。此时映像文件 init.img 中包含了文件 zlg_ffs。

④ 在 μClinux 根文件系统中的/dev 目录下，建立块设备节点文件 ZLG_FFS。在 μClinux 根文件系统中的/dev 目录下，使用“mknod zlg_ffs b 126 0”命令。此时映像文件 init.img 中包

更多高清PDF图书　www.gqpdf.com

含了设备节点。

⑤ 在 μClinux 根文件系统中的根目录建立目录 zlg，用来加载驱动程序支持的块设备。

⑥ 修改 μClinux 根文件系统中的/etc/rc。这个文件是 μClinux 再启动时自动执行的命令，类似于 DOS 的 autoexec.bat 这个文件。修改后的 RC 文件内容见程序清单 12.3，其中程序清单 12.3(5)、(6)为新添加的代码：程序清单 12.3(5)为加载块设备驱动程序；程序清单 12.3(6)为把 NAND FLASH 加载到目录/zlg 目录下。

程序清单 **12.3　rc** 文件的内容

```
hostname GDB-ARMulator                          (1)
/bin/expand /etc/ramfs.img /dev/ram0            (2)
mount -t proc proc /proc                        (3)
mount -t ext2 /dev/ram0 /var                    (4)
insmod /bin/zlg_ffs                             (5)
mount /dev/zlg_ffs  /zlg                        (6)
mkdir /var/tmp                                  (7)
mkdir /var/log                                  (8)
mkdir /var/run                                  (9)
mkdir /var/lock                                 (10)
mkdir /var/empty                                (11)
cat /etc/motd                                   (12)
```

⑦ 编写文件 zlg_boot.ini。这是 ZLG/BOOT 的要求。实际的代码见程序清单 12.4。除程序清单 12.4(3)外，其他参见参考文献[6]相关章节很容易理解。12.1.2 小节提到驱动程序有一个 bin 文件，它就是 k9f2808.bin。这个文件为芯片 k9f2808 的最底层操作代码。从理论上说，这个代码应编译到驱动程序中。不过，芯片 k9f2808 对时序的速度有要求，在 DeviceARM2200 的外部 RAM 中执行很难达到其速度要求，所以需要把它放在片内 RAM 中。由于 k9f2808 的驱动与本章关系不大，因此这里不介绍 k9f2808.bin 的生成方法。

程序清单 **12.4　zlg_boot.ini** 内容

```
load 0x81008000 a:\\linux.bin                   (1)
load 0x81700000 a:\\init.img                    (2)
load 0x40003000 a:\\k9f2808.bin                 (3)
set PC=0x81008000                               (4)
run                                             (5)
```

⑧ 下载文件 linux.bin、init.img、k9f2808.bin 和 zlg_boot.ini 到 DeviceARM2200 的 NAND FLASH 中(根目录)。这一步参见参考文献[6]相关章节。

⑨ 启动 μClinux。这一步参见参考文献[6]相关章节。

⑩ 执行“ls zlg”，应出现类似于图 12.1 的图，说明块设备已可使用，在上面建立文件、目录等操作也可正确执行了。

图 12.1　块设备上的文件已加载到文件系统中

注意：由于 ZLG/FS 为 FAT 兼容文件系统，所以 μClinux 内核必须配置成支持所有 FAT 文件系统；否则由于文件系统不兼容，不能把设备加载到 μClinux 的文件系统中。

12.2　范例程序源码分析

事实上，第 11 章已介绍了编写块驱动程序一般知识，而本驱动程序也是在那个驱动程序的基础上修改而来的，因此，应先阅读第 11 章，否则不易理解本节内容。

12.2.1　关于主设备号

Linux（μClinux）的字符设备和块设备驱动程序都需要主设备号。与主设备号相关的代码一般分散在源代码的几个地方，程序清单 12.5 把本章介绍的驱动程序与主设备号相关的代码放在了一起，注释说明了其在源代码中的位置。

更多高清PDF图书　www.gqpdf.com

程序清单 **12.5**　与主设备号相关的一些代码

```
/* 以下代码在 config.h 中 */
#define ZLG_FFS_MAJOR_NR          126                                    (1)

/* 以下代码在驱动程序主文件中,必须在#include <linux/blk.h> 之前 */
#define MAJOR_NR major                                                   (2)

ODULE_PARM(major, "i");                                                  (3)
static int major = ZLG_FFS_MAJOR_NR;                                     (4)

/* 以下代码在驱动程序的初始化函数中 */

    result = register_blkdev(MAJOR_NR, DEVICE_NAME, &zlg_ffs_fops);      (5)
    if (result < 0)                                                      (6)
    {
        printk(KERN_ERR DEVICE_NAME ": Unable to get major % d\\n", MAJOR_NR );
                                                                         (7)
        return(result);                                                  (8)
    }
    if (MAJOR_NR == 0)                                                   (9)
    {
        MAJOR_NR = result; /* dynamic */                                 (10)
    }
```

这部分代码比 11.2 节介绍的驱动程序的相关部分复杂很多,这是为了让用户可通过 insmod 命令参数改变主设备号而增加的。insmod 命令可改变驱动程序的整型变量和字符型变量的初始值,但驱动程序必须声明这个变量可修改,所以程序清单 12.5(4)定义一个变量保存设备的主设备号并初始化,并通过程序清单 12.5(3)来声明这个变量。宏 MODULE_PARM 的第 1 个参数为变量本身,第 2 个参数用字符串声明参数的类型,其中 i 表明其为整型变量。目前,MODULE_PARM 支持(也就是 insmod 命令支持)5 种变量: b——一个字节;h——短整型;i——整型;l——长整型;s——字符串。

这样做后,无论 insmod 命令是否给出驱动的主设备号,驱动程序都能顺利加载。关于如何使用 insmod 命令初始化驱动程序的变量,请参考 insmod 命令的帮助文档。

另外,主设备号可能设置为 0,而事实上这个主设备号是保留的。当驱动程序以 0 为第一个参数调用函数 register_blkdev()时(程序清单 12.5(5)、(9)),Linux(μClinux)会返回一个空闲的主设备号(当然是正数,负数表示错误),驱动程序需要保存它(程序清单 12.5(10))。

12.2.2 几个数据结构

Linux(μClinux)内核中有几个数组保存每个块设备的信息,其中在驱动程序中比较常用的是 blk_size\[\]、blksize_size\[\]和 hardsect_size\[\],其详细介绍见表 12.1。由表 12.1 中可看出,它们均定义二维数值,事实上,分别使用主设备号和次设备号作为其第一下标和第二下标即可访问特定的设备信息。另外,虽然它们形式上是二维数组,实质上是一维数组指针,实际的存储空间由驱动程序提供。

表 12.1 内核中的一些表

数组名称	数组原型	数组作用	备 注
blk_size\[\]	int blk_size\[\]\[\]	每个设备的大小,以千字节为单位	如果 blk_size\[\] = =NULL,则不检测设备大小
blksize _ size \[\]	int blksize _ size \[\]\[\]	每个设备的块大小,以字节为单位,必须为 2 的幂	如果 blksize_size\[\] = =NULL,则假定为 BLOCK_SIZE(当前为 1 KB)
hardsect _ size \[\]	int hardsect_size\[\]\[\]	硬件扇区大小,以字节为单位,必须为大于 512 的 2 的幂	默认值为 512 字节

因此,需要定义几个数组与之对应。另外,在程序清单 11.7、程序清单 11.8 中使用了设备使用计数器,这也需要定义相应的数组。这几个数组的定义见程序清单 12.6,其中 ZLG_FFS_blocksizes\[\]对应于 blksize_size\[\],ZLG_FFS_sizes\[\]对应于 blk_size\[\],而 ZLG_FFS_usage\[\]为设备使用计数器。hardsect_size 使用默认值,而 MAX_DRIVES 是 congfig.h 定义的宏,用于声明本驱动支持的物理设备的个数。

程序清单 **12.6** 几个数组

```
static int ZLG_FFS_blocksizes\[MAX_DRIVES\];          /* bytes per device */
static int ZLG_FFS_sizes\[MAX_DRIVES\];                /* bytes per device's
block */
                                                       /* must be (1 < < n) */
static unsigned int ZLG_FFS_usage\[MAX_DRIVES\];       /* device using count */
```

12.2.3 驱动程序的初始化函数

驱动程序的初始化函数代码见程序清单 12.7。可以看出,程序清单 12.7(2)~(7)在 12.2.1 小节已介绍,而通过 12.2.2 小节,程序清单 12.7(9)、(10)也很容易理解,关键代码是程序清单 12.7(1)和(8)。程序清单 12.7(1)是初始化 ZLG/FFS 的,与本章介绍的内容没有多大联系,这里不介绍了,感兴趣的读者可阅读参考文献\[8\]。而程序清单 12.7(8)向 Linux

更多高清PDF图书 www.gqpdf.com

(μClinux)内核注册了一个函数(zlg_ffs_make_request),这个函数是用来实际传输数据的,这一部分会在12.2.10小节详细论述。

程序清单**12.7** 驱动程序的初始化函数代码

```
int  zlg_ffs_init(void)
{
    int  result;

    init_flash();                                                    (1)

    result = register_blkdev(MAJOR_NR,  DEVICE_NAME,  &zlg_ffs_fops);    (2)
    if (result < 0)                                                  (3)
    {
        printk(KERN_ERR DEVICE_NAME ": Unable to get major % d\\n", MAJOR_NR );
                                                                     (4)
        return(result);                                              (5)
    }
    if (MAJOR_NR == 0)                                               (6)
    {
        MAJOR_NR = result; /* dynamic */                             (7)
    }

    blk_queue_make_request(BLK_DEFAULT_QUEUE(MAJOR_NR), zlg_ffs_make_request);
                                                                     (8)

    blk_size\[MAJOR_NR\] = ZLG_FFS_sizes;                            (9)
    blksize_size\[MAJOR_NR\] = ZLG_FFS_blocksizes;                   (10)

    printk(KERN_INFO DEVICE_NAME ": zlg_ffs_init OK\\n");            (11)
    return(0);                                                       (12)
}
```

12.2.4 驱动程序的清除函数

驱动程序的清除函数代码见程序清单12.8。它与程序清单11.1中代码一样,不再介绍。

程序清单**12.8** 清除函数代码

```
void zlg_ffs_cleanup(void)
{
    unregister_blkdev(MAJOR_NR, DEVICE_NAME);
}
```

12.2.5 open()方法

本驱动程序 open()方法的代码见程序清单 12.9。这些代码完全是按照程序清单 11.7 编写的。本驱动程序的 open()方法代码与 open()方法的示意代码对照见表 12.2。

程序清单 **12.9** **open()方法代码**

高清PDF藏 qqpdf.com

更多高清PDF图书 www.gqpdf.com

```
static int  zlg_ffs_open(struct inode *inode, struct file *filp)
{
    int num;
    unsigned long flag;
    Disk_RW_Parameter dp;

num = MINOR(inode->i_rdev);                                          (1)
if (num >= MAX_DRIVES)                                               (2)
{
    return -ENODEV;                                                  (3)
}

local_irq_save(flag);                                                (4)
ZLG_FFS_usage\[num\]++;                                              (5)
if (ZLG_FFS_usage\[num\]==1)                                         (6)
{
    FFSDiskInfo\[num\].Drive=&(FlashDriverInfo\[num\]);              (7)
    dp.RsvdForLow=FFSDiskInfo + num;                                 (8)
    ZLG_FFS(DISK_INIT, &dp);                                         (9)
    ZLG_FFS_sizes\[num\]=FFSDiskInfo\[num\].BlockSum * FlashDriverInfo\
                         [num\].
                         BytsPerSec * FlashDriverInfo\[num\].SecsPre-
                         Block/1024;                                 (10)
    ZLG_FFS_blocksizes\[num\]=FlashDriverInfo\[num\].BytsPerSec;     (11)
    register_disk(NULL, inode->i_rdev, 1, &zlg_ffs_fops,
                  FFSDiskInfo\[num\].BlockSum * FlashDriverInfo\[num\].Sec-
sPreBlock);                                                          (12)
    }
    local_irq_restore(flag);                                         (13)

    MOD_INC_USE_COUNT;                                               (14)
    return 0;          /* success */                                 (15)
}
```

表 12.2　open() 方法的示意代码与驱动代码对照说明

示意代码(程序清单 11.7)	实际代码	说　明

(1) 从设备号 = MINOR(inode->i_rdev);	程序清单 12.9(1)	
(2) 通过从设备号判断需要操作的设备;	无	从设备号就代表物理设备
(3) 通知内核进入临界区;	程序清单 12.9(4)	
(4) 需要操作的设备的使用计数器++;	程序清单 12.9(5)	
(5) if(需要操作的设备的使用计数器==1)	程序清单 12.9(6)	
{	{	一般情况下程序清单 12.9(10)~(12)是必需的
(6) 初始化设备(包括申请资源);	程序清单 12.9(7)~(12)	
}	}	
(7) 通知内核退出临界区;	程序清单 12.9(13)	
(8) MOD_INC_USE_COUNT;	程序清单 12.9(14)	
(9) return 0;	程序清单 12.9(15)	

由表 12.2 可看出,示意代码(程序清单 11.7)的(2)行没有对应的代码。这是因为在本驱动程序中,物理设备与从设备号是一一对应的。如果不是一一对应,则需要进行相应的转换。而实际代码(程序清单 12.9(2)、(3))是错误处理,示意代码中也不会写出,可归为示意代码(程序清单 11.7)的(1)。

在实际代码中,使用关本地中断方式实现保护临界区代码(示意代码即程序清单 11.7(3)、(7)),这在嵌入式系统中是可行的,因为在嵌入式系统中几乎不会出现对称多处理器系统。当然,如果临界区代码需要执行的时间很长,则需要使用其他机制。具体需要使用哪个机制,可参见参考文献[4]。

在实际代码初始化设备中,程序清单 12.9(7)~(9)与具体设备密切相关,在这里与 ZLG/FFS 相关,不在本章讨论范围之内。程序清单 12.9(10)、(11)为初始化两个数组的某个元素,关于这两个数组的含义见前面介绍的“几个数据结构”。这具体初始化的值也与实际的设备相关,这里不再说明。函数 register_disk()(程序清单 12.9(12))是 Linux(μClinux)提供的函数,用来检测设备的分区。本驱动不支持分区的设备,本来可不调用,不过,Linux(μClinux)建议调用。关于支持可分区的块设备驱动程序的编写方法可参见参考文献[2]。函数 register_disk()的函数原型见程序清单 12.10。其中参数 dev 为一个结构体,它传送 NULL,事实上是不执行分区检测;参数 first 为设备编号,它包含主设备号和从设备号;参数 minors 为所支持的分区数目;参数 ops 就是 11.2.5 小节提到的结构体指针,应指向本驱动的这个结构体变量;size 为以扇区为单位的设备大小。

更多高清PDF图书 www.gqpdf.com

程序清单 **12.10**　函数 **register_disk**()的原型

```
void register_disk(struct gendisk *dev, kdev_t first, unsigned minors, struct
block_device_operations *ops, long size);
```

12.2.6　release()方法

本驱动程序 release()方法的代码见程序清单 12.11。这些代码完全是按照程序清单 11.8 编写的。本驱动程序 release()方法的代码与 release()方法的示意代码对照见表 12.3。

程序清单 **12.11**　**release**()方法代码

```
static int  zlg_ffs_release(struct inode *inode, struct file *filp)
{
    int num;
    Disk_RW_Parameter dp;
    unsigned long flag;

    num = MINOR(inode->i_rdev);                                    (1)
    if (num >= MAX_DRIVES)                                         (2)
    {
        return -ENODEV;                                            (3)
    }

    local_irq_save(flag);                                          (4)
    ZLG_FFS_usage\[num\]--;                                        (5)
    if (ZLG_FFS_usage\[num\] == 0)                                 (6)
    {
        fsync_dev(inode->i_rdev);                                  (7)
        dp.RsvdForLow = FFSDiskInfo + num;                         (8)
        ZLG_FFS(DISK_CLOSE, &dp);                                  (9)
    }
    local_irq_restore(flag);                                       (10)

    MOD_DEC_USE_COUNT;                                             (11)
    return(0);                                                     (12)
}
```

表 12.3　release()方法的示意代码与驱动代码对照说明

示意代码(程序清单 11.8)	实际代码	说　明
(1) 从设备号 = MINOR(inode->i_rdev);	程序清单 12.11(1)	
(2) 通过从设备号判断需要操作的设备;	无	从设备号就代表物理设备
(3) 通知内核进入临界区;	程序清单 12.11(4)	
(4) 需要操作的设备的使用计数器--;	程序清单 12.11(5)	
(5) if(需要操作的设备的使用计数器 = =0)	程序清单 12.11(6)	
{	{	
(6) 关闭设备处理(包括释放资源);	程序清单 12.11(7) ~ (9)	
}	}	
(7) 通知内核退出临界区;	程序清单 12.11(10)	
(8) MOD_DEC_USE_COUNT;	程序清单 12.11(11)	
(9) return 0;	程序清单 12.11(12)	

由 12.3 可看出,示意代码(程序清单 11.8)的(2)行没有对应的代码。这是因为在本驱动中,物理设备与从设备号是一一对应的。如果不是一一对应,则需要进行相应的转换。而实际代码(程序清单 12.11(2)、(3))是错误处理,示意代码中也不会写出,可归为示意代码(程序清单 11.8)的(1)。

在实际代码中,使用关本地中断方式实现保护临界区代码(示意代码即程序清单 11.8(3)、(7)),这在嵌入式系统中是可行的,因为在嵌入式系统中几乎不会出现对称多处理器系统。当然,如果临界区代码需要执行的时间很长,则需要使用其他机制。具体需要使用哪个机制,可参见参考文献[4]。

在实际代码中,关闭设备处理的代码与具体设备密切相关,在这里程序清单 12.11(8)、(9)与 ZLG/FFS 相关,不在本章讨论范围之内。至于程序清单 12.11(7),调用函数 fsync_dev()将释放由内核保存在各种高速缓存中的对该设备的引用。函数 fsync_dev()的原型见程序清单 12.12,惟一的参数 dev 为设备编号。

程序清单 **12.12**　函数 **fsync_dev**()的原型

```
int fsync_dev(kdev_t dev);
```

12.2.7　ioctl()方法

本驱动程序的 ioctl()方法代码见程序清单 12.13。这个代码与 ioctl()方法的示意代码几乎完全相同(参见程序清单 11.9),仅把 BLKGETSIZE 和 HDIO_GETGEO 命令(返回设备物理

更多高清PDF图书　www.gqpdf.com

参数)具体化了。BLKGETSIZE 命令的更改(程序清单 12.13(1))很简单,这里不再说明。对于 HDIO_GETGEO 命令,ZLG/FFS 没有磁头、柱面这些参数,因此,需要伪造。驱动程序固定磁头数为 4(程序清单 12.13(4)),每磁道扇区数为 16(程序清单 12.13(5)),进而可计算柱面数(程序清单 12.13(3))。计算柱面数是由下面公式推导而来:

```
总扇区数 = 柱面数 × 磁头数 × 每磁道扇区数
```

最后,设置数据开始扇区为 0(程序清单 12.13(6))。

程序清单 **12.13　ioctl**()方法代码

```
static int zlg_ffs_ioctl(struct inode *inode, struct file *filp,
                         unsigned int cmd, unsigned long arg)
{
    long size;
    struct hd_geometry geo;

    switch(cmd)
    {
    case BLKGETSIZE:
        /* Return the device size, expressed in sectors */
        if (arg == 0)
        {
            return -EINVAL; /* NULL pointer: not valid */
        }
        if(access_ok(VERIFY_WRITE, arg, sizeof(long)) == 0)
        {
            return -EFAULT;
        }
        size = 1024 * ZLG_FFS_sizes\[MINOR(inode->i_rdev)\] /
             ZLG_FFS_blocksizes\[MINOR(inode->i_rdev)\];                (1)
        if (copy_to_user((long *) arg, &size, sizeof (long)))
        {
            return -EFAULT;
        }
        return 0;
    case BLKRRPART: /* re-read partition table: can't do it */
        return -ENOTTY;
    case HDIO_GETGEO:
```

```
        if (access_ok(VERIFY_WRITE, arg, sizeof(geo)) == 0)
        {
            return -EFAULT;
        }
        size = 1024 * ZLG_FFS_sizes\[MINOR(inode->i_rdev)\] /
             ZLG_FFS_blocksizes\[MINOR(inode->i_rdev)\];                    (2)
            geo.cylinders = (size & ~0x3f) >> 6;                            (3)
            geo.heads = 4;                                                  (4)
            geo.sectors = 16;                                               (5)
            geo.start = 0;                                                  (6)
            if (copy_to_user((void *) arg, &geo, sizeof(geo)))              (7)
            {
                return -EFAULT;                                             (8)
            }
            return 0;                                                       (9)
        default:
            return blk_ioctl(inode->i_rdev, cmd, arg);
    }
        return -ENOTTY; /* unknown command */
    }
```

12.2.8 check_media_change()方法

本驱动程序的 check_media_change()方法代码见程序清单 12.14。因为本驱动程序不支持可移动设备,所以它仅仅返回 0。

程序清单 **12.14** **check_media_change()方法代码**

```
static int check_zlg_ffs_change(kdev_t dev)
{
    return 0;
}
```

12.2.9 revalidate()方法

本驱动程序的 revalidate()方法代码见程序清单 12.15。因为本驱动程序不支持可移动设备,所以它仅仅返回 0。

更多高清PDF图书 www.gqpdf.com

程序清单 **12.15**　**revalidate**()方法代码

```
static int zlg_ffs_revalidate(kdev_t dev)
{
    return 0;
}
```

12.2.10　关于数据传输

11.2.5 小节中 ioctl()方法已提及，Linux 最初的块设备是基于机械寻址的硬盘的，且目前硬盘也是其最常用和最重要的块设备。对于硬盘，最耗时的操作是磁头在不同的轨道上移动，移动的距离越长，需要的时间便越长，而且这个操作所需要的时间远远大于其他操作所需要的时间，并成为整个系统的性能瓶颈。因此，Linux 需要优化代码（在一定的公平原则下）尽量减少这个操作所消耗的时间。减少时间的方法很多，其不在本章讨论范围之内。但所有方法的共同点是不按照实际请求的顺序操作硬盘，而是使用某种规则将请求排序，按使用排序后的顺序操作硬盘。事实上，Linux 使用请求队列来缓存和排序上层软件发出的访问请求。μClinux 是 Linux 的变体，这方面是兼容的。驱动程序可使用函数 blk_init_queue()来初始化一个请求队列，并把实际的传输函数传递给队列。关于使用请求进行队列传输的块设备驱动程序的编写方法可参见参考文献[2]。

但在嵌入式系统中，大部分使用电子存储设备作为块设备，它们没有“寻道”的问题，使用请求队列不但不会提高效率，还会增加系统资源的占用（包括 CPU 时间）。Linux（μClinux）也对这种方式提供了支持。在 Linux（μClinux）2.4 中，块 I/O 请求默认由函数__make_request()放入请求队列中，而驱动程序可用自己的“make request”函数替换系统提供的函数，在这个“make request”中就可实现实际的传输操作。此时，不需要调用函数 blk_init_queue()，因为实际上并未使用请求队列。

当然，即使用自己的“make request”函数使用请求队列也是可以的。这主要是使用自己的算法来对请求排序。这种方法比较深入，在嵌入式系统中也几乎不会使用，这里不详细介绍。

要使用自己的“make request”函数，最主要是在驱动程序的初始化函数中调用函数 blk_queue_make_request()（见程序清单 12.7(8)）。blk_queue_make_request()的函数原型见程序清单 12.16。其中，参数 q 为块设备请求队列，而 mfn 为驱动的“make request”函数。

程序清单 **12.16**　**blk_queue_make_request**()函数原型

```
void blk_queue_make_request(request_queue_t * q, make_request_fn * mfn)
```

“make request”的函数原型见程序清单 12.17，其参数见表 12.4。

程序清单 **12.17**　**make request** 函数原型

高清藏 PDF gqpdf.com

```
static int make_request(request_queue_t *queue, int rw, struct buffer_head *bh)
```

表 12.4 “make request”函数的参数

参 数	类 型	说 明	备 注
queue	request_queue_t *	指向请求队列的指针	可不使用
rw	int	请求类型：READ 读请求 READA 提前读请求 WRITE 写请求	
bh	struct buffer_head *	指向缓冲区头信息结构体的指针	

“make request”函数有一个参数 bh，它是指向 struct buffer_head 类型结构体的指针。这个结构体包含大量的成员，但只有少量成员给“make request”函数使用，这些字段见表 12.5。不使用请求队列时成员 b_reqnext 没有用。

表 12.5 “make request”函数可能用到的 buffer_head 结构体成员

字段名称	类 型	说 明
b_data	char *	与缓冲区相关的实际数据块
b_size	unsigned short	所指向的数据块大小
b_rdev	kdev_t	次设备号 = MINOR (bh - > b_rdev)
b_rsector	unsigned long	该数据块在设备上的扇区编号
b_reqnext	struct buffer_head *	指向请求队列中缓冲区头结构链表的指针
b_end_io	void (*)(struct buffer_head *, int)	I/O 结束时调用此函数。第 1 个参数为缓冲区头本身，第 2 个参数一般为非 0 值(I/O 操作成功时)

不使用请求队列驱动的“make request”函数示意代码见程序清单 12.18。程序清单 12.18 中出现两个函数 refile_buffer () 和 mark_buffer_uptodate ()。驱动程序调用函数 refile_buffer () 告诉核心准备改变缓冲块的状态；调用 mark_buffer_uptodate () 告诉核心已完成对缓冲区的操作，用于保证缓冲高速缓存保持正确，防止有错误指向的指针。函数 mark_buffer_uptodate () 的第 2 个参数为 1 表示传送成功；为 0 表示传送失败。

程序清单 **12.18** **make_request** () 示意代码

```
static int make_request(request_queue_t *queue, int rw, struct buffer_head *bh)
{
    从设备号 = MINOR(bh->b_rdev) ;                                    (1)
    通过从设备号判断需要操作的设备和分区;                              (2)
    获取每扇区字节数;                                                 (3)
```


更多高清PDF图书 www.gqpdf.com

```
开始扇区号 = bh -> b_rsector + 本分区起始扇区号;                    (4)
缓冲区地址 = bh -> b_data;                                          (5)
缓冲区字节数 = bh -> b_size;                                        (6)
switch(rw)                                                          (7)
{
    case READ:                                                      (8)
case READA:                                                         (9)
    while (缓冲区字节数 > = 每扇区字节数)                           (10)
    {
        通知内核进入临界区;                                        (11)
        读指定扇区(开始扇区号);                                    (12)
        通知内核退出临界区;                                        (13)
        缓冲区字节数 = 缓冲区字节数-每扇区字节数;                   (14)
        开始扇区号 ++ ;                                            (15)
        缓冲区地址 += 每扇区字节数;                                (16)
    }
    bh -> b_end_io(bh, 1);                                          (17)
    break;                                                          (18)
case WRITE:                                                         (19)
    refile_buffer(bh);                                              (20)
    while (缓冲区字节数 > = 每扇区字节数)                           (21)
    {
        通知内核进入临界区;                                        (22)
        写指定扇区(开始扇区号);                                    (23)
        通知内核退出临界区;                                        (24)
        缓冲区字节数 = 缓冲区字节数-每扇区字节数;                   (25)
        开始扇区号 ++ ;                                            (26)
        缓冲区地址 += 每扇区字节数;                                (27)
    }
    mark_buffer_uptodate(bh, 1);                                    (28)
        bh -> b_end_io(bh, 1);                                      (29)
        break;                                                      (30)
    default:
        bh -> b_end_io(bh, 0);                                      (31)
        break;                                                      (32)
    }
    return 0;                                                       (33)
}
```

本驱动程序“make request”函数实际的代码见程序清单 12.19。“make request”函数的代码基本是按照示意代码编写的，其与示意代码的对照见表 12.6。可看出，程序清单 12.18 (2) ~ (7) 在示意代码中没有对应的程序。它是错误处理，示意代码中也不会写出，可归为程序清单 12.19 (1)。另外，程序清单 12.19 (3) 也是 GCC 的扩展功能，标准 C 不一定支持。程序清单 12.19 (4)、(5) 是仅在前 5 次出错时显示出错信息，以保证控制台的整洁。

在实际代码中，使用关本地中断方式实现保护临界区代码（程序清单 12.18 (11)、(13)、(22)、(24)），这在嵌入式系统中是可行的，因为在嵌入式系统中几乎不会出现对称多处理器系统。当然，如果临界区代码需要执行的时间很长，则需要使用其他机制。具体需要使用哪个机制，可参见参考文献[4]。

程序清单 **12.19　“make request”** 处理函数代码

更多高清PDF图书　www.gqpdf.com

```
static int zlg_ffs_make_request(request_queue_t *queue, int rw, struct buffer_
head *bh)
{
    uint32 temp, SecSize;
    int32 size;
    unsigned long flag;

    Disk_RW_Parameter dp;
    FFSDisk   *fdp;

    temp = MINOR(bh->b_rdev);                                          (1)
    if (temp >= MAX_DRIVES)                                            (2)
    {
        static int count = 0;                                          (3)
        if (count++ < 5)                                               (4)
        {
            printk(KERN_WARNING DEVICE_NAME ": request past end of device\\n");
                                                                       (5)
        }
        bh->b_end_io(bh, 0);                                           (6)
        return 0;                                                      (7)
    }
    fdp = FFSDiskInfo + temp;                                          (8)
    dp.RsvdForLow = fdp;                                               (9)
    SecSize = fdp->Drive->BytsPerSec;                                  (10)
    dp.SectorIndex = bh->b_rsector;                                    (11)
    dp.Buf = bh->b_data;                                               (12)
```

高清PDF gqpdf.com

```
    size = bh->b_size;                                                    (13)

    switch(rw)                                                            (14)
    {
        case READ:                                                        (15)
        case READA:   /* Readahead */                                     (16)
        while (size >= SecSize)                                           (17)
        {
            local_irq_save(flag);                                         (18)
            ZLG_FFS(DISK_READ_SECTOR, &dp);                               (19)
            local_irq_restore(flag);                                      (20)
            size = size - SecSize;                                        (21)
            dp.SectorIndex ++;                                            (22)
            dp.Buf += SecSize;                                            (23)
        }
        bh->b_end_io(bh, 1);                                              (24)
        break;                                                            (25)
    case WRITE:                                                           (26)
        refile_buffer(bh);                                                (27)
        while (size >= SecSize)                                           (28)
        {
            local_irq_save(flag);                                         (29)
            ZLG_FFS(DISK_WRITE_SECTOR, &dp);                              (30)
            local_irq_restore(flag);                                      (31)
            size = size - SecSize;                                        (32)
            dp.SectorIndex ++;                                            (33)
            dp.Buf += SecSize;                                            (34)
        }
        mark_buffer_uptodate(bh, 1);                                      (35)
        bh->b_end_io(bh, 1);                                              (36)
            break;                                                        (37)
        default:
            bh->b_end_io(bh, 0);                                          (38)
            break;                                                        (39)
    }
    return 0;                                                             (40)
}
```


更多高清PDF图书 www.gqpdf.com

表 12.6 “make request”的示意代码与驱动代码对照说明

示意代码	实际代码	说　明
程序清单 12.18 (1)	程序清单 12.19 (1)	
程序清单 12.18 (2)	无	从设备号就代表物理设备,不支持分区
程序清单 12.18 (3)	程序清单 12.19 (10)	
程序清单 12.18 (4)	程序清单 12.19 (11)	不支持分区,所以起始扇区号为 0
程序清单 12.18 (5)	程序清单 12.19 (12)	
程序清单 12.18 (6)	程序清单 12.19 (13)	
程序清单 12.18 (7)	程序清单 12.19 (14)	
程序清单 12.18 (8)	程序清单 12.19 (15)	
程序清单 12.18 (9)	程序清单 12.19 (16)	
程序清单 12.18 (10)	程序清单 12.19 (17)	
程序清单 12.18 (11)	程序清单 12.19 (18)	
程序清单 12.18 (12)	程序清单 12.19 (19)	
程序清单 12.18 (13)	程序清单 12.19 (20)	
程序清单 12.18 (14)	程序清单 12.19 (21)	
程序清单 12.18 (15)	程序清单 12.19 (22)	
程序清单 12.18 (16)	程序清单 12.19 (23)	
程序清单 12.18 (17)	程序清单 12.19 (24)	
程序清单 12.18 (18)	程序清单 12.19 (25)	
程序清单 12.18 (19)	程序清单 12.19 (26)	
程序清单 12.18 (20)	程序清单 12.19 (27)	
程序清单 12.18 (21)	程序清单 12.19 (28)	
程序清单 12.18 (22)	程序清单 12.19 (29)	
程序清单 12.18 (23)	程序清单 12.19 (30)	
程序清单 12.18 (24)	程序清单 12.19 (31)	
程序清单 12.18 (25)	程序清单 12.19 (32)	
程序清单 12.18 (26)	程序清单 12.19 (33)	
程序清单 12.18 (27)	程序清单 12.19 (34)	
程序清单 12.18 (28)	程序清单 12.19 (35)	
程序清单 12.18 (29)	程序清单 12.19 (36)	
程序清单 12.18 (30)	程序清单 12.19 (37)	
程序清单 12.18 (31)	程序清单 12.19 (38)	
程序清单 12.18 (32)	程序清单 12.19 (39)	
程序清单 12.18 (33)	程序清单 12.19 (40)	

高清PDF gqpdf.com

12.2.11 其他说明

本驱动程序没有涉及中断、动态内存分配和按需加载等相关问题。关于这些部分的进一步信息请参见参考文献[2]和[4]。

另外，关于修改版权及作者声明，请参考6.3.6小节。

本章小结

本章以NAND FLASH K9f2808为例介绍了实际的Linux（μClinux）块设备驱动程序的相关知识。首先简单介绍该驱动程序，然后对源码进行分析。

相对于第11章，本章主要增加了数据传输的方法（参考12.2.10小节）。

高清PDF gqpdf.com

更多高清PDF图书 www.gqpdf.com

第13章 基于 ZLG/FS 驱动程序的通用块设备驱动程序

13.1 背景知识

13.1.1 驱动程序简介

本驱动程序是与硬件无关的通用驱动程序，是 Linux（μClinux）块设备驱动程序与 ZLG/FS V1.10 文件系统驱动程序的接口。本驱动程序不能单独使用，必须与硬件相关的驱动程序一起使用。这个与硬件相关的驱动程序很容易编写。广州致远电子有限公司提供一个驱动程序模板，读者只需少量改动模板代码，极少改动（主要修改与 GCC 不兼容的地方）或不改动 ZLG/FS V1.10 文件系统驱动程序源代码，即可生成与硬件相关的驱动程序。

驱动程序特点如下：

- 支持分区设备，每个设备最多支持 15 个分区；
- 同时最多可支持 16 个与硬件相关的驱动程序；
- 提供驱动程序模板，使 ZLG/FS V1.10 文件系统驱动程序源代码直接用到 Linux（μClinux）驱动程序中；
- 支持可移动设备。

13.1.2 ZLG/FS 简介

ZLG/FS 是广州致远电子有限公司开发的面向嵌入式系统的小型文件系统，是 ZLG 系列中间件的重要成员之一。它是与 FAT12、FAT16、FAT32 高度兼容的文件系统，可直接与个人电脑交换文件。它是可移植、可固化的文件系统，可用于前后台系统，也可用于多任务环境。在撰写本书时，ZLG/FS 的最新版本为 V1.10。

ZLG/FS 的特点如下：

- 高度兼容 FAT12、FAT16、FAT32。ZLG/FS 可正确地访问由 Windows 98 建立的 FAT12、FAT16、FAT32 逻辑盘，ZLG/FS 建立的逻辑盘也可被 Windows 98 正确地 访问。
- 直接支持硬盘格式和软盘格式设备。用户设备上的数据可按照硬盘格式存储（支持分

区），也可按照软盘格式存储（没有分区）。

- 可移植。全部代码用 ANSI C 编写，并与目标处理器的存储器结构无关（即无论存储器是大端结构还是小端结构，均不影响程序的执行），方便移植到自己的目标系统中。
- 可固化。ZLG/FS 为嵌入系统设计，如果有固化手段，则它可嵌入到用户的产品中成为产品的一部分。
- 支持多任务操作系统。提供 ZLG/FS 在 μC/OS－II 中使用的接口代码，参考这些代码就可很方便地在其他多任务环境下使用 ZLG/FS。
- 兼容多种介质。ZLG/FS 提供一个底层驱动程序的接口，只需提供相应介质的扇区访问代码，就可在相应的介质上使用 ZLG/FS。
- 提供源代码。需要购买源码的用户，可与广州致远电子有限公司联系。
- 可配置。得到源码的用户可对一些参数进行配置。

已实现的特性如下：

- 支持多个逻辑盘；
- 不同的逻辑盘可在不同的介质上；
- 支持树型目录结构，子目录层数不受限制；
- 支持以 FAT12、FAT16、FAT32 格式化的逻辑盘；
- 支持 8.3 文件名格式；
- 支持文件读/写和目录操作；
- 支持日期时间特性；
- 支持同步更新多 FAT 表；
- 直接支持硬盘格式和软盘格式设备；
- 提供格式化（format）范例代码；
- μC/OS－II 中实现不同任务的“当前目录”。

暂时未实现的特性如下：

- 对文件、目录名的限制不够严格。这一版本的 ZLG/FS 有一些 FAT 规范限制使用的字符没有过滤掉。
- 忽略文件（目录）属性中的只读、存档、隐含、系统属性。这一版本的 ZLG/FS 没有对带有这几个属性的文件、目录进行特殊处理。
- 忽略长文件名。这一版本的 ZLG/FS 不支持创建和访问长文件名的文件。不过，具有长文件名的文件还是可通过短文件名访问。

ZLG/FS 的升级版将逐步解决上述问题。

13.2 ZLG/FS V1.10 驱动程序使用指南

更多高清PDF图书 www.gqpdf.com

13.2.1 接口函数

ZLG/FS V1.10 驱动程序仅有一个接口函数，这个函数对于不同的驱动有不同的函数名，但参数相同。ZLG/FS V1.10 驱动程序接口函数原型可参考程序清单 13.1。

程序清单 **13.1　ZLG/FS V1.0** 驱动程序接口函数示例

```
extern uint16 Empty(uint8 Cammand, void * Parameter);
```

13.2.2 参　数

可以看出，驱动程序仅有一个接口简单的函数。它有两个参数，其中第 1 个参数Cammand 为 ZLG/FS 对驱动程序请求的操作。目前版本支持的命令见表 13.1。各命令的具体说明见后续内容。

表 13.1　底层驱动程序支持的命令

命　令	功　能	备　注
DISK_INIT	初始化驱动程序	必须实现
DISK_CLOSE	关闭驱动程序	必须实现
DISK_READ_SECTOR	读物理扇区	必须实现
DISK_WRITE_SECTOR	写物理扇区	必须实现
DISK_DRIVER_VER	查看驱动程序接口版本号，即 ZLG/FS 的版本号	必须实现
DISK_CHECK_CMD	查看指定命令是否实现	必须实现
DISK_LOW_FORMAT	低级格式化	可选命令
DISK_FREE_SECTOR	释放扇区	可选命令
DISK_GET_SECTOR_NUMBER	获得设备总扇区数	可选命令
DISK_GET_BYTES_PER_SECTOR	获得每扇区字节数	可选命令
DISK_CHECK_CHANGE	查看介质是否改变	可选命令
DISK_GET_SECTORS_PER_BLOCK	获取每块扇区数	可选命令
DISK_READ_BLOCK	读数据块	可选命令
DISK_WRITE_BLOCK	写数据块	可选命令

驱动程序的第 2 个参数 Parameter 虽然定义为无符号指针，但实质是一个结构体指针。这个结构体中保存着其他必需的参数。这个结构体的定义见程序清单 13.2。

程序清单 **13.2**　传递给底层驱动的参数结构体

```
typedef struct _Disk_RW_Parameter
{
    uint32          SectorIndex;        /* 操作的物理扇区号 */
    void            *RsvdForLow;        /* 保留给底层驱动程序 */
    uint8           *Buf;               /* 数据存储位置 */
}Disk_RW_Parameter;
```

除了 RsvdForLow 这个成员外，源代码的注释已经很清楚，这里不再介绍。RsvdForLow 是逻辑盘信息管理结构体的一个成员 RsvdForLow 的拷贝，它在加载驱动程序时由用户指定。它的使用由驱动程序自己决定，不是必需的。一般情况下，它用于存储驱动程序支持的物理设备相关信息，如使用的资源等。使用它，可使一个驱动程序支持多个物理设备。

要求驱动程序编写者尽量使驱动程序通用化，一些必需的硬件信息不要保存在驱动程序中，而是在使用时通过成员 RsvdForLow 指出。具体可参见参考文献[6]。

1. 驱动程序初始化（DISK_INIT 请求）

在使用驱动程序之前，必须调用 DISK_INIT 请求。DISK_INIT 请求仅使用了 Parameter 参数的 RsvdForLow 成员，其他成员没有使用。当初始化成功时，程序返回 DISK_INIT_OK；否则返回 DISK_INIT_NOT_OK。

注意：此请求与 DISK_CLOSE 请求配对使用，并且不允许嵌套。

2. 关闭驱动程序（DISK_CLOSE 请求）

在驱动程序使用完毕，应调用 DISK_CLOSE 请求。DISK_CLOSE 请求仅使用了 Parameter 参数的 RsvdForLow 成员，其他成员没有使用。

注意：此请求与 DISK_INIT 请求配对使用，并且不允许嵌套。

3. 读扇区（DISK_READ_SECTOR 请求）

用户程序通过 DISK_READ_SECTOR 请求读取一个硬件扇区的数据，驱动程序将请求转化为实际的硬件操作。DISK_READ_SECTOR 请求使用了 Parameter 参数的所有成员：SectorIndex 指出需要读取的物理扇区号；Buf 指出读取的数据保存的位置；RsvdForLow 指出物理设备的相关信息。当读取数据成功时，程序返回 DISK_READ_OK；否则返回 DISK_READ_NOT_OK。

4. 写扇区（DISK_WRITE_SECTOR 请求）

用户程序通过 DISK_WRITE_SECTOR 请求把一个扇区的数据写到硬件扇区，驱动程序将请求转化为实际的硬件操作。DISK_WRITE_SECTOR 请求使用了 Parameter 参数的所有成员：SectorIndex 指出需要写的物理扇区号；Buf 指出将要存储的数据；RsvdForLow 指出物理设备的相关信息。当写数据成功时，程序返回 DISK_WRITE_OK；否则返回 DISK_WRITE_NOT_OK。

5. 获取驱动程序接口版本号（DISK_DRIVER_VER 请求）

不同版本的 ZLG/FS 具有不同的驱动程序接口，之间并不一定兼容。目前 V1.0 版本的接

更多高清PDF图书 www.gqpdf.com

口与 V1.10 版本的接口不兼容。我们会尽量保证以后的接口与 V1.10 版本的接口兼容(也比较容易保证)。为了确保不错用驱动程序,V1.10 版本以后的驱动程序均提供 DISK_DRIVER_VER 请求。DISK_DRIVER_VER 请求使用 Parameter 参数的 SectorIndex 成员返回驱动程序接口版本号,110 表示为 V1.10,即版本号乘以 100。

6. 检测命令是否存在(DISK_CHECK_CMD 请求)

驱动程序有许多可选命令,这些命令在具体的驱动程序中不一定实现。这样,驱动程序需要提供某种手段让上层程序知道驱动程序已实现了哪些可选命令,这通过 DISK_CHECK_CMD 请求实现。DISK_CHECK_CMD 请求使用了 Parameter 参数的 SectorIndex 成员指出命令的编号。当命令已实现时,程序返回 DISK_TRUE;否则返回 DISK_FALSE。

7. 低级格式化(DISK_LOW_FORMAT 请求)

这是可选的命令。

这个请求对应硬盘的低级格式化。一般的介质不需要此命令,驱动程序根据实际情况决定是否支持此请求(ZLG/FFS 使用了此请求)。DISK_CLOSE 请求仅使用了 Parameter 参数的 RsvdForLow 成员指出实际使用的物理设备,其他成员没有使用。DISK_CLOSE 请求与具体的设备有关,由驱动程序实现。

8. 释放扇区(DISK_FREE_SECTOR 请求)

这是可选的命令。

当驱动程序知道上层软件使用了哪些扇区会提高效率时,可以实现此命令。DISK_FREE_SECTOR 请求使用了 Parameter 参数的 SectorIndex 成员指出不再使用的扇区编号,而 RsvdForLow 成员指出实际的物理设备。

9. 获得设备总扇区数(DISK_GET_SECTOR_NUMBER 请求)

这是可选的命令。

此请求使用 Parameter 参数的 SectorIndex 成员返回设备总扇区数,RsvdForLow 成员指出实际使用的物理设备。

10. 获得每扇区字节数(DISK_GET_BYTES_PER_SECTOR 请求)

这是可选的命令。

此请求使用 Parameter 参数的 SectorIndex 成员返回每扇区字节数,RsvdForLow 成员指出实际使用的物理设备。

11. 查看介质是否改变(DISK_CHECK_CHANGE 请求)

这是可选的命令。

此请求用于支持可移动设备。此请求使用 Parameter 参数的 RsvdForLow 成员指出实际访问的物理设备。当设备支持的介质在上层软件上次发送 DISK_CHECK_CHANGE 请求后已改变(如拔出 SD 卡),则返回 DISK_TRUE;否则返回 DISK_FALSE。

12. 获取每块扇区数(DISK_GET_SECTORS_PER_BLOCK 请求)

这是可选的命令。

此请求使用 Parameter 参数的 SectorIndex 成员返回设备每块扇区数,RsvdForLow 成员指出实际使用的物理设备。此请求为读/写数据块作准备。

13. 读数据块(DISK_READ_BLOCK 请求)

这是可选的命令。

一些介质可通过块读和块写提高性能,此时驱动程序可提供此命令。这个请求与硬件密切相关,用户必须自己实现它。DISK_READ_SECTOR 请求使用了 Parameter 参数的所有成员:SectorIndex 指出需要读取的数据块的第一个物理扇区号;Buf 指出读取的数据保存的位置;RsvdForLow 指出物理设备的相关信息。当读取数据成功时,程序返回 DISK_READ_OK;否则返回 DISK_READ_NOT_OK。

14. 写数据块(DISK_WRITE_BLOCK 请求)

这是可选的命令。

一些介质可通过块读和块写提高性能,此时驱动程序可提供此命令。这个请求与硬件密切相关,用户必须自己实现它。DISK_WRITE_SECTOR 请求使用了 Parameter 参数的所有成员:SectorIndex 指出需要写的数据块的第一个物理扇区号;Buf 指出将要存储的数据;RsvdForLow 指出物理设备的相关信息。当写数据成功时,程序返回 DISK_WRITE_OK;否则返回 DISK_WRITE_NOT_OK。

13.3 使用范例

13.3.1 编译驱动程序

整个驱动程序包含 1 个 C 语言文件、3 个头文件和 1 个 Makefile 文件。在编译时除 Makefile 文件外,其他文件无须修改。程序清单 13.3 为 Makefile 文件的内容。其中程序清单 13.3(4)为需要修改的地方(方框内的代码)。这一条指出系统头文件所在的目录,范例为/work/uClinux-dist/linux-2.4.x/include,表明 μClinux 源代码安装在/work/uClinux-dist/目录下,可根据实际情况修改。Makefile 文件其他语句不需要修改,具体介绍请看 4.3 节。

程序清单 **13.3** **Makefile** 文件内容

更多高清PDF图书 www.gqpdf.com

```
EXEC = zlg_fs                                                                    (1)
OBJS = zlg_fs.o                                                                  (2)
SRC = zlg_fs.c                                                                   (3)

INCLUDE = /work/uClinux-dist/linux-2.4.x/include                                 (4)
CC = arm-elf-gcc                                                                 (5)
LD = arm-elf-ld                                                                  (6)
MODCFLAGS = -D__KERNEL__ -I $(INCLUDE) -O2 -fno-strict-aliasing -fno-common -pipe
-fno-builtin -D__linux__ -g -DNO_MM -mapcs-32 -march=armv4 -mtune=arm7tdmi
-mshort-load-bytes -msoft-float -nostdinc -iwithprefix include                   (7)
LDFLAGS = -m armelf -r                                                           (8)

all: $(EXEC)                                                                     (9)

$(EXEC): $(OBJS)                                                                 (10)
    $(LD) $(LDFLAGS) -o $@  $(OBJS)                                              (11)

% .o:% .c                                                                        (12)
    $(CC) $(MODCFLAGS) -mapcs -c $< -o $@                                        (13)

clean:                                                                           (14)
    -rm -f $(EXEC) *.elf *.gdb *.o                                               (15)
```

修改完 Makefile 文件后，将所有文件放在同一个目录中。在此目录中执行程序清单 12.4(1)所示的命令时，可编译驱动程序。使用程序清单 13.4(2)所示命令可清除编译结果。执行 make 命令后，会生成 zlg_fs.o 和 zlg_fs 两个文件。其中 zlg_fs.o 为中间文件，而 zlg_fs 为生成的驱动程序，是需要的文件。

程序清单 **13.4**　编译命令

```
make                                                                             (1)
make clean                                                                       (2)
```

13.3.2　使用驱动程序

1. 概　述

本设备驱动程序也是一个标准的块设备驱动程序，与普通块设备驱动程序的区别主要是：本驱动程序不带操作硬件部分代码，操作硬件部分代码由其他驱动程序实现。因此，至少需要

多加载 1 个硬件驱动程序，才能使用本驱动程序。关于块设备的一般使用方法请参考 11.3 节。下面简单介绍本驱动的使用方法。

2. 建立设备文件

本驱动程序最多可支持 16 个设备，每个设备最多可有 15 个分区，根据 Linux（μClinux）的规范，需要建立多达 256（即 16 ×（15 +1））个设备文件。如果仅仅使用 mknod 命令来建立，则是一个繁琐的工作。笔者提供了一个 shell 脚本用来建立这些设备文件。这个 shell 脚本文件内容见程序清单 13.5。

注意：此 shell 脚本文件在“红帽子 LINUX 9.0”上通过，μClinux 的 shell 不一定支持。

关于 shell 脚本的知识不是本书讨论的范畴，有兴趣的读者可自行参考相关书籍。假设脚本文件名为 xmknod.sh，则执行以下命令建立设备文件：

```
sh xmknod.sh
```

程序清单 **13.5**　建立设备文件的脚本

```
#! /bin/sh
devid = 0
subname = /dev/zlg_fs

for nameadd in a b c d e f g h i j k l m n o p
do
    mknod   $ subname $ nameadd  b  125   $ devid
    let devid = $ devid + 1
    i = 1
    while
        test $ i -lt 16
    do
        mknod   $ subname $ nameadd $ i  b  125   $ devid
        let i = i + 1
        let devid = $ devid + 1
    done
done
```

3. 加载驱动程序

假设本驱动程序的目标文件名为 zlg_fs，并且所有驱动程序都在当前目录下，则使用程序清单 13.6 所示命令序列加载驱动程序，其中程序清单 13.6（3）~（17）为可选命令。

更多高清PDF图书　www.gqpdf.com

程序清单 **13.6**　加载驱动程序

```
insmod  zlg_fs                                          (1)
insmod   第 1 个硬件驱动程序名                          (2)
insmod   第 2 个硬件驱动程序名                          (3)
insmod   第 3 个硬件驱动程序名                          (4)
insmod   第 4 个硬件驱动程序名                          (5)
insmod   第 5 个硬件驱动程序名                          (6)
insmod   第 6 个硬件驱动程序名                          (7)
insmod   第 7 个硬件驱动程序名                          (8)
insmod   第 8 个硬件驱动程序名                          (9)
insmod   第 9 个硬件驱动程序名                          (10)
insmod   第 10 个硬件驱动程序名                         (11)
insmod   第 11 个硬件驱动程序名                         (12)
insmod   第 12 个硬件驱动程序名                         (13)
insmod   第 13 个硬件驱动程序名                         (14)
insmod   第 14 个硬件驱动程序名                         (15)
insmod   第 15 个硬件驱动程序名                         (16)
insmod   第 16 个硬件驱动程序名                         (17)
```

4. 把块设备(的某个分区)映射到指定的目录

加载驱动程序后即可映射块设备。以下为几个加载的例子(不要输入“;”及以后部分):

```
mount   /dev/zlg_fsa /zlg          ;第 1 个块设备为不分区设备,加载它
mount   /dev/zlg_fsa1 /zlg         ;第 1 个块设备为分区设备,加载第 1 个分区
mount   /dev/zlg_fsc2 /zlg         ;加载第 3 个块设备的第 2 个分区
```

5. 取消目录映射

详情请参考 11.3.4 小节。

6. 卸载驱动程序

详情请参考 11.3.5 小节。惟一需要注意的是,必须把所有硬件驱动程序卸载后,才能卸载本驱动程序。

13.4　范例程序源码分析

事实上,第 11 章已介绍了编写块驱动程序的一般知识,而本驱动程序也是在那个驱动程序的基础上修改而来的,因此,应先阅读第 11 章,否则不容易理解本节内容。同时,12.2.2 小

高清PDF gqpdf.com

节和 12.2.10 小节也有重要的内容，应先仔细阅读。

13.4.1 块设备信息结构体

本驱动程序中有一个重要的变量 DiskInfo，其定义见程序清单 13.7，其中 MAX_DISKS 是一个常量，默认值为 16，是驱动程序支持的块设备数目。可以看出，它是结构体 fs_drivers 数组指针。事实上，这个结构体保存着物理设备的信息，本驱动通过这个结构体指针数组记录已加载的物理设备，并通过这个结构体指针数组访问硬件驱动程序。

程序清单 **13.7** **DiskInfo** 定义

```
fs_drivers *DiskInfo\[MAX_DISKS\];                                    (1)
```

结构体 fs_drivers 的定义见程序清单 13.8。结构体 fs_drivers 实际上记录了块设备的一些参数和硬件驱动程序的一些接口函数参数。下面简单介绍各成员的含义。

BytesPerSec：每扇区字节数，用于保存块设备的每个扇区包含的字节数目。

SecPerDisk：设备扇区数总数，用于保存块设备的扇区总数。

RsvdForLow：硬件驱动程序的一个可选参数。硬件驱动程序一般用它来识别自己支持的物理设备。

DiakCommand：硬件驱动程序接口函数。实际上，这是 13.2 节介绍的 ZLG/FS V1.10 驱动程序接口函数。

open：硬件驱动程序的 open () 方法，在使用硬件驱动程序前必须调用它。该方法与块设备驱动程序的 open () 方法具有同样的功能、参数和返回值。

release：硬件驱动程序的 release () 方法，在使用完硬件驱动程序后必须调用它。该方法与块设备驱动程序的 release () 方法具有同样的功能、参数和返回值。

get_info：硬件驱动程序的 get_info () 方法，在调用硬件的 open () 方法后必须马上调用此方法。在介质改变后必须调用它。硬件驱动程序会把块硬件信息填充到 get_info () 函数的参数指向的结构体变量中。

程序清单 **13.8** 硬件设备信息结构体

```
typedef struct _zlg_driver_Info
{
        u16         BytesPerSec;                                      (1)
        u32         SecPerDisk;                                       (2)
        void        *RsvdForLow;                                      (3)
        u16         (* DiakCommand)(uint8 Cammand, void *Parameter);  (4)
        int         (*open) (struct inode *, struct file *);          (5)
```


更多高清PDF图书 www.gqpdf.com

```
        int         (*release) (struct inode *, struct file *);          (6)
        int         (*get_info) (struct _zlg_driver_Info *);             (7)
    }fs_drivers;                                                         (8)
```

13.4.2 注册 ZLG/FS V1.10 驱动程序

本驱动程序不能单独使用，必须与操作硬件的驱动程序配合才能使用。因此，本驱动程序需要知道加载了哪些物理块设备（当然也就加载了对应的硬件驱动程序），这通过硬件驱动程序调用函数 zlg_fs_add_driver() 来实现。这个函数实质上是记录 13.4.1 小节所说结构体指针。因此，此函数的惟一参数为结构体 fs_drivers 指针，它保存相应的块设备信息。当加载成功时，此函数返回 0；否则返回 - 1。

函数 zlg_fs_add_driver() 的代码见程序清单 13.9。程序很简单，它首先在结构体指针数组 DiskInfo 中找到空闲的成员（程序清单 13.9(2)、(3)），即值为 NULL 的成员。若找到，则它指向参数指向的结构体变量（程序清单 13.9(4)），返回成功（程序清单 13.9(5)）；若结构体指针数组 DiskInfo 满，则返回 - 1，表示不成功（程序清单 13.9(6)）。

程序清单 **13.9** 注册 **ZLG/FS V1.10** 驱动程序

```
int zlg_fs_add_driver(fs_drivers * info)                                 (1)
{
    int i;

    for (i = 0; i < MAX_DISKS; i ++)                                     (2)
    {
        if (DiskInfo\[i\] == NULL)                                       (3)
        {
            DiskInfo\[i\] = info;                                        (4)
            return 0;                                                    (5)
        }
    }
    return -1;                                                           (6)
}
```

13.4.3 注销 ZLG/FS V1.10 驱动程序

硬件驱动程序可随时卸载，但在卸载前要通知本驱动程序取消对相应块设备的支持，这通过调用函数 zlg_fs_remove_driver() 来实现。函数 zlg_fs_remove_driver() 具有惟一的参数info，

它是一个指针，指向保存块设备信息的结构体变量。若操作成功，则函数 zlg_fs_remove_driver() 返回 0；否则返回 - 1。函数 zlg_fs_remove_driver() 的代码见程序清单 13.10。函数 zlg_fs_remove_driver() 很简单，只是在结构体指针数组 DiskInfo 中查找与参数相同的程序，然后将这个成员赋值为 NULL，这里不再详细介绍，请自行阅读源代码。

程序清单 **13.10**　注销 **ZLG/FS V1.10** 驱动程序

```
int zlg_fs_remove_driver(fs_drivers *info)
{
    int i;

    for (i=0; i < MAX_DISKS; i++)                                    (1)
    {
        if (DiskInfo\[i\] == info)                                   (2)
        {
            DiskInfo\[i\] = NULL;                                    (3)
            return 0;                                                (4)
        }
    }
    return -1;                                                       (5)
}
```

13.4.4　关于分区设备

很多块设备不会以一整块的方式使用，相反，系统管理员通常希望对该设备进行分区，也就是说，将整个设备划分为若干独立的伪设备。实际上，这是通过分区软件 fdisk 建立分区实现的，但也要驱动程序支持才可以实现。

为了能对设备分区，必须给每个物理设备分配几个次设备号。一个数字用来访问整个设备（如/dev/hda），其他用来访问不同的分区（如/dev/hda1）。fdisk 产生分区名的办法是在全盘设备名后加一个数字后缀，块设备驱动程序应遵循同样的命名规则。

对于本驱动程序来说，次设备号是分区使用的。在默认情况下，其中高 4 位用于识别物理块设备，而低 4 位用于识别某个物理块设备的分区。具体的使用方法如表 13.2 所列。

给每个物理设备分配几个次设备号，所以宏 DEVICE_NR（参考 11.2.6 小节）也需要改变，其定义见程序清单 13.11，其中的 ZLG_FS_SHIFT 是一个常量，默认情况下为 4。

更多高清PDF图书　www.gqpdf.com

表 13.2　次设备号的使用

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
物理块设备索引				块设备索引			
0000b 对应 DiskInfo\[0\]		0001b 对应 DiskInfo\[1\]		0000b 对应整个设备		0001b 对应第 1 个分区	
0010b 对应 DiskInfo\[2\]		0011b 对应 DiskInfo\[3\]		0010b 对应第 2 个分区		0011b 对应第 3 个分区	
0100b 对应 DiskInfo\[4\]		0101b 对应 DiskInfo\[5\]		0100b 对应第 4 个分区		0101b 对应第 5 个分区	
0110b 对应 DiskInfo\[6\]		0111b 对应 DiskInfo\[7\]		0110b 对应第 6 个分区		0111b 对应第 7 个分区	
1000b 对应 DiskInfo\[8\]		1001b 对应 DiskInfo\[9\]		1000b 对应第 8 个分区		1001b 对应第 9 个分区	
1010b 对应 DiskInfo\[10\]		1011b 对应 DiskInfo\[11\]		1010b 对应第 10 个分区		1011b 对应第 11 个分区	
1100b 对应 DiskInfo\[12\]		1101b 对应 DiskInfo\[13\]		1100b 对应第 12 个分区		1101b 对应第 13 个分区	
1110b 对应 DiskInfo\[14\]		1111b 对应 DiskInfo\[15\]		1110b 对应第 14 个分区		1111b 对应第 15 个分区	

程序清单 **13.11**　从 **kdev** 设备编号获得设备的物理编号

```
#define DEVICE_NR(device) (MINOR(device) > > ZLG_FS_SHIFT)
```

13.4.5　检测分区

1. 概　述

每个可分区设备需要知道它是如何分区的。这个信息可从分区表中得到。驱动程序应当包括解码分区表的代码，并及时更新内部数据结构，以反映分区信息。

这个解码并不容易。不过，核心提供可被所有块设备驱动程序使用的“通用硬盘”支持，它明显地减少了处理分区驱动程序的代码。这个普通支持的另一个好处是，编写驱动程序的程序员不必理解分区是如何完成的，并且不需要修改驱动程序的代码即可在核心中支持新的分区方式。若要支持分区的块设备驱动程序，则应包含 < linux/genhd. h > 。

2. 函数 register_disk ()

“通用硬盘”最主要的是支持调用函数 register_disk () 来检测分区。函数 register_disk () 的原型见程序清单 13.12。

程序清单 **13.12**　分区检测函数原型

```
void register_disk(struct gendisk *gdev, kdev_t dev, unsigned minors, struct
block_device_operations *ops, long size);
```

函数 register_disk () 有 5 个参数，下面分别说明。

gdev：指向结构体 gendisk 变量的指针。该结构体变量有驱动程序分配。后面会专门介绍。

dev：从设备号。

minors：支持的分区数目。

ops：指向块设备的保存设备信息的结构体变量的指针，可参考 11.2.5 小节。

size：以扇区为单位的设备大小。

3. 结构体 gendisk

若要支持分区的块设备驱动程序，则必须拥有结构体 gendisk 变量（可静态定义或动态分配）。所有这样的结构体变量被组织在一个单向链表中，它的头是全局指针 gendisk_head。

在 Linux（μClinux）2.4.x 中，结构体 gendisk 的定义见程序清单 13.13。

程序清单 **13.13**　结构体 **gendisk** 的定义

```
struct gendisk {
    int major;
    const char *major_name;
    int minor_shift;
    int max_p;

    struct hd_struct *part;
    int *sizes;
    int nr_real;

    void *real_devices;
    struct gendisk *next;
    struct block_device_operations *fops;

    devfs_handle_t *de_arr;
    char *flags;
}
```

尽管结构体 gendisk 有 12 个成员，但驱动程序并不需要关心那么多，下面介绍驱动程序关心的成员。

major：确定这个结构所指的设备驱动程序的主设备号。

major_name：属于这个主设备号的设备基本名。每个设备名是通过在这个名字后为每个单元加一个字母并为每个分区加一个数字得到的。例如，hd 是用来构成/dev/hda1 和/dev/hda3 的基本名。Linux（μClinux）2.4.x 中，磁盘名称的总长可达 32 个字符。这样，基本名长度小于 27 个字符都是安全的。在本驱动中，它为 zlg_fs。

minor_shift：从设备的次设备号中获取驱动器号要进行移位的次数。在本驱动程序中，默认

更多高清PDF图书 www.gqpdf.com

为4。这个域中的值应与宏 DEVICE_NR(device)中的定义一致(参考 13.4.4 小节)。

max_p: 分区的最大数目。在本驱动程序中默认为(1 < < ZLG_FS_SHIFT)(参考 13.4.4 小节)。

part: 设备的解码后的分区表。驱动程序用这一项确定,通过次设备号访问块设备时,哪些范围的磁盘扇区是可访问的。这个指针指向的结构体数组由驱动程序自己分配空间。在核心解码分区表之前,驱动程序应将数组初始化为零。

sizes: 这个域指向一个整数数组。这个数组保持着与 blk_size 同样的信息。驱动程序负责分配和释放该数据区域。注意设备的分区检查把这个指针拷贝到 blk_size,因此处理可分区设备的驱动程序不必分配后一个数组。

nr_real: 存在的真实设备(单元)的个数。

real_devices: 这个指针被那些需要保存一些额外私有信息的驱动程序内部使用(这与 filp- > private_data 类似)。

next: 用于构造单向链表。

4. 结构体 hd_struct

前面提到,结构体 gendisk 有一个成员 part,它是指向结构体 hd_struct 的指针。该结构体变量用于保存分区解码后的数据。每个分区对应一个结构体 hd_struct 变量。对于驱动程序来说,只有两个成员有用,它们是 start_sect 和 nr_sects,都是无符号长整型变量。

start_sect: 以扇区为单位的分区起始偏移。

nr_sects: 以扇区为单位的分区大小。

5. 操作链表

前面提到,内核使用一个链表来管理可分区块设备。但是,“通用硬盘”并不是为内核模块设计的,而是为编译进内核的块设备驱动程序设计的,因此,内核没有提供操作这个链表的函数,需要驱动程序自己操作这个链表。对于驱动程序来说,有两个操作:把一个元素插入链表和把一个特定的元素从链表中删除。

把一个元素插入链表很简单,参考程序清单 13.14,其中 gendisk_head 为要插入的元素。程序清单 13.14 是把这个元素插入链表头。

程序清单 **13.14** 把一个元素插入链表

```
zlg_fs_gendisk.next = gendisk_head;
gendisk_head = &zlg_fs_gendisk;
```

从链表中删除一个指定的元素参考程序清单 13.15。这段代码删除链表中与 zlg_fs_gendisk 一致的元素。删除原理也很简单:如果对单向量表的原理很熟悉,则程序很容易读懂,这里仅简单介绍:当指定要删除的元素为链表头时,用程序清单 13.15(3)删除;否则用程序清单 13.15(4) ~ (9)删除。如果想详细了解单向链表的相关知识,请参考关于数据结构的书籍。

程序清单 **13.15**　删除链表中的一个元素

```
struct gendisk *gdp;                                                    (1)

if (gendisk_head == &zlg_fs_gendisk)                                    (2)
{
    gendisk_head = zlg_fs_gendisk.next;                                 (3)
}
else
{
    gdp = gendisk_head;                                                 (4)
    while (gdp->next != NULL)                                           (5)
    {
        if (gdp->next == &zlg_fs_gendisk)                               (6)
        {
            gdp->next = zlg_fs_gendisk.next;                            (7)
            break;                                                      (8)
        }
        gdp = gdp->next;                                                (9)
    }
}
```

13.4.6　驱动程序的初始化函数

驱动程序的初始化函数代码见程序清单 13.16。对比程序清单 12.7 可看出，程序清单 13.16 没有程序清单 12.7(1)，但多了程序清单 13.16(7) ~ (16)。程序清单 12.7(1)是初始化 ZLG/FFS 的，与特定的驱动程序有关，当然在程序清单 13.16 找不到，而程序清单 13.16(7) ~ (16)与分区检测相关。程序清单 13.16(7) ~ (14)是对 13.4.5 小节所介绍的结构体 gendisk 变量赋值，而程序清单 13.16(15) ~ (16)就是把结构体 gendisk 变量插入链表中(参见程序清单 13.14)。关于这两个部分代码请参考 13.4.5 小节。

程序清单 **13.16**　初始化函数

```
int  zlg_fs_init(void)
{
    int  result, i;

    result = register_blkdev(MAJOR_NR,  DEVICE_NAME,  &zlg_fs_fops);    (1)
    if (result < 0)                                                     (2)
```


更多高清PDF图书　www.gqpdf.com

```
    {
        printk(KERN_ERR DEVICE_NAME ": Unable to get major % d\\n", MAJOR_NR );   (3)
        return result;                                                            (4)
    }
    if (MAJOR_NR == 0)                                                            (5)
    {
        MAJOR_NR = result;                                                        (6)
    }

    zlg_fs_gendisk.major = major;                                                 (7)
    zlg_fs_gendisk.major_name = DEVICE_NAME;                                      (8)
    zlg_fs_gendisk.minor_shift = ZLG_FS_SHIFT;                                    (9)
    zlg_fs_gendisk.max_p = 1 < < ZLG_FS_SHIFT;                                    (10)
    zlg_fs_gendisk.fops = &zlg_fs_fops;                                           (11)
    zlg_fs_gendisk.sizes = ZLG_FS_sizes;                                          (12)
    zlg_fs_gendisk.nr_real = MAX_DISKS;                                           (13)
    zlg_fs_gendisk.part = part;                                                   (14)

    zlg_fs_gendisk.next = gendisk_head;                                           (15)
    gendisk_head = &zlg_fs_gendisk;                                               (16)

    blk_queue_make_request(BLK_DEFAULT_QUEUE(MAJOR_NR), zlg_fs_make_request);
                                                                                  (17)
    blk_size\[MAJOR_NR\] = ZLG_FS_sizes;                                          (18)
    blksize_size\[MAJOR_NR\] = ZLG_FS_blocksizes;                                 (19)

    printk(KERN_INFO DEVICE_NAME ": init OK\\n");                                 (20)
    return 0;                                                                     (21)
}
```

13.4.7 驱动程序的清除函数

驱动程序的清除函数代码见程序清单 13.17。代码是按照程序清单 11.4 编写的。程序清单 11.4 对应于程序清单 13.17(1) ~ (8),而程序清单 11.4(2)对应于程序清单 13.17(9)。程序清单 13.17(1) ~ (8)即是程序清单 13.15 所示代码,具体请参考 13.4.5 小节。

程序清单 **13.17** 清除函数

```
void zlg_fs_cleanup(void)
{
    struct gendisk *gdp;

    if (gendisk_head == &zlg_fs_gendisk)                    (1)
    {
        gendisk_head = zlg_fs_gendisk.next;                 (2)
    }
    else
    {
        gdp = gendisk_head;                                 (3)
        while (gdp->next != NULL)                           (4)
        {
            if (gdp->next == &zlg_fs_gendisk)               (5)
            {
                gdp->next = zlg_fs_gendisk.next;            (6)
                break;                                      (7)
            }
            gdp = gdp->next;                                (8)
        }
    }
    unregister_blkdev(MAJOR_NR, DEVICE_NAME);               (9)
}
```

13.4.8 open()方法

驱动程序的 open()方法代码见程序清单 13.18。这些代码完全是按照程序清单 11.7 编写的。本驱动程序的 open()方法代码与 open()方法的示意代码对照见表 13.3。

程序清单 **13.18** **open()方法**

更多高清PDF图书 www.gqpdf.com

```
static int  zlg_fs_open(struct inode *inode, struct file *filp)
{
    int num;
    unsigned long flag;
    Disk_RW_Parameter dp;

    num = DEVICE_NR(inode->i_rdev);                                    (1)
    if (num >= MAX_DISKS)                                              (2)
    {
        return -ENODEV;                                                (3)
    }

    if (DiskInfo\[num\] == NULL)                                       (4)
    {
      return -ENODEV;                                                  (5)
    }

    ZLG_FS_usage\[num\]++;                                             (6)
    if (ZLG_FS_usage\[num\] == 1)                                      (7)
    {
        dp.RsvdForLow = DiskInfo\[num\]->RsvdForLow;                   (8)
        DiskInfo\[num\]->DiakCommand(DISK_INIT, &dp);                  (9)

        if (DiskInfo\[num\]->open != NULL)                             (10)
        {
            DiskInfo\[num\]->open(inode, filp);                        (11)
        }
        zlg_fs_revalidate(inode->i_rdev);                              (12)
    }
    MOD_INC_USE_COUNT;                                                 (13)
    return 0;                                                          (14)
}
```

表 13.3　open() 方法的示意代码与驱动代码对照说明

示意代码(程序清单 11.7)	实际代码	说　明

(1) 从设备号 = MINOR (inode - > i_rdev);	程序清单 13.18 (1)	宏 DEVICE_NR () 为获取物理设备号(参考 13.4.4 小节)
(2) 通过从设备号判断需要操作的设备;		
(3) 通知内核进入临界区;	无	硬件驱动应有这部分代码,这里不需要
(4) 需要操作的设备的使用计数器 + +;	程序清单 13.18 (6)	
(5) if (需要操作的设备的使用计数器 = =1)	程序清单 13.18 (7)	
{	{	程序清单 13.18 (8) ~ (11) 为初始化硬件驱动程序。程序清单 13.18 (12) 为自己的初始化
(6) 初始化设备(包括申请资源);	程序清单 13.18 (8) ~ (12)	
}	}	
(7) 通知内核退出临界区;	无	
(8) MOD_INC_USE_COUNT;	程序清单 13.18 (13)	
(9) return 0;	程序清单 13.18 (14)	

可以看出,程序清单 13.18 (2) ~ (5) 在示意代码中没有对应的语句。事实上,它是出错处理代码。程序清单 13.18 (2)、(3) 用于处理物理设备号超出范围的情况,而程序清单 13.18 (4) 则查看对应的物理设备是否被支持,不被支持也要返回错误代码(程序清单 13.18 (5))。

下面详细介绍程序清单 13.18 (8) ~ (12)。程序清单 13.18 (12) 为调用自己的 revalidate () 方法。revalidate () 方法是在介质被改变时调用,关于其具体介绍请参考 11.2.5 小节。本驱动程序 revalidate () 方法的分析请参考 13.4.12 小节。

程序清单 13.18 (8) ~ (11) 用于初始化硬件驱动程序。其中程序清单 13.18 (8)、(9) 为 ZLG/FS V1.10 硬件驱动程序的初始化,而程序清单 13.18 (10)、(11) 为调用硬件驱动程序的 open () 方法。

13.4.9 release () 方法

本驱动程序的 release () 方法代码见程序清单 13.19。这些代码完全是按照程序清单 11.8 编写的。本驱动程序的 release () 方法代码与 release () 方法的示意代码对照见表 13.4。

程序清单 **13.19** **release** () 方法代码

更多高清PDF图书 www.gqpdf.com

```
static int  zlg_fs_release(struct inode *inode, struct file *filp)
{
    int num;
    Disk_RW_Parameter dp;
    unsigned long flag;
    int part1, npart, i;

    num = DEVICE_NR(inode->i_rdev);                                    (1)
    if (num >= MAX_DISKS)                                              (2)
    {
        return -ENODEV;                                                (3)
    }

    if (DiskInfo\[num\] == NULL)                                       (4)
    {
        return -ENODEV;                                                (5)
    }

    ZLG_FS_usage\[num\]--;                                             (6)
    if (ZLG_FS_usage\[num\] == 0)                                      (7)
    {
```

高清藏 PDF gqpdf.com

```
        part1 = (num < < ZLG_FS_SHIFT);                                    (8)
        npart =1 < < ZLG_FS_SHIFT;                                         (9)
        for (i =0; i < npart; i ++)                                        (10)
        {
            fsync_dev(MKDEV(major, part1 ++));                             (11)
        }

        dp.RsvdForLow = DiskInfo\[num\] -> RsvdForLow;                     (12)
        DiskInfo\[num\] -> DiakCommand(DISK_CLOSE, &dp);                   (13)

        if (DiskInfo\[num\] -> release != NULL)                            (14)
        {
            DiskInfo\[num\] -> release(inode, filp);                       (15)
        }
    }

    MOD_DEC_USE_COUNT;                                                     (16)
    return 0;                                                              (17)
}
```

表 13.4　release () 方法的示意代码与驱动代码对照说明

示意代码(程序清单 11.7)	实际代码	说　明
(1) 从设备号 = MINOR (inode - > i_rdev);	程序清单 13.19 (1)	宏 DEVICE_NR () 为获取物理设备号(参考 13.4.4 小节)
(2) 通过从设备号判断需要操作的设备;		
(3) 通知内核进入临界区;	无	硬件驱动应有这部分代码,这里不需要
(4) 需要操作的设备的使用计数器 - - ;	程序清单 13.19 (6)	
(5) if (需要操作的设备的使用计数器 = =0)	程序清单 13.19 (7)	
{	{	
(6) 关闭设备处理(包括释放资源);	程序清单 13.19 (8) ~ (15)	
}	}	
(7) 通知内核退出临界区;	无	
(8) MOD_DEC_USE_COUNT;	程序清单 13.19 (16)	
(9) return 0;	程序清单 13.19 (17)	

可以看出,程序清单 13.19 (2) ~ (5) 在示意代码中没有对应的语句。事实上,它是出错处理代码。程序清单 13.19 (2)、(3) 用于处理物理设备号超出范围的情况,而程序清单 13.19 (4) 则查看对应的物理设备是否被支持,不被支持也要返回错误代码(程序清单 13.19 (5))。

下面详细介绍程序清单 13.19 (8) ~ (15)。关于函数 fsync _dev () (程序清单 13.19

更多高清PDF图书 www.gqpdf.com

(11))，在 12.2.6 小节介绍过，它是用于释放由内核保存在各种高速缓存中的对该设备的引用的，其惟一的参数 dev 为设备编号。

注意：这个设备编号既不是主设备号，也不是从设备号；相反，它是由主设备号和从设备号通过某种方式生成的。

通过主设备号和从设备号生成设备编号的方法就是调用宏 MKDEV()。宏 MKDEV() 的第一个参数为主设备号，第二个参数为从设备号。因为一个物理块设备对应有几个从设备号，所以需要多次调用函数 fsync_dev()。程序清单 13.19(8)用于计算物理设备的第一个从设备号，而程序清单 13.19(9)用于计算物理块设备占用的从设备号数目。因为一个物理块设备占用的从设备号是连续的，所以使用一个 for() 循环来调用函数 fsync_dev()（程序清单 13.19(10)、(11)）。

既然硬件被关闭，就需要调用 ZLG/FS V1.10 驱动程序的 DISK_CLOSE 命令（程序清单 13.19(12)、(13)）。关于 ZLG/FS V1.10 驱动程序的接口请参考 13.2 节。而程序清单 13.19(14)、(15) 就是调用硬件驱动程序的 release() 方法。

13.4.10 ioctl() 方法

本驱动程序的 ioctl() 方法代码见程序清单 13.20。这个代码与 ioctl() 方法的示意代码几乎完全相同（参考程序清单 11.9），仅增加了一点错误处理代码（程序清单 13.20(1) ~ (5)）和把各个命令具体化了。这些代码都很简单，参考 11.2.5 小节和 12.2.7 小节容易读懂，这里不再说明（其他信息参考 13.2 节和 13.4.1 小节）。

程序清单 **13.20** ioctl() 方法

```
static int zlg_fs_ioctl(struct inode *inode, struct file *filp,
                        unsigned int cmd, unsigned long arg)
{
    long err, size;
    struct hd_geometry geo;
    int num;

    num = DEVICE_NR(inode->i_rdev);                          (1)
    if (num >= MAX_DISKS)                                    (2)
    {
        return -ENODEV;                                      (3)
    }

    if (DiskInfo\[num\] == NULL)                             (4)
    {
```

高清藏 PDF gqpdf.com

```
        return -ENODEV;                                                    (5)
    }

    switch(cmd)                                                            (6)
    {
        case BLKGETSIZE:                                                   (7)
        if (arg == 0)                                                      (8)
        {
            return -EINVAL;                                                (9)
        }
        if(access_ok(VERIFY_WRITE, arg, sizeof(long)) == 0)                (10)
        {
            return -EFAULT;                                                (11)
        }
        size = zlg_fs_gendisk.part\[MINOR(inode -> i_rdev) \].nr_sects;
                                                                           (12)
        if (copy_to_user((long *) arg, &size, sizeof (long)))              (13)
        {
            return -EFAULT;                                                (14)
        }
        return 0;                                                          (15)
    case BLKRRPART:                                                        (16)
        return zlg_fs_revalidate(inode -> i_rdev);                         (17)
    case HDIO_GETGEO:                                                      (18)
        if (access_ok(VERIFY_WRITE, arg, sizeof(geo)) == 0)                (19)
        {
            return -EFAULT;                                                (20)
        }
        size = DiskInfo\[num\] -> SecPerDisk;                              (21)
        geo.cylinders = (size & ~0x3f) > > 6;                              (22)
        geo.heads = 4;                                                     (23)
        geo.sectors = 16;                                                  (24)
        geo.start = 0;
        if (copy_to_user((void *) arg, &geo, sizeof(geo)))                 (25)
        {
            return -EFAULT;                                                (26)
        }
        return 0;                                                          (27)
```


更多高清PDF图书 www.gqpdf.com

```
        default:                                                    (28)
            return blk_ioctl(inode->i_rdev, cmd, arg);              (29)
    }
    return -ENOTTY;                                                 (30)
}
```

13.4.11 check_media_change()方法

由13.2.2小节中的第11条可知，调用ZLG/FS V1.10驱动程序的DISK_CHECK_CHANGE命令可以检查介质是否改变。很自然，本驱动程序的check_media_change()方法的核心是调用ZLG/FS V1.10驱动程序的DISK_CHECK_CHANGE命令（程序清单13.21(6)、(7)），并根据返回值的不同返回0（程序清单13.21(9)）或1（程序清单13.21(8)）。程序清单13.21的其他部分主要是错误处理代码，本章前面几个小节都多次提到，这里不再描述。

程序清单**13.21　check_media_change()方法代码**

```
static int check_zlg_fs_change(kdev_t dev)
{
    int num;
    Disk_RW_Parameter dp;

    num = DEVICE_NR(dev);                                          (1)
    if (num > = MAX_DISKS)                                         (2)
    {
        return -ENODEV;                                            (3)
    }

    if (DiskInfo\[num\] == NULL)                                   (4)
    {
        return -ENODEV;                                            (5)
    }

    dp.RsvdForLow = DiskInfo\[num\] -> RsvdForLow;                 (6)
    if (DiskInfo\[num\] -> DiakCommand(DISK_CHECK_CHANGE, &dp) == DISK_TRUE)
                                                                   (7)
    {
        return 1;                                                  (8)
    }
    return 0;                                                      (9)
}
```

13.4.12 revalidate()方法

本驱动程序的 revalidate()方法代码见程序清单 13.22。根据 13.4.1、13.4.4、13.4.5 和 13.4.6 小节的介绍，本驱动程序的 revalidate()方法的核心是完成以下 3 个工作：

- 调用硬件驱动程序的 get_info()方法(程序清单 13.22(8))。
- 给数组 ZLG_FS_blocksizes\[\]和 ZLG_FS_sizes\[\]对应成员赋值(程序清单 13.22(12)~(15))。关于这两个数组的赋值可参考 12.2.2 小节(ZLG_FS_blocksizes 对应于 blksize_size，ZLG_FS_sizes 对应于 blk_size)。
- 调用内核提供的函数 register_disk()(程序清单 13.22(18))。

程序清单 13.22(1)~(5)是前几节多次提到获得硬件块设备的索引和错误处理代码。程序清单 13.22(9)获得物理块设备的起始从设备号。程序清单 13.22(10)获得物理块设备占用的从设备号数目。

更多高清PDF图书 www.gqpdf.com

程序清单 **13.22**　**revalidate**()方法代码

```
static int zlg_fs_revalidate(kdev_t dev)
{
    int num;
    unsigned long flag;
    int part1, npart, i;

    num = DEVICE_NR(dev);                                          (1)
    if (num >= MAX_DISKS)                                          (2)
    {
        return -ENODEV;                                            (3)
    }

    if (DiskInfo\[num\] == NULL)                                   (4)
    {
        return -ENODEV;                                            (5)
    }

    if (DiskInfo\[num\]->get_info == NULL)                         (6)
    {
        return -ENODEV;                                            (7)
    }
    DiskInfo\[num\]->get_info(DiskInfo\[num\]);                    (8)
```

关键代码

高清PDF藏 gqpdf.com

```
    part1 = num < < ZLG_FS_SHIFT;                                          (9)
    npart = 1 < < ZLG_FS_SHIFT;                                            (10)

    memset(zlg_fs_gendisk.part + part1, 0, npart * sizeof(struct hd_struct)); (11)

    for (i = 0; i < npart; i ++)                                           (12)
    {
        ZLG_FS_blocksizes\[part1\] = DiskInfo\[num\] -> BytesPerSec;        (13)
        ZLG_FS_sizes\[part1\] = (DiskInfo\[num\] -> SecPerDisk * ZLG_FS_blocksi-
zes\[num\])
                                /1024;                                     (14)
        part1 ++;                                                          (15)
    }

    zlg_fs_gendisk.part\[num < < ZLG_FS_SHIFT\].nr_secta = DiskInfo\[num\] ->
SecPerDisk;                                                                (16)

    register_disk(&zlg_fs_gendisk, dev, 1 < < ZLG_FS_SHIFT,
                  &zlg_fs_fops, DiskInfo\[num\] -> SecPerDisk);            (17)
    return 0;                                                              (18)
}
```

关键代码

关键代码

由 13.4.5 小节可知，调用 register_disk () 之前要把 zlg_fs_gendisk 的 sizes 成员和 part 成员的对应部分清 0，所以需要执行程序清单 13.22 (11)。至于程序清单 13.22 (16)，这是因为函数 register_disk () 会（使用硬件块设备的第一个从设备号）调用本驱动的“make request”函数（参考 13.4.13 小节）读取分区表，而本驱动程序的“make request”函数会依靠这个成员检查扇区号是否超出范围，所以这里需要给该成员赋一个正确的值。

13.4.13 数据传输

关于数据传输请参考 12.2.10 小节。由 12.2.10 小节可知，块设备的数据传输可通过“make request”函数实现，如程序清单 13.23 所示。“make request”函数的示意代码可参考程序清单 12.18。

程序清单 **13.23** “**make request**”函数

更多高清PDF图书 www.gqpdf.com

```
static int zlg_fs_make_request(request_queue_t *queue, int rw, struct buffer_
head *bh)
{
    u32 temp, SecSize;
    u32 size;
```

高清PDF藏 gqpdf.com

```
    int part, num, nr;
    Disk_RW_Parameter dp;
    fs_drivers *fs;

    num = DEVICE_NR(bh->b_rdev);                                    (1)
    nr = MINOR(bh->b_rdev);                                         (2)
    if (num >= MAX_DISKS)                                           (3)
    {
        static int count = 0;                                       (4)
        if (count++ < 5)                                            (5)
        {
            printk(KERN_WARNING DEVICE_NAME ": request past end of device\\n");
                                                                    (6)
        }
        bh->b_end_io(bh, 0);                                        (7)
        return 0;                                                   (8)
    }

    fs = DiskInfo\[num\];                                           (9)
    dp.RsvdForLow = fs->RsvdForLow;                                 (10)
    SecSize = fs->BytesPerSec;                                      (11)
    dp.SectorIndex = bh->b_rsector + zlg_fs_gendisk.part\[nr\].start_sect;
                                                                    (12)
    dp.Buf = bh->b_data;                                            (13)
    size = bh->b_size;                                              (14)
    if (bh->b_rsector + size /SecSize >= zlg_fs_gendisk.part\[nr\].nr_sects)
                                                                    (15)
    {
        bh->b_end_io(bh, 0);                                        (16)
        return 0;                                                   (17)
    }

    switch(rw)                                                      (18)
    {
        case READ:                                                  (19)
        case READA:                                                 (20)
            while (size >= SecSize)                                 (21)
            {
                DiskInfo\[num\]->DiakCommand(DISK_READ_SECTOR, &dp);    (22)
```

更多高清PDF图书 www.gqpdf.com

```
            size = size - SecSize;                                          (23)
            dp.SectorIndex ++;                                              (24)
            dp.Buf  += SecSize;                                             (25)
        }
        bh -> b_end_io(bh, 1);                                              (26)
        break;                                                              (27)
    case WRITE:                                                             (28)
        refile_buffer(bh);                                                  (29)
        while (size > = SecSize)                                            (30)
        {
            DiskInfo\[num\] -> DiakCommand(DISK_WRITE_SECTOR, &dp);         (31)
            size = size - SecSize;                                          (32)
            dp.SectorIndex ++;                                              (33)
            dp.Buf  += SecSize;                                             (34)
        }
        mark_buffer_uptodate(bh, 1);                                        (35)
        bh -> b_end_io(bh, 1);                                              (36)
        break;                                                              (37)
    default:
        bh -> b_end_io(bh, 0);                                              (38)
        break;                                                              (39)
    }
    return 0;                                                               (40)
}
```

本驱动程序“make request”函数实际的代码见程序清单 13.23。“make request”函数的代码基本是按照示意代码编写的，其与示意代码的对照见表 13.5。可以看出，程序清单 13.23 (3) ~ (8)、(15) ~ (17) 在示意代码中没有对应的语句。它是错误处理，示意代码中也不会写出。另外，程序清单 13.23 (4) 也是 GCC 的扩展功能，标准 C 不一定支持。程序清单 13.23 (5)、(6) 是仅在前 5 次出错时显示出错信息，以保证控制台的整洁。

真正的数据传输是调用 ZLG/FS V1.10 驱动程序的 DISK_READ_SECTOR 命令（程序清单 13.23 (22)）和 DISK_WRITE_SECTOR 命令（程序清单 13.23 (31)）实现的，程序清单 13.23 (9)、(10) 为它们提供初始参数。关于 ZLG/FS V1.10 驱动程序的相关信息请参考 13.2 节。

表 13.5 “make request”的示意代码与驱动代码对照说明

示意代码	实际代码	说 明
程序清单 12.18(1)	程序清单 13.23(1)	直接获得要操作的物理设备索引
程序清单 12.18(2)	程序清单 13.23(2)	获得分区号
程序清单 12.18(3)	程序清单 13.23(11)	
程序清单 12.18(4)	程序清单 13.23(12)	
程序清单 12.18(5)	程序清单 13.23(13)	
程序清单 12.18(6)	程序清单 13.23(14)	
程序清单 12.18(7)	程序清单 13.23(18)	
程序清单 12.18(8)	程序清单 13.23(19)	
程序清单 12.18(9)	程序清单 13.23(20)	
程序清单 12.18(10)	程序清单 13.23(21)	
程序清单 12.18(11)		硬件驱动自己保护
程序清单 12.18(12)	程序清单 13.23(22)	
程序清单 12.18(13)		硬件驱动自己保护
程序清单 12.18(14)	程序清单 13.23(23)	
程序清单 12.18(15)	程序清单 13.23(24)	
程序清单 12.18(16)	程序清单 13.23(25)	
程序清单 12.18(17)	程序清单 13.23(26)	
程序清单 12.18(18)	程序清单 13.23(27)	
程序清单 12.18(19)	程序清单 13.23(28)	
程序清单 12.18(20)	程序清单 13.23(29)	
程序清单 12.18(21)	程序清单 13.23(30)	
程序清单 12.18(22)		硬件驱动自己保护
程序清单 12.18(23)	程序清单 13.23(31)	
程序清单 12.18(24)		硬件驱动自己保护
程序清单 12.18(25)	程序清单 13.23(32)	
程序清单 12.18(26)	程序清单 13.23(33)	
程序清单 12.18(27)	程序清单 13.23(34)	
程序清单 12.18(28)	程序清单 13.23(35)	
程序清单 12.18(29)	程序清单 13.23(36)	
程序清单 12.18(30)	程序清单 13.23(37)	
程序清单 12.18(31)	程序清单 13.23(38)	
程序清单 12.18(32)	程序清单 13.23(39)	
程序清单 12.18(33)	程序清单 13.23(40)	

更多高清PDF图书 www.gqpdf.com

13.4.14 其他说明

本驱动程序没有涉及中断、动态内存分配和按需加载等相关问题。关于这些部分的进一步信息请参见参考文献[2]和[4]。

另外，关于修改版权及作者声明，请参考 6.3.6 小节。

本章小结

本章以一个通用块设备驱动程序为例介绍了实际的 Linux（μClinux）块设备驱动程序的相关知识。首先介绍该驱动程序的使用方法，然后对源码进行分析。

相对于第 11 章，本章主要增加了以下知识：

- 支持分区设备（参考 13.4.4 小节）。
- 提供函数给其他内核模块使用（参考 13.4.1 和 13.4.3 小节）。

高清藏 PDF gqpdf.com

第14章　转换 ZLG/FS V1.10 驱动为 μClinux 驱动

本章为第13章的延续，主要讲述如何编写第13章提到的与硬件相关的驱动程序，实际上是讲述如何把基于 ZLG/FS V1.10 的驱动程序转换为基于 μClinux 的驱动，这个基于 μClinux 的驱动与第13章提到的驱动程序一起实现完整的块设备驱动程序。

笔者提供一个驱动程序模板，通过这个模板，基于 ZLG/FS V1.10 的驱动程序只需改动很少的代码，就可以生成这个与硬件相关的驱动程序。

14.1　转换步骤

使用笔者提供的模板，很容易把基于 ZLG/FS V1.10 的驱动程序转换为基于 μClinux 的驱动，转换步骤如下：

① 将模板提供的所有文件与基于 ZLG/FS V1.10 的驱动程序源代码拷贝到同一个目录下。这主要是方便编译程序。

② 修改 Makefile 文件。

模板提供的 Makefile 文件内容请参考程序清单14.1。程序清单14.1实际上是在程序清单4.2的基础上修改而来的，其修改方法请参考4.3节。

程序清单 **14.1**　模板提供的 **Makefile** 文件

```
EXEC = zlg_fs_driver                                          (1)
OBJS = zlg_fs_driver.o                                        (2)
SRC = zlg_fs_driver.c                                         (3)

INCLUDE = /work/uClinux-dist/linux-2.4.x/include              (4)
CC = arm-elf-gcc                                              (5)
LD = arm-elf-ld                                               (6)
```

与程序清单4.2一样的修改方法

与程序清单4.2一样的修改方法

更多高清PDF图书　www.gqpdf.com

```
MODCFLAGS = -D__KERNEL__ -I $(INCLUDE) -O2 -fno-strict-aliasing -fno-common -pipe -
fno-builtin -D__linux__ -g -DNO_MM -mapcs-32 -march=armv4 -mtune=arm7tdmi -mshort-
load-bytes -msoft-float -nostdinc -iwithprefix include                          (7)
LDFLAGS = -m armelf -T                                                          (8)
                                  特殊的连接参数，不能修改

all: $(EXEC)                                                                    (9)

$(EXEC): $(OBJS)                                                                (10)
    $(LD) $(LDFLAGS) -o $@  $(OBJS)                                             (11)

%.o:%.c                                                                         (12)
    $(CC) $(MODCFLAGS) -mapcs -c $< -o $@                                       (13)

clean:                                                                          (14)
    -rm -f $(EXEC) *.elf *.gdb *.o                                              (15)
```

③ 修改 config.h 文件。

模板提供的 config.h 文件内容请参考程序清单 14.2。config.h 文件看起来很长，但大部分不需要修改。一般情况下，用户只需修改程序清单 14.2(42) ~ (53) 部分。

程序清单 **14.2**　模板提供的 **config.h** 文件

```
#ifndef __CONFIG_H                                                              (1)
#define __CONFIG_H                                                              (2)

//This segment should not be modified

#ifndef TRUE                                                                    (3)
#define TRUE   1                                                                (4)
#endif                                                                          (5)
                                  辅助宏定义

#ifndef FALSE                                                                   (6)
#define FALSE 0                                                                 (7)
#endif                                                                          (8)
```

```
#ifndef __KERNEL__                                          (9)
    #define __KERNEL__                                      (10)
#endif                                                      (11)
#ifndef MODULE                                              (12)
    #define MODULE                                          (13)
#endif
```

驱动程序需要的宏定义

```
#include <linux/config.h>                                   (14)
#include <linux/module.h>                                   (15)

#include <linux/sched.h>                                    (16)
#include <linux/kernel.h>                                   (17)
//#include <linux/malloc.h>                                 (18)
#include <linux/slab.h>                                     (19)
#include <linux/fs.h>                                       (20)
#include <linux/errno.h>                                    (21)
#include <linux/timer.h>                                    (22)
#include <linux/types.h>                                    (23)
#include <linux/fcntl.h>                                    (24)
#include <linux/hdreg.h>                                    (25)

#include <asm/system.h>                                     (26)
#include <linux/blkpg.h>                                    (27)
#include <linux/iobuf.h>                                    (28)
#include <linux/ioctl.h>                                    (29)
#include <asm/uaccess.h>                                    (30)
#include <linux/init.h>                                     (31)
```

驱动程序需要的一些头文件

```
typedef u8          uint8;                                  (32)
typedef s8          int8;                                   (33)
typedef u16         uint16;                                 (34)
typedef s16         int16;                                  (35)
typedef u32         uint32;                                 (36)
typedef s32         int32;                                  (37)
typedef float       fp32;                                   (38)
typedef double      fp64;                                   (39)

#include "../zlg_fs/zlg_fs.h"                               (40)
```

ZLG/FS V1.10 驱动程序需要的一些定义

第13章的驱动程序提供的头文件

更多高清PDF图书 www.gqpdf.com

```
#include "fat.h"                                                    (41)

#define DEVICE_NAME "zlg/fs-test"                                   (42)
#define MAX_DISK_DRIVES          1                                  (43)

#define DEVICE_BYTES_PER_SEC     512                                (44)
#define DEVICE_SEC_PER_DISK      (1024 * 32)                        (45)

#define COMMAND_FUNCTION         NULL                               (46)
#define DISK_INFO                zlg_fs_disk                        (47)

#ifdef IN_ZLG_FS_DRIVER                                             (48)

void *zlg_fs_disk\[MAX_DISK_DRIVES\];                               (49)

      static void drive_init(void)                                  (50)
{                                                                   (51)
}                                                                   (52)

#endif                                                              (53)

#define malloc(a)    kmalloc(a, GFP_KERNEL)                         (54)
#define free(a)      kfree(a)                                       (55)

#define OS_ENTER_CRITICAL()                                         (56)
#define OS_EXIT_CRITICAL()                                          (57)

#endif                                                              (58)
```

ZLG/FS V1.10提供的头文件

需要修改的地方

转换动态内存分配函数

转换开关中断函数

程序清单 14.2(42)为驱动程序名称，为一个用户自定义的字符串。

程序清单 14.2(43)为驱动程序支持的块设备数目，根据实际情况配置。

程序清单 14.2(44)为以字节为单位的块设备默认扇区大小，根据实际情况配置。如果驱动程序支持 DISK_GET_BYTES_PER_SECTOR 命令(参考 13.2.2 小节第 10 条)，则此定义无效，可定义成任意值。程序清单 14.2(45)为块设备默认扇区数目，根据实际情况配置。如果驱动程序支持 DISK_GET_SECTOR_NUMBER 命令(参考 13.2.2 小节第 9 条)，则此定义无效，可定义成任意值。程序清单 14.2(46)为驱动程序接口函数名，根据实际情况定义。程序清单

14.2(47)、(49)为给驱动程序的保留参数 RsvdForLow(参考 13.2.2 小节),根据实际情况定义。如果驱动程序不需要这个参数,则可不定义它。程序清单 14.2(48)、(53)为保证程序清单 14.2(49)~(52)此段代码在主文件中有效。程序清单 14.2(50)~(52)为在驱动程序被加载时调用的一个函数,用户可用它实现特定的操作。

④ 修改 ZLG/FS V1.10 驱动程序源文件。

如果 ZLG/FS V1.10 驱动程序源文件兼容性比较强,则这些代码无须修改。但 Linux(μClinux)的文件系统是区分大小写的,GCC 编译工具也对文件名(包括路径)中的大小写敏感,并且 GCC 中的 C 编译器对源代码中的文件名(包括路径)大小写敏感,而 Windows 下大多数编译器不区分文件名(包括路径)中的大小写。这样,用户可能需要修改 ZLG/FS V1.10 驱动程序源文件的这部分代码(主要是#include 语句)。同时,GCC 毕竟是一个特定的编译器,特定版本的 GCC 编译器(中的 C 编译器)可能有与特定版本的 ANSI C 不兼容的地方。如果 ZLG/FS V1.10 驱动程序源文件使用了这些特性,源代码也需要进行相应的修改。

经过上述步骤后,即可编译驱动程序了。编译驱动程序请参考 4.3.1 小节。

14.2 转换实例之 CF 卡驱动

关于 CF 卡/IDE 硬盘驱动的知识可参见参考文献[5]的第 3 章。

本节假设工作目录为当前目录。下面介绍转换步骤。

① 拷贝模板到当前目录。当前版本包含 config.h、Fat.h、Makefile 和 zlg_fs_driver.c。

② 将 zlg_fs_driver.c 改名为 zlg_fs_cf.c。

③ 新建 CF_IDE 目录,将 ZLG/FS V1.10 的 CF 卡驱动源代码拷贝到新建的目录。当前版本包含 ATA.c、IDE.c、IDE.h、SysATA.c 和 SysATA.H。可将 SysATA.H 改名为 SysATA.h。

④ 修改 config.h 文件。这个 config.h 的修改部分见程序清单 14.3。至于详细修改哪些部分,请看程序清单 14.3 中的注释。

程序清单 **14.3** **CF** 卡驱动 **config.h** 的修改部分

```
#include "CF_IDE/SysATA.H"                                    (1)
#include "CF_IDE/IDE.H"                                       (2)
#include "LPC2294.h"                                          (3)

#define DEVICE_NAME          "zlg/fs/cf"                      (4)
#define MAX_DISK_DRIVES      1                                (5)
```

驱动程序需要新增的头文件

驱动程序名称

更多高清PDF图书 www.gqpdf.com

```
#define DEVICE_BYTES_PER_SEC          512                              (6)
#define DEVICE_SEC_PER_DISK           (1024 * 32)                      (7)

#define COMMAND_FUNCTION              CFCammand                        (8)
//#define DISK_INFO                   zlg_fs_disk                      (9)

#ifdef IN_ZLG_FS_DRIVER                                                (10)
extern uint16 CFCammand(uint8 Cammand, void *Parameter);
                                                                       (11)
//void *zlg_fs_disk\[MAX_DISK_DRIVES\];                                (12)

static void drive_init(void)                                           (13)
{                                                                      (14)
}                                                                      (15)
#endif
```

驱动程序接口函数名

驱动程序不需要保留参数 RsvdForLow

驱动程序接口函数

驱动程序不需要保留参数 RsvdForLow

⑤ 修改 Makefile 文件。修改后的 Makefile 参考程序清单 14.4，修改的部分已标出。

程序清单 **14.4**　**CF** 卡驱动的 **Makefile** 文件

```
EXEC = zlg_fs_cf
OBJS = zlg_fs_cf.o CF_IDE/IDE.o CF_IDE/SysATA.o CF_IDE/ATA.o
SRC = zlg_fs_cf.c CF_IDE/IDE.c CF_IDE/SysATA.c CF_IDE/ATA.c
INCLUDE = /work/uClinux-dist/linux-2.4.x/include
USEINC = .
CC = arm-elf-gcc
LD = arm-elf-ld
MODCFLAGS = -D__KERNEL__ -I $(INCLUDE) -I $(USEINC) -O2 -fno-strict-aliasing -fno-
common -pipe -fno-builtin -D__linux__ -g -DNO_MM -mapcs-32 -march = armv4 -mtune =
arm7tdmi -mshort-load-bytes -msoft-float -nostdinc -iwithprefix include
LDFLAGS = -m armelf -r

all: $(EXEC)

$(EXEC): $(OBJS)
```

修改的部分

修改的部分

```
    $(LD) $(LDFLAGS) -o $@  $(OBJS)

%.o:%.c
    $(CC) $(MODCFLAGS) -mapcs -c $< -o $@

clean:
    -rm -f $(EXEC) *.elf *.gdb *.o修改的部分
    -rm -f CF_IDE/*.o
```

⑥ 修改 CF_IDE 下的源代码。源代码中部分文件名与实际文件名大小写不对应，改正即可。

⑦ 编译驱动程序，生成 zlg_fs_cf 文件就是需要的目标文件。

14.3 转换实例之 SD 卡驱动

关于 SD 卡驱动的知识可参见参考文献[6]。

本节假设工作目录为当前目录。下面介绍转换步骤。

① 拷贝模板到当前目录。当前版本包含 config.h、Fat.h、Makefile 和 zlg_fs_driver.c。

② 将 zlg_fs_driver.c 改名为 zlg_fs_sd.c。

③ 新建 SDMMC 目录，将 ZLG/FS V1.10 的 SD 卡驱动源代码拷贝到新建的目录。当前版本包含 sd.c、sdcmd.c、sdcmd.h、sdcrc.c、sdcrc.h、sddriver.c、sddriver.h、sdhal.c 和 sdhal.h。

④ 修改 config.h 文件。这个 config.h 的修改部分见程序清单 14.5。至于详细修改哪些部分，请看程序清单 14.5 中的注释。

程序清单 **14.5** **SD** 卡驱动 **config.h** 的修改部分

```
#include "sdconfig.h"
#include "SDMMC/sddriver.h"
#include "LPC2294.h"

#define DEVICE_NAME                 "zlg/fs/sd"
#define MAX_DISK_DRIVES             1
#define DEVICE_BYTES_PER_SEC        512
#define DEVICE_SEC_PER_DISK         (1024 * 32)

#define COMMAND_FUNCTION            SDCammand
//#define DISK_INFO                 zlg_fs_disk
```

驱动程序需要新增的头文件

驱动程序名称

驱动程序接口函数名

驱动程序不需要保留参数 RsvdForLow

更多高清PDF图书 www.gqpdf.com

```
#ifdef IN_ZLG_FS_DRIVER
extern uint16 SDCammand(uint8 Cammand, void * Parameter);

//void * zlg_fs_disk\[MAX_DISK_DRIVES\];

static void drive_init(void)
{
}
```

驱动程序接口函数

驱动程序不需要保留参数 RsvdForLow

⑤ 修改 Makefile 文件。修改后的 Makefile 参考程序清单 14.6，修改的部分已标出。

程序清单 **14.6**　**SD** 卡驱动的 **Makefile** 文件

高清藏 PDF gqpdf.com

```
EXEC = zlg_fs_sd
OBJS = zlg_fs_sd.o SDMMC/sd.o SDMMC/sdcmd.o SDMMC/sdcrc.o SDMMC/sddriver.o SDMMC/
sdhal.o
SRC = zlg_fs_sd.c SDMMC/sd.c SDMMC/sdcmd.c SDMMC/sdcrc.c SDMMC/sddriver.c SDMMC/
sdhal.c

INCLUDE = /work/uClinux-dist/linux-2.4.x/include
USEINC = .
CC = arm-elf-gcc
LD = arm-elf-ld
MODCFLAGS = -D__KERNEL__ -I $(INCLUDE) -I $(USEINC) -O2 -fno-strict-aliasing -fno-com-
mon -pipe -fno-builtin -D__linux__ -g -DNO_MM -mapcs-32 -march = armv4 -mtune = arm7tdmi -
mshort-load-bytes -msoft-float -nostdinc -iwithprefix include
LDFLAGS = -m armelf -r

all: $(EXEC)

$(EXEC): $(OBJS)
    $(LD) $(LDFLAGS) -o $@  $(OBJS)

%.o:%.c
    $(CC) $(MODCFLAGS) -mapcs -c $< -o $@

clean:
    -rm -f $(EXEC) *.elf *.gdb *.o
    -rm -f SDMMC/*.o
```

修改的部分

修改的部分

⑥ 修改 SDMMC 下的源代码。arm-elf-gcc V2.95.3 中的 C 编译器不支持#define 语句中使用连接符“\\”，而源代码中使用了这个特性，所以需要修改。

⑦ 编译驱动程序，生成 zlg_fs_sd 文件就是需要的目标文件。

14.4 转换实例之 ZLG/FFS 驱动

ZLG/FFS 驱动是具有写平衡和坏块管理的 NAND FLASH 的驱动程序。

更多高清PDF图书 www.gqpdf.com

关于 ZLG/FFS 驱动的知识可参见参考文献[6]。

本节假设工作目录为当前目录。下面介绍转换步骤。

① 拷贝模板到当前目录。当前版本包含 config. h、Fat. h 、Makefile 和 zlg_fs_driver. c。

② 将 zlg_fs_driver. c 改名为 zlg_fs_ffs. c。

③ 新建 ZLG_FFS 目录,将 ZLG/FFS 驱动源代码拷贝到新建的目录。当前版本包含 ZLG_FFS. c、k9fxx08\\k9fxx08. c、k9fxx08\\k9fxx08. h 和 ZLG_FFS. h。

④ 修改 config. h 文件。这个 config. h 的修改部分见程序清单 14.7。至于详细修改哪些部分,请看程序清单 14.7 中的注释。一部分代码仅与 ZLG/FFS 驱动相关,这里不详细说明。

程序清单 **14.7　ZLG/FFS** 驱动 **config. h** 的修改部分

```
#include "ZLG_FFS/k9fxx08/k9fxx08.h"
#include "ZLG_FFS/ZLG_FFS.h"

#define DEVICE_NAME                     "zlg/fs-ffs"
#define MAX_DISK_DRIVES                 1

#define DEVICE_BYTES_PER_SEC            512
#define DEVICE_SEC_PER_DISK             (1024 * 32)

#define COMMAND_FUNCTION                ZLG_FFS
#define DISK_INFO                       FFSDiskInfo

#define K9F28_CLE    ((volatile uint8 *)0x80D00000)
#define K9F28_ALE    ((volatile uint8 *)0x80C80000)
#define K9F28_DATA   ((volatile uint8 *)0x80C00000)
#define K9FXX_WAIT()

#ifdef IN_ZLG_FS_DRIVER
```

驱动程序需要新增的头文件

驱动程序名称

驱动程序接口函数名

驱动程序的需要保留参数 RsvdForLow

驱动程序需要的定义

```
static FFSDisk FFSDiskInfo\[MAX_DISK_DRIVES\];
static u8 FFSbuf\[512\];
#define K9F28_BASE_ADDR                 0x40003000
static K9fXX08Hardware ChipInfo\[MAX_DISK_DRIVES\];
static FlashDriver FlashDriverInfo\[MAX_DISK_DRIVES\];
```

驱动程序需要的定义

```
static void drive_init(void)
{
    void * *vp;

    vp = (void *)K9F28_BASE_ADDR;
    ChipInfo\[0\].CLE = K9F28_CLE;
    ChipInfo\[0\].ALE = K9F28_ALE;
    ChipInfo\[0\].Data = K9F28_DATA;
    ChipInfo\[0\].Buf = FFSbuf;
    ChipInfo\[0\].BytsPerSec = 512;
    ChipInfo\[0\].BytsPerSecC = 16;
    ChipInfo\[0\].SecPerBlock = 32;
    ChipInfo\[0\].BlockPerChip = 1024;
    ChipInfo\[0\].DataToK9fxx08 = vp\[0\];
    ChipInfo\[0\].DataToMCU = vp\[0\];

    FlashDriverInfo\[0\].BytsPerSec = 512;
    FlashDriverInfo\[0\].SecsPreBlock = 32;
    FlashDriverInfo\[0\].SecPreDisk = 1024 * 32;
    FlashDriverInfo\[0\].BlockPreDisk = 1024;
    FlashDriverInfo\[0\].BufA = FFSbuf;
    FlashDriverInfo\[0\].Index = (void *)ChipInfo;
    FlashDriverInfo\[0\].SectorRead = vp\[2\];
    FlashDriverInfo\[0\].ExSectorRead = vp\[3\];
    FlashDriverInfo\[0\].SectorWrite = vp\[4\];
    FlashDriverInfo\[0\].ExSectorWrite = vp\[5\];
    FlashDriverInfo\[0\].BlockErase = vp\[6\];
    FlashDriverInfo\[0\].BlockCheck = vp\[9\];
    FlashDriverInfo\[0\].SectorCheck = vp\[10\];
    FlashDriverInfo\[0\].ExSectorCheck = vp\[11\];

    FFSDiskInfo\[0\].Drive = &(FlashDriverInfo\[0\]);
```

初始化保留参数 RsvdForLow

更多高清PDF图书 www.gqpdf.com

```
}

#endif
```

⑤ 修改 Makefile 文件。修改后的 Makefile 请参考程序清单 14.6，修改的部分已标出。

程序清单 **14.8**　**ZLG/FFS** 驱动的 **Makefile** 文件

```
EXEC = zlg_fs_ffs
OBJS = zlg_fs_ffs.o ZLG_FFS/ZLG_FFS.o
SRC = zlg_fs_ffs.c ZLG_FFS/ZLG_FFS.c

INCLUDE = /work/uClinux-dist/linux-2.4.x/include

USEINC = .

CC = arm-elf-gcc
LD = arm-elf-ld

MODCFLAGS = -D__KERNEL__ -I $(INCLUDE) -I $(USEINC) -O2 -fno-strict-aliasing -fno-
common -pipe -fno-builtin -D__linux__ -g -DNO_MM -mapcs-32 -march = armv4 -mtune =
arm7tdmi -mshort-load-bytes -msoft-float -nostdinc -iwithprefix include

LDFLAGS = -m armelf -r

all: $(EXEC)

$(EXEC): $(OBJS)
    $(LD) $(LDFLAGS) -o $@  $(OBJS)

%.o:%.c
    $(CC) $(MODCFLAGS) -mapcs -c $< -o $@

clean:
    -rm -f $(EXEC) *.elf *.gdb *.o
    -rm -f ZLG_FFS/*.o
```

修改的部分

修改的部分

修改的部分

⑥ 不需要 ZLG_FFS 目录下的源代码。

⑦ 编译驱动程序，生成 zlg_fs_ffs 文件就是需要的目标文件。

14.5 模板源码分析

因为本章介绍的驱动程序也是内核模块，所以模板中也有初始化函数和清除函数。另外，根据 13.4 节可知，本节介绍的驱动程序的最重要的数据结构为结构体 fs_drivers（参考程序清单 13.8），其中它有几个方法 DiakCommand()、open()、release() 和 get_info()，而 DiakCommand() 由 ZLG/FS V1.10 驱动程序提供，详情请参考 13.2 节，这里不再介绍。下面分别介绍这几个函数。

14.5.1 驱动程序的初始化函数

驱动程序的初始化函数代码见程序清单 14.9。可以看出，本函数主要是调用用户特定的初始化函数 drive_init()（程序清单 14.9(2)）和初始化结构体 fs_drivers 变量数组 zlg_driver_driver\[\]（程序清单 14.9(4) ~ (12)）。关于函数 drive_init()，请参考程序清单 14.2；关于结构体 fs_drivers 的定义，请参考 13.4.1 小节。DISK_INFO（程序清单 14.9(4) ~ (6)）和 COMMAND_FUNCTION（程序清单 14.9(9)）请参考程序清单 14.2。zlg_fs_driver_open（程序清单 14.9(10)）是本驱动程序的 open() 方法。zlg_fs_driver_release（程序清单 14.9(11)）是本驱动程序的 release() 方法。zlg_fs_driver_get_info（程序清单 14.9(12)）是本驱动程序的 get_info() 方法。至于程序清单 14.9(13)，是调用第 13 章介绍的驱动程序提供给硬件驱动的函数，调用它也是第 13 章介绍的驱动程序的要求，具体参考 13.4.2 小节。其他部分就是标准内核模块，详情请参考 4.2.3 小节。

注意：程序清单 14.9(1) 和其他部分在源代码中并不在一起，为了表述方便而把它们放在一起。

程序清单 **14.9** 初始化函数

```
static fs_drivers zlg_driver_driver\[MAX_DISK_DRIVES\];                    (1)

int  zlg_fs_driver_init(void)
{
    int i;

    drive_init();                                                          (2)

    for (i=0; i < MAX_DISK_DRIVES; i++)                                    (3)
    {
#ifdef DISK_INFO                                                           (4)
```


更多高清PDF图书 www.gqpdf.com

```
        zlg_driver_driver\[i\].RsvdForLow = &(DISK_INFO\[i\]);                    (5)
#else                                                                             (6)
        zlg_driver_driver\[i\].RsvdForLow = NULL;                                 (7)
#endif                                                                            (8)
        zlg_driver_driver\[i\].DiakCommand = COMMAND_FUNCTION;                    (9)
        zlg_driver_driver\[i\].open = zlg_fs_driver_open;                        (10)
        zlg_driver_driver\[i\].release = zlg_fs_driver_release;                  (11)
        zlg_driver_driver\[i\].get_info = zlg_fs_driver_get_info;                (12)
        zlg_fs_add_driver(&(zlg_driver_driver\[i\]));                            (13)
    }

    printk(KERN_INFO DEVICE_NAME ": init OK\\n");                                (14)
    return 0;                                                                    (15)
}
```

14.5.2 驱动程序的清除函数

驱动程序的清除函数代码见程序清单 14.10。程序很简单，只是按照第 13 章的要求调用函数 zlg_fs_remove_driver()，具体请参考 13.4.3 小节。

程序清单 **14.10** 清除函数

```
void zlg_fs_driver_cleanup(void)
{
    int i;

    for (i = 0; i < MAX_DISK_DRIVES; i ++)                                       (1)
    {
        zlg_fs_remove_driver(&(zlg_driver_driver\[i\]));                         (2)
    }
}
```

14.5.3 open() 方法

驱动程序的 open() 方法代码见程序清单 14.11。代码仅仅调用 MOD_INC_USE_COUNT，没有其他操作。关于宏 MOD_INC_USE_COUNT 的说明请参考 5.2.4 小节。

程序清单 **14.11**　**open**()方法

```
static int  zlg_fs_driver_open(struct inode *inode, struct file *filp)
{
    MOD_INC_USE_COUNT;
    return 0;
}
```

14.5.4　release()方法

本驱动程序的 release()方法代码见程序清单 14.12。代码仅仅调用宏 MOD_DEC_USE_COUNT,没有其他操作。关于宏 MOD_DEC_USE_COUNT 的说明请参考 5.2.4 小节。

程序清单 **14.12**　**release**()方法代码

```
static int  zlg_fs_driver_release(struct inode *inode, struct file *filp)
{
    MOD_DEC_USE_COUNT;
    return 0;
}
```

14.5.5　get_info()方法

get_info()方法主要用于获取当时的块设备的一些参数,其实就是获得块设备的扇区大小(以字节为单位)和块设备的大小(以扇区为单位)。get_info()方法的参数为结构体 fs_drivers 指针(通过程序清单 13.8 可知,struct _zlg_driver_Info 就是 fs_drivers),它指向的结构体变量保存着块设备的接口函数及这两个参数(参考 13.4.1 小节)。在 get_info()方法被调用以前,这个结构体变量除 BytesPerSec 和 SecPerDisk 两个成员外其他成员均初始化正确。成员 BytesPerSec 就是以字节为单位的块设备的扇区大小,SecPerDisk 就是以扇区为单位的块设备的大小,get_info()方法的任务就是把正确的值写入这两个成员。get_info()方法返回 0 表示成功,返回其他值表示失败。

get_info()方法的代码见程序清单 14.13。该方法首先查看驱动程序是否支持 DISK_GET_BYTES_PER_SECTOR(程序清单 14.13(3)):如果支持此命令,则调用此命令获取以字节为单位的块设备的扇区大小(程序清单 14.13(4)),并填充到结构体变量中(程序清单 14.13(4));如果不支持此命令,则结构体变量的 BytesPerSec 填充默认值(程序清单 14.13(7))。关于 DISK_GET_BYTES_PER_SECTOR 命令的说明请参考 13.2.2 小节第 10 条。DEVICE_BYTES_PER_SEC 在 config.h 中定义(参见程序清单 14.2)。

更多高清PDF图书　www.gqpdf.com

然后，get_info()方法查看驱动程序是否支持 DISK_GET_SECTOR_NUMBER（程序清单 14.13(9)）：如果支持此命令，则调用此命令获取以扇区为单位的块设备的大小（程序清单 14.13(10)），并填充到结构体变量中（程序清单 14.13(11)）；如果不支持此命令，则结构体变量的 SecPerDisk 填充默认值（程序清单 14.13(13)）。关于 DISK_GET_SECTOR_NUMBER 命令的说明请参考 13.2.2 小节第 10 条。DEVICE_SEC_PER_DISK 在 config.h 中定义（参见程序清单 14.2）。

程序清单 **14.13　get_info()方法**

```
static int  zlg_fs_driver_get_info(struct _zlg_driver_Info *Disk)
{
    Disk_RW_Parameter dp;

    dp.RsvdForLow = Disk->RsvdForLow;                                    (1)
    dp.SectorIndex = DISK_GET_BYTES_PER_SECTOR;                          (2)
    if (Disk->DiakCommand(DISK_CHECK_CMD, &dp) == DISK_TRUE)             (3)
    {
        Disk->DiakCommand(DISK_GET_BYTES_PER_SECTOR, &dp);               (4)
        Disk->BytesPerSec = dp.SectorIndex;                              (5)
    }
    else                                                                 (6)
    {
        Disk->BytesPerSec = DEVICE_BYTES_PER_SEC;                        (7)
    }

    dp.SectorIndex = DISK_GET_SECTOR_NUMBER;                             (8)
    if (Disk->DiakCommand(DISK_CHECK_CMD, &dp) == DISK_TRUE)             (9)
    {
        Disk->DiakCommand(DISK_GET_SECTOR_NUMBER, &dp);                  (10)
        Disk->SecPerDisk = dp.SectorIndex;                               (11)
    }
    else                                                                 (12)
    {
        Disk->SecPerDisk = DEVICE_SEC_PER_DISK;                          (13)
    }
    return 0;                                                            (14)
}
```

高清PDF gqpdf.com

14.5.6 其他说明

本驱动程序没有涉及中断、动态内存分配和按需加载等相关问题。关于这些部分的进一步信息请参见参考文献[2]和[4]。

另外，关于修改版权及作者声明，请参考6.3.6小节。

本章小结

本章是前一章的延续，主要是介绍如何把基于ZLG/FS V1.10的驱动程序转换成μClinux的驱动程序。

相对于第11章，本章主要增加了使用其他内核模块提供的函数（参考14.5.1和14.5.2小节）。

高清PDF gqpdf.com

更多高清PDF图书 www.gqpdf.com

第15章　网络设备驱动程序初步

Linux（μClinux）系统的设备主要有字符设备、块设备和网络设备3种。字符设备和块设备比较相似，在前面的章节已经介绍。Linux（μClinux）是一个与网络紧密结合的操作系统，为了提高效率，它的网络设备与字符设备和块设备有很大的不同，进行了专门的处理。

Linux（μClinux）网络设备驱动程序是Linux（μClinux）操作系统网络应用中的一个重要组成部分。访问网络设备的驱动程序不需要使用设备节点。Linux（μClinux）的网络系统主要是基于BSD Unix的socket机制。在系统和驱动程序之间定义有专门的数据结构（sk_buff）进行数据的传递。系统内部支持对发送数据和接收数据的缓存，提供流量控制机制，提供对多协议的支持。因此，选择哪个驱动程序是基于内核内部的其他决定，而不是调用open()。对网络设备的使用通常由系统调用socket接口引入。

一般驱动程序中不对发送数据进行缓存，而是直接使用硬件的发送功能把数据发送出去。接收数据一般是通过硬件中断来通知的。在中断处理程序里，把硬件帧信息填入一个skbuff结构中，然后调用netif_rx()传递给上层处理。

15.1　最简单的网络设备驱动程序

15.1.1　驱动程序代码

该驱动程序的代码见程序清单15.1。程序清单15.1为一个简单的网络设备驱动程序的完整源码。理论上，这个代码还可以简化，但再简化就不容易使用了，所以这里仍然称为最简单的网络设备驱动程序（注意是特指网络设备驱动程序）。其实，这仅仅是一个可编译、加载和卸载的驱动程序，不能完成任何实际的工作。不过，可把它当作模板，网络设备驱动程序就从这里开始编写。

gqpdf.com

程序清单**15.1**　最简单的网络设备驱动程序

```
#ifndef __KERNEL__
    #define __KERNEL__                                              (1)
#endif
#ifndef MODULE
```

```
    #define MODULE                                                    (2)
#endif

#include <linux/module.h>                                             (3)
#include <linux/sched.h>                                              (4)
#include <linux/kernel.h>                                             (5)
#include <linux/init.h>                                               (6)

#include <linux/in.h>                                                 (7)
#include <linux/netdevice.h>                                          (8)
#include <linux/etherdevice.h>                                        (9)
#include <linux/ip.h>                                                (10)
#include <linux/tcp.h>                                               (11)

static int test_init(struct net_device *dev);                        (12)
static int test_open(struct net_device *dev);                        (13)
static int test_release(struct net_device *dev);                     (14)
static int test_config(struct net_device *dev, struct ifmap *map);
                                                                     (15)
static int test_tx(struct sk_buff *skb, struct net_device *dev);
                                                                     (16)
int  test_init_module(void);                                         (17)
void test_cleanup(void);                                             (18)

#define DEVICE_NAME "test"                                           (19)

module_init(test_init_module);                                       (20)
module_exit(test_cleanup);                                           (21)

static struct net_device net_test =                                  (22)
{
    init: test_init,                                                 (23)
};
static int test_init(struct net_device *dev)                         (24)
{
    ether_setup(dev);                                                (25)
```

内核模块名称

定义结构体变量并初始化必需的成员

初始化结构体变量与硬件无关的成员

更多高清PDF图书 www.gqpdf.com

```
    dev->name               ="eth0",                                    (26)
    dev->open               =test_open;                                 (27)
    dev->stop               =test_release;                              (28)
    dev->set_config         =test_config;                               (29)
    dev->hard_start_xmit    =test_tx;                                   (30)
    dev->flags              &=~(IFF_BROADCAST|IFF_LOOPBACK|IFF_MULTICAST);
                                                                        (31)
    SET_MODULE_OWNER(dev);                                              (32)

    return 0;                                                           (33)
}

static int test_config(struct net_device *dev, struct ifmap *map)       (34)
{
    return 0;                                                           (35)
}

static int test_tx(struct sk_buff *skb, struct net_device *dev)         (36)
{
    return 0;                                                           (37)
}

static int  test_open(struct net_device *dev)                           (38)
{
    MOD_INC_USE_COUNT;                                                  (39)

    netif_start_queue(dev);                                             (40)

    return 0;                                                           (41)
}

static int  test_release(struct net_device *dev)                        (42)
{
    netif_stop_queue(dev);                                              (43)

    MOD_DEC_USE_COUNT;                                                  (44)
```

初始化结构体变量与硬件相关的成员

第一次调用open( )方法必须调用

最后一次调用stop( )方法必须调用

配对使用

必需的代码，配对使用

高清藏 PDF gqpdf.com

```
    return 0;                                                          (45)
}

int  test_init_module(void)                                            (46)
{
    int  result;                                                       (47)

    result = register_netdev(&net_test);                               (48)
    if (result < 0)                                                    (49)
    {
        printk(KERN_ERR DEVICE_NAME ": error % i registering device \\"% s \\" \\n",
                                                                       (50)
               result, net_test.name);                                 (51)
        return result;                                                 (52)
    }

    printk(KERN_ERR DEVICE_NAME ": init OK \\n");                      (53)
    return 0;                                                          (54)
}

void test_cleanup(void)                                                (55)
{
    unregister_netdev(&net_test);                                      (56)
}
```

程序清单 15.1 看起来比较长，但实际上去掉宏定义、包含文件及函数声明外内容并不算多。事实上，核心部分仅有两个函数（初始化函数和清除函数）和一个数据结构变量（net_test）。

15.1.2 驱动程序的初始化函数

一切变化从初始化函数开始。

Linux（μClinux）在加载内核模块时会调用驱动程序的初始化函数。网络设备驱动程序初始化函数的示意代码见程序清单 15.2。驱动程序的初始化函数用来初始化驱动程序本身，其返回值为负数时表示出错；负数的绝对值为错误编码，错误编码在 < linux/errno. h > 中定义。程序清单 15.1（46）~（54）就是驱动程序初始化函数的例子。其实，这个函数最重要的是调用系统提供的函数 register_netdev（）（程序清单 15.1（48））来注册字符设备驱动程序，其他代码都是错误处理和显示处理。相对于程序清单 4.1，调用这个函数其实也是所有变化的根源。

更多高清PDF图书 www.gqpdf.com

函数 register_netdev () 的原型见程序清单 15.3，它只有 1 个参数，为结构体 net_device 指针，这个结构体保存驱动程序操作网络设备的方法。这个结构体后面还会介绍，这里不再说明。

程序清单 **15.2**　网络设备初始化函数的示意代码

```
int  init_module (void)
{
    初始化模块自身(包括请求一些资源);
    return  register_netdev(&net_test);
}
```

程序清单 **15.3**　函数 **register_netdev** () 的原型

```
int register_netdevice(struct net_device *dev);
```

15.1.3　驱动程序的清除函数

Linux (μClinux) 在卸载内核模块时会调用驱动程序的清除函数。驱动程序清除函数的示意代码见程序清单 15.4。驱动程序的清除函数主要用来释放自身获得的资源。清除函数是一个没有参数和返回值的函数。程序清单 15.1 (55)、(56) 就是清除函数的例子，它仅调用系统提供的函数 unregister_netdev () 通知内核对应的网络设备已被卸载 (程序清单 15.1 (56))。函数 unregister_netdev () 是与函数 register_netdev () 配对的函数。如果驱动程序在初始化或工作时向内核申请了其他资源 (如内存)，在卸载时如果还没有被释放，也需要在这个函数释放。**特别注意**：释放的顺序应按申请的相反顺序执行。函数 unregister_netdev () 的原型见程序清单 15.5，它仅有一个参数，为一个结构体变量指针，指向保存设备信息的结构体变量。

程序清单 **15.4**　网络设备清除函数的示意代码

```
void  cleanup_module (void)
{
    关闭模块自身(包括释放资源);
    unregister_netdev(&net_test);
}
```

程序清单 **15.5**　函数 **unregister_netdev** () 的原型

```
int unregister_netdevice(struct net_device *dev);
```

15.1.4　保存设备信息的结构体

在块设备驱动程序初始化时需要调用函数 register_netdev ()，其惟一的参数为指向结构体

net_device 的指针，它存储了网络设备的操作方法和其他信息。结构体 net_device 的定义见程序清单 15.6。程序清单 15.6 仅列出部分常用的成员，对于普通的网络设备驱动程序，这些成员已足够，其他成员仅在特殊情况下使用，感兴趣的读者可自行阅读源码或参见参考文献[2]。

程序清单 **15.6**　保存网络设备驱动程序信息的结构体

```
struct net_device
{
    char                name\[IFNAMSIZ\];
    unsigned short      flags;
    unsigned long       base_addr;
    unsigned int        irq;
    unsigned char       dev_addr\[MAX_ADDR_LEN\];
    unsigned char       addr_len;
    unsigned long       trans_start;
    unsigned long       last_rx;
    int                 watchdog_timeo;
    void                *priv;
    struct module       *owner;

    int                 (*init)(struct net_device *dev);
    int                 (*open)(struct net_device *dev);
    int                 (*stop)(struct net_device *dev);
    int                 (*set_config)(struct net_device *dev, struct ifmap *
map);
    int                 (*hard_start_xmit) (struct sk_buff *skb, struct net_de-
vice *dev);
    int                 (*set_mac_address)(struct net_device *dev, void *ad-
dr);
    ......
};
```

程序清单 15.1(22)、(23)就是定义了这个结构体变量并初始化其中最重要的部分，程序清单 15.1(25)～(33)对这个结构体的其他成员进行初始化。程序清单 15.1(22)、(23)较易看懂：仅对结构体指定的成员初始化，未指定的成员用默认值初始化。这是 GCC 对标准 C 的扩展，好处是结构体成员增加时不需要更改初始化代码。

下面分别介绍每个成员的意义。

更多高清PDF图书　www.gqpdf.com

1. name

指向保存网络接口名称的字符串，应在 init() 方法中设置它。在一个系统中，网络接口名称是惟一的。内核提供某种机制用于生成惟一的名称，但本书不讨论它。感兴趣的读者可参见参考文献[2]。

2. flags

接口标志。这个域含有下列位值。前缀 IFF 为“接口标志”(InterFace Flags)之意。有些标志由核心管理，有些则是在初始化时由接口设置，以确认接口的能力。这些标志在 <linux/if.h> 中定义，有效的标志解释如下：

IFF_UP：当网络端口是活跃时，核心置上该标志。这个标志对驱动程序是只读的。

IFF_BROADCAST：该标志表明网络端口的广播地址是有效的。以太网卡支持广播。

IFF_DEBUG：调试模式。该标志控制 printk() 调用的详细程度，还用于其他一些查错目的。尽管目前没有正式驱动程序使用它，但用户程序可通过 ioctl() 来对其置位或清除，驱动程序可使用它。misc-progs/netifdebug 程序可用来将这个标志打开或关闭。

IFF_LOOPBACK：该标志在环回网络端口中被置位。核心检测该标志，而不是将名字 lo 作为特殊接口硬写入程序。

IFF_POINTOPOINT：该标志表明网络端口连接到点对点链路。该标志由 ifconfig 设置。

IFF_NOARP：该标志表明网络端口不能执行 ARP。

IFF_PROMISC：设置该标志将激活混杂模式。在默认情况下，以太网接口使用硬件过滤器来保证它只接收广播数据包，以及直接发送到网络端口硬件地址的数据包。而像 tcpdump 一类包监视器，则在网络端口上设置混杂模式，以获取传输介质上的所有数据包。

IFF_MULTICAST：能进行组播传送的网络端口要置该标志。ether_setup 在缺省情况下对其置位。所以如果驱动程序不支持组播，则它必须在初始化时清除该标志。

IFF_ALLMULTI：该标志告诉网络端口接收所有组播数据包。只有当 IFF_MULTICAST 被置位，而主机又进行主播路由时，核心对其置位。它对驱动程序是只读的。

IFF_MASTER、IFF_SLAVE：这些标志被加载均衡代码使用。驱动程序无须了解这个标志。

IFF_PORTSEL、IFF_AUTOMEDIA：这些标志表明设备可在多种介质类型(如双绞线、同轴电缆)之间切换。如果 IFF_AUTOMEDIA 被设置，则设备会自动选择正确的介质类型。

IFF_DYNAMIC：该标志表示网络端口地址可改变。

IFF_RUNNING：该标志表示网络端口正在运行。Linux(μClinux)中不使用这个标志，它只是为了与 BSD 兼容而存在。

IFF_NOTRAILERS：Linux(μClinux)中不使用该标志，它只是为了与 BSD 兼容而存在。

3. base_addr

此成员用于保存网络接口的 I/O 基地址，应在 init () 方法中设置它。ifconfig 命令可显示和修改它。内核不会使用该字段。

4. irq

此成员用于保存网络接口的中断号，应在 init () 方法中设置它。ifconfig 命令可显示和修改它。内核不会使用该字段。

5. dev_addr\[\]和 addr_len

网络接口设备的物理地址（即 MAC 地址）和地址长度，应在 init () 方法中设置它。ifconfig 命令可显示和修改它。

6. trans_start

hard_start_xmit () 方法必须把这个成员改变为全局变量 jiffies 的值。全局变量 jiffies 实际上是记录系统运行的时间（以时钟节拍为单位）。Linux（μClinux）内核使用这个成员处理与发送时间相关的一些任务（如超时检测等）。

7. last_rx

当网络接口接收到数据包时，驱动程序应把这个成员改变为全局变量 jiffies 的值。全局变量 jiffies 实际上是记录系统运行的时间（以时钟节拍为单位）。last_rx 成员在 2.4.x 版本的 Linux（μClinux）中没有使用，但以后可能会使用。

8. priv

一个通用指针，内核不会使用它。关于 priv 的处理完全由驱动程序控制。一般来说，驱动程序会使用它来区分自己支持的多个物理设备。

9. owner

用来表明驱动程序的所有者。

10. init ()

对于内核来说，在使用网络设备驱动程序之前会调用它的 init () 方法，使用的语句是“dev->init (dev);”或等价的语句。

init () 方法惟一的参数（见程序清单 15.7）为指向保存网络设备信息的结构体变量的指针，这个变量已保存有这个 init () 方法本身。init () 方法返回 0 表示成功，返回负值表示出错；这个值就是错误编码，错误编码的绝对值在 <linux/errno.h> 中定义。init () 方法主要用于修改保存设备信息的结构体变量的各个成员。这些成员中有一个比较特殊，它就是 name 成员。同一个系统中每个网络接口必须有一个惟一的名字，它保存在 name 成员中。同时，内核提供一种机制自动生成惟一的字符串。本章不讨论这个问题，感兴趣的读者可参见参考文献[2]。

程序清单 **15.7** **init** () 方法示意代码

更多高清PDF图书 www.gqpdf.com

```
static int test_init(struct net_device *dev)
{
    ether_setup(dev);                                    (1)
    设置 dev 中必需的成员;                                (2)
    设置 dev 中驱动程序感兴趣的成员;                      (3)
    SET_MODULE_OWNER(dev);                               (4)
    return 0;                                            (5)
}
```

程序清单 15.7(1)为调用网络设备的通用初始化函数 ether_setup(),其原型见程序清单 15.8,它会初始化与硬件驱动程序无关的成员。函数 ether_setup()使用很简单,按照程序清单 15.7(1)的方式使用即可。程序清单 15.7(2)主要包括设置结构体变量的 name 成员、open()成员、stop()成员、set_config()成员、hard_start_xmit()成员、set_mac_address()成员(是可选的,但一般需要)和 flags 成员。至于其他成员,在程序清单 15.7(3)中初始化,一般包括 base_addr 成员、irq 成员、watchdog_timeo 成员、priv 成员和其他感兴趣的成员。至于程序清单 15.7(5),SET_MODULE_OWNER(dev)这个宏是设置 dev 结构中 owner 成员,使得它指向本模块本身。

程序清单 **15.8**　函数 **ether_setup**()的原型

```
void    ether_setup(struct net_device *dev)
```

11. open()

当 Linux(μClinux)内核和/或应用程序使用设备传输数据之前,会调用驱动程序的这个方法。open()方法最重要的是调用宏 MOD_INC_USE_COUNT(程序清单 15.1(39))和第一次使用设备时调用函数 netif_start_queue()(程序清单 15.1(40))。宏 MOD_INC_USE_COUN 主要用来增加驱动程序使用计数器,以避免不正确的卸载驱动程序。open()方法一般还用于(在第一次使用驱动程序支持的设备时)对驱动程序支持的设备进行初始化。一般情况下,open()方法的示意代码见程序清单 15.9。这个代码针对一个驱动管理多个设备的情况(常见);如果一个驱动仅管理一个设备,就不需要这么复杂了,仅调用宏 MOD_INC_USE_COUNT 即可,而设备初始化(包括调用函数 netif_start_queue())可在驱动程序初始化时进行。

程序清单 **15.9**　**open**()方法示意代码

```
static int  open(struct net_device *dev)
{
    if (需要操作的设备的使用计数器 == 0)                                    (1)
    {
        初始化设备(包括申请资源);                                           (2)
        netif_start_queue(dev);                                             (3)
    }
    需要操作的设备的使用计数器 ++;                                           (4)

    MOD_INC_USE_COUNT;                                                      (5)
    return 0;                                                               (6)
}
```

必需的语句

必需的语句

open()方法的惟一参数为指向保存设备信息的结构体变量的指针。open()方法返回0表示成功;返回负值表示出错,这个值就是错误编码,错误编码的绝对值在<linux/errno.h>中定义。

函数 netif_start_queue()是告诉内核可使用发送数据队列了,也就是可发送数据了。函数 netif_start_queue()使用简单,按照程序清单15.9(3)方式调用即可。

12. stop()

当Linux(μClinux)内核和/或应用程序停止发送数据时,会调用驱动程序的这个方法。release()方法最重要的是调用宏 MOD_DEC_USE_COUNT(程序清单15.1(44))和最后一次使用设备时调用函数 netif_stop_queue()。宏 MOD_DEC_USE_COUNT 与宏 MOD_INC_USE_COUNT 对应,主要用来减少驱动程序使用计数器。stop()方法一般还用于(在驱动程序支持的设备使用完毕时)对驱动程序支持的设备进行关闭操作。一般情况下 stop()方法的示意代码见程序清单15.10。这个代码针对一个驱动管理多个设备的情况(常见);如果一个驱动仅管理一个设备,就不需要这么复杂了,仅调用宏 MOD_DEC_USE_COUNT 即可,而关闭设备(包括调用函数 netif_stop_queue())可在卸载驱动程序时进行。

程序清单 **15.10**　**stop()**方法示意代码

更多高清PDF图书　www.gqpdf.com

```
static int  test_release(struct net_device *dev)
{
    MOD_DEC_USE_COUNT;                                          (1)
                                    必需的语句

    需要操作的设备的使用计数器--;                                  (2)
    if (需要操作的设备的使用计数器 == 0)                           (3)
    {
                                    必需的语句
        netif_stop_queue(dev);                                  (4)
        关闭设备处理(包括释放 revalidate()方法获取的资源);           (5)
    }

    return(0);                                                  (6)
}
```

stop()方法的惟一参数为指向保存设备信息的结构体变量的指针。stop()方法返回0表示成功;返回负值表示出错,这个值就是错误编码,错误编码的绝对值在<linux/errno.h>中定义。

函数netif_stop_queue()是让内核暂停向网络端口请求发送数据。此时,Linux(μClinux)内核不会调用驱动程序的hard_start_xmit()方法。函数netif_stop_queue()使用简单,按照程序清单15.9(3)方式调用即可。

13. set_config()

当网络设备驱动程序不能自动识别硬件配置时,系统管理员会使用相关的配置工具将正确的硬件配置传递给驱动程序。事实上,这些配置工具会调用驱动程序的set_config()方法把硬件配置信息传递给驱动程序。因为在嵌入式系统中,硬件配置信息是十分固定的,一般不会有系统管理员来参与驱动程序的安装,所以一般情况下按照程序清单15.11来编写这个方法即可。

set_config()方法的有两个参数:第一个参数为指向保存网络设备信息的结构体变量的指针;第二个参数为保存硬件信息的结构体变量指针。set_config()方法返回0表示成功;返回负值表示出错,这个值就是错误编码,错误编码的绝对值在<linux/errno.h>中定义。

关于set_config()方法的详细介绍可参见参考文献[2]。

程序清单**15.11　set_config()**方法示意代码

```
static int test_config (struct net_device *dev, struct ifmap *map)
{
    return -EBUSY;
}
```

14. hard_start_xmit ()

当 Linux(μClinux)内核需要发送一个数据包时,它调用 hard_start_xmit () 方法将数据放到一个输出队列。核心处理的每个包包含在一个套接字缓冲区结构体(struct sk_buff)变量中,其定义见 < linux/skbuff. h > 。套接字缓冲区是一个复杂的结构,但对于驱动程序来说,套接字缓冲区仅仅是一个数据包,驱动程序无须改变其中的数据。驱动程序仅需要知道 sk_buff 的一些基本成员即可正常工作: data 成员指向被发送的包; len 成员为以字节为单位的数据表长度。这个结构体变量由内核负责动态分配,由网络驱动程序负责释放。

一般来说,hard_start_xmit () 方法会直接把数据写到网络接口设备的硬件缓冲区中,然后启动发送。用这样的方式的示意代码见程序清单 15.12。

hard_start_xmit () 方法的具有两个参数:第一个参数就是指向上面提到的套接字缓冲区结构体变量的指针,这个变量中包含要发送的数据包;第二个参数是指向保存设备信息的结构体变量的指针。stop () 方法返回 0 表示成功;返回负值表示出错,这个值就是错误编码,错误编码的绝对值在 < linux/errno. h > 中定义。

程序清单 **15.12　hard_start_xmit** () 方法示意代码

```
static int test_tx(struct sk_buff *skb, struct net_device *dev)
{
    int len;

    netif_stop_queue(dev);                                          (1)
    len = skb->len < ETH_ZLEN ? ETH_ZLEN : skb->len;                (2)
    把 len 个数据写入网络设备硬件缓冲区并启动发送;                  (3)
    dev->trans_start = jiffies;                                     (4)
    dev_kfree_skb(skb);                                             (5)
    等待数据发送完毕;                                               (6)
    netif_wake_queue(dev)                                           (7)
    return 0;                                                       (8)
}
```

所有代码都为必需的代码

一般在中断服务程序中完成

函数 netif_stop_queue () (程序清单 15.12 (1)) 是让内核暂停向网络端口请求发送数据。此时,Linux(μClinux)内核不会调用驱动程序的 hard_start_xmit () 方法。而 netif_wake_queue () 函数(程序清单 15.11 (7)) 则告诉内核暂停已结束,可向网络端口请求发送数据。这两个函数使用很简单,按照程序清单 15.12 所示方式调用即可。

因为以太网不能发送太小的数据包,所以当数据包太小时需要调整数据包的大小(程序清单 15.12 (2))。宏 ETH_ZLEN 就是内核定义以太网数据包的最小值。如果编写的网络驱

更多高清PDF图书　www.gqpdf.com

动程序不是针对以太网的，则可根据实际情况改写这一句。

函数 dev_kfree_skb()（程序清单 15.12(5)）是用于释放由内核分配的套接字缓冲区结构体变量，这个变量保存着将要发送的数据。

至于程序清单 15.12(4)，前面介绍 trans_start 成员时已说明。

15. set_mac_address()

如果网络端口允许改变 mac 地址，则在重新配置网络端口的 mac 地址后，Linux（μClinux）内核会调用驱动程序的 set_mac_address() 方法。该方法的示意代码见程序清单 15.13。

set_mac_address() 方法有两个参数：第一个参数为指向保存网络设备信息的结构体变量的指针；第二个参数虽然被定义为一个无符号指针，但实际是指向结构体 sockaddr 的一个结构体变量的指针。对于网络驱动程序来说，结构体 sockaddr 仅一个成员有用，它就是字符数组 sa_data，它保存有实际的 MAC 地址。

set_mac_address() 方法返回 0 表示成功；返回负值表示出错，这个值就是错误编码，错误编码的绝对值在 <linux/errno.h> 中定义。

程序清单 **15.13　set_mac_address()** 方法示意代码

```
static int test_set_mac_address(struct net_device *dev, void *addr)
{
    struct sockaddr *mac_addr;

    mac_addr = addr;                                                    (1)
    if (netif_running(dev))                                             (2)
    {
        return -EBUSY;                                                  (3)
    }
    memcpy(dev->dev_addr, mac_addr->sa_data, dev->addr_len);            (4)
    改变网络接口的 mac 地址为 dev->dev_addr;                              (5)
    return 0;                                                           (6)
}
```

netif_running() 函数是一个 inline 函数，用于检测网络端口是否处于运行状态。如果 netif_running() 函数返回 0，表示网络端口不处于运行状态；否则处于运行状态。而网络端口处于运行状态是不能改变 MAC 地址的，需要返回 - EBUSY（程序清单 15.13(2)、(3)）。程序清单 15.13(4) 是把 MAC 地址保存到保存设备信息的结构体变量中。程序清单 15.13(5) 是实际改变物理 MAC 地址，它与具体的设备有关。

15.1.5 数据接收与中断服务程序

一般驱动程序不对发送数据进行缓存，而是直接使用硬件的发送功能把数据发送出去。接收数据一般是通过硬件中断来通知的。在中断处理程序里，把硬件帧信息填入一个 skbuff 结构中，然后调用 netif_rx() 传递给上层处理。根据一般的情况，中断服务程序的示意代码见程序清单 15.14。当然，在使用中断程序之前，需要向系统注册中断服务程序。关于这方面请参考 7.3.1 小节。中断服务程序的 dev_id 也要配置为指向保存硬件信息的结构体变量的指针，配置的方法参考 7.3.1 小节。

程序清单 **15.14** 中断服务程序示意代码

```
static void net_irq_handle(int irq, void *dev_id, struct pt_regs *regs)
{
    struct net_device *dev;
    struct sk_buff *skb;
```

高清PDF
gqpdf.com

更多高清PDF图书 www.gqpdf.com

```
unsigned int length
u8 *dec;

dev = (struct net_device *)dev_id;                                    (1)
if (发送成功)                                                          (2)
{
    清除发送中断标志;                                                  (3)
    netif_wake_queue(dev);                                            (4)
}
if (接收成功)                                                          (5)
{
    清除接收中断标志;                                                  (6)
    length = 数据包长度;                                               (7)
    skb = dev_alloc_skb(length + 2);                                  (8)
    if (! skb)                                                        (9)
    {
        return;                                                       (10)
    }
    skb_reserve(skb, 2);                                              (11)
    dec = skb->data;                                                  (12)
    把数据包(length 个数据)读到 dec 指向的内存中;                      (13)
    skb->dev = dev;                                                   (14)
    skb->protocol = eth_type_trans(skb, dev);                         (15)
    skb->ip_summed = CHECKSUM_UNNECESSARY;                            (16)
    netif_rx(skb);                                                    (17)
    dev->last_rx = jiffies;                                           (18)
}
if (有错误出错)                                                        (19)
{
    错误处理程序;                                                     (20)
}
清除中断源;                                                           (21)
}
```

主要的代码

关键代码

当中断为通知发生错误时(程序清单 15.14(16)),进行错误处理(程序清单 15.14(17)),这部分与硬件相关,不同的驱动程序这部分差别很大。当中断为通知发送成功(程序清单15.14(2)),程序很简单,清除中断源(程序清单 15.14(3))后,调用函数 netif_wake_

queue ()（程序清单 15.14 (4)）即可。关于函数 netif_wake_queue () 的说明请参考 15.1.4 小节中关于 hard_start_xmit () 方法的说明。

中断服务程序比较复杂的部分为接收数据包的处理（程序清单 15.14 (5) ~ (18)）。其实，复杂的部分主要是分配套接字缓冲区结构体变量（参考 15.1.4 小节中关于 hard_start_xmit () 方法的说明）。函数 dev_alloc_skb ()（程序清单 15.14 (8)）就是动态分配一个套接字缓冲区结构体变量。函数 dev_alloc_skb () 是一个 inline 函数，其惟一的参数为指定以字节为单位的缓冲区的长度。当分配成功时，函数 dev_alloc_skb () 返回一个有效的指针。至于程序清单 15.14 (8) 的 (length +2) 个字节，是因为内核需要两个字节保存自己的信息。函数 skb_reserve ()（程序清单 15.14 (11)）是用于保存数据前移动头指针，以保留若干字节空间。函数 skb_reserve () 的第一个参数为函数 dev_alloc_skb () 的返回值，第二个参数为需要保留的字节数。

至于程序清单 15.14 (13)，完全与硬件相关了。程序清单 15.14 (14) ~ (17) 是处理接收到的数据包。程序清单 15.14 (14) 为设置套接字缓冲区结构体变量的所有者。程序清单 15.14 (15) 为设置数据包使用的协议，用户可根据实际情况调用相关的宏，程序清单 15.14 说明协议为以太网协议。程序清单 15.14 (16) 说明数据包不需要校验和。程序清单 15.14 (14) ~ (16) 均为调用函数 netif_rx ()（程序清单 15.14 (17)）作准备的。函数 netif_rx () 是内核的数据包处理函数，按照程序清单 15.14 (17) 调用即可。关于程序清单 15.14 (18) 的说明，请参考 15.1.4 小节中关于 last_rx 成员的说明。

15.1.6 关于头文件

本节介绍的驱动程序使用了最少的系统头文件，而真正的驱动程序一般需要包含更多的系统头文件。当不知道一个宏或函数在哪个系统头文件中声明时，一般是直接参考源代码（可利用搜索工具来查找）。

15.1.7 其他说明

本章没有涉及超时处理和多个数据包同时发送的知识，感兴趣的读者可参见参考文献 [2] 和阅读源码。

编译此驱动程序主要编写 Makefile 文件，具体请参考 4.3.1 小节。

15.2 使用网络设备驱动程序

用户一般会通过应用程序（如网页浏览器）来使用网络驱动程序，不会直接接触到网络驱动程序。对于程序员，他们通过 socket 编程接口使用网络驱动程序，也不会直接接触到驱动程序，所以，只有系统管理员会直接接触到网络驱动程序，他们要加载和卸载驱动程序并配置网络。

更多高清PDF图书 www.gqpdf.com

1. 加载驱动程序

本书介绍的驱动程序都编译成内核模块，这就必须把驱动程序加载到内核中才能被使用。加载内核模块的命令为 insmod，一般使用方法如下：

```
insmod   内核模块文件名(可包含路径)
```

输入“insmod --help”可得到 mknod 的详细帮助。

2. 卸载驱动程序

不再使用驱动程序时可以卸载它，这通过 rmmod 命令实现。rmmod 命令的一般使用方法如下：

```
rmmod   模块名
```

输入“rmmod --help”可得到 rmmod 的详细帮助。

3. 配置网络

在使用网络之前必须配置网络端口，这一般通过 ifconfig、route 等实用程序完成。这些实用程序使用比较复杂，同时介绍的资料很多，这里就不详细介绍了。

本章小结

本章主要介绍了 Linux(μClinux)网络驱动程序的一般编写方法和相关知识，包括源码分析和编译使用。这些是 Linux(μClinux)网络驱动程序最基础的公共知识，也是编写网络驱动程序必须掌握的知识。

本章介绍的知识是第 16 和 17 章的基础。

高清PDF藏 gqpdf.com

第16章 RTL8019驱动程序编写范例

16.1 硬件原理

RTL8019AS是一种高度集成的以太网芯片，能简单地实现Plug and Play功能并兼容NE2000、掉电等特性。由于它拥有3种等级的掉电模式，所以是作为绿色电脑的网络设备的理想选择。在全双工模式下，如果连接到一个同样是全双工的交换机或集线器，则可实现同时接收和发送。这个特性虽然不能把传输速率从10 Mbps提高到20 Mbps，但是在执行以太网CSMA/CD协议时，可以避免更多的冲突发生。而微软的即插即用功能就可为用户减轻对资源配置的烦恼（如IRQ、I/O Memory address等）。在一些特殊的场合，为了对一些不支持微软的即插即用的器件的兼容，RTL8091AS还可选择跳线模式或非跳线模式。

RTL8019AS与LPC2200系列ARM芯片的接口电路图参考图16.1，它是从SmartARM2200开发板（广州致远电子有限公司设计、生产）的原理图中截取出来的。由于RTL8019AS工作电源是5 V，而LPC2210的I/O电压为3.3 V，所以在总线上串接了470 Ω保护电阻。RTL8019AS芯片的输入高电平的最小值为2.0 V，所以与LPC2210连接不需要加电平转换芯片。RTL8019AS芯片工作在跳线模式。SA5与片选信号NET_nCS相连，该片选信号由LPC2200的CS3、A23、A22和A21进行地址译码后，得到最终RTL8019AS芯片的操作地址为0x83400000～0x8340001F。另外，使用了P0.8控制RTL8019AS芯片复位，RTL8019AS芯片的中断输出与P0.9（EINT3）相连。

RTL8019AS的应用及连接方法的详细说明请参考RTL8019AS芯片的数据手册。

更多高清PDF图书 www.gqpdf.com

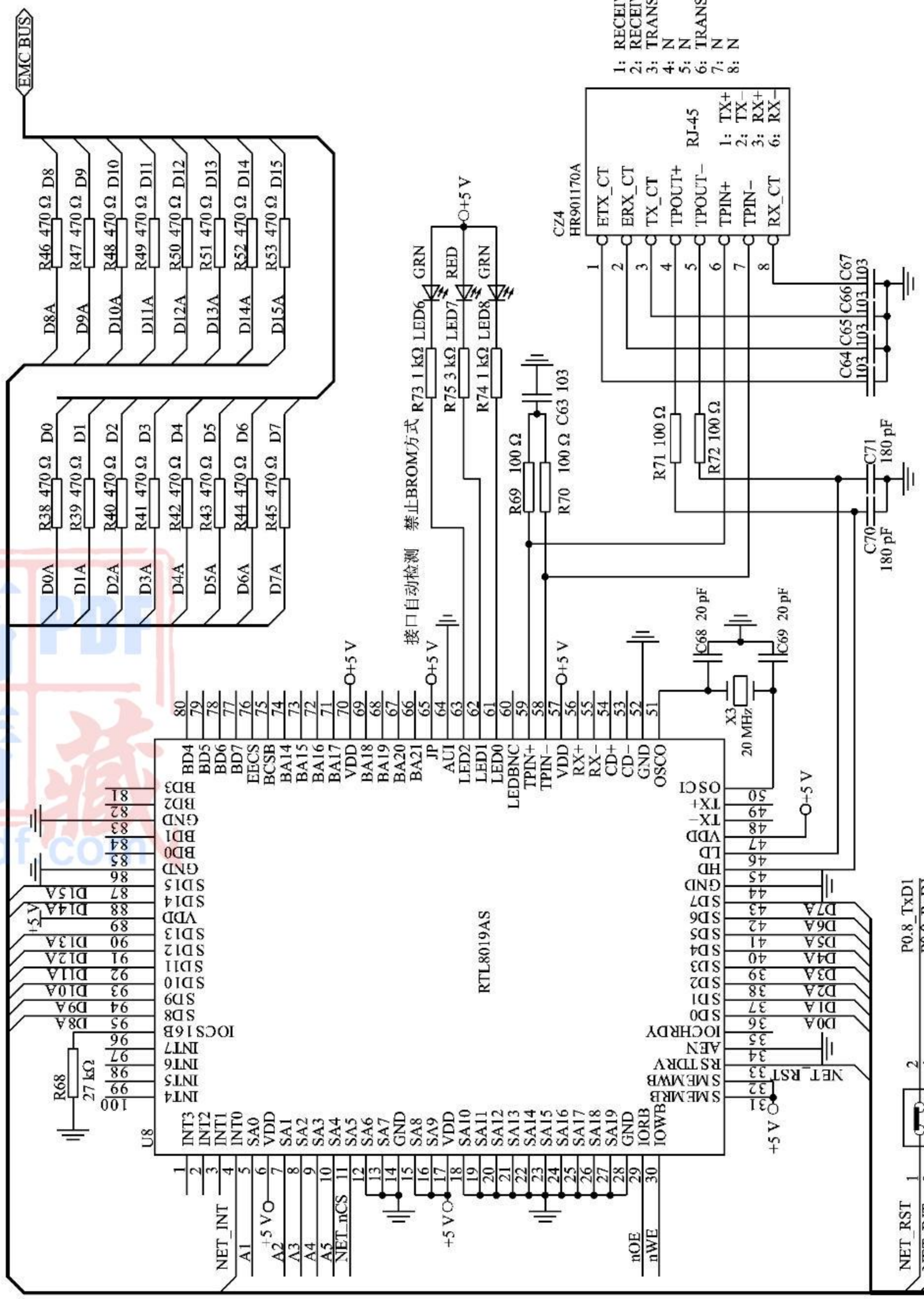

图 16.1　以太网接口电路

16.2 使用源代码

RTL8019 驱动程序源代码由 4 个文件组成，它们是 config. h、rtl8019. h、rtl8019. c 和 Makefile。其中 config. h 中配置了一些硬件信息，而 Makefile 也保存了一些编译器相关的信息。因此，对于使用者来说，这两个文件可能需要改动。

1. 修改 config. h 文件适合用户硬件

config. h 的代码请参考程序清单 16. 1。config. h 文件比较长，程序清单 16. 1 仅列出其与硬件相关的部分。程序清单 16. 1 已列出各部分的含义，用户按照实际的硬件修改即可。惟一要注意的是程序清单 16. 1(3)，宏 NET_TIMEOUT 暂时没有使用，但以后的升级版本可能会用到。

程序清单 **16. 1** **config. h** 文件内容

```
......
#define IOaddress                    0x83400000          (1)
                                                         硬件基地址

#define NET_IRQ                      IRQ_EXT3            (2)
                                                         中断号

#define NET_TIMEOUT                  10000               (3)
                                                         超时设置，没有使用

#define NET_HARD_REG()  outl(1 < < 8, IOSET), mdelay(7), outl(1 < < 8, IOCLR)
                                                         (4)

                                                         硬件复位RTL8019
#ifdef IN_RTL8019                                        (5)
                                                         设置默认MAC地址
static const char MyMacID\[\] = {0x52, 0x54, 0x4c, 0x33, 0xf7, 0x42, 0x00};
                                                         (6)

static u32 PinSel0Save;                                  (7)

inline void mac_hard_open(void)                          (8)
{                                                        驱动程序加载后被调用，用于初始化与硬件相关的部分
    u32 temp;

temp = inl(PINSEL0);                                     (9)
PinSel0Save = temp & (0x0f < < (8 * 2));                 (10)
                                                         保存引脚连接配置
```


更多高清PDF图书 www.gqpdf.com

```
    temp |= (3 << (9 * 2));                                          (11)
    temp &= ~(3 << (8 * 2));                                         (12)
    outl(temp, PINSEL0);                                             (13)

    temp = inl(IO0DIR);                                              (14)
    temp |= 1 << 8;                                                  (15)
    outl(temp, IO0DIR);                                              (16)
    outl(1 << 8, IO0CLR);                                            (17)

    temp = inl(VPBDIV);                                              (18)
    temp = inl(VPBDIV);                                              (19)
    outl(0, VPBDIV);                                                 (20)
    outl(inl(EXTMODE) & (~(1 << 3)),                                 (21)
    EXTMODE);                                                        (22)
    outl(inl(EXTPOLAR) | (1 << 3), EXTPOLAR);                        (23)
    outl(temp, VPBDIV);
}

inline void mac_hard_close(void)                                     (24)
{
    u32 temp;

    temp = inl(PINSEL0);                                             (25)
    temp &= ~(3 << (9 * 2));                                         (26)
    temp |= PinSel0Save;                                             (27)
    outl(temp, PINSEL0);                                             (28)
}

#endif
......
```

P0.8 设置为 GPIO，P0.9 设置为EINT3

P0.8 设置为输出，并输出低电平

设置外部中断为高电平触发

驱动程序卸载前被调用，用于与硬件相关退出处理

恢复硬件连接配置

2. 修改 **Makefile** 文件及编译

编译 RTL8019 驱动程序主要是编写 Makefile 文件，具体内容请参考 4.3.1 小节。

16.3 源码分析

16.3.1 初始化函数

本驱动程序的初始化函数代码见程序清单 16.2。可以看出，程序清单 16.2 是按照程序

清单 15.2 编写的，仅仅是在程序清单 15.2 上增加了信息显示而已（程序清单 16.2（6）、（8））。

注意：在源代码文件中，程序清单 16.2（1）、（2）与其他部分并不在一起，这里为了表述方便而放在一起。

程序清单 **16.2**　初始化函数代码

```
static struct net_device net_net =                                    (1)
{
    init: net_init,                                                   (2)
};

int net_init_module(void)                                             (3)
{
    int result;

    result = register_netdev(&net_net);                               (4)
    if (result < 0)                                                   (5)
    {
        printk(KERN_ERR "eth0: error % i registering device \\"% s\\" \\n",
              result, net_net.name);                                  (6)
        return result;                                                (7)
    }

    printk(KERN_ERR "eth0: init OK\\n");                              (8)
    return 0;                                                         (9)
}
```

16.3.2　清除函数

本驱动程序的清除函数代码见程序清单 16.3。可以看出，程序清单 16.3 完全是按照程序清单 15.4 编写的。

程序清单 **16.3**　清除函数代码

```
void net_cleanup(void)
{
    unregister_netdev(&net_net);
}
```


更多高清PDF图书　www.gqpdf.com

16.3.3 init()方法

本驱动程序的 init()方法代码见程序清单 16.4。可以看出，程序清单 16.4 完全是按照程序清单 15.7 编写的，两者的对照见表 16.1。

程序清单 **16.4** **init()**方法代码

```
static int net_init(struct net_device *dev)
{
    ether_setup(dev);                                                    (1)

    strcpy(dev->name, "eth0");                                           (2)
    dev->open               =net_open;                                   (3)
    dev->stop               =net_release;
                                                                         (4)
    dev->set_config         =net_config;                                 (5)
    dev->hard_start_xmit    =net_tx;                                     (6)
    dev->set_mac_address    =net_set_mac_address;                        (7)
    dev->flags              &=~(IFF_LOOPBACK|IFF_MULTICAST);             (8)
    dev->priv               =NULL;                                       (9)
    dev->base_addr          =IOaddress;                                  (10)
    dev->irq                =NET_IRQ;                                    (11)

    memcpy(dev->dev_addr, MyMacID, dev->addr_len);                       (12)
    SET_MODULE_OWNER(dev);                                               (13)

    return 0;                                                            (14)
}
```

参考 15.1.4 小节、程序清单 15.7 和程序清单 15.6，程序清单 16.4 的代码很容易读懂，这里不再说明。

表 16.1 **init()**方法的示意代码与驱动代码对照说明

示意代码（程序清单 15.7）	实际代码
(1) ether_setup(dev)；	程序清单 16.4 (1)
(2) 设置 dev 中必需的成员；	程序清单 16.4 (3) ~ (8)
(3) 设置 dev 中驱动程序感兴趣的成员；	程序清单 16.4 (9) ~ (12)

(4) SET_MODULE_OWNER (dev) ;	程序清单 16.4 (13)
(5) return 0;	程序清单 16.4 (14)

16.3.4 open () 方法

本驱动程序的 open () 方法代码见程序清单 16.5。可以看出，程序清单 16.5 完全是按照程序清单 15.9 编写的，两者的对照见表 16.2。

程序清单 **16.5** **open** () 方法代码

```
static int  net_open(struct net_device *dev)
{
    unsigned long flag;

    if (usage == 0)                                                    (1)
    {
        local_irq_save(flag);                                          (2)
        mac_hard_open();                                               (3)
        device_init(dev);                                              (4)
        local_irq_restore(flag);                                       (5)
        request_irq(dev->irq, net_irq_handle, SA_INTERRUPT |SA_SAMPLE_RANDOM,
                    "eth0", dev);                                      (6)
        netif_start_queue(dev);                                        (7)
    }
    usage++;                                                           (8)
    MOD_INC_USE_COUNT;                                                 (9)
    return 0;                                                         (10)
}
```

表 16.2 **open** () 方法的示意代码与驱动代码对照说明

示意代码 (程序清单 15.9)	实际代码
(1) if (需要操作的设备的使用计数器 = =0)	程序清单 16.5 (1)
(2) {	{
(3) 初始化设备 (包括申请资源) ;	程序清单 16.5 (2) ~ (6)
(4) netif_start_queue (dev) ;	程序清单 16.5 (7)
(5) }	}

更多高清PDF图书 www.gqpdf.com

(6) 需要操作的设备的使用计数器 + + ;	程序清单 16.5 (8)
(7) MOD_INC_USE_COUNT;	程序清单 16.5 (9)
(4) return 0;	程序清单 16.5 (10)

程序清单 16.5 (2)、(5) 是开关中断代码,可参考 7.3.2 小节。程序清单 16.5 (3) 的代码是与用户电路相关的代码,前面介绍过,详情请参考 16.2 节第 1 条。程序清单 16.5 (6) 为驱动程序申请中断服务程序,具体可参考 7.3.1 小节。至于程序清单 16.5 (4),函数 device_init () 是初始化 RTL8019 这个芯片的,其代码见程序清单 16.6。函数 device_init () 用到的一些宏分别在 config. h 和 rtl8019. h 中定义。config. h 的代码参考程序清单 16.1,rtl8019. h 的代码参考程序清单 16.7。程序清单 16.7 中使用了一些 I/O 操作函数,这部分参考 5.2.7 小节。由于函数 device_init () 完全与硬件相关,与编写驱动程序本身关系不大,这里不详细介绍,感兴趣的读者可参考源代码注释、芯片数据手册和参考文献[5]。

程序清单 16.5 的其他代码都很简单,这里不再说明。

程序清单 **16.6　RTL8019** 芯片初始化代码

```
static int device_init(struct net_device *dev)
{
    unsigned long flag;

    local_irq_save(flag);
    /* 硬件复位 */
    NET_HARD_REG();

    /* 软件复位 */
    WriteToNet(0x1f, 0x00);
    mdelay(11);

    WriteToNet(0x00, 0x21);                    //使芯片处于停止模式,这时进行寄存器设置

    mdelay(11);
    page(0);

    WriteToNet(0x0a, 0x00);                    //清 rbcr0
    WriteToNet(0x0b, 0x00);                    //清 rbcr1
    WriteToNet(0x0c, 0xe0);                    //RCR,监视模式,不接收数据包
    WriteToNet(0x0d, 0xe2);                    //TCR,loop back 模式
    page(0);
    WriteToNet(0x01, 0x4c);                    //PSTART =0x4c
    WriteToNet(0x02, 0x80);                    //PSTOP =0x80
    WriteToNet(0x03, 0x4c);                    //bnry =0x4c;
```


更多高清PDF图书 www.gqpdf.com

```
    page(0);
    WriteToNet(0x04, 0x40);          //TPSR,发送起始页寄存器
    WriteToNet(0x07, 0xff);          //清除所有中断标志位,中断状态寄存器
    WriteToNet(0x0f, 0x13);          //允许相关的中断

    WriteToNet(0x0e, 0xcb);          //数据配置寄存器,16 位 DMA 方式
    page(1);

    WriteToNet(0x07, 0x4d);          //curr =0x4d
    WriteToNet(0x08, 0x00);
    WriteToNet(0x09, 0x00);
    WriteToNet(0x0a, 0x00);
    WriteToNet(0x0b, 0x00);
    WriteToNet(0x0c, 0x00);
    WriteToNet(0x0d, 0x00);
    WriteToNet(0x0e, 0x00);
    WriteToNet(0x0f, 0x00);
    WriteToNet(0x00, 0x22);          //这时让芯片开始工作

    SetMacID(dev);                   //将芯片物理地址写入到 MAR 寄存器

    page(0);
    WriteToNet(0x0c, 0xc4);          //将芯片设置成正常模式,与外部网络连接
    WriteToNet(0x0d, 0xe0);
    WriteToNet(0x00, 0x22);          //启动芯片开始工作
    WriteToNet(0x07, 0xff);          //清除所有中断标志位

    local_irq_restore(flag);
    return 0;
}
```

程序清单 **16.7**　**rtl8019.h** 内容

```
#ifndef __RTL8019_H
#define __RTL8019_H

#define RTL8019_REG(reg)                (dev->base_addr + (reg) * 2)
#define ReadFromNet(addr)               inw(RTL8019_REG(addr))
#define WriteToNet(addr, data)          outw(data, RTL8019_REG(addr))
```

```
#define page(pagenumber)          WriteToNet(0, (ReadFromNet(0) & 0x3b) |((pa-
genumber) < < 6))
#define savepage()                ((ReadFromNet(0) & 0x3c) > > 6)
#endif
```

16.3.5 stop()方法

本驱动程序的 stop()方法代码见程序清单 16.8。可以看出，程序清单 16.8 完全是按照程序清单 15.10 编写的，两者的对照见表 16.3。

程序清单 **16.8** **stop()方法代码**

```
static int net_release(struct net_device *dev)
{
    unsigned long flag;

    MOD_DEC_USE_COUNT;                                          (1)
    usage--;                                                    (2)
    if (usage == 0)                                             (3)
    {
        netif_stop_queue(dev);                                  (4)
        local_irq_save(flag);                                   (5)
        mac_hard_close();                                       (6)
        local_irq_restore(flag);                                (7)
        free_irq(dev->irq, dev);                                (8)
    }
    return 0;                                                   (9)
}
```

表 16.3 stop()方法的示意代码与驱动代码对照说明

示意代码(程序清单 15.10)	实际代码
(1) MOD_DEC_USE_COUNT;	程序清单 16.8 (1)
(2) 需要操作的设备的使用计数器 - - ;	程序清单 16.8 (2)
(3) if (需要操作的设备的使用计数器 = =0)	程序清单 16.8 (3)
(4) {	{
(5) netif_stop_queue (dev) ;	程序清单 16.8 (4)
(6) 关闭设备处理(包括释放 revalidate()方法获取的资源);	程序清单 16.8 (5) ~ (8)
(7) }	}

更多高清PDF图书 www.gqpdf.com

(8) return 0;	程序清单 16.8 (9)

程序清单 16.8 (5)、(7) 是开关中断代码，可参考 7.3.2 小节。程序清单 16.6 (6) 的代码是与用户电路相关的代码，前面介绍过，详情参考 16.2 节第 1 条。程序清单 16.6 (8) 为释放驱动程序申请的中断服务程序，具体可参考 7.3.1 小节。

程序清单 16.8 的其他代码都很简单，这里不再说明。

16.3.6 set_config () 方法

本驱动程序的 set_config () 方法代码见程序清单 16.9。可以看出，程序清单 16.9 与程序清单 15.11 完全相同，这里不再说明。

程序清单 **16.9** **set_config** () 方法代码

```
static int net_config(struct net_device *dev, struct ifmap *map)
{
    return -EBUSY;
}
```

16.3.7 hard_start_xmit () 方法

本驱动程序的 hard_start_xmit () 方法代码见程序清单 16.10。可以看出，程序清单 16.10 完全是按照程序清单 15.12 编写的，两者的对照见表 16.4。

程序清单 **16.10** **hard_start_xmit** () 方法代码

```
static int net_tx(struct sk_buff *skb, struct net_device *dev)
{
    unsigned long flag;
    int len;
    u16 *data;

    netif_stop_queue(dev);                                          (1)

    len = skb->len < ETH_ZLEN ? ETH_ZLEN : skb->len;                (2)
    data = (u16 *)skb->data;                                        (3)
    len = (len + 1) & (~1);                                         (4)

    local_irq_save(flag);                                           (5)

    page(0);                                                        (6)
    WriteToNet(0x09, 0x40);                  //设置发送页地址       (7)
```

高清PDF gqpdf.com

更多高清PDF图书 www.gqpdf.com

```
    WriteToNet(0x08, 0x00);          //写入 RSAR0 DMA 起始地址低位          (8)
    WriteToNet(0x0b, len >> 8);      //写入 RSCR1 DMA 计数器高位            (9)
    WriteToNet(0x0a, len & 0x00ff);  //写入 RSCR0 DMA 计数器低位           (10)
    WriteToNet(0, 0x12);             //启动 DMApage0                       (11)

    outsw(RTL8019_REG(0x10), data, len >> 1);                              (12)

    WriteToNet(0x0b, 0x00);                                                (13)
    WriteToNet(0x0a, 0x00);                                                (14)
    WriteToNet(0x00, 0x22);          //结束或放弃 DMA 操作                 (15)

    WriteToNet(0x06, len >> 8);                                            (16)
    WriteToNet(0x05, len & 0x00ff);                                        (17)
    WriteToNet(0x07, 0xff);                                                (18)
    WriteToNet(0x00, 0x3e);                                                (19)

    dev->trans_start = jiffies;                                            (20)
    local_irq_restore(flag);                                               (21)

    dev_kfree_skb(skb);                                                    (22)

    return 0;                                                              (23)
}
```

表 16.4　hard_start_xmit()方法的示意代码与驱动代码对照说明

示意代码(程序清单 15.12)	实际代码	说　明
(1) netif_stop_queue(dev);	程序清单 16.10(1)	
(2) len = skb->len < ETH_ZLEN ? ETH_ZLEN : skb->len;	程序清单 16.10(1)	
(3) 把 len 个数据写入网络设备硬件缓冲区并启动发送;	程序清单 16.10(3) ~ (19)、(21)	
(4) dev->trans_start = jiffies;	程序清单 16.10(20)	
(5) dev_kfree_skb(skb);	程序清单 16.10(22)	
(6) 等待数据发送完毕;	无	用中断实现
(7) netif_wake_queue(dev)	无	在中断服务程序中,见程序清单 16.13(10)
(8) return 0;	程序清单 16.10(23)	

程序清单 16.10(4)是把发送的尺寸转换成偶数字节,这是 RTL8019 的要求。程序清单 16.10(5)、(21)是开关中断代码,可参考 7.3.2 小节。程序清单 16.10(12)使用 I/O 操作函数,这部分请参考 5.2.7 小节。程序清单 16.10(3)~(19)的其他代码与硬件相关,与编写驱动程序本身关系不大,这里不详细介绍,感兴趣的读者可参考源代码注释、芯片数据手册和参考文献[5]。

程序清单 16.10 的其他代码都很简单,这里不再说明。

16.3.8 set_mac_address()方法

本驱动程序的 set_mac_address()方法代码见程序清单 16.11。可以看出,程序清单16.11 完全是按照程序清单 15.13 编写的,仅仅是把程序清单 15.13(5)换成了程序清单 16.11(5)。至于程序清单 16.11(5),函数 SetMacID()的代码参考程序清单 16.12。因为函数 SetMacID()完全与硬件相关,与编写驱动程序本身关系不大,这里不详细介绍,感兴趣的读者可参考源代码注释、芯片数据手册和参考文献[5]。

程序清单 **16.11** set_mac_address()方法代码

```
static int net_set_mac_address(struct net_device *dev, void *addr)
{
    struct sockaddr *mac_addr;

    mac_addr = addr;                                                    (1)
    if (netif_running(dev))                                             (2)
    {
        return -EBUSY;                                                  (3)
    }
    memcpy(dev->dev_addr, mac_addr->sa_data, dev->addr_len);
                                                                        (4)
    SetMacID(dev);                                                      (5)
    return 0;                                                           (6)
}
```

程序清单 **16.12** 函数 **SetMacID()**的代码

更多高清PDF图书 www.gqpdf.com

```
void SetMacID(struct net_device *dev)
{
    u8 *mac_ptr;

    mac_ptr = (u8 *)dev->dev_addr;
    //把 MAC 地址写入 MY——MAC——ID 中
    page(1);

    WriteToNet(1 , *mac_ptr++);
    WriteToNet(2 , *mac_ptr++);
    WriteToNet(3 , *mac_ptr++);
    WriteToNet(4 , *mac_ptr++);
    WriteToNet(5 , *mac_ptr++);
    WriteToNet(6 , *mac_ptr);
    page(0);
}
```

16.3.9 中断服务程序

本驱动程序的中断服务程序代码见程序清单 16.13。可以看出，程序清单 16.13 基本是按照编程序清单 15.14 编写的，两者的对照见表 16.5。

程序清单 **16.13** 中断服务程序代码

高清PDF gqpdf.com

```
static void net_irq_handle(int irq, void *dev_id, struct pt_regs *regs)
{
    unsigned int page_save, flag;
    unsigned int bnry, curr;
    struct net_device *dev;

dev = (struct net_device *)dev_id;                                    (1)
page_save = savepage();                                               (2)
while(1)                                                              (3)
{
    page(0);                                                          (4)
    flag = ReadFromNet(0x07) & 0xff;                                  (5)
    WriteToNet(0x07, flag);                    /*清除中断标志*/        (6)
    if ((flag & ((1 < < 1) | (1 < < 4) | (1 < < 0))) ==0)
                                                                      (7)
    {
        break;                                                        (8)
    }

    if ((flag & (1 < < 1)) !=0)                /*发送成功*/
                                                                      (9)
    {
```


更多高清PDF图书 www.gqpdf.com

```
            netif_wake_queue(dev);                                          (10)
        }

        if ((flag & (1 < < 4)) !=0)       /* 接收缓冲区溢出 */
                                                                            (11)
        {
            page(1);                                                        (12)
            curr =ReadFromNet(0x07);      //page1 读取 CURR 的值          (13)
            curr =curr & 0xff;                                              (14)
            page(0);                      //切换回 page0
                                                                            (15)
            bnry =curr -1;                //把 bnry 恢复为下 16K 中的空余部分
                                                                            (16)
            if(bnry < 0x4c)                                                 (17)
            {
                bnry =0x7f;                                                 (18)
            }
            WriteToNet(0x03, bnry);       //把 BNRY 恢复到指向下一帧 write to bnry
                                                                            (19)
            WriteToNet(0x07, 0xff);       //清除中断标志                  (20)
        }

        if ((flag & (1 < < 0)) !=0)       /* 接收成功 */                    (21)
        {
            if (device_rx(dev) == -1)                                       (22)
            {
                RTL8019Dev =dev;                                            (23)
                tasklet_schedule (&ZLG_net_tasklet);
                                                                            (24)
                break;                                                      (25)
            }
        }
        page(page_save);                                                    (26)

        outl(1 < < (dev->irq -14), EXTINT);                                 (27)
    }
```

表 16.5　中断服务程序示意代码与驱动代码对照说明

示意代码(程序清单 15.14)	实际代码	说　明
(1) dev = (struct net_device *)dev_id;	程序清单 16.13 (1)	
(2) if (发送成功)	程序清单 16.13 (9)	
{	{	
(3) 清除发送中断标志;	程序清单 16.13 (6)	
(4) netif_wake_queue (dev);	程序清单 16.13 (10)	
}	}	
(5) f (接收成功)	程序清单 16.13 (21)	
{	{	
(6) 清除接收中断标志;	程序清单 16.13 (6)	
(7) length = 数据包长度;	程序清单 16.13 (22)	在函数 device_rx () 中实现
(8) skb = dev_alloc_skb (length + 2);		
(9) if (! skb)		
{		
(10) return;		
}		
(11) skb_reserve (skb, 2);		
(12) dec = skb - > data;		
(13) 把数据包 (length 个数据) 读到 dec 指向的内存中;		
(14) skb - > dev = dev;		
(15) skb - > protocol = eth_type_trans (skb, dev);		
(16) skb - > ip_summed = CHECKSUM_UNNECESSARY;		
(17) netif_rx (skb);		
(18) dev - > last_rx = jiffies;		
}	}	
(19) if (有错误出错)	程序清单 16.13 (11)	
{	{	
(20) 错误处理程序;	程序清单 16.13 (12) ~ (20)、(23) ~ (25)	
}	}	
(21) 清除中断源;	程序清单 16.13 (27)	

更多高清PDF图书　www.gqpdf.com

可以看出，程序清单 16.13 (2)、(26) 没有对应语句。其实程序清单 16.13 (2)、(26) 是为了保证中断退出后，驱动程序操作 RTL8019 依然正常而加入的。具体可参考芯片数据手册。

程序清单 15.14 (7) ~ (18) 由函数 device_rx () 实现。device_rx () 返回 0 表示成功，返回 - 1 表示有错误。由于函数 device_rx () 完全与硬件相关，这里不作介绍，感兴趣的读者可阅读源码和参见芯片数据手册及参考文献[5]。

程序清单 16.13 (12) ~ (20) 为错误处理代码，完全与硬件相关，这里不作介绍，感兴趣的读者可阅读源码和参见芯片数据手册及参考文献[5]。

至于程序清单 16.13 (23) ~ (25)，涉及 Linux (μClinux) 中断系统的一个重要概念：中断的下半部分。Linux (μClinux) 把中断分为上半部分和下半部分。中断的上半部分为执行需要立即完成的事物，而中断的下半部分会延迟执行。中断的上半部分需要尽快执行完毕，而且很多内核都不能使用。中断的下半部分就没有这些限制。关于中断下半部分的说明可参见参考文献[5]。本书前面提到的中断均值使用了中断的上半部分，这是因为它们都会很快地执行完毕，没有必要使用下半部分。device_rx ()（程序清单 16.13 (22)）执行出错时，表明需要重新初始化 RTL8019，而初始化 RTL8019 需要较长的时间，所以需要中断的下半部分。

本驱动程序使用 tasklet 方式实现中断的下半部分，其相关代码见程序清单 16.14。

注意：程序清单 16.14 (1)、(2) 和其他部分之间在源代码中分散在不同的地方。

程序清单 **16.14**　中断下半部分相关代码

```
static struct net_device *RTL8019Dev;                                    (1)

DECLARE_TASKLET(ZLG_net_tasklet, net_tasklet, (unsigned long) (&RTL8019Dev));
                                                                         (2)

static void net_tasklet(unsigned long data)
                                                                         (3)
{
struct net_device *dev;
    dev = *((struct net_device **)data);                                 (4)
    device_init(dev);                                                    (5)
}
```

使用 tasklet 必须使用宏 DECLARE_TASKLET () 建立并初始化 tasklet 结构体变量（参考程序清单 16.14 (2)）。宏 DECLARE_TASKLET 的使用方法参考程序清单 16.15。其中 name 为 tasklet 结构体变量名称，以后就通过这个变量使用这个 tasklet；function 为执行 tasklet 时调用的函数；data 是一个用来传递给 tasklet 函数的 unsigned long 类型的值。在建立了 tasklet 变量

和编写了 tasklet 处理函数（程序清单 16.16 为 tasklet 处理函数原型）后，就可以使用 tasklet 了，这通过调用函数 tasklet_schedule () 实现。函数 tasklet_schedule () 是一个 inline 函数，其惟一的参数为指向 tasklet 结构体变量的指针。函数 tasklet_schedule () 的使用请参考程序清单 16.13 (24)。

程序清单 16.14 很简单，对比相关代码（函数 device_init () 代码请参考程序清单 16.6）很容易读懂，这里不再说明。

程序清单 **16.15**　声明 **tasklet**

```
DECLARE_TASKLET(name, function, data);
```

程序清单 **16.16**　**tasklet** 处理函数原型

```
void tasklet_function(unsigned long data);
```

本章小结

本章以 RTL8029 网络接口芯片为例介绍了 Linux（μClinux）网络驱动程序的编写方法和相关知识，包括源码分析和编译使用。

针对本书前面的章节，本章主要增加了中断下半部分 tasklet 的介绍。

高清PDF藏 gqpdf.com

更多高清PDF图书 www.gqpdf.com

第17章 DM9000 驱动程序编写范例

17.1 硬件原理

DM9000 为 MAC、PHY、MMU 三合一的网络芯片，是一种高度集成的快速以太网芯片，具有 10/100M 自适应、16 KB 大容量 FIFO、4 路多功能 GPIO、掉电特性、全双工工作等功能。由于它拥有 3 种等级的掉电模式，所以是作为绿色电脑的网络设备的理想选择。在全双工模式下，如果连接到一个同样是全双工的交换机或集线器，则可实现同时接收和发送。

DM9000 与 LPC2200 系列 ARM 芯片的接口电路图参考图 17.1，它是从 MagicARM2200 开发板（广州致远电子有限公司设计生产）的原理图中截取出来的。电路使用 16 位总线方式进行控制，即数据总线 D0 ~ D15 与芯片的 SD0 ~ SD15 连接，地址线也进行相对应的连接，片选线与芯片的 AEN 相连。DM9000E 10/100M 以太网控制器的工作基地址为 0x300，而总线的地址线 A2 与芯片的命令/数据使能端 CMD 相连，所以对其进行操作时的地址是 0x300（地址端口）或 0x304（数据端口）。片选信号 IO_nCS1 由译码电路生成，其地址为 0x83000000 ~ 0x833FFFFF，所以操作 DM9000 的 32 位地址为 0x83000300（地址端口）或 0x83000304（数据端口）。

DM9000E 的应用及连接方法的详细说明请参考 DM9000E 芯片的数据手册。

17.2 使用源代码

DM9000 驱动程序源代码由 4 个文件组成，它们是 config. h、dm9000. h、dm9000. c 和 Makefile。其中 config. h 中配置了一些硬件信息，而 Makefile 也保存了一些编译器相关的信息。因此，对于使用者来说，这两个文件可能需要改动。

1. 修改 config. h 文件适合用户硬件

config. h 的代码请参考程序清单 17.1。config. h 文件比较长，程序清单 17.1 仅列出其与硬件相关的部分。程序清单 17.1 已列出各部分的含义，用户按照实际的硬件修改即可。惟一要注意的是程序清单 17.1（3），宏 NET_TIMEOUT 暂时没有使用，但以后的升级版本可能会用到。

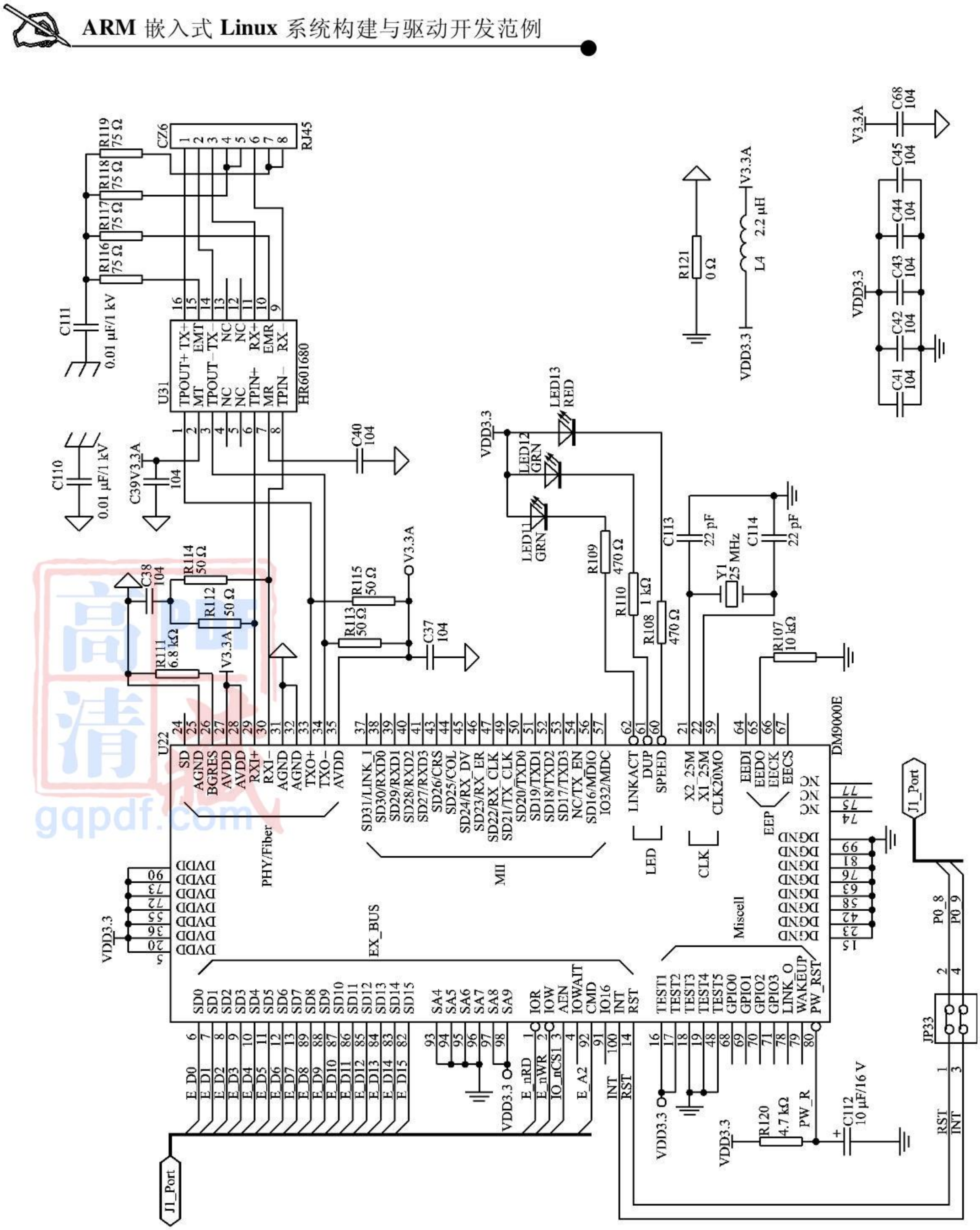

图 17.1　以太网接口电路

更多高清PDF图书　www.gqpdf.com

程序清单 **17.1**　**config.h** 文件内容

```
......
#define IOaddress0      0x83000000                                  (1)

#define NET_IRQ         IRQ_EXT3                                    (2)

#define NET_TIMEOUT     10000                                       (3)

#define NET_RST0                        0X00000100                  /*复位引脚*/
#define  Hardware_Initial0()            outl(inl(IO0DIR) |NET_RST0, IO0DIR);/*设置为输出*/
#define  Hardware_Reset_Clr0()          outl(NET_RST0, IO0CLR)      /*设置为低电平*/
#define  Hardware_Reset_Set0()          outl(NET_RST0, IO0SET)      /*设置为高电平*/

#define   BUS_DELAY   200                                           (8)

#ifdef IN_DM9000                                                    (9)
char MyMacID\[\] = {0x52, 0x54, 0x4c, 0x33, 0xf7, 0x42, 0x00};      (10)

static u32 PinSel0Save;                                             (11)

inline void mac_hard_open(void)                                     (12)
{
    u32 temp;

    temp = inl(PINSEL0);                                            (13)
    PinSel0Save = temp & (0x0f << (8 * 2));                         (14)
    temp |= (3 << (9 * 2));                                         (15)
    temp &= ~(3 << (8 * 2));                                        (16)
    outl(temp, PINSEL0);                                            (17)

    temp = inl(IO0DIR);                                             (18)
    temp |=1 << 8;                                                  (19)
    outl(temp, IO0DIR);                                             (20)
    outl(1 << 8, IO0CLR);                                           (21)
```

硬件基地址

中断号

超时设置，没有使用

硬件复位DM9000相关操作

操作间延时，一般无须修改

设置默认MAC地址

驱动程序加载后被调用，用于初始化与硬件相关的部分

保存引脚连接配置

P0.8 设置为 GPIO，P0.9 设置为 EINT3

P0.8设置为输出，并输出低电平

```
    temp = inl(VPBDIV);
    temp = inl(VPBDIV);
    outl(0, VPBDIV);
    outl(inl(EXTMODE) & (~(1 << 3)), EXTMODE);
    outl(inl(EXTPOLAR) |(1 << 3), EXTPOLAR);
    outl(temp, VPBDIV);
}
```

设置外部中断为高电平触发

```
inline void mac_hard_close(void)
{
    u32 temp;
    temp = inl(PINSEL0);
    temp &= ~(3 << (9 * 2));
    temp |= PinSel0Save;
    outl(temp, PINSEL0);
}
```

驱动程序卸载前被调用，用于与硬件相关的退出处理

恢复硬件连接配置

```
#endif
......
```

2. 修改 Makefile 文件及编译

编译 DM9000 驱动程序主要是编写 Makefile 文件，具体请参考 4.3.1 小节。

17.3 源码分析

17.3.1 头文件 DM9000.h

本节讲述的很多代码都用到了 DM9000.h 中的内容，为读者查阅方便，程序清单 17.2 给出其代码。因为这些内容与硬件相关，与驱动程序编写关系不大，这里不再说明。

程序清单 **17.2** **DM9000.h** 文件内容

```
#ifndef __DM9000_H
#define __DM9000_H

#define DM9000_REG00            0x00
#define DM9000_REG05            0x0c
```


更多高清PDF图书 www.gqpdf.com

```
#define DM9000_REG08            0x37
#define DM9000_REG09            0x38
#define DM9000_REG0A            0xff
#define DM9000_REGFF            0x83

#define DM9000_DWORD_MODE       1
#define DM9000_BYTE_MODE        2
#define DM9000_WORD_MODE        0

#define DM9801_NOISE_FLOOR      0x08
#define DM9802_NOISE_FLOOR      0x05

#define DM9000_PHY              0x40
#define DM9000_PKT_MAX          1536
#define DM9000_PKT_RDY          0x01
#define DM9000_MIN_IO           0x300
#define DM9000_MAX_IO           0x370
#define DM9000_INT_MII          0x00
#define DM9000_EXT_MII          0x80

#define DM9000_RX_INTR          0x01
#define DM9000_TX_INTR          0x02
#define DM9000_OVERFLOW_INTR    0x04

enum DM9000_PHY_mode {
    DM9000_10MHD      =0,
    DM9000_100MHD     =1,
    DM9000_10MFD      =4,
    DM9000_100MFD     =5,
    DM9000_AUTO       =8,
    DM9000_1M_HPNA    =0x10
};

enum DM9000_NIC_TYPE {
    FASTETHER_NIC     =0,
    HOMERUN_NIC       =1,
    LONGRUN_NIC       =2
};
```

```
typedef struct board_info {

    uint32 ioaddr;
    uint32 io_data;
    uint16 irq;

    uint16 queue_pkt_len;
    uint8 reg0, reg5, reg8, reg9, rega;
    uint8 op_mode;
    uint8 io_mode;
    uint8 phy_addr;
    uint8 link_failed;
    uint8 nic_type;
    uint8 nic_mac\[6\];
} board_info_t;

#endif
```

17.3.2 底层硬件操作函数

本节讲述的很多代码都用到了几个底层操作函数，为读者查阅方便，程序清单 17.3 给出其代码。因为这些内容与硬件相关，与驱动程序编写关系不大，这里不再说明。程序清单 17.3 中使用了一些 I/O 操作函数，详细内容请参考 5.2.7 小节。

程序清单 **17.3** 底层操作函数

```
static uint8 ior(board_info_t *db, uint8 reg)
{
    unsigned int i;
    unsigned int rt;

    outb(reg, db->ioaddr);
    for (i=0; i < BUS_DELAY; i++)
    {
    }

    rt = inb(db->io_data);
    for (i=0; i < BUS_DELAY; i++)
```


更多高清PDF图书 www.gqpdf.com

```
    {
    }

    return rt;
}
static void iow(board_info_t *db, uint8 reg, uint8 value)
{
    unsigned int i;

    outb(reg, db->ioaddr);
    for(i=0; i<BUS_DELAY; i++)
    {
    }

    outb(value, db->io_data);
    for(i=0; i<BUS_DELAY; i++)
    {
    }
}

static void phy_write(board_info_t *db, uint8 reg, uint16 value)
{
    /* Fill the phyxcer register into REG_0C */
    iow(db, 0xc, DM9000_PHY|reg);

    /* Fill the written data into REG_0D & REG_0E */
    iow(db, 0xd, (value & 0xff));
    iow(db, 0xe, ((value >> 8) & 0xff));

    iow(db, 0xb, 0xa);
    udelay(500);
    iow(db, 0xb, 0x0);
}

static void set_PHY_mode(board_info_t *db)
{
    unsigned int phy_reg4=0x01e1, phy_reg0=0x1000;
```

```
    if ( ! (db->op_mode & DM9000_AUTO))
    {
        switch(db->op_mode)
        {
            case DM9000_10MHD:
                phy_reg4 = 0x21;
                phy_reg0 = 0x0000;
                break;
            case DM9000_10MFD:
                phy_reg4 = 0x41;
                phy_reg0 = 0x1100;
                break;
            case DM9000_100MHD:
                phy_reg4 = 0x81;
                phy_reg0 = 0x2000;
                break;
            case DM9000_100MFD:
                phy_reg4 = 0x101;
                phy_reg0 = 0x3100;
                break;
            default:
                break;
        }
        phy_write(db, 4, phy_reg4);
        phy_write(db, 0, phy_reg0);
    }

    iow(db, 0x1e, 0x01);
    iow(db, 0x1f, 0x00);
}
```

高清 PDF 藏 gqpdf.com

17.3.3 初始化函数

本驱动程序的初始化函数代码见程序清单 17.4。可以看出，程序清单 17.4 是按照程序清单 15.2 编写的，仅仅是在程序清单 17.4 上增加了信息显示（程序清单 17.4（6）、（8））。

注意：在源代码文件中，程序清单 17.4（1）、（2）与其他部分并不在一起，这里为了表述方便而放在一起。

更多高清PDF图书 www.gqpdf.com

程序清单 **17.4**　初始化函数代码

```
static struct net_device net_net =                                    (1)
{
    init: net_init,                                                   (2)
};
 int net_init_module(void)                                            (3)
{
    int result;

    result = register_netdev(&net_net);                               (4)
    if (result < 0)                                                   (5)
    {
        printk(KERN_ERR "eth0: error % i registering device \\"% s\\" \\n",
              result, net_net.name);                                  (6)
        return result;                                                (7)
    }

    printk(KERN_ERR "eth0: init OK\\n");                              (8)
    return 0;                                                         (9)
}
```

17.3.4　清除函数

本驱动程序的清除函数代码见程序清单 17.5。可以看出，程序清单 17.5 完全是按照程序清单 15.4 编写的，这里不再说明。

程序清单 **17.5**　清除函数代码

```
void net_cleanup(void)
{
    unregister_netdev(&net_net);
}
```

17.3.5　init() 方法

本驱动程序的 init() 方法代码见程序清单 17.6。可以看出，程序清单 17.6 完全是按照程序清单 15.7 编写的，两者的对照见表 17.1。

高清PDF藏 gqpdf.com

程序清单 17.6　init() 方法代码

```
static board_info_t NetDriverInfo\[1 \];                                    (1)

static int net_init(struct net_device *dev)
{
    ether_setup(dev);                                                       (2)
    strcpy(dev->name, "eth0");                                              (3)
    dev->open               =net_open;                                      (4)
    dev->stop               =net_release;                                   (5)
    dev->set_config         =net_config;                                    (6)
    dev->hard_start_xmit    =net_tx;                                        (7)
    dev->set_mac_address    =net_set_mac_address;                           (8)
    dev->flags              &=~(IFF_LOOPBACK | IFF_MULTICAST);              (9)
    dev->priv               =NetDriverInfo;                                 (10)
    dev->base_addr          =IOaddress0;                                    (11)
    dev->irq                =NET_IRQ;                                       (12)

    memcpy(dev->dev_addr, MyMacID, dev->addr_len);                          (13)
    SET_MODULE_OWNER(dev);                                                  (14)

    return 0;                                                               (15)
}
```

结构体 board_info_t(程序清单 17.6(1))是保存硬件信息的,消息请参考程序清单 17.2。参考 15.1.4 小节、程序清单 15.7 和程序清单 15.6,程序清单 17.6 的代码很容易读懂,这里不再说明。

表 17.1　init() 方法的示意代码与驱动代码对照说明

示意代码(程序清单 15.7)	实际代码
(1) ether_setup(dev);	程序清单 17.6(2)
(2) 设置 dev 中必需的成员;	程序清单 17.6(4) ~ (9)
(3) 设置 dev 中驱动程序感兴趣的成员;	程序清单 17.6(10) ~ (13)
(4) SET_MODULE_OWNER(dev);	程序清单 17.6(14)
(4) return 0;	程序清单 17.6(15)

更多高清PDF图书　www.gqpdf.com

17.3.6 open()方法

本驱动程序的 open()方法代码见程序清单 17.7。可以看出,程序清单 17.7 完全是按照程序清单 15.9 编写的,两者的对照见表 17.2。

程序清单 **17.7** **open()方法代码**

```
static int  net_open(struct net_device *dev)
{
    unsigned long flag;

    if (usage == 0)                                                    (1)
    {
        local_irq_save(flag);                                          (2)
        mac_hard_open();                                               (3)
        device_init(dev);                                              (4)
        local_irq_restore(flag);                                       (5)
        request_irq(dev->irq, net_irq_handle, SA_INTERRUPT | SA_SAMPLE_RANDOM,
                    "eth0", dev);                                      (6)
        netif_start_queue(dev);                                        (7)
    }
    usage++;                                                           (8)
    MOD_INC_USE_COUNT;                                                 (9)
    return 0;                                                          (10)
}
```

表 17.2 open()方法的示意代码与驱动代码对照说明

示意代码(程序清单 15.9)	实际代码
(1) if(需要操作的设备的使用计数器 = =0)	程序清单 17.7(1)
(2){	{
(3) 初始化设备(包括申请资源);	程序清单 17.7(2) ~ (6)
(4) netif_start_queue(dev);	程序清单 17.7(7)
(5)}	}
(6) 需要操作的设备的使用计数器 + +;	程序清单 17.7(8)
(7) MOD_INC_USE_COUNT;	程序清单 17.7(9)
(4) return 0;	程序清单 17.7(10)

高清PDF藏 gqpdf.com

程序清单 17.7 (2)、(5) 是开关中断的代码，可参考 7.3.2 小节。程序清单 17.7 (3) 的代码是与用户电路相关的代码，前面已介绍，详情请参考 16.2 节第 1 条。程序清单 17.7 (6) 为驱动程序申请中断服务程序，具体可参考 7.3.1 小节。至于程序清单 17.7 (4)，函数 device_init () 是初始化 DM9000 这个芯片的，其代码见程序清单 17.8。函数 device_init () 用到的一些宏和函数分别在 17.2 节第 1 条、17.3.1 小节和 17.3.2 小节中有介绍。程序清单 17.8 中使用了一些 I/O 操作函数，这部分请参考 5.2.7 小节。由于函数 device_init () 完全与硬件相关，与编写驱动程序本身关系不大，这里不详细介绍，感兴趣的读者可参考源代码注释和芯片数据手册。

程序清单 17.7 的其他代码都很简单，这里不再说明。

程序清单 **17.8　DM9000** 芯片初始化代码

```
static int device_init(struct net_device *dev)
{
    unsigned long flag;
    uint8   i;
    board_info_t *db = dev->priv;

    local_irq_save(flag);
    /* 硬件复位 */

    Hardware_Initial0();
    db->ioaddr = IOaddress0;
    db->io_data = IOaddress0 + 4;
    /* RESET device */
    /*硬件复位*/
    Hardware_Reset_Clr0();
    udelay(500);
    Hardware_Reset_Set0();
    udelay(500);
    Hardware_Reset_Clr0();
    udelay(500);

    /*软件复位*/
    iow(db, 0, 1);
    udelay(500);                /* 延时100 μs */
    /* I/O mode */
```


更多高清PDF图书 www.gqpdf.com

```
    db->io_mode = ior(db, 0xfe) >> 6;

    /* NIC Type: FASTETHER, HOMERUN, LONGRUN */
    db->nic_type = FASTETHER_NIC;

    /* GPIO0 on pre-activate PHY */
    iow(db, 0x1f, 0x00);

    /* Set PHY */
    db->op_mode = DM9000_AUTO;
    set_PHY_mode(db);

    /* Init needed register value */
    db->reg0 = DM9000_REG00;
    if ((db->nic_type != FASTETHER_NIC) && (db->op_mode & DM9000_1M_HPNA) )

    {
        db->reg0 |= DM9000_EXT_MII;
    }

    /* User passed argument */
    db->reg5 = DM9000_REG05;
    db->reg8 = DM9000_REG08;
    db->reg9 = DM9000_REG09;
    db->rega = DM9000_REG0A;

    /* Program operating register */
    iow(db, 0x00, db->reg0);
    iow(db, 0x02, 0x14);
    iow(db, 0x08, 0x3f);
    iow(db, 0x09, db->reg9);
    iow(db, 0x0a, db->rega);
    iow(db, 0x2f, 0);
    iow(db, 0x01, 0x2c);
```

局部PDF 清晰藏 gqpdf.com

```
    iow(db, 0xfe, 0x0f);

    /* Set address filter table */
    for(i=0; i < 6; i++)
    {
        db->nic_mac\[i\]=MyMacID\[i\];
    }

    for(i=0; i < 6; i++)
    {
        iow(db, 0x10 + i, db->nic_mac\[i\]);   //将芯片物理地址写入到 MAC 寄存器
    }

    /* Activate DM9000 */
    iow(db, 0x05, db->reg5 |1);
    iow(db, 0xff, DM9000_REGFF);
    //udelay(5000);

        local_irq_restore(flag);

        return 0;
    }
```

17.3.7 stop()方法

本驱动程序的 stop()方法代码见程序清单 17.9。可以看出,程序清单 17.9 完全是按照程序清单 15.10 编写的,两者的对照见表 17.3。

程序清单 **17.9** **stop()方法代码**

```
    static int  net_release(struct net_device *dev)
    {
        unsigned long flag;

        MOD_DEC_USE_COUNT;                                          (1)
        usage--;                                                    (2)
        if (usage == 0)                                             (3)
```


更多高清PDF图书 www.gqpdf.com

```
    {
        netif_stop_queue(dev);                                    (4)
        local_irq_save(flag);                                     (5)
        mac_hard_close();                                         (6)
        local_irq_restore(flag);                                  (7)
        free_irq(dev->irq, dev);                                  (8)
    }
    return 0;                                                     (9)
}
```

表 17.3　stop() 方法的示意代码与驱动代码对照说明

示意代码(程序清单 15.10)	实际代码
(1) MOD_DEC_USE_COUNT;	程序清单 17.9(1)
(2) 需要操作的设备的使用计数器 - - ;	程序清单 17.9(2)
(3) if(需要操作的设备的使用计数器 = =0)	程序清单 17.9(3)
(4) {	{
(5) netif_stop_queue(dev);	程序清单 17.9(4)
(6) 关闭设备处理(包括释放 revalidate() 方法获取的资源);	程序清单 17.9(5) ~ (8)
(7)}	}
(8) return 0;	程序清单 17.9(9)

程序清单 17.9(5)、(7)是开关中断的代码,可参考 7.3.2 小节。程序清单 17.9(6)的代码是与用户电路相关的代码,前面已介绍,详情请参考 17.2 节第 1 条。程序清单 17.9(8)为释放驱动程序申请的中断服务程序,具体可参考 7.3.1 小节。

程序清单 17.9 的其他代码都很简单,这里不再说明。

17.3.8　set_config() 方法

本驱动程序的 set_config() 方法代码见程序清单 17.10。可以看出,程序清单 17.10 与程序清单 15.11 完全相同,这里不再说明。

程序清单 **17.10**　**set_config()** 方法代码

```
static int net_config(struct net_device *dev, struct ifmap *map)
{
    return -EBUSY;
}
```

17.3.9 hard_start_xmit () 方法

本驱动程序的 hard_start_xmit () 方法代码见程序清单 17.11。可以看出，程序清单 17.11 完全是按照程序清单 15.12 编写的，两者的对照见表 17.4。

程序清单 **17.11** **hard_start_xmit** () 方法代码

```
static int net_tx(struct sk_buff *skb, struct net_device *dev)
{
    unsigned long flag;
    int len;
    u16 *data;
    board_info_t *db = dev->priv;

    netif_stop_queue(dev);                                          (1)

    len = skb->len < ETH_ZLEN ? ETH_ZLEN : skb->len;                (2)

    data = (u16 *)skb->data;                                        (3)
    len = (len + 1) & (~1);                                         (4)

    local_irq_save(flag);                                           (5)

    iow(db, 0xff, 0x80);                                            (6)
    outb(0xf8, db->ioaddr);                                         (7)

    outsw(db->io_data, data, len >> 1);                             (8)

    iow(db, 0xfc, len & 0xff);                                      (9)
    iow(db, 0xfd, (len >> 8) & 0xff);                               (10)
    iow(db, 0x2, 0x1);                                              (11)
    iow(db, 0xff, 0x83);                                            (12)

    dev->trans_start = jiffies;                                     (13)
    local_irq_restore(flag);                                        (14)

    dev_kfree_skb(skb);                                             (15)
```


更多高清PDF图书 www.gqpdf.com

```
    return 0;                                                          (16)
}
```

表 17.4　hard_start_xmit () 方法的示意代码与驱动代码对照说明

示意代码(程序清单 15.12)	实际代码	说　明
(1) netif_stop_queue(dev);	程序清单 17.11(1)	
(2) len = skb->len < ETH_ZLEN ? ETH_ZLEN : skb->len;	程序清单 17.11(2)	
(3) 把 len 个数据写入网络设备硬件缓冲区并启动发送;	程序清单 17.11(3)~(12)、(14)	
(4) dev->trans_start = jiffies;	程序清单 17.11(13)	
(5) dev_kfree_skb(skb);	程序清单 17.11(15)	
(6) 等待数据发送完毕;	无	用中断实现
(7) netif_wake_queue(dev)	无	在中断服务程序中,见程序清单 17.13(10)
(8) return 0;	程序清单 17.11(16)	

程序清单 17.11(4)是把发送的尺寸转换成偶数字节,这是 DM9000 的要求。程序清单 17.11(5)、(14)是开关中断的代码,可参考 7.3.2 小节。程序清单 17.11(6)~(12)均为调用 I/O 操作函数,这部分请参考 5.2.7 小节。程序清单 17.11(6)~(12)与硬件相关,与编写驱动程序本身关系不大,这里不详细介绍,感兴趣的读者可参考芯片数据手册。

程序清单 17.11 的其他代码都很简单,这里不再说明。

17.3.10　set_mac_address () 方法

本驱动程序的 set_mac_address () 方法代码见程序清单 17.12。可以看出,程序清单 17.12 完全是按照程序清单 15.13 编写的,仅仅是把程序清单 15.13(5)换成了程序清单 17.12(5)~(7)。程序清单 17.12(5)~(7)与硬件相关,与编写驱动程序本身关系不大,这里不详细介绍,感兴趣的读者可参考芯片数据手册。

程序清单 17.12　set_mac_address () 方法代码

```
static int net_set_mac_address(struct net_device *dev, void *addr)
{
    struct sockaddr *mac_addr;
    int i;
    mac_addr = addr;                                                   (1)
    if (netif_running(dev))                                            (2)
```

```
    {
        return -EBUSY;                                          (3)
    }
    memcpy(dev->dev_addr, mac_addr->sa_data, dev->addr_len);
                                                                (4)
    for(i=0; i < 6; i++)                                        (5)
    {
        MyMacID\[i\]=mac_addr->sa_data\[i\];                    (6)
    }
    device_init(dev);                                           (7)

    return 0;                                                   (8)
}
```

17.3.11 中断服务程序

本驱动程序的中断服务程序代码见程序清单 17.13。可以看出，程序清单 17.13 基本是按照程序清单 15.14 编写的，两者的对照见表 17.5。

程序清单 **17.13** 中断服务程序代码

更多高清PDF图书 www.gqpdf.com

```
static void net_irq_handle(int irq, void *dev_id, struct pt_regs *regs)
{
    unsigned int int_status;
    struct net_device *dev;
    board_info_t *db;

    dev = (struct net_device *)dev_id;                          (1)
    db = dev->priv;                                             (2)

    int_status = ior(db, 0xfe);                                 (3)
    iow(db, 0xfe, int_status);                                  (4)
    outl(1 << (dev->irq -14), EXTINT);                          (5)

    if (int_status & DM9000_TX_INTR)                            (6)
    {
        netif_wake_queue(dev);                                  (7)
    }
```

gqpdf.com

```
    if (int_status & DM9000_RX_INTR)                                    (8)
    {
        if (device_rx(dev) == -1)                                       (9)
        {
            DM9000Dev = dev;                                            (10)
            tasklet_schedule(&ZLG_net_tasklet);                         (11)
        }
    }
}
```

表 17.5　中断服务程序示意代码与驱动代码对照说明

示意代码(程序清单 15.14)	实际代码	说　明
(1) dev = (struct net_device *)dev_id;	程序清单 17.13 (1)	
(2) if (发送成功)	程序清单 17.13 (6)	
{	{	
(3) 清除发送中断标志;	程序清单 17.13 (2)	
(4) netif_wake_queue(dev);	程序清单 17.13 (7)	
}	}	
(5) if (接收成功)	程序清单 17.13 (8)	
{	{	
(6) 清除接收中断标志;	程序清单 17.13 (2)	
(7) length = 数据包长度;	程序清单 17.13 (9)	在函数 device_rx () 中实现
(8) skb = dev_alloc_skb (length + 2);		
(9) if (! skb)		
{		
(10) return;		
}		
(11) skb_reserve (skb, 2);		
(12) dec = skb->data;		
(13) 把数据包 (length 个数据) 读到 dec 指向的内存中;		
(14) skb->dev = dev;		
(15) skb->protocol = eth_type_trans (skb, dev);		
(16) skb->ip_summed = CHECKSUM_UNNECESSARY;		
(17) netif_rx (skb);		
(18) dev->last_rx = jiffies;		
}	}	
(19) if (有错误出错)	程序清单 17.13 (9)	
{	{	
(20) 错误处理程序;	程序清单 17.13 (10)、(11)	
}	}	
(21) 清除中断源;	程序清单 17.13 (5)	

更多高清PDF图书　www.gqpdf.com

程序清单 15.14 (7) ~ (18) 由函数 device_rx () 实现。device_rx () 返回 0 表示成功，返回 - 1 表示有错误。由于函数 device_rx () 完全与硬件相关，这里不再介绍，感兴趣的读者可阅读源码和参考芯片数据手册。

至于程序清单 16.13 (23) ~ (25)，涉及 Linux (μClinux) 中断系统的一个重要概念：中断的下半部分。关于这部分请参考 16.3.9 小节。device_rx () (程序清单 17.13 (9)) 执行出错时，表明需要重新初始化 DM9000，而初始化 DM9000 需要较长的时间，所以需要中断的下半部分。

本驱动程序使用 tasklet 方式实现中断的下半部分，其相关代码见程序清单 17.14。

注意：DM9000Dev (1)、(2) 和其他部分之间在源代码中分散在不同的地方。

参考 16.3.9 小节这部分很容易读懂，这里不详细说明了。

程序清单 **17.14**　中断下半部分相关代码

```
static struct net_device *DM9000Dev;                                    (1)

DECLARE_TASKLET(ZLG_net_tasklet, net_tasklet, (unsigned long) (&DM9000Dev));
                                                                        (2)

static void net_tasklet(unsigned long data)                             (3)
{
    struct net_device *dev;

    dev = *((struct net_device **)data);                                (4)
    device_init(dev);                                                   (5)
}
```

本章小结

本章以 DM9000 网络接口芯片为例介绍了 Linux (μClinux) 网络驱动程序的编写方法和相关知识，包括源码分析和编译使用。

附录A 版权声明及许可协议

1. 关于版权

本书提到的设备驱动程序及本书的大部分文字、图片、表格等内容的著作权、版权和知识产权属于广州致远电子有限公司所有，并受《中华人民共和国著作权法》、《计算机软件保护条理》、《知识产权保护条例》和相关国际版权条约、法律、法规，以及其他知识产权法律和条约的保护。本书中部分文字、图片、表格的著作权、版权和知识产权不完全属于广州致远电子有限公司，这些部分将在本文最后列出。

本书提到的驱动程序的知识产权和其他所有权利属于广州致远电子有限公司所有。任何单位和个人未经广州致远电子有限公司同意，不能使用、修改、再发布设备驱动程序的任何部分；否则将视为非法侵害，广州致远电子有限公司保留依法追究其责任的权利，此条款同样适用于本书中广州致远电子有限公司拥有完全权利的文字、图片、表格等内容。

2. 关于修改与升级

广州致远电子有限公司保留在不通知用户的情况下对本书及本书提到的设备驱动程序进行修改、升级的权力，并保留对升级版本修改许可协议的权力。

3. 关于许可协议

用户可通过两种途径获取本书提到的设备驱动程序的使用许可证：

（1）在授权经销商处购买 SmartARM2200、MagicARM2200、MagicARM2200－S 教学实验开发平台（第 17 章提到的设备驱动程序不是针对所有套件，与具体型号有关）。

SmartARM2200、MagicARM2200、MagicARM2200－S 教学实验开发平台是广州致远电子有限公司开发的 ARM7 开发学习套件，一套 SmartARM2200、MagicARM2200、MagicARM2200－S 教学实验开发平台搭配一份许可证，用户在广州致远电子有限公司授权的经销商处购买即可获得。一份许可证可在一个项目或产品中使用这些驱动程序。许可证不可转让，且以用户在广州致远电子有限公司登记的名称为准。一旦用户购买了 SmartARM2200、MagicARM2200、MagicARM2200－S 教学实验开发平台，表示接受了本许可协议。如果不同意本许可协议，请将其退回原购买处。

（2）直接与制造商联系。

更多高清PDF图书 www.gqpdf.com

4. 没有担保

由于本书提到的软件只是最终产品的程序的一部分，SmartARM2200 不提供任何类型的担保。包括但不限于合适特定用途的保证。全部的风险由使用者来承担。如果程序出现缺陷，使用者承担所有必要的服务、修改和改正的费用。

注 1：本书提到的软件是指以下部分：第 4 ~ 17 章提到的设备驱动程序。

2：本书中广州致远电子有限公司不完全拥有所有权的部分：第 1 ~ 3 章。

广州致远电子有限公司联系方式如下：

地址：广州市天河区东陂路黄洲工业区三栋二楼

邮编：510660

电话：(020) 22644399

传真：(020) 38601859

联系人：周先生

高清 PDF 藏 gqpdf.com

附录B SmartARM2200 教学实验开发平台

B.1 产品概述

SmartARM2200 是一款基于 LPC2200 系列 ARM 微控制器的、功能强大的 ARM7 教学实验开发平台。它支持多种嵌入式操作系统，提供丰富的配套教材和众多的实验例程，并提供多种商业化软件包，还配备了精心设计的多媒体教学课件，既适用于进行 ARM7 嵌入式系统开发的工程师，也适用于各高等院校的嵌入式系统教学。

SmartARM2200 产品外观图如图 B.1 所示。

图 B.1 SmartARM2200 产品外观图

更多高清PDF图书 www.gqpdf.com

SmartARM2200 标准配置有 ISP1161BM PACK 板，其外观图如图 B.2 所示。

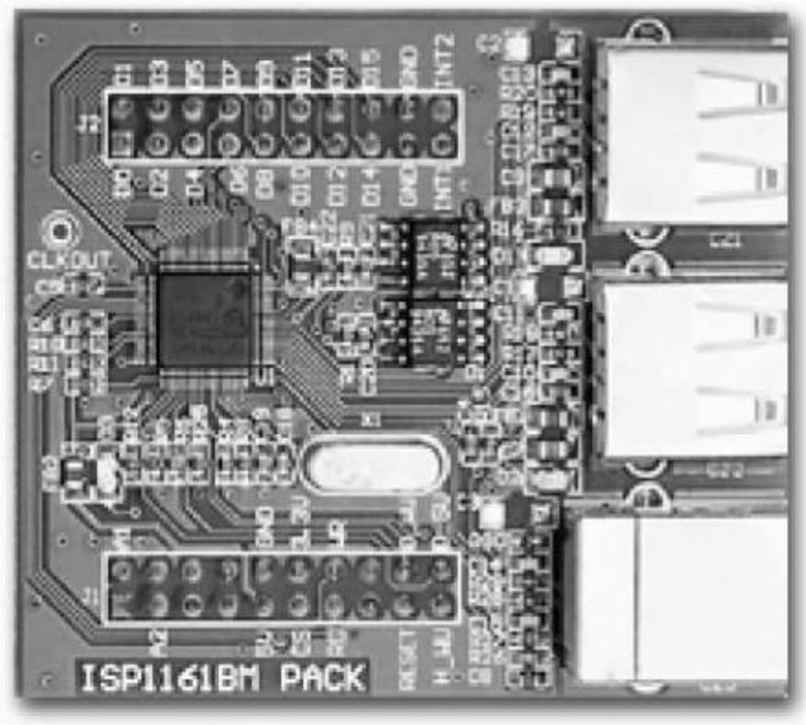

图 B.2　ISP1161BM PACK 板外观图

B.2　产品目标

SmartARM2200 是为了给用户提供一个学习 32 位 ARM 嵌入式系统的教学实验开发平台而开发的。其硬件上设计有充足的存储资源，可运行 μClinux、μC/OS－II 双操作系统和其他较大规模的用户程序；设计有嵌入式系统常见的功能电路，如彩色液晶屏、数码管、键盘和温度测量等；还有 RS－232 接口、Modem 接口、以太网接口、CF 卡接口、IDE 硬盘接口和 SD 卡接口等；软件上提供了 μC/OS－II & μClinux 移植代码、多种商业化软件包（如 ZLG/FS、ZLG/IP 和 ZLG/PPP）、μClinux 驱动程序和众多实验例程，并提供了生动的多媒体教学课件，形成一个较为完整的、切合实际应用的 ARM 嵌入式系统的教学实验开发平台。

B.3　产品特点

1. SmartARM2200 功能特点

- 支持 μClinux、μC/OS－II 双操作系统，搭建用户系统平台更方便。
- 完全自主设计的软硬件，拥有自主版权的 JTAG 仿真技术，支持 ADS1.2 集成开发环境及 PHILIPS 所有通用 ARM 微控制器的仿真和开发。
- 采用“主板＋CPU PACK 适配器＋多种可选配置适配器”的形式构成 SmartARM2200 教学实验开发平台。标准配置的 CPU PACK 主芯片为 LPC2210FBD，附送空 CPU PACK 一块。
- 标准配置有 TFT6758 PACK 板，板上包含一个 2.2 英寸 240×RGB×320 TFT 液晶屏，

真彩色 262K 色，用户界面更华丽。

- 主板核心配置为 2 MB NOR Flash（可用于存放 BootLoader）、8 MB RAM、16 MB NAND Flash 和 256 字节 E^2PROM。
- 全面支持 11 种型号的 64 引脚和 144 引脚 ARM7 微控制器：
  — LPC2100（LPC2114/2124/2119/2129/2194）；
  — LPC2200（LPC2210/2212/2214/2290/2292/2294）。
- 板上的功能部件与 CPU 之间可使用跳线器选择连接。
- 具有外设 PACK 和 GPIO 输出接口，方便用户连接外部电路的开发和使用。
- 标准配置有 ISP1161 PACK 板，支持 USB1.1 HOST/Device 固件编程与驱动程序的开发。
- 具有 SD 卡接口，支持 SD/MMC 卡读/写。
- 具有 RTL8019AS 网卡芯片。
- 具有 IDE 硬盘接口和 CF 存储卡接口。
- 可与标准 Modem 直接相接，进行远程数据通信。
- 具有 RS－232 转换电路，可与上位机进行串行通信，完成 UART 通信实验。
- 具有 3 个 I^2C 器件，完成 I^2C 总线的实验。
- 具有温度测量电路，可用于温度控制相关实验。
- 4 个独立 LED，17 个按键（包含 1 个独立按键），1 个蜂鸣器。
- 多种可选配置适配器：
  — 可自制各种型号 CPU PACK，按需求和喜好配置主 ARM 芯片；
  — MG240128 单色点阵图形液晶模块；
  — CAN 接口板，方便组装现场总线；
  — GPS 模块，进行地理定位实验；
  — GPRS 模块，进行 GPRS 实验；
  — PDIUSBD12 USB1.1 Device PCAK；
  — ISP1181 USB1.1 Device PACK；
  — SP1160 USB1.1 HOST PACK。
- 多种免费商业化软件包及其详细的开发文档：
  — 移植 μC/OS－II 到 ARM7 软件包；
  — 移植 μClinux 到 LPC2200 软件包；
  — 数据队列软件包；
  — 串口驱动软件包；
  — Modem 接口软件包；
  — SPI 总线软件包；

更多高清PDF图书 www.gqpdf.com

— I^2C 总线软件包；
— ZLG/GPS 软件包(选配)；
— ZLG/GPRS 软件包(选配)；
— ZLG/FS 文件管理系统软件包；
— MiniGUI 图形用户界面学习版软件包(μC/OS－II & μClinux)；
— ZLG/IP TCP/IP 软件包；
— ZLG/PPP 协议软件包；
— ZLG/CF CF 卡及 IDE 硬盘软件包；
— ZLG/SD SD/MMC 读/写软件包；
— ZLG/FFS 支持写平衡的 NAND Flash 驱动软件包；
— ZLG/HOST & Device 固件程序及其驱动程序软件包；
— ZLG/FTP－S 嵌入式 FTP 服务器软件包；
— ZLG/BOOT BootLoader 程序。

- 提供 μClinux 驱动程序编程实验和 μClinux 应用程序编程实例。
- 自主设计的 ZLG/BOOT BootLoader 程序，使用 ZLG/FS 文件管理系统，支持 FTP 下载，下载程序速度更快。
- 提供基于 μC/OS－II & μClinux 的实验例子，如串口通信、I^2C 存储器读/写和Modem通信等。
- 提供 MiniGUI for μC/OS－II 和 MiniGUI for μClinux 移植实验及 MiniGUI 应用实例，免费提供 MiniGUI－STR 软件包。
- 提供基于 PC 的人机界面软件，方便调试实时时钟和串口通信等功能。
- 可进行 GPIO 的控制实验，如 LED 闪烁控制、按键输入和蜂鸣器控制等。
- 可进行外部中断实验，学习向量中断控制器(VIC)。
- 可进行定时器控制实验和实时时钟控制实验。
- 可进行 WDT 及低功耗控制实验。
- 可进行 EMC 设置实验、REMAP 操作实验和 ADC 数据采集实验。
- 大量的实验程序，详细的配置教材(高等学校嵌入式系统通用教材——《ARM 嵌入式系统系列教材》，我们还针对 SmartARM2200 & MagicARM2200 教学实验开发平台配套编写了《ARM 嵌入式 Linux 系统构建与驱动开发范例》与《ARM 嵌入式 MiniGUI 初步与应用开发范例》两本专著)，以及完整的多媒体教学课件。

2. SmartARM2200 主板硬件特点

1) 主　板

主板采用 4 层 PCB 板，抗干扰能力强。

2）板上资源

- 2 MB NOR Flash；
- 8 MB PSRAM；
- 16 MB NAND Flash；
- 256 字节 E^2PROM；
- 全面支持 7 种型号的 144 引脚 ARM7 微控制器：LPC2200（LPC2210/2220/2212/2214/2290/2292/2294）。

3）众多扩展部件

- CF 卡/IDE 硬盘接口；
- SD/MMC 卡模块；
- 2 路 10 位 A/D；
- LM75 数字温度传感器；
- ZLG7290 键盘管理和显示控制；
- MG240128 单色点阵图形液晶接口；
- GPIO 引出，方便功能扩展；
- 各功能部件可通过跳线灵活选择。

4）多种通信接口

- 双串口，串口 1 带 Modem 接口；
- 10M 以太网接口；
- 2 个 USB HOST 2.0（全速）；
- 1 个 USB Device 2.0（全速）。

5）多种人机交互界面

- 1 个 2.2 英寸 240 ×320 262K 色彩色液晶屏；
- 4 个独立 LED；
- 1 个蜂鸣器；
- 1 个独立按键；
- 4 ×4 矩阵键盘。

6）精美布局，人性化设计

- 精心设计外壳，完美感观享受；
- 专业、进取完美结合。

更多高清PDF图书 www.gqpdf.com

B.4 产品组成

1. 硬件部分

产品硬件部分组成如表 B.1 所列。

表 B.1 产品硬件部分组成

序 号	名 称	数 量	单 位	备 注
1	SmartARM2200 开发板	1	块	CPU 用 LPC2210
2	EasyJTAG 仿真头	1	只	
3	ISP1161BM PACK 板	1	块	
4	TFT6758 PACK 板	1	块	TFT 液晶屏
5	20P 仿真电缆	1	条	
6	USB 线	1	条	2H/USB - A + USB - B
7	RS－232 串行通信线	1	条	
8	并口延长线	1	条	
9	以太网交叉电缆	1	条	
10	DC 9 V 电源适配器	1	只	
11	产品配套光盘	1	张	
12	产品保修卡	1	张	
13	CPU PACK(不含 CPU)	1	块	附送
14	《ARM 嵌入式系统基础教程》	1	本	
15	《ARM 嵌入式系统实验教程(二)》	1	本	

2. 软件部分

软件部分包括如下各种软件包和实验程序(在产品配套光盘上):

- 移植 μC/OS－II 到 ARM7 软件包;
- 移植 μClinux 到 LPC2200 软件包;
- 数据队列软件包;
- 串口驱动软件包;
- Modem 接口软件包;
- SPI 总线软件包;
- I^2C 总线软件包;
- ZLG/GPS 软件包(选配);
- ZLG/GPRS 软件包(选配);
- ZLG/FS 文件管理系统软件包;

高清PDF藏 gqpdf.com

- MiniGUI 图形用户界面学习版软件包（μC/OS－II & μClinux）；
- ZLG/IP TCP/IP 软件包；
- ZLG/PPP 协议软件包；
- ZLG/CF CF 卡及 IDE 硬盘软件包；
- ZLG/SD SD/MMC 读/写软件包；
- ZLG/FFS 支持写平衡的 NAND Flash 驱动软件包；
- ZLG/HOST & Device 固件程序及其驱动程序软件包；
- ZLG/FTP－S 嵌入式 FTP 服务器软件包；
- ZLG/BOOT BootLoader 程序。

B.5 硬件结构说明

详见参考文献[7]的第 1 章。

B.6 应用环境

硬件环境：带有 USB、10M/100M 以太网、9 针串口和 25 针并口的 PC 机；根据实验需求，可能会用到 CF 卡、SD 卡、IDE 硬盘、U 盘和 Modem 等。

软件环境：Windows 98/XP/2000 操作系统、RedHat 9.0 操作系统和 ADS1.2 集成开发环境等。

B.7 产品部件规格描述

产品部件规格描述如表 B.2 所列。

表 B.2 产品部件规格描述

主要部件名称	参数/规格	备 注
SmartARM2200 开发板	CPU：LPC2210 RAM：MT45W4MW16 NOR Flash：SST39VF1601 NAND Flash：K9F2808U0C	
TFT 液晶屏	2.2 英寸 240×320 液晶屏	真彩色 262K 色
ISP1161 PACK 板	芯片：ISP1161	按 SmartARM2200 开发板上的外设 PACK 接口设计
GPRS PACK 板	核心：BENQ 公司的 M22 GPRS 模块	选配的模件

更多高清PDF图书 www.gqpdf.com

附录C MagicARM2200 教学实验开发平台

C.1 产品概述

MagicARM2200 是专为高等院校嵌入式系统教学精心打造的 ARM 嵌入式教学实验开发平台，支持多种嵌入式操作系统，提供丰富的配套教材和众多的实验例程，提供多种商业化软件包，并配备精心设计的多媒体教学课件，是高等院校进行嵌入式系统教学的最佳选择。

MagicARM2200 产品外观图如图 C.1 所示。

图 C.1　MagicARM2200 产品外观图

C.2 产品目标

MagicARM2200 是为了给用户提供一个学习 32 位 ARM 嵌入式系统的教学实验开发平台而开发的。其硬件上设计有充足的存储资源，可运行 μClinux、μC/OS－II 双操作系统和其他较大规模的用户程序；设计有嵌入式系统常见的功能电路，如彩色液晶屏（触摸屏）、数码管、键盘、直流电机和步进电机等；还有 RS－232 接口、Modem 接口、RS－485 接口、GPRS 接口、CF 卡接口、IDE 硬盘接口和 SD 卡接口等；软件上提供了 μC/OS－II & μClinux 移植代码、多种商业化软件包（如 ZLG/FS、ZLG/IP 和 ZLG/PPP）、μClinux 驱动程序和众多实验例程，并提供了生动的多媒体教学课件，形成一个较为完整的、切合实际应用的 ARM 嵌入式系统的教学实验开发平台。

C.3 产品特点

1. MagicARM2200 功能特点

- 支持 μClinux、μC/OS－II 双操作系统，搭建用户系统平台更方便。
- 完全自主设计的软硬件，拥有自主版权的 JTAG 仿真技术，支持 ADS1.2 集成开发环境及 PHILIPS 所有通用 ARM 微控制器的仿真和开发。
- 采用 DeviceARM2200 嵌入式工控板（即核心板），标准配置为 PHILIPS 公司的 ARM7TDMI－S 微控制器 LPC2290、2 MB NOR Flash（BootLoader）、8 MB RAM、16 MB NAND Flash 和 256 字节 E^2PROM。
- 标准配置 5.2 英寸 320×240 彩色液晶屏，带触摸屏，人机界面更华丽。
- 全面支持 6 种型号的 144 引脚 ARM7 微控制器：LPC2200（LPC2210/2212/2214/2290/2292/2294）。
- 灵活的跳线器选择，让用户可以重新分配 I/O 资源。
- 具有外设 PACK 和 GPIO 输出接口，方便用户连接外部电路的开发和使用。
- 标准配置有 ISP1161 PACK 板，支持 USB1.1 HOST/Device 固件编程与驱动程序的开发。
- 具有 SD 卡接口，支持 SD/MMC 卡读/写。
- 具有 DM9000E 网卡芯片，支持 10/100M 以太网通信。
- 具有 IDE 硬盘接口和 CF 存储卡接口。
- 可与标准 Modem 直接连接，进行远程数据通信。
- 具有 RS－232 转换电路，可与上位机进行串行通信，完成 UART 通信实验。
- 包含 2 路 CAN 接口，方便组装现场总线。

更多高清PDF图书 www.gqpdf.com

- 多种可选配置适配器：
  — GPS 模块，进行地理定位实验；
  — GPRS 模块，进行 GPRS 实验；
  — PDIUSBD12 USB1.1 Device PCAK；
  — ISP1181 USB1.1 Device PACK；
  — ISP1160 USB1.1 HOST PACK。
- 使用 ZLG7290 扩展了 8 位动态 LED 数码管和一个 16 键小键盘。采用德国 CHERRY 按键，具有极佳的手感和超长的使用寿命。
- 8 个独立 LED 灯，1 个独立按键，1 个蜂鸣器。
- 包含 1 路 RS－485 接口电路，可做 RS－485 通信实验。
- 具有打印机接口电路，提供打印机软件包。
- 具有 PS/2 键盘、鼠标接口和扩展系统的输入设备。
- 串行 DAC 电路，可同步输出 4 路 D/A 信号。
- 具有四相步进电机和直流电机驱动电路，可做步进电机、直流电机控制实验。
- 多种免费商业化软件包及其详细的开发文档：
  — 移植 μC/OS－II 到 ARM7 软件包；
  — 移植 μClinux 到 LPC2200 软件包；
  — 数据队列软件包；
  — 串口驱动软件包；
  — Modem 接口软件包；
  — SPI 总线软件包；
  — I^2C 总线软件包；
  — 支持 RS－485/422 MODBUS RTU 协议软件包；
  — CAN－bus 协议软件包；
  — 打印机接口软件包；
  — ZLG/GPS 软件包（选配）；
  — ZLG/GPRS 软件包（选配）；
  — ZLG/FS 文件管理系统软件包；
  — MiniGUI 图形用户界面学习版软件包（μC/OS－II & μClinux）；
  — ZLG/IP TCP/IP 软件包；
  — ZLG/PPP 协议软件包；
  — ZLG/CF CF 卡及 IDE 硬盘软件包；
  — ZLG/SD SD/MMC 读/写软件包；
  — ZLG/FFS 支持写平衡的 NAND Flash 驱动软件包；

高清PDF藏 qqpdf.com

— ZLG/HOST & Device 固件程序及其驱动程序软件包；

— ZLG/FTP－S 嵌入式 FTP 服务器软件包；

— ZLG/BOOT BootLoader 程序。

- 提供 μClinux 驱动程序编程实验和 μClinux 应用程序编程实例。
- 自主设计的 ZLG/BOOT BootLoader 程序，使用 ZLG/FS 文件管理系统，支持 FTP 下载，下载程序速度更快。
- 提供基于 μC/OS－II & μClinux 的实验例子，如串口通信、I^2C 存储器读/写和 Modem 通信等。
- 提供 MiniGUI for μClinux 移植实验和 MiniGUI 应用实例，免费提供 MiniGUI-STR 软件包。
- 提供基于 PC 的人机界面软件，方便调试实时时钟和串口通信等功能。
- 可进行 GPIO 的控制实验，如 LED 闪烁控制、按键输入和蜂鸣器控制等。
- 可进行外部中断实验，学习向量中断控制器（VIC）。
- 使用实验箱主板内的 I^2C 器件，可完成 I^2C 总线的实验。
- 可进行定时器控制实验和实时时钟控制实验。
- 可进行 WDT 及低功耗控制实验。
- 可进行 EMC 设置实验、REMAP 操作实验和 ADC 数据采集实验。
- 大量的实验程序，详细的配置教材（高等学校嵌入式系统通用教材——《ARM 嵌入式系统系列教材》，针对 SmartARM2200 & MagicARM2200 教学实验平台配套编写了《ARM 嵌入式 Linux 系统构建与驱动开发范例》与《ARM 嵌入式 MiniGUI 初步与应用开发范例》两本专著），以及完整的多媒体教学课件。

2. MagicARM2200 主板硬件特点

1）核心板

核心板采用 DeviceARM2200 工控板（6 层 PCB 设计），其特点如下：

- 2 MB NOR Flash；
- 8 MB PSRAM；
- 16 MB NAND Flash；
- 256 字节 E^2PROM；
- 采用标准 SO－DIMM144 接口；
- 全面支持 7 种型号的 144 引脚 ARM7 微控制器：LPC2200（LPC2210/2220/2212/2214/2290/2292/2294）。

2）主　板

主板采用 4 层 PCB 板，抗干扰能力强。

更多高清PDF图书 www.gqpdf.com

3）众多扩展部件

- CF 卡/IDE 硬盘接口；
- SD/MMC 卡模块；
- 1 个打印机接口；
- 2 路 10 位 A/D；
- 4 路可同步串行 D/A；
- ZLG7290 键盘管理和显示控制；
- 1 个直流电机；
- 1 个步进电机；
- 各功能部件可通过跳线灵活选择。

4）多种通信接口

- 双串口，串口 1 带 Modem 接口；
- 10M/100M 以太网接口；
- 2 个 USB HOST 2.0（全速）；
- 1 个 USB Device 2.0（全速）；
- 1 路 RS－485；
- 2 路 CAN。

5）多种人机交互界面

- 1 个 5.2 寸 320×240 彩色液晶屏；
- 4 线电阻式触摸屏，带触摸笔；
- 8 个独立 LED；
- 1 个蜂鸣器；
- 8 个 7 段数码管；
- 1 个独立按键；
- 16 个德国 CHERRY 按键，超长使用寿命，手感极佳；
- PS/2 键盘和鼠标接口。

6）精美布局，人性化设计

- 精心设计液晶屏模具，完美感观享受；
- 精心设计触摸笔，独显个性；
- 专业、进取完美结合。

3. 多种可选核心板

- ARM7 微处理器（LPC2210/2220/2212/2214/2290/2292/2294 ARM 微处理器），其外观如图 C.2 所示。

高清PDF藏 gaopdf.com

- ARM9 微处理器(LPC3000 系列 ARM 微处理器),其外观图如图 C.3 所示。
- Altera Nios II 软核 SOPC 32 位 CPU 处理器,其外观图如图 C.4 所示。
- Actel ARM7 硬核 SoC 处理器,其外观图如图 C.5 所示。

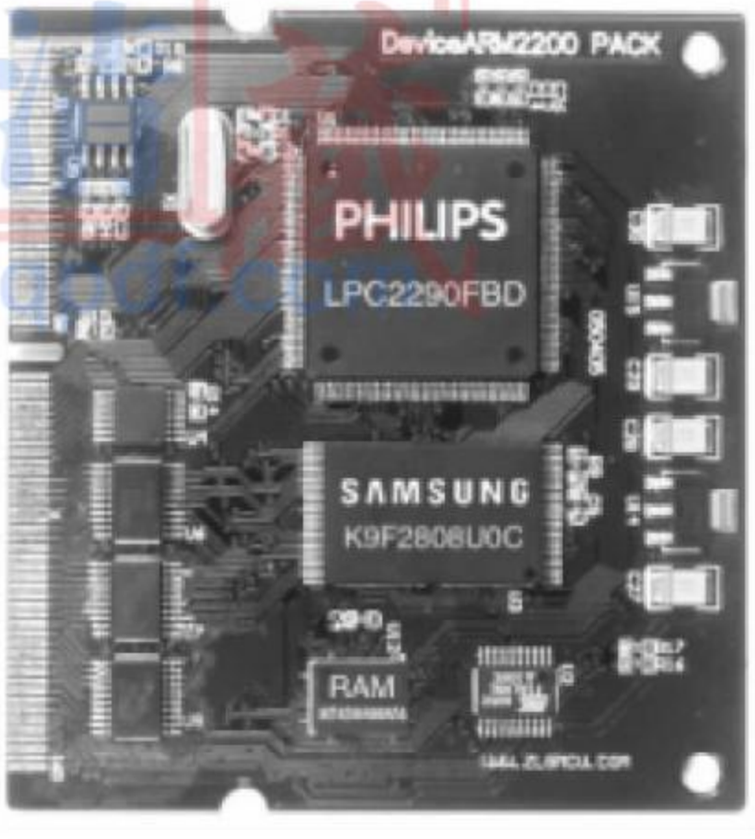

图 C.2　DeviceARM2200(标配)

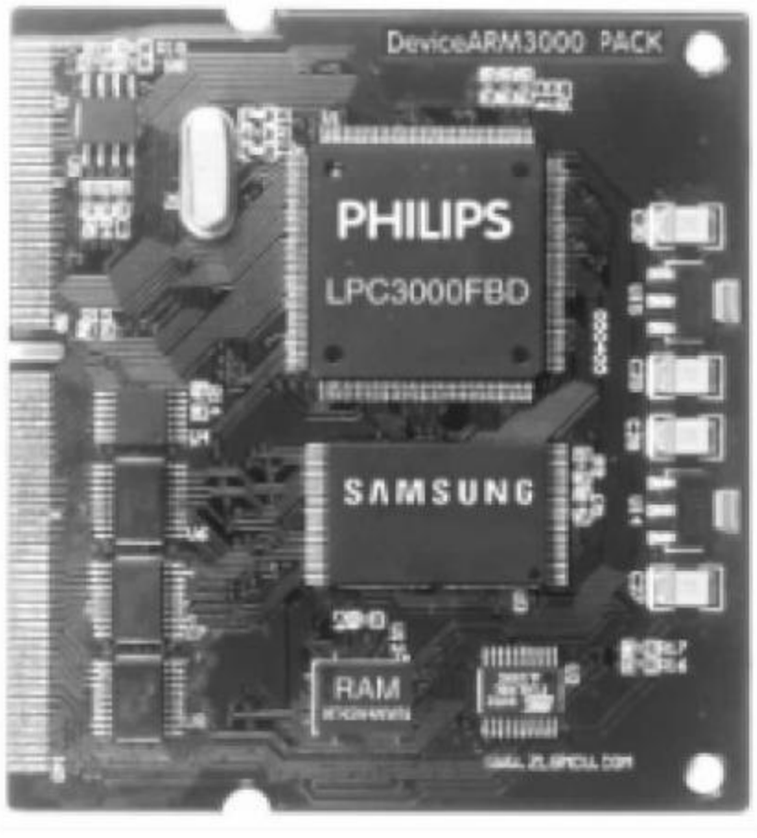

图 C.3　DeviceARM3000(选配)

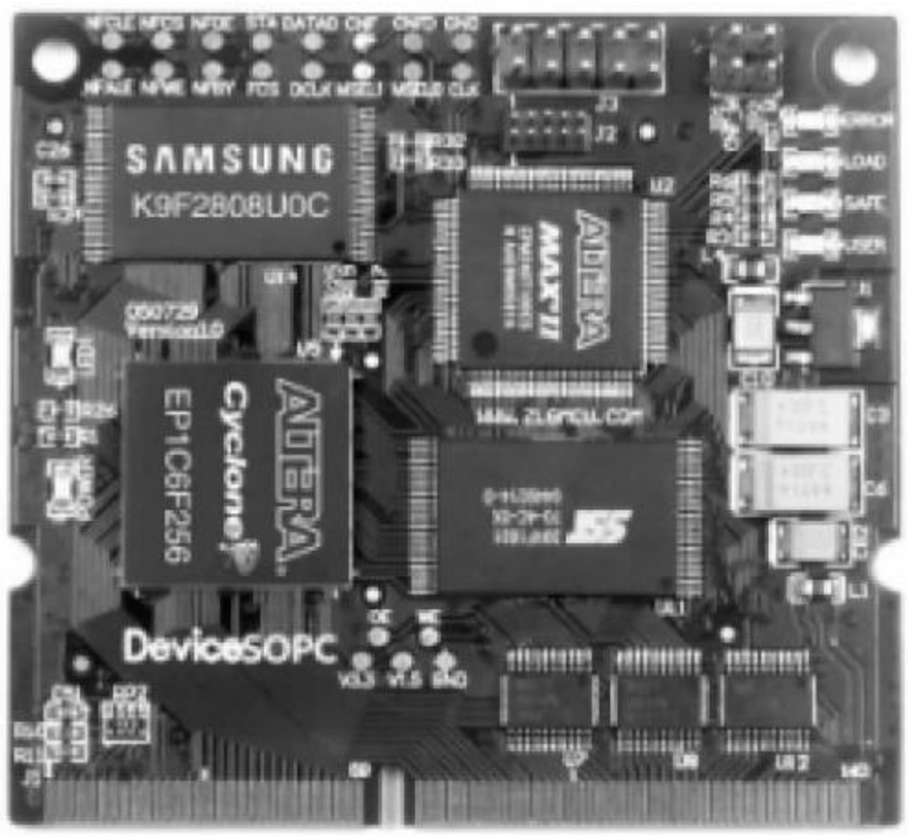

图 C.4　Altera Nios II SOPC(标配)

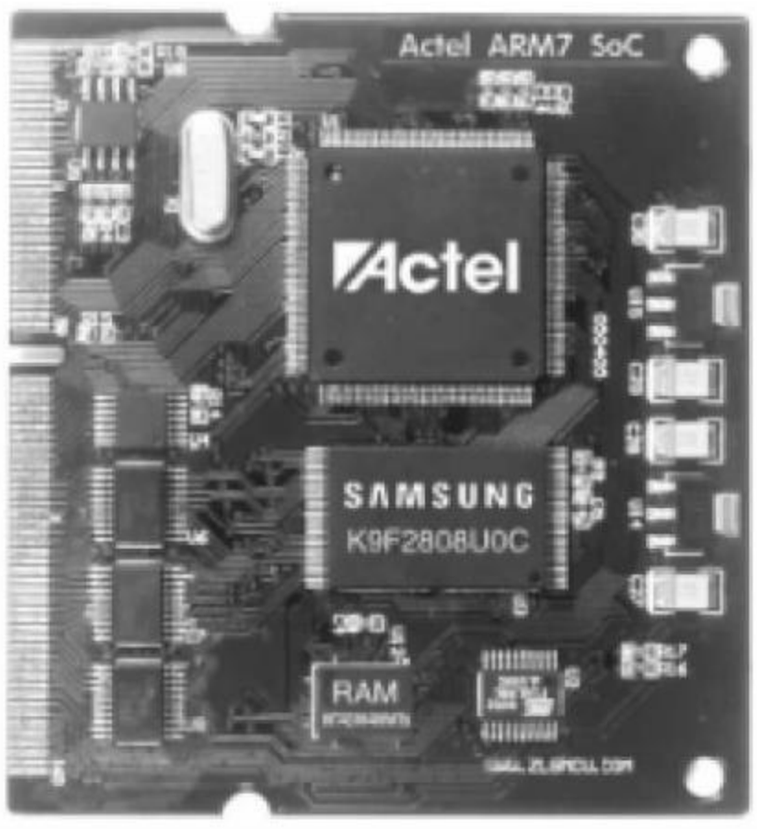

图 C.5　Actel ARM7 SoC(选配)

更多高清PDF图书　www.gqpdf.com

C.4 产品组成

1. 硬件部分

产品硬件部分组成如表 C.1 所列。

表 C.1 产品硬件部分组成

序 号	名 称	数 量	单 位	备 注
1	MagicARM2200 实验箱	1	台	包括主板、电源和机箱
2	EasyJTAG 仿真头	1	只	
3	DeviceARM2200 嵌入式工控板	1	块	CPU 用 LPC2290
4	ISP1161BM PACK 板	1	块	
5	20P 仿真电缆	1	条	
6	USB 线	1	条	2H/USB - A + USB - B
7	RS－232 串行通信线	1	条	
8	并口延长线	1	条	
9	以太网交叉电缆	1	条	
10	电源线	1	条	
11	产品光盘	1	张	
12	产品保修卡	1	张	
13	《ARM 嵌入式系统基础教程》	1	本	
14	《ARM 嵌入式系统实验教程(三)》	1	本	
15	《ARM 嵌入式系统实验教程(三)——扩展实验》	1	本	

2. 软件部分

软件部分包括如下各种软件包和实验程序(在产品配套光盘上):

- 移植 μC/OS－II 到 ARM7 软件包;
- 移植 μClinux 到 LPC2200 软件包;
- 数据队列软件包;
- 串口驱动软件包;
- Modem 接口软件包;
- SPI 总线软件包;
- I^2C 总线软件包;
- 支持 RS－485/422 MODBUS RTU 协议软件包;
- CAN－bus 协议软件包;
- 打印机接口软件包;
- ZLG/GPS 软件包(选配);

- ZLG/GPRS 软件包(选配);
- ZLG/FS 文件管理系统软件包;
- MiniGUI 图形用户界面学习版软件包(μC/OS－II & μClinux);
- ZLG/IP TCP/IP 软件包;
- ZLG/PPP 协议软件包;
- ZLG/CF CF 卡及 IDE 硬盘软件包;
- ZLG/SD SD/MMC 读/写软件包;
- ZLG/FFS 支持写平衡的 NAND Flash 驱动软件包;
- ZLG/HOST & Device 固件程序及其驱动程序软件包;
- ZLG/FTP－S 嵌入式 FTP 服务器软件包;
- ZLG/BOOT BootLoader 程序。

C.5 硬件结构说明

详见参考文献[8]的第 1 章。

C.6 应用环境

硬件环境:带有 USB、10M/100M 以太网、9 针串口和 25 针并口的 PC 机;根据实验需求,可能会用到 CF 卡、IDE 硬盘、U 盘和 Modem 等。

软件环境:Windows 98/XP/2000 操作系统、RedHat 9.0 操作系统和 ADS1.2 集成开发环境等。

C.7 产品部件规格描述

产品部件规格描述如表 C.2 所列。

表 C.2 产品部件规格描述

主要部件名称	参数/规格	备 注
DeviceARM2200 嵌入式工控板	CPU:LPC2290 RAM:MT45W4MW16 NOR Flash:SST39VF1601 NAND Flash:K9F2808U0C	核心板
液晶屏	5.2 英寸 320×240 彩色液晶屏	带触摸屏(4 线电阻式)
ISP1161 PACK 板	芯片:ISP1161	按 MagicARM2200 主板上的外设 PACK 接口设计
GPRS PACK 板	核心:BENQ 公司的 M22 GPRS 模块	选配的模件

更多高清PDF图书 www.gqpdf.com

附录D　MagicARM2200－S 教学实验开发平台

D.1　产品概述

MagicARM2200－S 是专为高等院校嵌入式系统教学精心打造的 ARM 嵌入式教学实验开发平台，支持多种嵌入式操作系统，提供丰富的配套教材和众多的实验例程，提供多种商业化软件包，并配备精心设计的多媒体教学课件，是高校进行嵌入式系统教学的最佳选择。

MagicARM2200－S 产品外观图如图 D.1 所示。

图 D.1　MagicARM2200－S 产品外观图

D.2 产品目标

MagicARM2200 - S 是为了给用户提供一个学习 32 位 ARM 嵌入式系统的教学实验开发平台而开发的。其硬件上设计有充足的存储资源，可运行 μClinux、μC/OS - II 双操作系统和其他较大规模的用户程序；设计有嵌入式系统常见的功能电路，如彩色液晶屏（触摸屏）、数码管、键盘和直流电机等；还有 RS - 232 接口、Modem 接口、GPRS 接口、CF 卡接口、IDE 硬盘接口和以太网接口等；软件上提供了 μC/OS - II & μClinux 移植代码、多种商业化软件包（如 ZLG/FS、ZLG/IP 和 ZLG/PPP）、μClinux 驱动程序和众多实验例程，并提供了生动的多媒体教学课件，形成一个较为完整的、切合实际应用的 ARM 嵌入式系统的教学实验开发平台。

D.3 产品特点

1. MagicARM2200 - S 功能特点

- 支持 μClinux、μC/OS - II 双操作系统，搭建用户系统平台更方便。
- 完全自主设计的软硬件，拥有自主版权的 JTAG 仿真技术，支持 ADS1.2 集成开发环境及 PHILIPS 所有通用 ARM 微控制器的仿真和开发。
- 采用 DeviceARM2200 嵌入式工控板（即核心板），标准配置为 PHILIPS 公司的 ARM7TDMI - S 微控制器 LPC2210、2 MB NOR Flash（BootLoader）、8 MB RAM、16 MB NAND Flash 和 256 字节 E^2PROM。
- 标准配置有 5.2 英寸 320 ×240 彩色液晶屏，带触摸屏，人机界面更华丽。
- 全面支持 7 种型号的 144 引脚 ARM7 微控制器：LPC2200（LPC2210/2220/2212/2214/2290/2292/2294）。
- 灵活的跳线器选择，让用户可重新分配 I/O 资源。
- 具有外设 PACK 和 GPIO 输出接口，方便用户连接外部电路的开发和使用。
- 标准配置有 D12 USB PACK 板，提供移动硬盘软件包。
- 具有 RT8019AS 网卡芯片，提供 TCP/IP 软件包。
- 具有 IDE 硬盘接口和 CF 存储卡接口。
- 可与标准 Modem 直接连接，进行远程数据通信。
- 具有 RS - 232 转换电路，可与上位机进行串行通信，完成 UART 通信实验。
- 多种可选配置适配器：
  - — GPS 模块，进行地理定位实验；
  - — GPRS 模块，进行 GPRS 实验；
  - — ISP1161BM PACK；

更多高清PDF图书 www.gqpdf.com

— ISP1181 USB1.1 Device PACK;
— ISP1160 USB1.1 HOST PACK;
— ARM-CAN 接口板;
— SD CARD PACK。

- 使用 ZLG7290 扩展了 8 位动态 LED 数码管和一个 16 键小键盘。
- 8 个独立 LED 灯,1 个独立按键,1 个蜂鸣器。
- 提供打印机接口电路图,提供打印机软件包。
- 具有 PS/2 键盘、鼠标接口和扩展系统的输入设备。
- 具有 PWM 滤波电路,实现 PWM D/A 转换。
- 具有直流电机驱动电路,可做直流电机控制实验。
- 多种免费商业化软件包及其详细的开发文档:
  — 移植 μC/OS-II 到 ARM7 软件包;
  — 移植 μClinux 到 LPC2200 软件包;
  — 数据队列软件包;
  — 串口驱动软件包;
  — Modem 接口软件包;
  — SPI 总线软件包;
  — I^2C 总线软件包;
  — 打印机接口软件包;
  — ZLG/GPS 软件包(选配);
  — ZLG/GPRS 软件包(选配);
  — ZLG/FS 文件管理系统软件包;
  — MiniGUI 图形用户界面学习版软件包(μC/OS-II & μClinux);
  — ZLG/IP TCP/IP 软件包;
  — ZLG/PPP 协议软件包;
  — ZLG/CF CF 卡及 IDE 硬盘软件包;
  — ZLG/SD SD/MMC 读/写软件包(选配);
  — ZLG/FFS 支持写平衡的 NAND Flash 驱动软件包;
  — ZLG/FTP-S 嵌入式 FTP 服务器软件包;
  — ZLG/BOOT BootLoader 程序;
  — ZLG/USBD12 固件程序及其驱动程序软件包。
- 提供 μClinux 驱动程序编程实验和 μClinux 应用程序编程实例。
- 自主设计的 ZLG/BOOT BootLoader 程序,使用 ZLG/FS 文件管理系统,支持 FTP 下载,下载程序速度更快。

- 提供基于 μC/OS - II & μClinux 的实验例子，如串口通信、I^2C 存储器读/写和 Modem 通信等。
- 提供 MiniGUI for μClinux 移植实验和 MiniGUI 应用实例，免费提供 MiniGUI - STR 软件包。
- 提供基于 PC 的人机界面软件，方便调试实时时钟和串口通信等功能。
- 可进行 GPIO 的控制实验，如 LED 闪烁控制、按键输入和蜂鸣器控制等。
- 可进行外部中断实验，学习向量中断控制器（VIC）。
- 使用实验箱主板内的 I^2C 器件，可完成 I^2C 总线的实验。
- 可进行定时器控制实验和实时时钟控制实验。
- 可进行 WDT 及低功耗控制实验。
- 可进行 EMC 设置实验、REMAP 操作实验和 ADC 数据采集实验。
- 大量的实验程序、详细的配置教材以及完整的多媒体教学课件。

2. MagicARM2200 - S 主板硬件特点

1）核心板

核心板采用 DeviceARM2200 工控板（6 层 PCB 设计），其特点如下：

- 2 MB NOR Flash；
- 8 MB PSRAM；
- 16 MB NAND Flash；
- 256 字节 E^2PROM；
- 采用标准 SO - DIMM144 接口；
- 全面支持 7 种型号的 144 引脚 ARM7 微控制器：LPC2200（LPC2210/2220/2212/2214/2290/2292/2294）。

2）众多扩展部件

- CF 卡/IDE 硬盘接口；
- 2 路 10 位 A/D；
- PWM DAC；
- ZLG7290 键盘管理和显示控制；
- 1 个直流电机；
- 各功能部件可通过跳线灵活选择。

3）多种通信接口

- 双串口，串口 1 带 Modem 接口；
- 10M 以太网接口；
- 1 个 USB Device（1.1）接口。

更多高清PDF图书 www.gqpdf.com

4）多种人机交互界面

- 1 个 5.2 寸 320 × 240 彩色液晶屏；
- 4 线电阻式触摸屏，带触摸笔；
- 8 个独立 LED；
- 1 个蜂鸣器；
- 8 个 7 段数码管；
- 1 个独立按键；
- 4 × 4 矩阵式键盘；
- PS/2 键盘和鼠标接口。

5）精美布局，人性化设计

- 精心设计液晶屏模具，完美感观享受；
- 精心设计触摸笔，独显个性；
- 专业、进取完美结合。

3. 多种可选核心板

- ARM7 微处理器（LPC2210/2220/2212/2214/2290/2292/2294 ARM 微处理器），其外观如图 C.2 所示。
- ARM9 微处理器（LPC3000 系列 ARM 微处理器），其外观如图 C.3 所示。
- Altera Nios II 软核 SOPC 32 位 CPU 处理器，其外观图如图 C.4 所示。
- Actel ARM7 硬核 SoC 处理器，其外观图如图 C.5 所示。

D.4 产品组成

1. 硬件部分

产品硬件部分组成如表 D.1 所列。

2. 软件部分

软件部分包括如下各种软件包和实验程序（在产品配套光盘上）：

- 移植 μC/OS - II 到 ARM7 软件包；
- 移植 μClinux 到 LPC2200 软件包；
- 数据队列软件包；
- 串口驱动软件包；
- Modem 接口软件包；
- SPI 总线软件包；
- I^2C 总线软件包；

表 D.1　产品硬件部分组成

序　号	名　称	数　量	单　位	备　注
1	MagicARM2200 - S 实验箱	1	台	包括主板、电源和机箱
2	EasyJTAG 仿真头	1	只	
3	DeviceARM2200 嵌入式工控板	1	块	CPU 用 LPC2210
4	D12 USB PACK 板	1	块	
5	20P 仿真电缆	1	条	
6	USB 线	1	条	
7	RS - 232 串行通信线	1	条	
8	并口延长线	1	条	
9	以太网交叉电缆	1	条	
10	电源线	1	条	
11	产品光盘	1	张	
12	产品保修卡	1	张	
13	《ARM 嵌入式系统基础教程》	1	本	
14	《MagicARM2200 - S 教学实验开发平台实验指导(上册)》	1	本	
15	《MagicARM2200 - S 教学实验开发平台实验指导(下册)》	1	本	

- 打印机接口软件包；
- ZLG/GPS 软件包(选配)；
- ZLG/GPRS 软件包(选配)；
- ZLG/FS 文件管理系统软件包；
- MiniGUI 图形用户界面学习版软件包(μC/OS - II & μClinux)；
- ZLG/IP TCP/IP 软件包；
- ZLG/PPP 协议软件包；
- ZLG/CF CF 卡及 IDE 硬盘软件包；
- ZLG/SD SD/MMC 读/写软件包(选配)；
- ZLG/FFS 支持写平衡的 NAND Flash 驱动软件包；

更多高清PDF图书 www.gqpdf.com

- ZLG/FTP－S 嵌入式 FTP 服务器软件包；
- ZLG/BOOT BootLoader 程序；
- ZLG/USBD12 固件程序及其驱动程序软件包。

D.5 硬件结构说明

参考《MagicARM2200－S 教学实验开发平台实验指导（上册）》的第 1 章。

D.6 应用环境

硬件环境：带有 USB、10M/100M 以太网、9 针串口和 25 针并口的 PC 机；根据实验需求，可能会用到 CF 卡、IDE 硬盘、U 盘和 Modem 等。

软件环境：Windows 98/XP/2000 操作系统、RedHat 9.0 操作系统和 ADS1.2 集成开发环境等。

D.7 产品部件规格描述

产品部件规格描述如表 D.2 所列。

表 D.2 产品部件规格描述

主要部件名称	参数/规格	备 注
DeviceARM2200 嵌入式工控板	CPU：LPC2210 RAM：MT45W4MW16 NOR Flash：SST39VF1601 NAND Flash：K9F2808U0C	核心板
液晶屏	5.2 英寸 320×240 彩色液晶屏	带触摸屏（4 线电阻式）
D12 USB PACK 板	芯片：PDIUSBD12	按 MagicARM2200－S 主板上的外设 PACK 接口设计
GPRS PACK 板	核心：BENQ 公司的 M22 GPRS 模块	选配的模件

参考文献

1 Griffith A. GCC 技术参考手册. 胡恩华译. 北京:清华大学出版社,2004

2 RUBINI A, CORBET J. LINUX 设备驱动程序(第二版). 魏永明, 骆刚, 姜君译. 北京:中国电力出版社,2002

3 周立功等. ARM 嵌入式系统基础教程. 北京:北京航空航天大学出版社,2005

4 BOVET D P., MARCOCESATI. 深入理解 LINUX 内核(第二版). 陈莉君,冯锐,牛欣源译. 孙玉芳审. 北京:中国电力出版社,2004

5 周立功等. ARM 嵌入式系统软件开发实例(一). 北京:北京航空航天大学出版社,2004

6 周立功等. ARM 嵌入式系统软件开发实例(二). 北京:北京航空航天大学出版社,2005

7 周立功等. ARM 嵌入式系统实验教程(二). 北京:北京航空航天大学出版社,2005

8 周立功等. ARM 嵌入式系统实验教程(三). 北京:北京航空航天大学出版社,2005

9 周立功等. ARM 嵌入式系统实验教程(三)——扩展实验. 北京:北京航空航天大学出版社,2006

10 周立功公司内部资料. MagicARM2200 - S 教学实验开发平台实验指导(上册). 2005

更多高清PDF图书 www.gqpdf.com

如果读者在阅读本书时发现有什么问题，或需要购买 **SmartARM2200**、**MagicARM2200** 或 **MagicARM2200 - S** 教学实验开发平台，可与广州周立功单片机发展有限公司联系。联系方式如下：

广州周立功单片机发展有限公司
地址：广州市天河区天河北路 689 号光大银行大厦 16 楼 D2 （邮编：510630）
电话：（020）38730916 38730917 38730976 38730977
传真：（020）38730925
电子邮箱：info@zlgmcu.com 网址：http://www.zlgmcu.com

技术支持
电话：（020）22644358 22644359 22644360 22644361
电子邮箱：ARM@zlgmcu.com
技术论坛：http://www.zlgmcu.com.cn http://www.21ICbbs.com

广州专卖店
地址：广州市天河区新赛格电子城 203 ~ 204 室
电话：（020）87578634 传真：（020）87578842

武汉周立功
地址：武汉市洪山区广埠屯珞瑜路 158 号 12128 室（华中电脑数码市场）
电话：（027）87168497 87168297 87168397
传真：（027）87163755

深圳周立功
地址：深圳市深南中路 2070 号电子科技大厦 A 座 22 楼 2201 室
电话：（0755）83781768 83781788 83782922 83781798
传真：（0755）83793285

成都周立功
地址：成都市一环路南一段 57 号金城大厦 612 室
电话：（028）85499320 85437446
传真：（028）85439505

重庆周立功
地址：重庆市石桥铺科园一路二号大西洋国际大厦 1116 室
电话：（023）68796438 68796439
传真：（023）68796439

北京周立功
地址：北京市海淀区知春路 113 号银网中心 715 室
电话：（010）62536178 62536179 82628073 82614433
传真：（010）82614433

上海周立功
地址：上海市北京东路 668 号科技京城东座 7E 室
电话：（021）53083452 53083453 53083496 53083497
传真：（021）53083491

杭州周立功
地址：杭州市登云路 428 号浙江时代电子商城 205 号（邮编：310000）
电话：（0571）88009275 88009932 88009933 88271834
传真：（0571）88009274

南京周立功
地址：南京市珠江路 280 号珠江大厦 2006 室
电话：（025）83613221 83613271 83603500 83603005
传真：（025）83613271

西安办事处
地址：西安市长安北路 54 号太平洋大厦 1201 室
电话：（029）87881296 87881295 83063000
传真：（029）87880865

深圳得技通 0755 - 83291449 广州得技通 020 - 87588300 上海得技通 021 - 54904533 郑州融泰 0371 - 67446736
济南启东 0531 - 86959007 青岛三友 0532 - 86961863 杭州炜煌 0571 - 88009778 兰州启东 0931 - 8279018
沈阳新华龙 024 - 23911896 北京金一倍 010 - 51659981 福州伟福启东 0591 - 3363562 上海凌煌 021 - 53081480
武汉合欣 027 - 87858706 天津启东 022 - 27412234 长沙中芯 0731 - 2237457 长沙中芯南宁 0771 - 3851552
济南历下诚信 0531 - 86123703 江西启天 0791 - 6260972 哈尔滨爱思 0451 - 82588391 合肥炜煌 0551 - 3663158
长春通州科 0431 - 5653784 昆明五华中晶 0871 - 5121865 台湾秉华 886 - 4 - 24629998 香港科技 852 - 23619975
台湾鼎丞 886 - 37 - 621880 飞碟实业 886 - 5 - 2163919

高清PDF藏 gqpdf.com